
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 14.09.2026, 22:52:26
Файлов исходного кода: 111
Суммарный объём кода: 1.96 MB
Глав/билетов в пособии: 36

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. 6. Графы. Представление графов, DFS, BFS [id: graph-basics]
7. 7. Графы. Планарные. Покраска [id: graph-planarity]
8. 7. Графы. Планарные. Покраска [id: planarity-euler]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 11. Графы. Мосты [id: bridges-code]
14. 12. Графы. Точки сочленения [id: graph-articulation-points]
15. 13. Графы. Эйлеров цикл [id: graph-euler]
16. 13. Графы. Эйлеров цикл [id: euler-path-vs-cycle]
17. Билет 14. Дейкстра [id: dijkstra]
18. Билет 15. Форд-Беллман [id: bellman-ford]
19. Билет 16. Флойд-Уоршелл [id: floyd]
20. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
21. 17. Графы. Поиск кратчайшего пути. Ускорения [id: shortest-path-accelerations]
22. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
23. 18–20. Графы. Остовное дерево: Краскал, Прима, Борувка
24. 19. Графы. Остовное дерево. Прима [id: mst-prim]
25. 20. Графы. Остовное дерево. Борувка [id: mst-kruskal]

- 18. src/data/content_temp.ts
- 19. src/data/content.ts
- 20. src/data/debugRunner.ts
- 21. src/data/glossary.ts
- 22. src/data/graphContent.ts
- 23. src/data/quizzes.ts
- 24. src/data/vizStepBus.ts
- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_segtree.ts
- 27. src/data/tickets/ticket14_20.ts
- 28. src/data/tickets/ticket18_20_mst.ts
- 29. src/data/tickets/ticket21_24.ts
- 30. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 31. src/components/ArticulationPointsViz.tsx
- 32. src/components/BellmanFordViz.tsx
- 33. src/components/BfsViz.tsx
- 34. src/components/ChapterImage.tsx
- 35. src/components/ChapterNav.tsx
- 36. src/components/ChapterQuiz.tsx
- 37. src/components/ChapterView.tsx
- 38. src/components/ComplexityViz.tsx
- 39. src/components/DfsBridgesSimulator.tsx
- 40. src/components/DijkstraViz.tsx
- 41. src/components/DPVisualizer.tsx
- 42. src/components/EulerSimulator.tsx
- 43. src/components/FloydViz.tsx
- 44. src/components/GraphComponentsViz.tsx
- 45. src/components/GraphTraversalViz.tsx
- 46. src/components/HeapViz.tsx
- 47. src/components/hints/demoKit.tsx
- 48. src/components/hints/demosExtra.tsx
- 49. src/components/hints/demosGraph.tsx
- 50. src/components/hints/demosStruct.tsx
- 51. src/components/hints/MiniDemo.tsx

86. src/layout/header.html

Утилиты

87. src/utils/cn.ts

88. src/utils/exportHtml.ts

89. src/utils/ticketLinks.ts

Скрипты сборки и экспорта

90. scripts/audit-coverage.mjs

91. scripts/audit-terms.mjs

92. scripts/export/build-guide-pdf.mjs

93. scripts/export/build-pdf.mjs

94. scripts/export/capture-viz-shots.mjs

95. scripts/export/collect-code.mjs

96. scripts/export/common.mjs

97. scripts/export/html-to-blocks.mjs

98. scripts/export/render-pages.mjs

99. scripts/verify/browser-checks.tsx

100. scripts/verify/check-links.mjs

101. scripts/verify/check-reference-code.mjs

102. scripts/verify/dom-env.mjs

103. scripts/verify/live-matrix.test.mjs

104. scripts/verify/probe-compiler.tsx

105. scripts/verify/README.md

106. scripts/verify/run-browser-checks.mjs

107. scripts/verify/run-probe.mjs

CI / автоматизация

108. .github/workflows/deploy-pages.yml

109. .github/workflows/rebuild-pdf.yml

110. .github/workflows/verify.yml

Прочее

111. .ai_skills/universal_guide_authoring/SKILL.md

```
contents: read
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
+ group: pages
+ cancel-in-progress: true
+
+jobs:
+  build:
+    name: Build static site
+    runs-on: ubuntu-latest
+    steps:
+      - name: Checkout
+        uses: actions/checkout@v6
+
+      - name: Setup Node.js
+        uses: actions/setup-node@v7
+        with:
+          node-version: 24
+          cache: npm
+
+      - name: Install dependencies
+        run: npm ci --no-audit --no-fund
+
+      - name: Build for the repository subpath
+        env:
+          VITE_BASE: /algv0/
+        run: npm run build
+
+      - name: Make GitHub Pages fallback
+        run: cp dist/index.html dist/404.html
+
+      - name: Upload Pages artifact
+        uses: actions/upload-pages-artifact@v5
+        with:
+          path: ./dist
+
+  deploy:
+    name: Publish to GitHub Pages
+    needs: build
+    runs-on: ubuntu-latest
+    environment:
```

```
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot]')"

    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommends imagemagick
          # На Ubuntu политика ImageMagick иногда запрещает конвертировать текст в изображение
          # разрешаем то, что нужно пайплайну (text -> png)
          for policy in /etc/ImageMagick-6/policy.xml /usr/share/ImageMagick-6/policy.xml; do
            if [ -f "$policy" ]; then
              sed -i 's/<policy domain="text" rights="none"/<policy domain="text" rights="write"/g' $policy
            fi
          done
```

```
      name: full-code-export
      path: |
        public/export/full_code_all_pages.pdf
        public/export/full_code_all_pages.txt
      if-no-files-found: error
      retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action@users.noreply.github.com"
    git add -- public/export/full_code_all_pages.*
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full-code-страниц"
    git push origin "HEAD:${GITHUB_REF_NAME}"
  ....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

 steps:
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

```
````html
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, >
    <link rel="icon" type="image/svg+xml" href="%BASE_URL%" />
    <title> Дискретка для тупых – Интерактивный гид по
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen" >
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
````
```

### Patch относительно `main`

```
````diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
   <head>
     <meta charset="UTF-8" />
     <meta name="viewport" content="width=device-width, >
+    <link rel="icon" type="image/svg+xml" href="%BASE_URL%" />
     <title> Дискретка для тупых – Интерактивный гид по
   </head>
   <body class="bg-slate-950 text-slate-100 min-h-screen" >
````
```

## `public/favicon.svg`

### Полное содержимое после изменений



```

import { SimulatorHub } from "../components/SimulatorHub
import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeTab, setActiveTab] = useState<"guide" |
 const [activeChapterId, setActiveChapterId] = useStat
 const [modalVizId, setModalVizId] = useState<string |

 const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
 const activeChapter = chapters[index] ?? chapters[0];
 const prev = index > 0 ? chapters[index - 1] : undefin
 const next = index < chapters.length - 1 ? chapters[in

 const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
 }, []);

 // Стрелки ← → листают темы (как на обучающих сайтах)
 useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id)
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown",
 }, [goTo, next, prev]);

 const progress = useMemo(() => ((index + 1) / chapter

 return (
 <div className="min-h-screen bg-slate-950 text-slate

```

```

 <ChapterView
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 />
 </main>
</div>
</>
) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
) : (
 <PythonCompiler />
)
}

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>
);
}
....

```

```
`src/components/Navbar.tsx`
```

```
Полное содержимое после изменений
```

```
```tsx
```

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, C
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
  activeTab: 'guide' | 'simulator' | 'compiler';
  setActiveTab: (tab: 'guide' | 'simulator' | 'compiler
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeT
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };
};
```

```
  return (
    <header className="bg-slate-900 border-b border-sla
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg
        ap-3 lg:gap-0">
          <div className="flex items-center gap-3 w-full
            <div className="bg-gradient-to-tr from-indigo
              k-0">
                <Sparkles className="w-6 h-6" />
              </div>
            <div className="min-w-0 flex-1">
```

```

        className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
            activeTab === 'compiler'
                ? 'bg-emerald-600 text-white shadow-lg
                : 'text-slate-400 hover:text-white'
        }}
    >
    <Terminal className="w-4 h-4" /> Python
</button>
<a
    id="download-pdf-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
    title="Скачать полный PDF сборник всех стра
    >
    <FileDown className="w-4 h-4" /> Скачать PD
</a>
<a
    id="download-txt-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
: bg-sky-600/20 border border-sky-500/30 tra
    title="Скачать полный TXT файл со всем кодом
    >
    <FileText className="w-4 h-4" /> TXT
</a>
<button
    id="download-html-btn"
    onClick={saveBook}
    disabled={busy}
    className="flex items-center justify-center

```



```

-----
export function PythonCompiler() {
  const [code, setCode] = useState(STARTER_CODE);
  const [stdin, setStdin] = useState("");
  const [output, setOutput] = useState("Нажмите «Запустить»");
  const [running, setRunning] = useState(false);
  const [runtimeReady, setRuntimeReady] = useState(false);
  const [runtimeMessage, setRuntimeMessage] = useState("");

  const runCode = useCallback(async () => {
    if (running) return;

    setRunning(true);
    setOutput("");
    setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не выполнила");

    const stdout: string[] = [];
    const stderr: string[] = [];
    const inputLines = stdin.split(/\r?\n/);

    try {
      const runtime = await getPythonRuntime();
      setRuntimeReady(true);
      setRuntimeMessage("Python готов — код выполняется");

      runtime.setStdout({ batched: (message) => stdout.push(message) });
      runtime.setStderr({ batched: (message) => stderr.push(message) });
      runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
      await runtime.runPythonAsync(code);

      const text = [...stdout, ...stderr].join("");
      setOutput(text || "Готово: программа ничего не выполнила");
    } catch (error) {
      const captured = [...stdout, ...stderr].join("");
      setOutput(`${captured}${captured && !captured.endsWith('.') ? '.' : ''}`);
      setRuntimeMessage("Не удалось выполнить код — подробности в консоли");
    } finally {
      setRunning(false);
    }
  }, [code, stdin]);
}

```

```

        <span className="text-[11px] text-slate
</div>
<div className="flex items-center gap-2">
  <button
    type="button"
    onClick={reset}
    className="inline-flex items-center gap-2 text-slate-800 transition-colors"
  >
    <RotateCcw className="w-3.5 h-3.5" />
  </button>
  <button
    type="button"
    onClick={() => void runCode()}
    disabled={running}
    className="inline-flex items-center gap-2 text-white text-xs font-bold transition-colors"
  >
    {running ? <Loader2 className="w-3.5 h-3.5" /> : "Запускаю..."}
    {running ? "Запускаю..." : "Запустить"}
  </button>
</div>
</div>

<textarea
  value={code}
  onChange={(event) => setCode(event.target.value)}
  onKeyDown={(event) => {
    if ((event.ctrlKey || event.metaKey) && event.key === "Enter") {
      event.preventDefault();
      void runCode();
    }
  }}
  spellCheck={false}
  aria-label="Код Python"
  className="block w-full min-h-[360px] resize-none focus:ring-2 focus:ring-inset focus:ring-slate-500"
  placeholder="Напишите Python-код..."

```

```
        <div className="flex gap-2 rounded-xl border border-gray-200 p-4">
          <Info className="w-4 h-4 shrink-0 text-indigo-600">
            <span>Ctrl или Cmd + Enter запускает код. Введите код в поле ниже</span>
          </div>
          <div className="flex gap-2 rounded-xl border border-gray-200 p-4">
            <Info className="w-4 h-4 shrink-0 text-amber-600">
              <span>Первый запуск скачивает примерно несколько килобайт кода</span>
            </div>
          <div className="flex gap-2 rounded-xl border border-gray-200 p-4">
            <Info className="w-4 h-4 shrink-0 text-emerald-600">
              <span>Код выполняется в браузере и не отправляется на сервер</span>
            </div>
          </div>
        </div>
      </div>
    </main>
  );
}
....
```

Patch относительно `main`

```
````diff
diff --git a/src/components/PythonCompiler.tsx b/src/components/PythonCompiler.tsx
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python и установит библиотеки
+name = "алгоритмы"
+
+def main():
+ print("Привет, мир!")
+ # Здесь ваш код
+
+main()
+
+`;
```



```

+
+ const runCode = useCallback(async () => {
+ if (running) return;
+
+ setRunning(true);
+ setOutput("");
+ setRuntimeMessage(runtimeReady ? "Выполняю код..." :
+
+ const stdout: string[] = [];
+ const stderr: string[] = [];
+ const inputLines = stdin.split(/\r?\n/);
+
+ try {
+ const runtime = await getPythonRuntime();
+ setRuntimeReady(true);
+ setRuntimeMessage("Python готов – код выполняется");
+
+ runtime.setStdout({ batched: (message) => stdout
+ runtime.setStderr({ batched: (message) => stderr
+ runtime.setStdin({ stdin: () => (inputLines.length
+ await runtime.runPythonAsync(code);
+
+ const text = [...stdout, ...stderr].join("");
+ setOutput(text || "Готово: программа ничего не выв
+ } catch (error) {
+ const captured = [...stdout, ...stderr].join("")
+ setOutput(`${captured}${captured && !captured.en
+ setRuntimeMessage("Не удалось выполнить код – пр
+ } finally {
+ setRunning(false);
+ }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+ setCode(STARTER_CODE);
+ setStdin("");
+ setOutput("Нажмите «Запустить», чтобы увидеть резу
+ };

```

```

late-800 transition-colors"
+
+ >
+ <RotateCcw className="w-3.5 h-3.5" />
+ </button>
+ <button
+ type="button"
+ onClick={() => void runCode()}
+ disabled={running}
+ className="inline-flex items-center
-60 text-white text-xs font-bold transition-colors"
+ >
+ {running ? <Loader2 className="w-3.5
+ {running ? "Запускаю..." : "Запустить"}
+ </button>
+ </div>
+ </div>
+
+ <textarea
+ value={code}
+ onChange={(event) => setCode(event.target
+ onKeyDown={(event) => {
+ if ((event.ctrlKey || event.metaKey) &
+ event.preventDefault();
+ void runCode();
+ }
+ }}
+ spellCheck={false}
+ aria-label="Код Python"
+ className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+ placeholder="Напишите Python-код..."
+ />
+
+ <div className="border-t border-slate-800
+ <label className="block px-4 pt-3 text-[
+ Ввод для input() • по одной строке на
+ </label>
+ <textarea

```

```

+ </div>
+ <div className="flex gap-2 rounded-xl border
+ <Info className="w-4 h-4 shrink-0 text-eme
+ Код выполняется в браузере и не отпра
+ </div>
+ </div>
+ </div>
+ </main>
+);
+}
+...

```

## `vite.config.ts`

### Полное содержимое после изменений

```

````typescript
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
  // На GitHub Pages приложение живёт в /algv0/, локальн
  // VITE_BASE можно переопределить, если репозиторий б
  base: process.env.VITE_BASE || "/",
  plugins: [react(), tailwindcss(), viteSingleFile()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "src"),
    },
  },
},

```

Универсальное пособие • полный исходный код

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7

AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

****CRITICAL DIRECTIVE****:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, styles and components required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML elements.

ФАЙЛ 3/111: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2

Gemini App Canvas Instructions

Ты — AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas).
При запросе пользователя создать или сгенерировать новую страницу, ты ****строго**** придерживаешься готовых шаблонов разметки, описанных в файле ``.ai_skills/universal_guide_authoring/SKILL.md``.

Правила для малых страниц (Билетов / Глав)

- **Единый стиль:**** Всегда используй Tailwind CSS и HTML-шаблоны, описанные в файле ``.ai_skills/universal_guide_authoring/SKILL.md``.
- **Структура JSON/TS:**** Каждая новая малая страница должна быть описана в файле ``.ai_skills/universal_guide_authoring/SKILL.md`` для вставки в ``.src/data/content.json`` или ``.src/data/content.ts``.
- **Строгая структура контента:****
 - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
 - Определение/правило в центре (``border-blue-500`` или ``border-blue-500``).
 - Обязательная сетка аналогий (2 колонки: неформальный язык и формальный язык).

ФАЙЛ 6/111: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 54 | РАЗМЕР: 2

```
{
  "name": "algv0-exam-guide",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "audit:content": "node scripts/audit-coverage.mjs",
    "audit:terms": "node scripts/audit-terms.mjs",
    "export:txt": "node scripts/export/collect-code.mjs",
    "export:png": "node scripts/export/render-pages.mjs",
    "export:pdf": "node scripts/export/build-pdf.mjs",
    "export": "npm run export:txt && npm run export:png",
    "export:guide": "node scripts/export/build-guide-pdf.mjs",
    "export:shots": "node scripts/export/capture-viz-shots.mjs",
    "verify:unit": "node scripts/verify/live-matrix.test.mjs",
    "verify:code": "node scripts/verify/check-reference.mjs",
    "verify:links": "node scripts/verify/check-links.mjs",
    "verify:dom": "node scripts/verify/run-browser-check.mjs",
    "verify:probe": "node scripts/verify/run-probe.mjs",
    "verify": "npm run verify:unit && npm run verify:links",
  },
  "dependencies": {
    "clsx": "2.1.1",
    "lucide-react": "^1.21.0",
    "motion": "^12.40.0",
    "pdfkit": "^0.20.1",
    "react": "19.2.6",
    "react-dom": "19.2.6",
  }
}
```

Универсальное пособие • полный исходный код

```
-----  
npm run dev          # предпросмотр на http://0.0.0.0:5  
npm run build        # сборка в dist/ (single-file)  
````
```

## Экспорт кода: код → txt → png → pdf

Пайплайн собирает **\*\*весь\*\*** исходный код репозитория в о

| Команда              | Шаг       | Результат                               |
|----------------------|-----------|-----------------------------------------|
| `npm run export:txt` | код → txt | `public/export/full_code_all_pages.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/`                     |
| `npm run export:pdf` | png → pdf | `public/export/full_code_all_pages.pdf` |
| `npm run export`     | всё сразу | txt + png + pdf                         |

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash  
sudo apt-get install -y imagemagick fonts-dejavu-core  
````
```

Настройки через переменные окружения:

- \* `EXPORT\_DENSITY=200` – DPI растеризации (по умолчанию 300)
- \* `EXPORT\_GRAYSCALE=1` – оттенки серого вместо 1-битного

### Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается **\*\*на каж**  
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док  
и коммитит обновлённые `public/export/full\_code\_all\_pages`  
(с меткой `[skip ci]`, чтобы не заиклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code`

## Аудит контента

```
```bash
```

Заголовки

- * Заголовки секций: Используют стандартный тег `<h2>` и
- * Подзаголовки: Используют тег `<h3>`.

Текст и параграфы

- * Обычный текст содержится в тегах `<p>`.
- * Блоки "Шиза" (Аналогии) и другая текстовая инфографика параграфы `<p>`.

Математические формулы

В приложении используется MathJax.

- * В самом исходном коде страницы (и внутри тегов) формулы.
- * Таким образом, при извлечении текстового содержимого математически остаются в валидном Markdown-формате латеха (и

Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторы

- * Они помещены внутрь оберток с классом `<div class="img">`
- * Все отрисовки находятся внутри векторного тега `<svg>`
- * Подпись к картинке, если она есть, идет сразу после `<img>`

Списки

- * Стандартные списки оформлены через `<ul class="list-n">`

Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `<li>` получают маркер `-`.
5. Результат извлекается в виде текста и чистится от лишнего

Универсальное пособие • полный исходный код

```
-----
* `stack-dfs` → `StackViz.tsx` — **Стек и DFS**
* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь и BFS**
* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой поиск**
* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Деревья отрезков**
* `dynamic-programming` → `DPVisualizer.tsx` — **Динамическое программирование**
---
```

Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухари»)

Глава	id	Комментарий
15. Планарность: K5, K3,3 и формула Эйлера	`planarity-euler`	т «на пальцах».
16. Флойд (Флойд–Уоршелл)	`floyd`	Есть матричный алгоритм
17. Алгоритм Джонсона	`johnson-algo`	Перевзвешивание
Обзор: Кратчайшие пути и Графы	`intro`	Вводная страница
Эйлер: Путь ≠ Цикл	`euler-path-vs-cycle`	Ключевая концепция

Уровень 4. Есть аналогия, но НЕТ 2D-визуализации («сложно»)

> Замечание: «5. Двумерная матрица префиксных сумм» пока не реализована

Глава	id	Чего не хватает
3. Декартово дерево (Treap)	`treap`	Нет ни визуализации, ни кода
8. Графы. Компоненты связности	`graph-components`	Нет кода
24. Классы сложности, сведение задач	`complexity-classes`	Нет кода

Сводная таблица по всем главам

#	Глава	Аналогий	2D-виз.	SVG	Картинка
---	---	---	---	---	---

ФАЙЛ 9/111: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

```
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "types": ["node"],

    /* Bundler mode */
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",

    /* Path mapping */
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"]
    },

    /* Linting */
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true
  },
  "include": ["src", "vite.config.ts"]
}
```

```
    },  
  });  
}
```

РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

ФАЙЛ 11/111: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 218 | РАЗМЕР: 11.3 КБ

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";  
import { PanelLeftClose, PanelLeftOpen } from "lucide-react";  
import { chapters } from "../data/content";  
import { Navbar } from "../components/Navbar";  
import { Sidebar } from "../components/Sidebar";  
import { MobileToc } from "../components/MobileToc";  
import { ChapterView } from "../components/ChapterView";  
import { ChapterNav } from "../components/ChapterNav";  
import { SimulatorModal } from "../components/hints/SimulatorModal";  
import { LiteGuide } from "../components/LiteGuide";  
import { PythonCompiler } from "../components/PythonCompiler";
```

```
/**
```

```
 * «Лёгкий» режим для печати и внешних парсеров: `?lite`
```

```
 * или `?lite=1&topic=mst` (одна тема). Без навигации, без поиска.
```

```
 */
```

```
function readLiteMode(): { lite: boolean; topic?: string } {  
  if (typeof window === "undefined") return { lite: false };  
  const q = new URLSearchParams(window.location.search);  
  const lite = q.has("lite") || q.get("view") === "lite";  
  return { lite, topic: q.get("topic") ?? undefined };  
}
```

```
export default function App() {  
  const liteMode = useMemo(readLiteMode, []);
```

```

    setSidebarMode((m) => {
      const next = m === "wide" ? "compact" : m === "co
      window.localStorage.setItem("sidebarMode", next);
      return next;
    });
  }, []);
const sidebarCollapsed = sidebarMode === "hidden";

// Десктопный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
  const apply = () => {
    const el = contentRef.current;
    if (!el) return;
    el.style.paddingRight = compilerOpen && window.inn
  };
  apply();
  window.addEventListener("resize", apply);
  return () => window.removeEventListener("resize", a
}, [compilerOpen, compilerWidth]);

const index = Math.max(
  0,
  chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefin
const next = index < chapters.length - 1 ? chapters[in

const goTo = useCallback((id: string) => {
  setActiveChapterId(id);
  window.scrollTo({ top: 0, behavior: "smooth" });
}, []);

// Открыть компилятор и подогнать визуализацию вверх:
const openCompilerWithViz = useCallback(() => {
  setCompilerOpen(true);
  requestAnimationFrame(() => {

```

```

<div className="flex flex-col lg:flex-row max-w-
  /* Сайдбар только на десктопе — на мобильном
  <div
    id="sidebar-root"
    className={`hidden lg:block shrink-0 border
      -all duration-200 print:hidden ${sidebarCol
    " : "lg:w-56"}` }
  >
    {!sidebarCollapsed && (
      <Sidebar selectedChapterId={activeChapterId}
    )}
  </div>
  /* Вкладка-переключатель содержания у левого
  <button
    type="button"
    onClick={toggleSidebar}
    className="hidden lg:flex items-center gap-1
    border-slate-700 bg-slate-900/90 px-1 py-3
    aria-label={sidebarMode === "wide" ? "Сжать
    ание"}
    title={sidebarMode === "wide" ? "Сжать содер
    }
  >
    {sidebarCollapsed ? <PanelLeftOpen className=
    <span className="text-[10px] font-bold [wri
  </button>

  <main id="main-content" className="flex-1 min
    /* Шапка главы с быстрыми переходами */
    <div className="flex items-center justify-b
      <div className="min-w-0">
        <div className="text-[10px] uppercase t
          {activeChapter.category || "Универсальн
        </div>
        <h2 className="text-base sm:text-xl fon
          {activeChapter.title}
        </h2>
      </div>
    </div>

```

```
        onHeightChange={setCompilerHeight}
        onClose={() => setCompilerOpen(false)}
      />
    ))
  </div>
);
}
```

ФАЙЛ 12/111: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 129 | РАЗМЕР: 5.1 K

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, text";

/** Числовые константы TreeWalker: глобального `NodeFile`
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

const escapeRe = (s: string) => s.replace(/[\.\*\+\?\^\$\{\}\(\)]/g, "\\$1");

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
```

```

    re.lastIndex = 0;
    if (!re.test(text)) continue;
    re.lastIndex = 0;

    const frag = document.createDocumentFragment();
    let last = 0;
    let m: RegExpExecArray | null;

    while ((m = re.exec(text)) !== null) {
        const surface = m[1].toLowerCase();
        const id = labelToId.get(surface);
        if (!id) continue;

        const usedTerm = perTerm.get(id) ?? 0;
        const usedSurface = perSurface.get(surface) ?? 0;
        if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
        perTerm.set(id, usedTerm + 1);
        perSurface.set(surface, usedSurface + 1);

        if (m.index > last) frag.appendChild(document.createElement("span"));
        const span = document.createElement("span");
        span.className = "term-hint";
        span.dataset.termId = id;
        span.tabIndex = 0;
        span.setAttribute("role", "button");
        span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
        span.textContent = m[1];
        frag.appendChild(span);
        last = m.index + m[1].length;
    }

    if (last === 0) continue;
    if (last < text.length) frag.appendChild(document.createElement("span"));
    textNode.parentNode?.replaceChild(frag, textNode);
}

/**

```

```
.neon-glow-blue {
  box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
}
.neon-glow-emerald {
  box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
}
.neon-glow-rose {
  box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
}
.neon-glow-yellow {
  box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
}
.no-scrollbar::-webkit-scrollbar {
  display: none;
}
.no-scrollbar {
  scrollbar-width: none;
}
}

@keyframes pulse-gold {
  0%,
  100% {
    stroke: #eab308;
    stroke-width: 5;
    filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5))
  }
  50% {
    stroke: #fef08a;
    stroke-width: 8;
    filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9))
  }
}

.animate-pulse-gold {
  animation: pulse-gold 1.5s infinite;
}
```

```
    color: #fff;
    outline: none;
}
@media (hover: none) {
    .term-hint {
        border-bottom-style: solid;
        background: rgba(99, 102, 241, 0.12);
    }
}

/* Ссылка на другой билет («...берите Форда–Беллмана (билет
.ticket-link {
    color: #a5b4fc;
    border-bottom: 1px dotted rgba(165, 180, 252, 0.7);
    text-decoration: none;
    border-radius: 3px;
    padding: 0 1px;
    cursor: pointer;
    transition:
        background-color 0.15s ease,
        color 0.15s ease;
}
.ticket-link:hover,
.ticket-link:focus-visible {
    background: rgba(99, 102, 241, 0.25);
    color: #fff;
    outline: none;
}

/* Контент главы приходит готовым HTML – чиним его под
.chapter-body {
    max-width: 100%;
    overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
    min-width: 0;
}
```



```
    line-height: 1.3 !important;
  }
  .chapter-body :where(h3) {
    font-size: 1.05rem !important;
    line-height: 1.35 !important;
  }
  .chapter-body :where(.p-6, .p-8) {
    padding: 0.9rem !important;
  }
  .chapter-body :where(.text-3xl) {
    font-size: 1.4rem !important;
  }
  .chapter-body :where(.text-2xl) {
    font-size: 1.2rem !important;
  }
  .chapter-body :where(.text-xl) {
    font-size: 1.05rem !important;
  }
}

/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
  width: 8px;
  height: 8px;
}
::-webkit-scrollbar-track {
  background: #0f172a;
}
::-webkit-scrollbar-thumb {
  background: #334155;
  border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
  background: #475569;
}

/* Индикатор длительности кадра в пошаговых визуализаци.
@keyframes demoProgress {
```

```
main {
  padding: 0 !important;
  max-width: none !important;
}

/* текст темы – чёрным по белому, без теней и плашек
article.chapter-body .prose,
article.chapter-body .prose * {
  color: #111827 !important;
  background-color: transparent !important;
  background-image: none !important;
  box-shadow: none !important;
  text-shadow: none !important;
  border-color: #d1d5db !important;
}

article.chapter-body .prose pre,
article.chapter-body .prose code {
  background-color: #f3f4f6 !important;
  color: #111827 !important;
  font-size: 9pt;
}

article.chapter-body .prose table {
  border-collapse: collapse;
}

article.chapter-body .prose th,
article.chapter-body .prose td {
  border: 1px solid #9ca3af !important;
}

article.chapter-body .prose h2,
article.chapter-body .prose h3,
article.chapter-body .prose h4 {
  page-break-after: avoid;
}
```

Универсальное пособие • полный исходный код

ФАЙЛ 14/111: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

ФАЙЛ 15/111: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

```
export interface Chapter {
  id: string;
  title: string;
  type: "html" | "markdown";
  content: string;
  description?: string;
  category?: string;
}
```

ФАЙЛ 16/111: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

```
/// <reference types="vite/client" />
```

```
<div class="flex items-center mb-6 flex-wrap gap-3">
  <span class="bg-indigo-600 text-white px-4 py-1">
    <h2 class="text-2xl sm:text-3xl font-bold text-white">
  </div>
```

```
<div class="space-y-8">
```

```
  <div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-blue-400">
```

```
    <div class="bg-slate-900 p-4 rounded-lg mb-4">
      <p class="text-lg text-blue-300 italic">
      <p class="text-xl font-mono text-white">
      <p class="text-sm font-mono text-slate-300">
    ] [k] </p>
  </div>
```

```
<p class="text-slate-300 text-sm mb-4">Ячейка
лины <span class="font-mono text-emerald-300">
">i</span>. Любой отрезок накрывается <b>двумя
перация <b>идемпотентна</b>: <span class="font-mono text-emerald-300">min</span>, <span class="font-mono text-emerald-300">
, побитовые И и ИЛИ.</p>
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-indigo-300 italic">
  <p class="text-xl font-mono text-white">
  <p class="text-sm font-mono text-slate-300">
вет = op из четырёх блоков kx × ky, накрывае
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Двумя
по столбцам. Отсюда и цена: уровней становится
s="font-mono text-emerald-300">log N</span>
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
  <div class="bg-slate-900 border border-slate-700">
    <p class="text-[10px] uppercase tracking-wider">
```

```

        <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
            Аналогия 2: Торт-пирамидка
        </div>
        <p class="text-slate-300 text-sm mb-4">
            ия, зато красивый, как пирамидка: каждый из
            сех блоков  $2^{k_x} \times 2^{k_y}$ ), и ответ на запрос
            ой операции нет: крем обратно в торт не запы
             $n(x, x) = x$ ), и цена  $O(1)$  – сто коржей вмес
            <div class="rounded-lg bg-slate-950">
                <div><span class="text-slate-500">
                     $n \ O(1)$ </span></div>
                <div><span class="text-slate-500">
                    = 100 копий матрицы</span></div>
            </div>
            <p class="text-[12px] text-slate-400">
                шает). Менять элементы тоже нельзя. Сумму ч
            </div>
        </div>
        <p class="text-slate-400 text-sm mt-4">Поря
            ерению) → <b>2D: построение</b> (уровни <span
            ос</b> (прямоугольник раскладывается на чет
        </div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
    <h3 class="text-xl font-bold text-rose-400">
    <p class="text-slate-300 text-sm mb-4">Эти
        за <span class="font-mono text-emerald-300">
        они решают разные задачи.</p>

<div class="bg-slate-800/70 p-5 rounded-lg">
    <p class="text-sm font-bold text-white">
    <p class="text-slate-300 text-sm mb-3">
        цы, у разреженной таблицы – размер, умножен
        <table class="w-full text-xs text-left">
            <thead>
                <tr class="text-slate-500 border">
                    <th class="py-1.5 pr-2 font
```

Почему префиксная тонкая

</div>

<p class="text-slate-300 text-sm mb-1" style="border: 1px solid #000; padding: 5px; display: inline-block;">P[i][j]</p>
ругой прямоугольник выражается через четыре

</div>

<div class="bg-slate-900 p-6 rounded-lg">

<div class="absolute -top-3 left-4 border-rose-500 shadow-md">

Почему разреженная жирная

</div>

<p class="text-slate-300 text-sm mb-1" style="border: 1px solid #000; padding: 5px; display: inline-block;">ум на его части не вычислишь. Приходится хранить
xt-emerald-300">log N × log M. На ка

</div>

</div>

<p class="text-slate-300 text-sm mt-4">Вывод:
лицу – перекрывающиеся блоки посчитают
 – у минимума нет обратной операции. Стр

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border">

<h3 class="text-xl font-bold text-amber-400">

<div class="bg-rose-950/30 p-5 rounded-lg border">

<ul class="list-disc pl-5 space-y-2 text">

Сумма через разреженную таблицу пре-
ефиксные (билет 5).

Память. N = M = 1000 даём массив, вы
считайте ячейки, потом пишите код.

Один log вместо двух. Вспомогательная таблица
span class="font-mono text-emerald-300">str[i][j] = st[i][j] + st[i-1][j-1]

Выход за границу в запросе. Вспомогательная таблица
l и str[i][j] = st[i][j] + st[i-1][j-1]

Нужны обновления. Разреженная таблица (билет 1), а не она.


```

        else
            st[r][c][kx][ky] = min(
    
```

```

// запрос минимума на прямоугольнике (R1,C1)..(R2,C2)
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 &lt;&lt; ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 &lt;&lt; kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);
    
```

Четыре запроса к четырем углам запроса и вместе с ним образуют минимальную сумму.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 = 100
- Префиксная сумма для той же матрицы

Почему это так много? Потому что для каждого запроса нужно вычислить сумму по всем уровням, а запросов очень много.

Вторая половина этого сравнения:

```
<div class="text-center my-6">
  <p class="text-slate-400 text-xs uppercase">
  <p class="my-1 font-bold leading-none">
    <span class="text-2xl text-slate-300">И</span>
  </p>
  <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
  <div class="bg-slate-800 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
      Аналогия 1: Split – таможня
    </div>
    <p class="text-slate-300 text-sm">S
х». Спуск от корня: узел, который вместе с
скрытия</b> – досматривается только одна ве
  </div>
```

```
  <div class="bg-slate-900 p-6 rounded-lg">
    <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
      Аналогия 2: Merge – стыковка
    </div>
    <p class="text-slate-300 text-sm">M
аг – одно сравнение: чей у у корня больше,
сохраняет порядок x). Итого  $O(h)$  сравнений
  </div>
</div>
```

```
<div class="bg-slate-900/40 rounded-lg bord
  <p class="text-slate-300 text-sm mb-2">
  <p class="text-slate-400 text-sm mb-1">
быстрее  $n \cdot \log n$ .</p>
  <p class="text-slate-400 text-sm mb-1">
  <p class="text-slate-400 text-sm">• «У6
```



```

<summary class="p-4 font-bold text-slate-700 border-slate-700/50">
    Код: вставка, merge, erase (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300">
    <pre class="bg-slate-950 p-4 rounded">
def insert(root, key, pri):
    if root is None:
        return Node(key, pri)
    if key <= root.key: # равные x — левый
        root.left = insert(root.left, key, pri)
    else:
        root.right = insert(root.right, key, pri)
    return root

for key, pri in pairs_sorted_by_pri_desc: # порядок э
    root = insert(root, key, pri)</pre>
    <pre class="bg-slate-950 p-4 rounded">
def merge(a, b): # все x в a < все x в b
    if not a or not b: return a or b
    if a.pri > b.pri:
        a.right = merge(a.right, b); return a
    b.left = merge(a, b.left); return b

def erase(root, key): # удалить произвольный элемент
    if key < root.key: root.left = erase(root.left, key)
    elif key > root.key: root.right = erase(root.right, key)
    else: root = merge(root.left, root.right)
    return root</pre>
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "splay-tree",
        title: "4. Splay-дерево",
    }

```

по факту

: ничего не хранит, дереву разрё

```
<div class="grid grid-cols-1 xl:grid-cols-2"
  <div class="bg-slate-800 p-6 rounded-lg"
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
```

Аналогия 1: Стопка бумаг

```
</div>
<p class="text-slate-300 text-sm">С
тировку не ведёшь, но через неделю самое ну
ерх».</p>
```

```
</div>
<div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
```

Аналогия 2: Тропинка через га.

```
</div>
<p class="text-slate-300 text-sm">И,
рно вдвое. Первый проход дорогой, зато опла
ы.</p>
```

```
</div>
</div>
```

```
<h4 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Повор
Среднее поддерево  $\beta$  (между  $x$  и  $p$ ) – единств
 $x$ , и под  $p$ .</p>
```

```
<pre class="bg-slate-950 p-4 rounded-lg ove
ading-relaxed">
```

```

      /  \ \
     x    y
    /  \ \
   α    β
      /  \ \
     x    y
    /  \ \
   α    β

      x выше
      -----> α
      /  \ \
     p    r
    /  \ \
   β    γ

      p выше
      <-----
      /  \ \
     p    r
    /  \ \
   β    γ
  </pre>
```

```
<p class="text-slate-300 text-sm mt-3">Обхо
омать дерево поиска поворотами невозможно в
омент «отстёжки», а врезка «Механика одного
```

```
<h4 class="text-lg font-bold text-white mt-3">
```

```
<pre class="bg-slate-950 p-4 rounded-lg">
    амбук из 8 узлов 8→7→6→5→4→3→2→1, ищем 1 (ч
наивно (x, потом x):    3 × «два зига» + Zig → высота 8
Zig-Zig (p, потом x):    3 × Zig-Zig    + Zig → высота 8
бамбук из 16 узлов:      наивно 16 → 16, Zig-Zig 16 → 10<
```

```
    <p class="text-slate-300 text-sm mt-3">
де: <code class="bg-slate-950 px-2 py-0.5 r
s="bg-slate-950 px-2 py-0.5 rounded font-mon
изации не будет.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-6">Полн
м 1. Спуск 8 узлов, затем <b>три Zig-Zig по
8 до 6 – линия перестала быть линией. Наивно
о перевёрнутой. Важно не врать себе: на кор
ван не на одном запросе, а в сумме. Следующ
по пути</b>. Отсюда амортизация: дорогой за
дом потенциала: потенциал  $\Phi$  = сумма по узла
ь его; отсюда же теорема о статической опти
ь запросов).</p>
```

```
<div class="bg-slate-900/40 rounded-lg bord
    <p class="text-slate-300 text-sm mb-2">
    <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
    <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечиваетс
    <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
ое <code class="bg-slate-950 px-2 py-0.5 ro
одиночная операция – до <code class="bg-sla
>O(n)</code>; n вставок подряд – амортизиро
-300 border border-slate-800">O(n log n)</c
```

```
<p class="text-slate-300 text-sm mt-4">Все
```

```

        rotate_up(x)
    else:                                     # ZIG-ZAG: НИЖНЯЯ
        rotate_up(x)
        rotate_up(x)
    return x

def find(root, key):                         # поиск: спуск + s
    cur, last = root, None
    while cur:
        last = cur
        if key == cur.key: break
        cur = cur.left if key < cur.key else cur.right
    return splay(last) if last else None</pre>
    <p class="text-xs text-slate-400">В
        Одна строка – и вся амортизация.</p>
    </div>
</details>
</div>
</div>
</section>`,
    },
    {
        id: "prefix-sums-2d",
        title: "5. Двумерная матрица префиксных сумм",
        type: "html",
        description: "Сумма подматрицы за O(1): 1D и 2D, дв
        category: "Алгоритмы матрицы",
        content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-indigo-600 text-white px-4 py-1">
        <h2 class="text-2xl sm:text-3xl font-bold text-v
    </div>

    <div class="space-y-8">

        <div class="bg-slate-700/50 p-6 rounded-xl bord
            <h3 class="text-xl font-bold text-emerald-4

```

```

        <td class="py-2 pr-3 font-mono">
      </td>
    </tr>
  </tbody>
</table>
<p class="text-slate-400 text-xs mt-2">
  ышли за границу» – поэтому в 2D почти всегда
</div>

<div class="grid grid-cols-1 sm:grid-cols-3">
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
    <p class="text-[11px] text-slate-500
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-emerald-300
    <p class="text-[11px] text-slate-500
  </div>
  <div class="bg-slate-900 border border-
    <p class="text-[10px] uppercase tra
    <p class="font-mono text-rose-300 t
    <p class="text-[11px] text-slate-500
  </div>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
  <h3 class="text-xl font-bold text-blue-400 m
  <div class="grid grid-cols-1 xl:grid-cols-2">
    <div class="bg-slate-800
      <div class="absolute -top-3 left-4 b
      rder border-emerald-500 shadow-md">
        Аналогия 1: Сетка из вафельных
      </div>
      <p class="text-slate-300 text-sm mb
    о накопилось от угла (0,0) до меня». Хочешь
    у и верни дважды отрезанный угол. Вычитание

```

```

<div class="bg-slate-800 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
  Тонкая: одна матрица
</div>
  <p class="text-slate-300 text-sm mb
одной матрицы (плюс рамка из нулей). Никаки
прямоугольника от угла. Всё остальное полу
</div>
  <div class="bg-slate-900 p-6 rounded-lg"
  <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
  Не умеет: всё, что нельзя вычес
</div>
  <p class="text-slate-300 text-sm mb
максимума или gcd вычитания не существует:
е запросы – это <a class="ticket-link" data
платит она за это памятью <span class="fon
</div>
</div>

<div class="bg-slate-800/70 p-5 rounded-lg"
  <p class="text-sm font-bold text-white"
  <table class="w-full text-xs text-left"
    <thead>
      <tr class="text-slate-500 borde
        <th class="py-1.5 pr-2 font
        <th class="py-1.5 pr-2 font
        <th class="py-1.5 pr-2 font
      </tr>
    </thead>
    <tbody>
      <tr class="border-b border-slate
        <td class="py-2 pr-3 font-b
        <td class="py-2 pr-3 font-m
        <td class="py-2 pr-3 font-m
      </tr>
      <tr class="border-b border-slate

```

```

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">
    Код: 1D (тот же, что выполняет пане
  </summary>
  <div class="p-5 text-sm text-slate-300
    <pre class="bg-slate-950 p-4 rounde
      = [3, 1, 4, 1, 5, 9, 2, 6] # исходный ма
n = len(a) # префиксов будет n + 1
P = [0] # P[0] = 0 – пустой преф
for i in range(1, n + 1):
    P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)</code></pre>
    <p class="text-slate-300 text-sm">П
  - часть демонстрации: лента подсвечивает, из
    </div>
</details>

<details class="bg-slate-900/40 rounded-lg l
  <summary class="p-4 font-bold text-slate
  -b border-slate-700/50">
    Код: 2D – построение и запрос прямо
  </summary>
  <div class="p-5 text-sm text-slate-300
    <pre class="bg-slate-950 p-4 rounde
      = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4

S = [] # рамка из нуле
for rr in range(n + 1):
    S.append([0] + [0] * m)
for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = A[i-1][j-1] + S[i-1][j] + S[i][j-1] -

r1, c1, r2, c2 = 1, 1, 2, 2 # прямоугольник
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1]
print("итого:", ans)</code></pre>

```

```

    },
    {
      id: "graph-planar-colors",
      title: "7. Графы. Планарные. Покраска",
      type: "html",
      description: "Формула Эйлера и теорема о 4 красках.",
      category: "Графы. Теория",
      content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">7
    <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Покраска графов
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.
        <p class="text-xl font-mono text-white">
          1. Если граф планарный, то существует такое разбиение его вершин на 4 класса, что
          любой граф можно раскрасить 4 цветами.
        </p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Карта мира
          </div>
          <p class="text-slate-300 text-sm mb-4">
            . Границы не пересекаются в одной точке по-
            ы не сливались цветами, Всегда можно всего 4
          </p>
        </div>
      </div>
    </div>
  </div>
</section>`,
    },
    {
      id: "top-sort",

```



```

    </div>
</section>`,
  },
  {
    id: "scc-kosaraju",
    title: "10. Графы. Компоненты сильной связности (SCC)",
    type: "html",
    description: "Разбиение орграфа на макро-вершины. Алгоритм Косарaju",
    category: "Графы. Продвинутое",
    content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">Графы
    <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
      <h3 class="text-xl font-bold text-emerald-400">10.1. Компоненты сильной связности (SCC)
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-emerald-300 italic">Сильная связность графа — это свойство, при котором для любых двух вершин u и v графа существует путь из u в v и из v в u. Другими словами, граф является сильно связным, если для любых двух вершин u и v графа существует путь из u в v и из v в u.
        <p class="text-xl font-mono text-white">Алгоритм Косарaju позволяет найти все компоненты сильной связности графа, инвертированному в порядке top-sort.</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
            Аналогия 1: Улицы с односторонним движением
          </div>
          <p class="text-slate-300 text-sm mb-4">Представьте себе карту города, где улицы имеют одностороннее движение. Если вы можете пройти по каждой улице в обоих направлениях, то город является сильно связным. В реальных городах это редко встречается, но в некоторых случаях (например, в старых городах с узкими улицами) это может быть так. В большинстве же случаев улицы имеют одностороннее движение, что делает граф не сильно связным. Однако, если вы знаете, как правильно ориентироваться, вы сможете найти путь из любой точки города в любую другую точку города, даже если улицы имеют одностороннее движение. Это и есть сильная связность графа.
        </div>
        <div class="bg-slate-800 p-6 rounded-lg">
          <p class="text-slate-300 text-sm mb-4">ВЫХ направлениях по односторонним улицам. Как это выглядит на карте.</p>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },

```

```

        description: "Вершины, удаление которых убивает свя
        category: "Графы. Продвинутые",
        content: `
<section id="graph-articulation" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-rose-400">
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-rose-300 italic">
        <p class="text-xl font-mono text-white">
          теи > 1).</p>
      </div>
      <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
            order border-emerald-500 shadow-md">
            Аналогия 1: Центральный роутер
          </div>
          <p class="text-slate-300 text-sm mb-4">
            правильный роутер - и два сегмента офиса б
          </div>
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "graph-euler",
    title: "13. Графы. Эйлеров цикл",
    type: "html",
    description: "Пройти по всем ребрам ровно 1 раз.",
    category: "Графы",
    content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">

```

```

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl border"
    <h3 class="text-xl font-bold text-blue-400"
    <div class="bg-slate-900 p-4 rounded-lg mb-1"
      <p class="text-lg text-blue-300 italic"
      <p class="text-xl font-mono text-white"
d-длина.</p>
    </div>
    <div class="grid grid-cols-1 gap-6">
      <div class="bg-slate-800 p-6 rounded-lg"
        <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
          Аналогия: Копаем туннель под .
        </div>
        <p class="text-slate-300 text-sm mb
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площ
        </div>
      </div>
    </div>
  </div>
</section>`,
  },
  {
    id: "mst-kruskal",
    title: "18. Графы. Остовное дерево. Краскал",
    type: "html",
    description: "",
    category: "Графы. Остовные",
    content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 r
    <h2 class="text-2xl sm:text-3xl font-bold text-
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-emerald-4

```

```

        которое торчит из посещенных в непосещенных
    </div>
    <div class="grid grid-cols-1 gap-6">
        <div class="bg-slate-800 p-6 rounded-lg">
            <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
                ✦ Аналогия: Заражение вирусом
            </div>
            <p class="text-slate-300 text-sm mb-2">
                ли вирус). Мы стартуем из одного города, и
                Сеть растет только из одного связного куска
            </p>
        </div>
    </div>
</div>
</section>`,
    },
    {
        id: "mst-boruvka",
        title: "20. Графы. Остовное дерево. Борувка",
        type: "html",
        description: "",
        category: "Графы. Остовные",
        content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">
        <span class="bg-blue-600 text-white px-4 py-1 rounded">
            <h2 class="text-2xl sm:text-3xl font-bold text-white">
                </div>
            <div class="space-y-8">
                <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
                    <h3 class="text-xl font-bold text-emerald-400">
                        <div class="bg-slate-900 p-4 rounded-lg mb-4">
                            <p class="text-lg text-emerald-300 italic">
                                <p class="text-xl font-mono text-white">
                                    ается с соседом.</p>
                                </div>
                            <div class="grid grid-cols-1 gap-6">

```

```

        rder border-emerald-500 shadow-md">
            Аналогия 1: Бумеранг (Префикс
        </div>
        <p class="text-slate-300 text-sm mb
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
меня перепроверок!</p>
    </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
    <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
        Код (Скрыто)
    </summary>
    <div class="p-5 text-sm text-slate-300
        <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
    int j = pi[i-1];
    while (j > 0 && s[i] != s[j]) j = pi[j-1];
    if (s[i] == s[j]) j++;
    pi[i] = j;
}</pre>
        </div>
    </details>
</div>
</div>
</section>`,
    },
    {
        id: "string-z-func",
        title: "22. Z-функция",
        type: "html",
        description: "",
        category: "Строки",
        content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
    <div class="flex items-center mb-6 flex-wrap gap-3">

```

```

    </div>
</section>`,
  },
  {
    id: "complexity-classes",
    title: "24. Классы сложности, сведение задач",
    type: "html",
    description: "",
    category: "Теория сложности",
    content: `
<section id="complexity-classes" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-purple-600 text-white px-4 py-1 rounded-md">24</span>
    <h2 class="text-2xl sm:text-3xl font-bold text-purple-400">Классы сложности</h2>
  </div>
  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
      <h3 class="text-xl font-bold text-purple-400">Сведение задач</h3>
      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-purple-300 italic">Пример: Сведение (Reduction) A->B: Если я умею решать A, то я умею решать B.</p>
        <p class="text-xl font-mono text-white">Сведение (Reduction) A->B: Если я умею решать A, то я умею решать B.</p>
      </div>
      <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
            Аналогия 1: Судoku (P vs NP)
          </div>
          <p class="text-slate-300 text-sm mb-4">Пример сортировка чисел). Класс <b>NP</b> – это класс задач, для которых существует быструю проверку (ответ), он БЫСТРО (за класс P) решить.</p>
        </div>
        <!-- Аналогия 2 -->
        <div class="bg-slate-900 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 right-4 bottom-4 border border-purple-500 shadow-md">
            Аналогия 2: Задача о рюкзаке (NP vs P)
          </div>
          <p class="text-slate-300 text-sm mb-4">Пример: Задача о рюкзаке. Класс <b>NP</b> – это класс задач, для которых существует быструю проверку (ответ), он БЫСТРО (за класс P) решить.</p>
        </div>
      </div>
    </div>
  </div>
</section>`
  }
]

```

```

<div class="bg-slate-900 p-4 rounded-lg mb-4">
  <p class="text-lg text-blue-300 italic">
    <div class="text-left font-mono text-sm">
      <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
      <p><strong class="text-white">Эйлер
    <div class="p-3 bg-slate-950 rounded">
      <p class="text-sm text-blue-300">
        <p><strong class="text-white">П
        <p><strong class="text-white">Ц
      </div>
    </div>
  </div>
</div>

```

```

<p class="text-slate-300 text-sm">
  <strong>Билет 13 (Различие пути и цикла
  <br>* <span class="text-green-400 font-
шел-вошёл-вышел).
  <br>* <span class="text-yellow-400 font-
финиш).
</p>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
  <div class="bg-slate-800 p-6 rounded-lg border">
    <div class="absolute -top-3 left-4 bg-slate-900
order-green-500 shadow-md">
      Аналогия 1: Путь – из дома в бар,
    </div>
    <p class="text-slate-300 text-sm mb-4">
      Ты вышел из дома <strong>(нечётная)</strong>.
      талый пришёл в бар <strong>(вторая нечётная)</strong>.
      Домой вернуться не смог, потому что
    </p>
  </div>

```

```

<div class="bg-slate-900 p-6 rounded-lg border">
  <div class="absolute -top-3 left-4 bg-slate-950

```

```

id: "bridges-code",
title: "11. Графы. Мосты",
type: "html",
description: "Мосты через DFS Тарьяна: tin/low, усл
category: "Графы. Продвинутое",
content: `<section id="bridges-code" class="mb-20 s
<div class="flex items-center mb-6">
  <span class="bg-emerald-600 text-white px-4 py-
  <h2 class="text-3xl font-bold text-white">Мосты
</div>

<div class="space-y-8">
  <div class="bg-slate-700/50 p-6 rounded-xl bord
    <h3 class="text-xl font-bold text-emerald-4

    <div class="bg-slate-900 p-4 rounded-lg mb-1
      <p class="text-lg text-emerald-300 italic
      <div class="text-left font-mono text-sm
        <p><strong class="text-white">tin[v
        <p><strong class="text-white">low[v
        <div class="p-3 bg-slate-950 rounde
          <span class="text-rose-400 font
          <p class="text-xl font-bold tex
          <p class="text-xs text-slate-400
        </div>
      </div>
    </div>
  </div>

  <p class="text-slate-300 text-sm">
    Если из сына u и всех его потомков <str
    Единственная ниточка. Порвётся — граф р
  </p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
  <div class="bg-slate-800 p-6 rounded-lg bord
    <div class="absolute -top-3 left-4 bg-s
      border-emerald-500 shadow-md">

```



```
using namespace std;

int n; // число вершин
vector<vector<int>>> adj; // список смежност
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++; // устанавливаем

    for (int to : adj[v]) {
        if (to == p) continue; // не идём назад

        if (visited[to]) {
            // обратное ребро: обновляем low
            low[v] = min(low[v], tin[to]);
        } else {
            // ребро дерева DFS
            dfs(to, v); // рекурсивный вызов
            low[v] = min(low[v], low[to]);

            if (low[to] > tin[v]) {
                // ЭТО МОСТ!
                // bridge_list.push_back({v, to});
            }
        }
    }
}

void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);

    for (int i = 0; i < n; ++i) {
```

```
<p class="text-slate-300 text-sm">  
  <strong>Следствие:</strong> Для планарного графа  $G$  с  $n$  вершинами и  $m$  ребрами:  
ode>.
```

Если ребер больше — граф гарантированно не планарен.

```
</p>
```

```
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">  
  <div class="bg-slate-800 p-6 rounded-lg border border-slate-700">  
    <div class="absolute -top-3 left-4 bg-slate-800 border border-slate-700  
order-green-500 shadow-md">
```

K5: Полный граф на 5 вершинах

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

У K5: `<code class="text-white">V=5, E=10 > 9</code>` — БАХ, непланарен!

Все 5 вершин соединены со всеми. На рисунке — K5.

Теорема Куратовского: K5 и K3,3 — минимальные непланарные графы.

Если внутри графа спрятано **K5** или **K3,3** — плоской раскладки не будет.

```
</p>
```

```
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg border border-slate-800">  
  <div class="absolute -top-3 left-4 bg-slate-900 border border-slate-800  
der-rose-500 shadow-md">
```

K3,3: 3 дома и 3 колодца

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">
```

Три дома хотят соединиться с тремя колодцами. Нарисовать без пересечений невозможно.

Нарисовать без пересечений невозможно. У него `<code class="text-white">V=6, E=9</code>`

`<code class="text-white">9 ≤ 12</code>` — проходит, но он не планарен. Почему? Потому что у двудольного графа $E ≤ V - 1$.

Отсюда более строгое ограничение: `<code class="text-white">9 > 8</code>`

```
</p>
```

```

border border-emerald-500 shadow-md">

```

Аналогия 1: Расписание экзамен

```

</div>

```

```

<p class="text-slate-300 text-sm">В
слот времени. Раскрасить граф = составить р
«берём предметы в каком-то порядке и ставим

```

```

</div>

```

```

<div class="bg-slate-900 p-6 rounded-lg

```

```

<div class="absolute -top-3 left-4
border border-fuchsia-500 shadow-md">

```

Аналогия 2: Частоты вышек

```

</div>

```

```

<p class="text-slate-300 text-sm">С
цвет, интерференция = ребро. Диапазон частот
регистров в компиляторе: «живые» одновременно

```

```

</div>

```

```

</div>

```

```

<div class="overflow-x-auto my-6">

```

```

<table class="w-full text-xs text-left

```

```

<thead>

```

```

<tr class="text-slate-500 border

```

```

<th class="py-2 pr-3 font-b

```

```

<th class="py-2 pr-3 font-b

```

```

<th class="py-2 font-bold">

```

```

</tr>

```

```

</thead>

```

```

<tbody class="text-slate-300">

```

```

<tr class="border-b border-slate

```

```

><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta + 1$ <
а свободен</td></tr>

```

```

<tr class="border-b border-slate

```

```

d><td class="py-2 pr-3 font-mono"> $\chi \leq \Delta$ </td>
/td></tr>

```

```

<tr class="border-b border-slate

```

```

<td class="py-2 pr-3 font-mono"> $\chi = 2 \Rightarrow$  нет
) – единственная «лёгкая» раскраска</td></tr>

```

```

<tr class="border-b border-slate

```

```
<details class="bg-slate-900/40 rounded-lg border-1
  <summary class="p-4 font-bold text-slate-300
    -b border-slate-700/50">
```

Код: жадная раскраска и проверка

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300 border-1
```

```
<pre class="bg-slate-950 p-4 rounded-lg border-1
```

я раскраска: не больше $\Delta + 1$ цвета

```
color = [-1] * n # -1 = ещё не покрашен
```

```
for v in order: # порядок влияет на чи
```

```
    used = set()
```

```
    for to in graph[v]:
```

```
        if color[to] != -1:
```

```
            used.add(color[to]) # цвета уже покрашены
```

```
    c = 0
```

```
    while c in used: # минимальный свободный
```

```
        c += 1
```

```
    color[v] = c
```

```
chi = max(color) + 1 #  $\chi$ , который выдал жад
```

```
# Проверка двудольности = раскраска в 2 цвета (BFS)
```

```
from collections import deque
```

```
col = [-1] * n
```

```
bipartite = True
```

```
for start in range(n):
```

```
    if col[start] != -1:
```

```
        continue
```

```
    col[start] = 0
```

```
    dq = deque([start])
```

```
    while dq:
```

```
        v = dq.popleft()
```

```
        for to in graph[v]:
```

```
            if col[to] == -1:
```

```
                col[to] = col[v] ^ 1 # второй цвет
```

```
                dq.append(to)
```

```
            elif col[to] == col[v]: # одноцветные
```

```
                bipartite = False
```

```
<p class="text-slate-400 text-xs">П
```

ег - это тупик со степенью 1).

</p>

</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-

<div class="bg-slate-800 p-6 rounded-lg border-

<div class="absolute -top-3 left-4 bg-slate-800 border-rose-500 shadow-md">

Аналогия: Главный перекресток

</div>

<p class="text-slate-300 text-sm mb-4">

Представь город, где два района соединены, а её перекроют (удалят вершину) – районы будут **г**хрупкость системы.

</p>

</div>

<div class="bg-slate-900 p-6 rounded-lg border-

<div class="absolute -top-3 left-4 bg-emerald-500 border-emerald-500 shadow-md">

Телеком: Центральный свитч

</div>

<p class="text-slate-300 text-sm mb-4">

У тебя несколько компьютеров в одной сети, соединённых **абелем** (мостом). Сами свитчи – это точки со

</p>

</div>

</div>

<details class="bg-slate-900/40 rounded-lg border-

<summary class="p-4 font-bold text-slate-400 border-slate-700/50">

Душный мод: Код и корень DFS (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300 space-

<p class="text-blue-400 font-bold">Особ

<p>

Для стартовой вершины обхода правил

```

        ависимые "конгломераты".</li>
        <li><strong>Как точка сочленения рушит
        Косарайю, свернув каждую SCC в одну супер-в
        (сделаем неориентированным), то любая <strong>
        о и есть критическая SCC! Если удалить эту
        . Одно слабое звено изолирует целые касты S
        </ul>
    </div>
</div>
</section>`,
    },
];

```

ФАЙЛ 19/111: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 145 | РАЗМ

```

import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { ticket1Segtree } from "../tickets/ticket1_segtr
import { tickets18to20Mst } from "../tickets/ticket18_20
import { chapters as newChapters } from "../content_temp

```

```
/**
```

```
 * Содержание пособия = билеты 1–24 из списка тем.
```

```
 *
```

```
 * Порядок массивов важен: при одинаковом id побеждает
```

```
 * версия главы. `newChapters` (content_temp) перетирае
```

```
 * поздние переписанные страницы (Эйлер, Мосты, Планарн
```

```
 *
```

```
 * Страниц в содержании меньше, чем тем: одна страница
```

```
 * несколько билетов сразу (18–20 «Остовное дерево: Кра

```

```

});

// Самые свежие версии страниц перетирают старые.
if (newChapters) {
  newChapters.forEach((c) => {
    if (!toRemove.includes(c.id)) dedup.set(c.id, c);
  });
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
  // Главы без номера билета (введение) стоят ДО билетов
  const numOf = (t: string) => {
    const m = t.match(/(\d+)/);
    return m ? parseInt(m[0], 10) : -1;
  };
  return numOf(a.title) - numOf(b.title);
});

export const chapters: Chapter[] = finalChapters;

/**
 * Билеты, закрытые каждой страницей (для шапки, экспор
 * Страница `mst`, например, закрывает сразу 18, 19 и 2
 */
export const chapterTopics: Record<string, number[]> =
  "segment-trees": [1],
  "sparse-table": [2],
  treap: [3],
  "splay-tree": [4],
  "prefix-sums-2d": [5],
  "graph-dfs-bfs": [6],
  "planarity-euler-formula": [7],
  "graph-components": [8],
  "top-sort": [9],
  "scc-kosaraju": [10],
  "bridges-code": [11],

```

```
-----  
    return SECTIONS.find((sec) => first >= sec.from && fi  
}  
  
export const activeVizIds = [  
    "euler-path-vs-cycle", "bridges-code", "planarity-euler  
    "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se  
    "dijkstra", "bellman-ford", "floyd", "aho-corasick",  
    "mst", "string-kmp", "string-z-func", "segment-trees", "s  
    "dynamic-programming", "johnson-algo", "graph-articulat  
    "complexity-classes", "treap", "splay-tree", "prefix-sum  
];
```

ФАЙЛ 20/111: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 207 | РАЗМ

```
-----  
/**  
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет  
 * под sys.settrace и на каждой строке записывает фрейм  
 * как панель Variables в обычном отладчике.  
 */  
  
export type DebugValue = null | boolean | number | string  
  
export interface DebugStep {  
    /** Номер строки в коде пользователя (1-based). */  
    line: number;  
    /** Имя функции-фрейма ("<module>" – верхний уровень)  
    func: string;  
    /** Локальные переменные: имя → герг (усечённый), для  
    locals: Record<string, string>;  
    /**  
    * JSON-снимок доступных имён (globals + locals). В о  
    * можно напрямую отдать визуализации: i/j остаются ч  
    */  
    values?: Record<string, DebugValue>;
```



```

__was_truncated = False

class __TraceLimit(Exception):
    pass

def __safe(v):
    try:
        r = repr(v)
    except Exception:
        r = "<...>"
    return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
    """Небольшой JSON-снимок: UI использует его без парсера
    if v is None or isinstance(v, (bool, int, str)):
        return v
    if isinstance(v, float):
        return v if v == v and v not in (float("inf"), float("-inf")) else "inf"
    if depth >= 4:
        return __safe(v)
    limit = 80 if depth < 2 else 32
    if isinstance(v, (list, tuple)):
        return [__snapshot(x, depth + 1) for x in list(v)]
    if isinstance(v, (set, frozenset)):
        return [__snapshot(x, depth + 1) for x in list(v)]
    if isinstance(v, dict):
        # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
        # предел 80 незаметно обрезал уровни 4x4 и 8x8
        dict_limit = 192 if depth == 0 else limit
        return {str(k): __snapshot(x, depth + 1) for k, x in list(v.items())}
    # deque, heap-подобные и другие итерируемые учебные
    if type(v).__name__ == "deque":
        return [__snapshot(x, depth + 1) for x in list(v)]
    return __safe(v)

def __visible(items):
    return {
        k: v for k, v in items
    }

```

```

        __steps[-1]["values"] = values
    else:
        __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
    if len(__steps) >= __CAP:
        # Нельзя просто отключить trace: бесконечный цикл
        # тогда продолжит работать вечно. Мягко прерываем
        __was_truncated = True
        raise __TraceLimit()
    # Финальные локалы программы (после последней строки)
    if event == "return" and frame.f_code.co_filename == __FILE__:
        loc, values = __capture(frame)
        __final.update(loc)
        __final_values.update(values)
    return __tracer

__err = ""
try:
    __sys.settrace(__tracer)
    exec(compile(__src, "<user-code>", "exec"), {"__name__": "__main__"})
except __TraceLimit:
    pass
except Exception:
    import traceback as __tb
    __err = __tb.format_exc(limit=3)
finally:
    __sys.settrace(None)

__OUT = __json.dumps({
    "steps": __steps,
    "finalLocals": __final,
    "finalValues": __final_values,
    "error": __err,
    "truncated": __was_truncated,
})
__OUT = __OUT + ";"
__OUT = __OUT + "}"

```

```
import type { DemoKind } from "../components/hints/MiniHints";

/**
 * Глоссарий «википедийных» подсказок.
 *
 * Любое слово из `labels`, встреченное в тексте главы,
 * в подчёркнутый термин: при наведении (или тапе на мобильном)
 * с объяснением на пальцах и мини-визуализацией.
 */
export interface GlossaryEntry {
  id: string;
  /** Варианты написания в тексте (регистр не важен). */
  labels: string[];
  title: string;
  /** Объяснение «на пальцах» – 1–2 фразы. */
  short: string;
  /** Строгая формулировка / что важно не забыть. */
  formal?: string;
  complexity?: string;
  /** Какая мини-анимация показывается в карточке. */
  demo?: DemoKind;
  /** id полного симулятора (см. vizRegistry) – кнопка «симулятор» */
  simulator?: string;
}

export const GLOSSARY: GlossaryEntry[] = [
  {
    id: "bfs",
    labels: ["BFS", "обход в ширину", "поиск в ширину"],
    title: "BFS – обход в ширину",
    short:
      "Волна от источника: сначала все соседи, потом соседи соседей и так далее.",
    formal:
      "Очередь (FIFO). Кладём старт, достаём вершину – её соседей, добавляем в очередь, достаём следующую вершину – её соседей, добавляем в очередь и так далее.",
  },
];
```

```

    demo: "heap",
    simulator: "heap-beam-search",
},
{
    id: "stack",
    labels: ["стек", "стека", "стеке", "стеком", "LIFO"],
    title: "Стек (LIFO)",
    short: "Стопка тарелок: кладёшь сверху и берёшь сверху",
    formal: "push / pop / top за  $O(1)$ . На стеке живёт рекурсия",
    demo: "stack",
    simulator: "stack-dfs",
},
{
    id: "queue",
    labels: ["очередь", "очереди", "очередью", "FIFO"],
    title: "Очередь (FIFO)",
    short: "Очередь в столовой: кто первым встал, тот первым и уходит",
    formal: "push_back / pop_front за  $O(1)$ . Основа BFS и поиска в ширину",
    demo: "queue",
    simulator: "queue-bfs",
},
{
    id: "dsu",
    labels: ["DSU", "СНМ", "система непересекающихся множеств"],
    title: "DSU — система непересекающихся множеств",
    short:
        "«Кто твой староста?» Каждый элемент знает своего старосту",
    formal:
        "find со сжатием путей + union по рангу/размеру. Инициализация за  $O(n)$ ",
    complexity: "почти  $O(1)$  — обратная функция Аккермана",
    demo: "dsu",
    simulator: "mst",
},
{
    id: "trie",
    labels: ["бор", "бора", "боре", "бору", "бором", "trie"],
    title: "Бор (trie, префиксное дерево)",
    short:

```

```

    demo: "suffix-link",
    simulator: "aho-corasick",
},
{
    id: "toposort",
    labels: ["топологическая сортировка", "топсорт", "топосорт"],
    title: "Топологическая сортировка",
    short:
        "Порядок «сначала штаны, потом ботинки»: каждая с
        циклов.",
    formal: "DFS + вершины в обратном порядке выхода, л
    complexity: "O(V + E)",
    demo: "toposort",
    simulator: "top-sort",
},
{
    id: "relaxation",
    labels: ["релаксация", "релаксацию", "релаксации",
    title: "Релаксация ребра",
    short:
        "«А через соседа не быстрее?» Если известный путь
        ,
    formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)
        ан и Флойд.",
    demo: "relax",
    simulator: "dijkstra",
},
{
    id: "prefix-sum",
    labels: ["префиксные суммы", "префиксная сумма", "префикс"],
    title: "Префиксные суммы",
    short:
        "Заранее посчитанные «сколько всего набегало от н
    formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r]-P[l-1]
    complexity: "препроцессинг O(n), запрос O(1)",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},

```

```
{
  id: "articulation",
  labels: ["точка сочленения", "точки сочленения", "т"],
  title: "Точка сочленения",
  short:
    "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
  formal: "Для не-корня:  $v$  — точка сочленения. Это ребро",
  demo: "articulation",
  simulator: "graph-articulation",
},
{
  id: "mst",
  labels: [
    "остовное дерево", "остовного дерева", "остовное",
    "минимальное остовное дерево", "MST",
  ],
  title: "Минимальное остовное дерево (MST)",
  short:
    "Связать все точки проводами так, чтобы суммарная",
  formal: " $V-1$  ребро, никаких циклов. Краскал (жадно и",
    "но).",
  demo: "mst",
  simulator: "mst",
},
{
  id: "amortized",
  labels: [
    "амортизированная", "амортизированный", "амортизи",
    "амортизированное", "амортизированного", "амортизи",
  ],
  title: "Амортизированная сложность",
  short:
    "Средняя цена операции «по абонементу»: одна опер",
  formal: "Метод потенциалов: дорогая операция оплачи",
    "ассив.",
  demo: "amortized",
},
{
```

```

    "префикс", "префикса", "суффиксом", "суффикс", "с",
    "подстроки", "подстроку", "подстрока",
  ],
  title: "Префикс-функция  $\pi$  (КМП)",
  short:
    "Для каждой позиции запоминаем: какой кусок начался",
    "от кусок.",
  formal:
    " $\pi[i]$  — длина наибольшего собственного префикса  $s$ ",
    "возвращая указатель по тексту.",
  complexity: "O(n + m)",
  demo: "prefix-function",
  simulator: "string-kmp",
},
{
  id: "z-function",
  labels: ["Z-функция", "Z-функции", "Z-функцию", "z-"],
  title: "Z-функция",
  short:
    "Для каждой позиции: сколько символов подряд, начинающихся",
    "с каждой буквы.",
  formal:
    " $z[i]$  = длина наибольшего общего префикса  $s$  и  $s[i..n]$ ",
  complexity: "O(n)",
  demo: "prefix-function",
  simulator: "string-z-func",
},
{
  id: "dp",
  labels: [
    "динамическое программирование", "динамики", "динамическое",
    "мемоизация", "мемоизацию", "подзадач", "подзадачи",
  ],
  title: "Динамическое программирование",
  short:
    "Решаем задачу через ответы на её меньшие копии и",
  formal:
    "Нужны: оптимальная подструктура + перекрывающиеся подзадачи",

```

```

    demo: "mst",
    simulator: "mst",
},
{
    id: "euler",
    labels: ["эйлеров цикл", "эйлеров путь", "эйлерова"],
    title: "Эйлеров путь и цикл",
    short:
        "Пройти по каждому ребру ровно один раз, не отрываясь",
    formal:
        "Связный граф: эйлеров цикл    все степени чётные;
    simulator: "euler-path-vs-cycle",
},
{
    id: "planarity",
    labels: ["планарный", "планарность", "планарного",
    title: "Планарность",
    short:
        "Граф планарен, если его можно нарисовать на листе бумаги",
    formal:
        " $V - E + F = 2$  для связного планарного графа. Курсив",
    simulator: "planarity-euler-formula",
},
{
    id: "splay",
    labels: [
        "splay", "splay-дерево", "AVL", "AVL-дерево", "всё",
    ],
    title: "Splay и повороты",
    short:
        "Каждый раз, когда к узлу обратились, его «вытягивают»",
    formal:
        "Три случая: zig (родитель – корень), zig-zig (узлы в одну
        сторону),
    demo: "amortized",
    simulator: "splay-tree",
}
```



```

    simulator: "bellman-ford",
},
{
  id: "floyd",
  labels: [
    "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уорше",
    "промежуточная вершина", "промежуточную вершину",
  ],
  title: "Флойд-Уоршелл: разрешаем вершину k",
  short:
    "Внешний цикл — это не «шаг», а «какие пересадки  

    шить пересадку в k?",
  formal:
    " $D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$ . Порядок  

    ами.",
  complexity: "O(V3) времени, O(V2) памяти",
  demo: "floyd",
  simulator: "floyd",
},
{
  id: "prim",
  labels: ["Прима", "Прим", "Prim"],
  title: "Алгоритм Прима",
  short:
    "Остов растёт как плесень из одной точки: на кажд
  formal:
    "Множество посещённых + куча рёбер на границе. До
    усок.",
  complexity: "O(E log V)",
  demo: "mst",
  simulator: "mst",
},
{
  id: "boruvka",
  labels: ["Борувка", "Борувки", "Борувку", "Boruvka"],
  title: "Алгоритм Борувки",
  short:
    "Все компоненты одновременно шлют «гонца» к ближай
```

```

        "вершины", "вершина", "вершин", "ребра с весом",
    ],
    title: "Граф",
    short:
        "Точки (вершины) и связи между ними (рёбра). Города и дороги",
    formal:
        "G = (V, E). Ребро бывает ориентированным (улицы с односторонним движением) и неориентированным (двустороннее движение).",
    demo: "graph",
    simulator: "graph-dfs-bfs",
},
{
    id: "coord-compress",
    labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
    title: "Сжатие координат",
    short:
        "Координаты бывают гигантские и дробные, а нам нужно хранить их в виде целых чисел. Решение — сортировать по порядку и хранить только порядковый номер.",
    formal:
        "sort + unique + binary search: значение → индекс",
    complexity: "O(n log n)",
    demo: "coord-compress",
},
{
    id: "treap",
    labels: ["Treap", "декартово дерево", "декартова д-во"],
    title: "Treap = Tree + Heap",
    short:
        "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
    formal:
        "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить две части). Ожидаемая высота O(log n).",
    complexity: "ожидаемо O(log n)",
    demo: "heap",
},
{
    id: "bst",

```

```

    labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
    title: "Запрос на отрезке",
    short:
        "Спрашиваем не про один элемент, а про кусок массива
        предподсчёты.",
    formal:
        "Суммы — префиксные суммы  $O(1)$ . Минимум без изменений.",
    demo: "prefix-sum",
    simulator: "prefix-sums-2d",
},
{
    id: "string",
    labels: ["строки", "строке", "строку", "строка", "с"],
    title: "Строка как массив символов",
    short:
        "Строковые алгоритмы держатся на трёх вопросах: где
        (Z-функция) и где встречаются слова словаря (бор).",
    formal:
        "Наивный поиск подстроки —  $O(N \cdot M)$ . Префикс-функция.",
    demo: "prefix-function",
    simulator: "string-kmp",
},
{
    id: "rotation",
    labels: ["поворот", "повороты", "поворота", "поворо"],
    title: "Поворот (rotation) в дереве",
    short:
        "Локальная перевеска двух узлов: ребёнок встаёт на
        дившемся месту. Порядок ключей при этом не ломается.",
    formal:
        "Правый поворот вокруг p:  $x = p.left; p.left = x.$ 
        g и Zig-Zag.",
    complexity: " $O(1)$ ",
    demo: "zig",
    simulator: "splay-tree",
},
{
    id: "zig",

```

```
export const GLOSSARY_LABELS: { label: string; id: string }[] =
  e.labels.map(({ label }) => ({ label, id: e.id }))
).sort((a, b) => b.label.length - a.label.length);
```

ФАЙЛ 22/111: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 404 | РАЗМЕР: 10.5 КБ

```
import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
  {
    id: "intro",
    title: "Введение. Базовая база: куча, очередь, бинарное дерево",
    type: "html",
    category: "Введение",
    content: `
<section id="intro-content" class="mb-12">
  <div class="flex items-center mb-6 flex-wrap gap-2">
    <span class="bg-indigo-600 text-white px-4 py-1">Введение
    <h2 class="text-3xl font-bold text-white">Базовая база
  </div>

  <div class="bg-slate-900 border border-indigo-500/40 p-6">
    <p class="text-slate-200 text-sm leading-relaxed">
      <b class="text-indigo-300">Базовая база</b> – это бинарное
      дерево как массив, куча, очередь и стек, бинарное дерево.
      Кодим по минимуму: там, где за нас уже написали.
    </p>
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border-l border-indigo-500">
      <h3 class="text-xl font-bold text-indigo-400 mb-4">Бинарное
      <p class="text-slate-300 text-sm mb-4">Дерево не имеет
      единицы, то родственников выдаёт простая арифметика
    </div>
  </div>
`
  }
];
```

```
<p class="text-slate-300 text-sm">Куча — двигатель, и список с <span class="font-mono text-sm">раз, и список с
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border-l border-slate-700">
  <h3 class="text-xl font-bold text-sky-400 mb-4">Очередь
  <p class="text-slate-300 text-sm mb-4"><b>Очередь (BFS)
    первый вышел) отличаются одной операцией снятия элемента
    е</span>: оба конца работают за  $O(1)$ . Обычный список
    an> сдвигает весь хвост и стоит  $O(n)$ .</p>
    <pre class="bg-slate-950 border border-slate-700 text-slate-300">
from collections import deque
q = deque([1, 2])
q.append(3); q.popleft() # 1 — очередь (BFS)
s = [1, 2]; s.append(3); s.pop() # 3 — стек (DFS)</pre>
    <p class="text-slate-300 text-sm">Вся разница между BFS (волнами),
    стек — DFS (вглубь).</p>
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border-l border-slate-700">
  <h3 class="text-xl font-bold text-amber-400 mb-4">Бинарный поиск
  <p class="text-slate-300 text-sm mb-4">Если массив отсортирован,
    ерединой — понял, в какой половине жить, — и в противном
    это 20 сравнений. В Python за это отвечает модуль bisect.
    <pre class="bg-slate-950 border border-slate-700 text-slate-300">
from bisect import bisect_left
a = [10, 20, 30, 40, 50]
bisect_left(a, 30) # 2 — индекс первого элемента, не меньше 30
    <p class="text-slate-300 text-sm">Без сортировки поиск по индексу
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border-l border-slate-700">
  <h3 class="text-xl font-bold text-rose-400 mb-4">Вставка в массив
  <p class="text-slate-300 text-sm mb-4">Писать сортировку в худшем
    o text-emerald-300"> $O(n \log n)$ </span> в худшем случае для неупорядоченных
    анных данных. Внутри он ищет уже упорядоченные данные и вставляет
    вставками на коротких хвостах. Сортировка <b>ускоряется</b> за счет
    <pre class="bg-slate-950 border border-slate-700 text-slate-300">
```

</div>

```
<div class="bg-slate-700/50 p-6 rounded-xl border-l
  <h3 class="text-xl font-bold text-teal-400 mb-4
  <p class="text-slate-300 text-sm mb-4">Оба обхо
    тару». Вся разница в таре: стек уходит вглубь
    ext-emerald-300">0(V + E)</span>: каждая вершн
    <pre class="bg-slate-950 border border-slate-700
def go(graph, start, tank):          # tank: список-стек
  seen, tank = {start}, tank([start])
  while tank:
    v = tank.pop() if isinstance(tank, list) else t
    for to in graph[v]:
      if to not in seen:
        seen.add(to); tank.append(to)
  return seen</pre>
  <p class="text-slate-300 text-sm">Дальше эта па
    надстройки над этими восемью строками.</p>
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border-l
  <h3 class="text-xl font-bold text-slate-300 mb-
  <div class="grid grid-cols-1 xl:grid-cols-2 gap
    <div class="bg-slate-800 p-6 rounded-lg bor
      <div class="absolute -top-3 left-4 bg-s
        border-indigo-500 shadow-md">
          Аналогия 1: Схема кинозала
        </div>
        <p class="text-slate-300 text-sm">Дерев
          азателей-«usher'ов» не нужно. Место n, дети
          рху вниз и слева направо.</p>
      </div>
      <div class="bg-slate-800 p-6 rounded-lg bor
        <div class="absolute -top-3 left-4 bg-s
          border-emerald-500 shadow-md">
            Аналогия 2: Турнирная сетка
          </div>
          <p class="text-slate-300 text-sm">Куча
```

```

<div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
    Аналогия 1: Позитивное планирование
  </div>
  <p class="text-slate-300 text-sm mb-2">
    (нельзя поехать и заработать). Он всегда выбирает кратчайший путь.
  </p>
  <div id="slot-chapter-image-dijkstra" style="height: 100px; background-color: #000; width: 100%;">
  <div class="absolute -top-3 left-4 right-4 bottom-4 border border-rose-500 shadow-md">
    Ограничения
  </div>
  <p class="text-slate-300 text-sm mb-2">
    (не может проходить по дорожке для пешеходов), Дейкстра сломается, так как не учитывает
    ограничения.
  </p>
</div>

<details class="bg-slate-900/40 rounded-lg" style="border: 1px solid #000; border-radius: 10px; padding: 10px;">
  <summary class="p-4 font-bold text-slate-300" style="border: 1px solid #000; border-radius: 10px; padding: 5px;">
    Строгое доказательство (Скрыто)
  </summary>
  <div class="p-5 text-sm text-slate-300">
    <p>Алгоритм использует приоритетную очередь (min-heap). Гарантирует корректность только для не-взвешенных графов (или с неотрицательными весами).
  </p>
  <p>Время работы:  $O(V \log V)$ .
  </p>
</div>
</details>

<div id="slot-dijkstra-viz" class="mt-8"></div>
</div>
</div>
</section>`
},
{

```

```
        г станет ЕЩЕ короче — значит мы попали во в
        </div>
    </div>

    <div id="slot-bellman-viz" class="mt-8"></div>
  </div>
</div>
</section>`
},
{
  id: "floyd",
  title: "Билет 16. Флойд-Уоршелл",
  type: "html",
  category: "Графы",
  content: `<section id="floyd-content" class="mb-12">
  <div class="flex items-center mb-6">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
    <h2 class="text-3xl font-bold text-white">Флойд
  </div>

  <div class="space-y-8">
    <div class="bg-slate-700/50 p-6 rounded-xl border">
      <h3 class="text-xl font-bold text-indigo-400">

      <div class="bg-slate-900 p-4 rounded-lg mb-4">
        <p class="text-lg text-indigo-300 italic">
        <p class="text-xl font-mono text-white">
      </div>

      <div class="grid grid-cols-1 xl:grid-cols-2">
        <!-- Аналогия 1 -->
        <div class="bg-slate-800 p-6 rounded-lg">
          <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
            Аналогия 1: Автомагистрали
          </div>
          <p class="text-slate-300 text-sm mb-4">
евле долететь из Москвы в Париж, если мы сд
```



```
<p class="text-slate-300 text-sm">Позволяет найти
  -mono text-amber-300 bg-black/30 px-1 rounded">
  у.</p>
</div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
  <div class="bg-slate-800 p-6 rounded-lg border bo
    <div class="absolute -top-3 left-4 bg-slate-700
      emerald-500 shadow-md">
      Аналогия 1: Паутина (Бор)
    </div>
    <p class="text-slate-300 text-sm mb-4">Представ
      мы двигаемся по веткам. Если нужного продолж
      ("нити страховки") в самый длинный из извест
    </div>

    <div class="bg-slate-900 p-6 rounded-lg border-2 l
      <div class="absolute -top-3 left-4 bg-rose-900
        -500 shadow-md">
        Аналогия 2: Матёшка (Терминальные)
      </div>
      <p class="text-slate-300 text-sm mb-4">Представ
        нашли и "he", потому что оно спрятано внутри
        минальные ссылки</b> (term_link) – прямые мос
      <p>
    </div>
  </div>

<details class="bg-slate-900/40 rounded-lg border b
  <summary class="p-4 font-bold text-slate-400 outl
    pen:border-slate-700/50">
    Внутренняя структура автомата и код BFS (Скры
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-4">
    <p>Каждая вершина имеет таблицу переходов <code>
      ссылку <code>term_link</code> (ближайшая мен
    <div class="bg-black/50 p-4 rounded-lg font-mono
```

```
        </div>
      </details>
    </div>
  </section>`
},
{
  id: "alg-map",
  title: "Алгоритмы на карте",
  type: "html",
  category: "Графы",
  content: `<section id="alg-map-content" class="mb-12">
    <div class="flex items-center mb-6">
      <span class="bg-purple-600 text-white px-4 py-1">
        <h2 class="text-3xl font-bold text-white">Алгоритмы на карте
      </span>
    </div>

    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-purple-400">Алгоритмы на карте
        <p class="text-slate-300 text-sm mb-4">Когда мы имеем дело с графами, которые могут
          содержать большие дробные числа (долгота и широта), мы часто хотим
          использовать целые числа, например, от 0 до N, и уже по ним строим графы.</p>
      </div>
    </div>
  </section>`
}
];
```

ФАЙЛ 23/111: src/data/quizzes.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 200 | РАЗМЕР: 1.2 КБ

```
export interface QuizQuestion {
  question: string;
  options: string[];
  correctIndex: number;
```

```

    },
    {
      question: "Между запросами элементы матрицы меняю
options: [
  "Разреженную таблицу: она перестраивается за  $O(N \times M)$ ",
  "Префиксную матрицу: достаточно поправить одну строку",
  "Дерево отрезков: и таблица, и префиксы статичны",
],
      correctIndex: 2,
      explanation: "Обе структуры статичны: изменение эл
        ксной. С обновлениями работает дерево отрезков,
    }
  ],
  "prefix-sums-2d": [
    {
      question: "Сколько памяти занимает двумерная преф
options: [
  " $N \times M$  — как исходная матрица, плюс рамка из нул",
  " $N \times M \times \log N \times \log M$ ",
  " $N + M$ : одна строка и один столбец",
],
      correctIndex: 0,
      explanation: "Сумма накапливается: каждая клетка
        ная здесь как раз разреженная таблица (билет 2)
    },
    {
      question: "Запрос суммы прямоугольника  $(r1, c1) \dots (r2, c2)$ 
options: [
  " $S[r2][c2] - S[r1][c2] - S[r2][c1] + S[r1][c1]$ ",
  " $S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1][c1]$ ",
  " $S[r2+1][c2+1] - S[r1][c1]$ ",
],
      correctIndex: 1,
      explanation: "С рамкой из нулей  $S[i][j]$  — это сум
        ариант без +1 даёт ответ со сдвигом на клетку —
    },
    {
      question: "Почему левый верхний угол в формуле пр

```

```

        "Ни пути, ни цикла нет — это полный провал"
    ],
    correctIndex: 1,
    explanation: "Если нечётных вершин ровно 2 — это нечётной, заканчиваем в другой."
},
{
    question: "Почему гамильтонов цикл — это NP-полная задача?",
    options: [
        "Потому что Гамильтон — болезнь, и лечить её тяжело",
        "Потому что для эйлерова цикла есть простой критерий",
        "Потому что Эйлер был умнее Гамильтона"
    ],
    correctIndex: 1,
    explanation: "Для эйлерова цикла работает критерий нечётности вершин. Это NP-полная задача, которую не решить за полиномиальное время."
},
{
    question: "Куда ведёт эйлеров путь, если нечётных вершин больше 2?",
    options: [
        "Всё ещё можно пройти по всем рёбрам ровно раз",
        "Такого графа не существует",
        "Эйлерова пути нет — нужно удалять лишние рёбра"
    ],
    correctIndex: 2,
    explanation: "Если нечётных вершин больше 2, эйлерова пути нет. Если ровно 2 нечётные или 0."
}
],
"bridges-code": [
    {
        question: "После DFS у нас low[u] = 4, а tin[v] = 2. Есть ли мост?",
        options: [
            "Да, так как low[u] (4) > tin[v] (2)",
            "Нет, low[u] должно быть меньше или равно",
            "Только если граф связный и неориентированный"
        ],
        correctIndex: 0,
    }
]

```

```

        options: [
            "3 грани",
            "5 граней",
            "10 граней"
        ],
        correctIndex: 1,
        explanation: "F = E - V + 2 = 10 - 7 + 2 = 5. Не
    },
    {
        question: "Почему K5 непланарен?",
        options: [
            "Потому что  $10 > 3*5 - 6 = 9$  — нарушение",
            "Потому что Эйлер был против",
            "Потому что 5 вершин — несчастливое число"
        ],
        correctIndex: 0,
        explanation: "У K5: V=5, E=10,  $3V - 6 = 9$ .  $10 > 9$ 
    }
}
];
};

```

ФАЙЛ 24/111: src/data/vizStepBus.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 292 | РАЗМ

```

import { createContext, useContext, useEffect, useState } from "react";
import type { DebugValue } from "../debugRunner";

```

```

/**

```

```

 * Прямая связь «компилятор ↔ открытая визуализация».

```

```

 *

```

```

 * Компилятор передаёт не «магический номер строки эталонного

```

```

 * выполненного Python-кода. Визуализация читает захардкоженные

```

```

 * (i, j, k, v, dist, P, st...) и подсвечивает соответствующие

```

```

 * любой пользовательский код работает, если в нём использованы

```

```

 */

```

```

-----
export function useVizRuntime(): VizStateEventDetail | null {
  const chapterId = useContext(VizChapterContext);
  const [runtime, setRuntime] = useState<VizStateEventDetail>()
    chapterId ? latestByChapter.get(chapterId) ?? null
  };

  useEffect(() => {
    setRuntime(chapterId ? latestByChapter.get(chapterId) : null);
    if (!chapterId) return;
    const handler = (e: Event) => {
      const detail = (e as CustomEvent<VizStateEventDetail>).detail;
      if (detail?.chapterId === chapterId) {
        setRuntime(detail.variables || detail.line !== null ? detail : null);
      }
    };
    window.addEventListener(CHANNEL, handler);
    return () => window.removeEventListener(CHANNEL, handler);
  }, [chapterId]);

  return runtime;
}

export type VizStepSelector = (variables: Record<string, string>) => number;

/**
 * Подписка старых покадровых демонстраций. Если передать номер кадра
 * выбирается по именам/значениям переменных.
 */
export function useVizStepSync(
  currentStep: number,
  goToStep: (n: number) => void,
  maxStep: number,
  selectByVariables?: VizStepSelector
) {
  const chapterId = useContext(VizChapterContext);

  useEffect(() => {
    if (!chapterId) return;

```

```

-----
/**
 * Нормализованная матрица, пришедшая из выполненного Р
 *
 * Размерность берётся из самих данных: если пользовате
 * вершин, визуализация рисует 5×5, а не подставляет ег
 * 7×7 (раньше ячейки «не той» размерности превращались
 *
 * Поддерживаются все учебные способы хранения:
 *   list[list]      d = [[0, 4], [999, 0]]
 *   list[dict]      d = [{0: 0, 1: 4}, {1: 0}]
 *   dict[dict]      d = {0: {0: 0, 1: 4}}
 *   dict[{i, j}]    d = {(0, 1): 4} — ключ после г
 */
export interface LiveMatrix {
  rows: number;
  cols: number;
  /** Значение ячейки; null — таких данных в коде польз
  at: (i: number, j: number) => number | null;
  /** Подпись размера для бейджа связи: «5×5». */
  sizeLabel: string;
  /** true, если хотя бы одна ячейка — число. */
  hasNumbers: boolean;
}

const MAX_MATRIX_SIDE = 64;

const toCellNumber = (value: DebugValue | undefined): number | null {
  if (typeof value === "number" && Number.isFinite(value)) return value;
  // float("inf") снапшотится строкой "inf" — показываем
  if (typeof value === "string") {
    const t = value.trim().toLowerCase();
    if (t === "inf" || t === "+inf" || t === "infinity") return Number.POSITIVE_INFINITY;
    if (t === "-inf" || t === "-infinity") return Number.NEGATIVE_INFINITY;
    const n = Number(t);
    return Number.isFinite(n) ? n : null;
  }
  return null;
};

```

```

        }
    });
} else {
    const record = vizRecord(value);
    if (!record) return null;
    for (const [key, cell] of Object.entries(record)) {
        const pair = parsePairKey(key);
        if (pair) {
            put(pair[0], pair[1], cell);
            continue;
        }
        const i = Number(key);
        if (!Number.isInteger(i)) continue;
        const inner = vizArray(cell);
        if (inner) {
            inner.slice(0, MAX_MATRIX_SIDE).forEach((v, j) => {
                put(i, j, v);
            });
        }
        const innerRecord = vizRecord(cell);
        if (innerRecord) {
            for (const [k2, v2] of Object.entries(innerRecord)) {
                const j = Number(k2);
                if (Number.isInteger(j)) put(i, j, v2);
            }
        }
    }
}

if (rows === 0 || cols === 0) return null;
return {
    rows,
    cols,
    at: (i, j) => (i >= 0 && j >= 0 && i < rows && j < cols) ? cells[i][j] : null,
    sizeLabel: `${rows}x${cols}`,
    hasNumbers: cells.size > 0,
};
}

```



```
* Сразу отдаём последнее значение: компилятор часто от
* как пользователь переключился с 1D на 2D.
*/
export function onVizDemo(chapterId: string, cb: (demoId: string) => void) {
  if (typeof window === "undefined") return () => {};
  const latest = latestDemoByChapter.get(chapterId);
  if (latest !== undefined) cb(latest);
  const handler = (e: Event) => {
    const detail = (e as CustomEvent<VizDemoEventDetail>).detail;
    if (detail && detail.chapterId === chapterId) cb(detail.demoId);
  };
  window.addEventListener(DEMO_CHANNEL, handler);
  return () => window.removeEventListener(DEMO_CHANNEL, handler);
}
```

ФАЙЛ 25/111: src/data/vizSync.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРOK: 1557 | РАЗМЕР: 10000 байт

```
/**
 * Захардкоженная «карта синхронизации» компилятора с визуализацией
 *
 * Для каждой страницы (главы) здесь вручную перечислены переменные,
 * которые понимает её визуализация: i, j, k, v, dist, ...
 * Компилятор выполняет любой Python-код и передаёт результат
 * имён напрямую; привязки к номерам или тексту строк эмулируются
 */
```

```
export interface SyncVariable {
  /** Имя переменной, как в коде визуализации: i, j, k, ... */
  name: string;
  /** Роль в алгоритме (показывается во всплывающей подпанели) */
  role: string;
  /** Характерный диапазон/значение на демонстрации. */
  range?: string;
}
```

Универсальное пособие • полный исходный код

```
-----
mid = 7                # разрез для split:  $x \leq \text{mid}$  у
key = 9                # какую точку удаляем в erase
step = 0               # номер шага – листает кадры д
height = 0             # высота построенного дерева
root = None
```

```
# --- тело алгоритма ---
```

```
class Node:
```

```
    def __init__(s, x, y):
        s.x = x; s.y = y; s.l = None; s.r = None
```

```
def build(pts):
```

```
    """Декартово дерево из отсортированных по x точек: """
```

```
    pts = sorted(pts, key=lambda p: p[0])
```

```
    stack = []
```

```
    for x, y in pts:
```

```
        cur = Node(x, y)
```

```
        last = None
```

```
        while stack and stack[-1].y < y:    # поднимаемс
```

```
            last = stack.pop()
```

```
        cur.l = last
```

```
        if stack:
```

```
            stack[-1].r = cur
```

```
        stack.append(cur)
```

```
    return stack[0] if stack else None
```

```
def split(t, m):
```

```
    """(L, R): в L все с  $x \leq m$ , в R – с  $x > m$ . """
```

```
    global step
```

```
    if t is None:
```

```
        return (None, None)
```

```
    step += 1
```

```
    if t.x <= m:
```

```
        a, b = split(t.r, m)
```

```
        t.r = a
```

```
        return (t, b)
```

```
    a, b = split(t.l, m)
```

```
    t.l = b
```

```

-----
print("merge(L, R) ->", inorder(back, []), "| шагов:",

step = 0
after = erase(build(points), key)
print("erase(", key, ") ->", inorder(after, []), "| выс
    variables: [
        { name: "points", role: "набор пар (x; y) – по не
          демо" },
        { name: "n", role: "число узлов = число пар", ran
        { name: "mid", role: "разрез для вкладки Split: в
        { name: "key", role: "какую точку удаляет вкладка
        { name: "step", role: "номер шага спуска/слияния
        { name: "height", role: "высота дерева: в среднем
        { name: "root", role: "корень – узел с максимальн
    ],
},
"splay-tree": {
    vizTitle: "Splay: Zig / Zig-Zig / Zig-Zag на узлах
    stepNote: "Шаг = одно применение случая: переменная
      рота.",
    code: `# Splay-дерево: три случая поворота на узлах
x, p, g = None, None, None      # узел, родитель, дед (о
xk, pk, gk = 0, 0, 0           # их ключи-числа: 20 / 4
case = "zig"                    # какой случай сработал:
step = 0                        # номер поворота
root = None                     # корень текущего дерева

# --- тело алгоритма ---
class N:
    def __init__(s, k):
        s.k = k; s.l = None; s.r = None; s.p = None

def rot(x):                      # один поворот: поднять
    p = x.p
    g = p.p if p else None
    if p and p.l is x:
        p.l = x.r
        if x.r: x.r.p = p

```

```
-----
splay(root.l)
```

```
# 2) Zig-Zig: бамбук 60 -> 40 -> 20, оба сына левые
root = N(60); root.l = N(40); root.l.p = root; root.l.l
splay(root.l.l)
```

```
# 3) Zig-Zag: змейка 60 -> 20 -> 40 (40 – правый сын 20)
root = N(60); root.l = N(20); root.l.p = root; root.l.r
splay(root.l.r)\,
```

```
    variables: [
        { name: "case", role: "какой случай сработал: \"z
          a" },
        { name: "step", role: "номер применения случая –
        { name: "xk / x", role: "ключ узла, который подни
        { name: "pk / p", role: "ключ родителя x перед по
        { name: "gk / g", role: "ключ деда – нужен в Zig-
        { name: "root", role: "корень дерева после расшей
    ],
```

```
},
"sparse-table": {
    vizTitle: "Разреженная таблица (1D и 2D)",
    stepNote: "Шаг = одна ячейка  $st[i][j] = \min(st[i][j - 1], st[i + (1 \ll (j - 1))][j - 1])$ ",
    code: `a = [2, 3, 5, 62, 3, 21, 1, 4]
```

```
n, LOG = 8, 4
```

```
i, j = 0, 0
```

```
st = [] # сюда складываем строки
```

```
for row in a:
    st.append([row] + [None] * (LOG - 1))
```

```
for j in range(1, LOG):
    for i in range(n - (1 << j) + 1):
        st[i][j] = min(st[i][j - 1], st[i + (1 << (j - 1))][j - 1])
```

```
for row in st:
    print(row) # готовая таблица: None т
```

```
# запрос min на отрезке за  $O(1)$ : отрезок покрывают ДВА
L, R = 1, 6
```

```

memo = {}

def fib(k):
    if k in memo:
        return memo[k]
    if k <= 2:
        return 1
    print(f"считаем fib({k-1}) + fib({k-2})")
    r1 = fib(k - 1)
    r2 = fib(k - 2)
    memo[k] = r1 + r2
    return memo[k]

print("итог:", fib(n))`,
    variables: [
        { name: "n", role: "целевой номер числа Фибоначчи"},
        { name: "k", role: "аргумент текущего рекурсивного"},
        { name: "memo", role: "кэш уже посчитанных значений"},
        { name: "r1, r2", role: "ответы двух рекурсивных"},
    ],
},
"heap-beam-search": {
    vizTitle: "Куча (priority queue)",
    stepNote: "Шаг = просеивание: вставленный элемент в",
    .",
    code: `a = [7, 3, 5, 1] # куча-массивом
n = len(a)
i = n - 1 # всплываемый элемент

parent = (i - 1) // 2 # родитель
kids = (2 * i + 1, 2 * i + 2) # дети
print(f"i={i} parent={parent} kids={kids}")`,
    variables: [
        { name: "n", role: "текущее число элементов в куче"},
        { name: "i", role: "индекс элемента, который сейчас"},
        { name: "(i-1)//2", role: "индекс родителя вершины"},
        { name: "2*i+1, 2*i+2", role: "индексы левого и правого"},
    ],
},

```

```

-----
st.append(" "); top = st.pop()
q.append(" "); q.append(" "); first = q.popleft()

i = 0 # счётчик выполненных операций
for op in ("push", "pop", "enqueue", "dequeue"):
    i += 1
print("тарелка:", top, "| первый из очереди:", first, "variables: [
    { name: "top / front", role: "верх стопки (стек)",
    { name: "i", role: "индекс в массиве кучи; родители",
],
},
"graph-dfs-bfs": {
    vizTitle: "DFS и BFS на графе",
    stepNote: "Шаг = переход из текущей вершины v к непосещенной",
    code: `from collections import deque

adj = {"A": ["B", "C"], "B": ["D"], "C": [], "D": []}
visited, order = set(), []

stack = ["A"] # DFS: свой стек вместо рекурсии
while stack:
    v = stack.pop() # шаг: достали вершину с вершины
    if v in visited:
        continue
    visited.add(v); order.append(v)
    for u in adj[v]: # соседи v → на стек
        stack.append(u)
print("DFS:", order)

visited, order = set(), []
q = deque(["A"]) # BFS: очередь, обход по слоям
while q:
    v = q.popleft() # шаг: достали из начала очереди
    if v in visited:
        continue
    visited.add(v); order.append(v)
    for u in adj[v]: # соседи v → в конец очереди

```

```

        used[to] = True
        stack.append(to)
print("компонент:", comps)
print("разметка comp:", comp)` ,
    variables: [
        { name: "n", role: "число вершин; в демонстрации"},
        { name: "adj", role: "списки смежности – тот же граф"},
        { name: "comp", role: "разметка: номер компоненты"},
        { name: "y", range: "-1 ... comps" },
        { name: "used", role: "булев вариант разметки – да/нет"},
        { name: "comps", role: "счётчик найденных компонент"},
        { name: "с", role: "номер компоненты, которую краем"},
        { name: "v", role: "вершина, где обход находится"},
        { name: "stack", role: "фронт обхода: стек (DFS)"},
        { name: "i / step", role: "номер шага – перематываем"},
    ],
},
"top-sort": {
    vizTitle: "Топологическая сортировка",
    stepNote: "Шаг = выход рекурсии из вершины v: она достигнута",
    code: `adj = {0: [1, 4], 1: [2, 3], 2: [], 3: [2], 4: []},
used, order = set(), []

def dfs(v):
    used.add(v)
    for to in adj[v]:
        # шаг: ребро v → to
        if to not in used:
            dfs(to)
    order.append(v)
    # выход из v!

for v in adj:
    if v not in used:
        dfs(v)

answer = order[::-1]
# разворот – топологический
for i, v in enumerate(answer):
    print(f"позиция {i}: вершина {v}")`,
    variables: [

```

```

-----
print("обратный граф:", adjT)

# фаза 2: идём по order С КОНЦА, DFS по adjT красит одну компоненту
comp = {}
c = 0
for i in range(len(order) - 1, -1, -1):
    v = order[i]
    if v in comp:
        continue
    stack = [v]
    comp[v] = c
    while stack:
        u = stack.pop()
        for to in adjT[u]:
            if to not in comp:
                comp[to] = c
                stack.append(to)
    print("SCC", c, "собрана из", v, ":", comp)
    c += 1

print("порядок выхода:", order, "| компонент:", c, "| переменные: [
    { name: "adj / adjT", role: "граф и его транспонированный граф" },
    { name: "v", role: "текущая вершина DFS (в фазе 1)" },
    { name: "to", role: "сосед по ребру: в фазе 1 прямой DFS" },
    { name: "used", role: "помеченные первым проходом" },
    { name: "order", role: "порядок ВЫХОДА из DFS первого прохода" },
    { name: "i", role: "индекс прохода по order на втором проходе" },
    { name: "stack", role: "стек DFS второго прохода" },
    { name: "u", role: "вершина, снятая со стека во втором проходе" },
    { name: "comp", role: "номер компоненты сильной связности" },
    { name: "c", role: "счётчик компонент: каждая новая компонента" },
    { name: "n", role: "число вершин", range: "5" },
],
},
"graph-articulation": {
    vizTitle: "Точки сочленения",
    stepNote: "Шаг = обновление low[v] по ребру; вершина v — кандидат на точку сочленения"
}

```



```

stepNote: "Шаг = переход по неиспользованному ребру
идти нельзя.",
code: `n = 5                                # вершин: 0...4 (на
edges = [(0, 1), (0, 2), (1, 3), (2, 4), (1, 2), (3, 4)]
adj = {v: [] for v in range(n)}
for i, (u, v) in enumerate(edges):
    adj[u].append((v, i))
    adj[v].append((u, i))
deg = {v: len(adj[v]) for v in range(n)}    # степени вер
odd = [v for v in range(n) if deg[v] % 2 == 1] # нечёт
start = odd[0] if odd else 0                # путь начинае
used = [False] * len(edges)                # какие рёбра
path = [start]                             # маршрут: пос
last = start                               # где мы сейча
stack = [start]                            # стек для отк
v = start
step = 0

# --- тело алгоритма ---
while stack:
    v = stack[-1]
    moved = False
    for to, i in adj[v]:
        if not used[i]:
            used[i] = True
            stack.append(to)
            path.append(to)
            last = to
            step += 1
            moved = True
            break
    if not moved:
        stack.pop()                        # из v все рёб
print("степени:", deg)
print("нечётных вершин:", len(odd), odd, "->", "цикл" i
print("маршрут:", path, "| рёбер пройдено:", sum(used),
    variables: [
        { name: "path", role: "маршрут – последовательнос

```

```

variables: [
    { name: "n", role: "число вершин графа на демонстраци"},
    { name: "adj", role: "списки смежности — тот же граф"},
    { name: "color", role: "массив цветов вершин (0 = неокрашено)"},
    { name: "v", role: "вершина, которую красим сейчас"},
    { name: "u", role: "сосед вершины v, чей цвет уже известен"},
    { name: "delta", role: "максимальная степень  $\Delta = \max_v \deg(v)$ "},
    { name: "chi", role: "сколько цветов реально понадобится"},
    { name: "i / step", role: "номер шага — перематываем таймер"},
    { name: "V, E, F", role: "для вкладки «Планарность графа»"},
    { name: "9", role: "для вкладки «Графы с 9 вершинами»"}],
},
dijkstra: {
    vizTitle: "Дейкстра",
    stepNote: "Шаг = старт, извлечение ближайшей вершины",
    code: `import heapq

g = {"A": [("B", 4), ("C", 2)], "B": [("G", 4), ("D", 5)],
     "C": [("D", 5), ("E", 1)], "D": [("F", 1)], "E": [], "F": []}
dist = {"A": 0}
prev = {}
done = set()
pq = [(0, "A")]
while pq:
    du, u = heapq.heappop(pq)
    if u in done:
        continue
    done.add(u)
    print(f"обрабатываем {u} (dist={du})")
    for v, w in g[u]:
        if v in done:
            continue
        print(f"смотрим ребро {u}→{v} (w={w})")
        if du + w < dist.get(v, float("inf")):
            dist[v] = du + w
            prev[v] = u
            heapq.heappush(pq, (dist[v], v))
    `}

```

```

-----
# контрольный (n-й) проход: улучшение здесь означает от
cycle_edge = None
for u, v, w in E:
    if dist[u] != INF and dist[u] + w < dist[v]:
        cycle_edge = (u, v, w)
        break

if cycle_edge:
    print("отрицательный цикл: улучшается ребро", cycle_edge)
else:
    print("отрицательных циклов нет:", {k: dist[k] for k in range(n)})
    variables: [
        { name: "n", role: "число вершин графа – ровно n"},
        { name: "m", role: "число рёбер, по которым проходим"},
        { name: "i", role: "номер итерации (после i итераций)"},
        { name: "u, v, w", role: "текущее ребро: откуда, куда, вес"},
        { name: "dist[v]", role: "расстояние; если обновилось, то новое"},
    ],
},
floyd: {
    vizTitle: "Флойд–Уоршелл",
    stepNote: "Шаг = инициализация матрицы, смена промежуточной вершины",
    code: `INF = 999

d = [[0, 4, INF, 2, INF, INF, INF], [INF, 0, 3, INF, INF, INF, INF],
      [INF, INF, INF, 0, INF, 1], [INF, INF, INF, INF, INF, INF, 0]]
n = 7
for k in range(n):
    print(f"посредник k={k}")
    for i in range(n):
        for j in range(n):
            if i != j and i != k and j != k and d[i][k] + d[k][j] < d[i][j]:
                d[i][j] = d[i][k] + d[k][j]
print("итог: все пары посчитаны")`,
    variables: [
        { name: "n", role: "число вершин – размер квадратной матрицы"},
        { name: "k", role: "промежуточная вершина – ВНЕШНЯЯ"},
    ],
}

```

--- тело алгоритма ---

```
def find(x):
    while parent[x] != x:
        parent[x] = parent[parent[x]]    # сжатие путей
        x = parent[x]
    return x

for i, (w, a, b) in enumerate(edges):
    u, v = a, b
    ra, rb = find(a), find(b)
    if ra != rb:                          # разные компоненты
        parent[ra] = rb
        if rank[ra] == rank[rb]:
            rank[rb] += 1
        mst.append((w, a, b))
        total += w
print("вес остова:", total, "| рёбер:", len(mst))
print(mst)`
variables: [
    { name: "n", role: "число вершин (A...I); в остов в"},
    { name: "edges", role: "список рёбер (вес, u, v) -"},
    { name: "i", role: "номер текущего ребра в отсорти"},
    { name: "u, v, w", role: "концы и вес ребра, котор"},
    { name: "parent", role: "DSU: представитель множе"},
    { name: "rank", role: "высота дерева DSU – union f"},
    { name: "mst", role: "взятые рёбра остова – рисуем"},
    { name: "total", role: "вес остова; на этом графе"}
],
},
"string-kmp": {
    vizTitle: "Префикс-функция (КМП)",
    stepNote: "Шаг = обработка символа s[i]: откаты по",
    code: `s = "aabaabaaba"
n = len(s)
i, j = 1, 0
pi = [None] * n                # пустые клетки: значения п
pi[0] = 0
```

```

        { name: "i", role: "позиция, для которой считаем",
        { name: "l, r", role: "правый крайний Z-блок: отпр
        { name: "z[i]", role: "длина наибольшего общего пр
    ],
},
"aho-corasick": {
    vizTitle: "Ахо-Корасик: бор, fail-ссылки и автомат"
    stepNote: "Шаг = один символ текста: state = go(sta
        овпадают с картинкой «водопада».",
    code: `words = ["HERS", "HIS", "SHE", "HE"]    # обр
text = "USHERS"                                # текст, в котор
n = len(text)                                # длина текста
state = 0                                    # текущее состоян
i = 0                                        # индекс текущег
c = ""                                       # сам символ
nodes = 1                                    # сколько вершин
queue = []                                  # очередь BFS, к
fail = {}                                    # fail[v] = верши
matches = []                                # найденные вхож

# --- тело алгоритма ---
nxt = {}                                    # nxt[(v, ch)] = u — переходы бора
term = {}                                  # term[v] = слово, которое заканчивае

def add_word(w):
    """Добавить образец в бор: недостающие вершины полу
    global nodes
    v = 0
    for ch in w:
        if (v, ch) not in nxt:
            nxt[(v, ch)] = nodes
            nodes += 1
        v = nxt[(v, ch)]
    term[v] = w

for w in words:
    add_word(w)

```

```

variables: [
    { name: "words", role: "набор образцов – по нему",
      },
    { name: "text", role: "сканируемый текст – подста",
    { name: "state", role: "текущее состояние автомата",
    { name: "i / c", role: "индекс и сам символ текста",
    { name: "nxt", role: "переходы бора: nxt[{v, ch}]",
    { name: "fail", role: "fail[v] – вершина наибольш",
    { name: "term", role: "в каких вершинах заканчива",
    { name: "queue", role: "очередь BFS, которой натя",
    { name: "matches", role: "найденные вхождения: {и",
    { name: "nodes", role: "сколько вершин в боре (ко",
],
},
"complexity-classes": {
    vizTitle: "Классы сложности: рост по n и ops",
    stepNote: "Шаг = одна точка (n, ops): визуализация",
    code: `n = 1                                # размер входа
ops = 0                                         # сколько операций насчитал
cls = "P"                                       # класс, к которому относим
step = 0                                       # кадр схемы: 0 вход → 1 алг
k = 0                                          # длина сертификата: сколько
cert = []                                     # сам сертификат – предполага
ok = True                                     # проверился ли сертификат з

# --- тело алгоритма ---
# Полиномиальный алгоритм: перебор всех пар –  $\Theta(n^2)$ .
# Точки (n, ops) сами ложатся на график вкладки «Рост».
for n in range(2, 11):
    step = 1                                # запустили алгоритм на вход
    ops = 0
    for i in range(n):
        for j in range(i, n):
            ops += 1
    step = 2                                # ответ готов
    cls = "P"                                #  $n^2/2$  операций – полином, з
    print("n =", n, "| ops =", ops, "|", cls)

```

```

-----
for v in range(n):
    used = set()
    for u in adj[v]:
        if color[u] != 0:
            used.add(color[u])
    c = 1
    while c in used:          # наименьший свободный цвет
        c += 1
    color[v] = c
    chi = max(chi, c)
    i += 1
    print("v =", v, "занято:", sorted(used), "-> цвет",
print("chi =", chi, "| delta =", delta, "| chi <= delta
variables: [
    { name: "color", role: "цвета вершин (0 = не покр
    { name: "v", role: "текущая вершина", range: "0 ...
    { name: "delta / chi", role: "максимальная степен
    { name: "i / step", role: "номер кадра", range: "0
],
},
"planarity-euler-formula#bipartite": {
    vizTitle: "Двудольность: BFS в 2 цвета",
    stepNote: "Шаг = сосед u получает цвет 3 - color[v]
    code: `n = 8
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4],
color = [0] * n          # 0 = не покрашена, иначе 1
queue = []                # очередь BFS
v = -1                    # вершина, из которой смотрим
u = -1                    # сосед, которого красим сейчас
ok = True                 # двудольный ли граф
i = 0

# --- тело алгоритма ---
for start in range(n):
    if color[start] != 0:
        continue
    color[start] = 1
    queue = [start]

```

```

while c in used:
    c += 1
ecolor[e] = c
inc[a][e] = c
inc[b][e] = c
chi_e = max(chi_e, c)
print("ребро", (a, b), "занято:", sorted(used), "-> ", c)
print("chi' =", chi_e, "| delta =", delta, "| delta <= ", delta)
variables: [
    { name: "ecolor", role: "цвета рёбер – рёбра на графе", range: 1..n },
    { name: "e", role: "номер текущего ребра", range: 1..n },
    { name: "v, u", role: "концы текущего ребра – подмножества V", range: 1..n },
    { name: "inc", role: "для каждой вершины: какие цвета имеют инцидентные рёбра", range: 1..n },
    { name: "chi_e / delta", role: "χ' – сколько цветов имеют инцидентные рёбра", range: 1..n }
],
},
"graph-components#dfs": {
    vizTitle: "Компоненты связности: DFS-покраска",
    stepNote: "Шаг = вершина v достаётся из стека; соседи v помечаются",
    code: `
n = 10 # вершин в графе
adj = [[1, 3], [0, 2, 3], [1, 3], [0, 1, 2], [5, 6], [4, 5]]
comp = [-1] * n # -1 = вершина ещё не покрашена
used = [False] * n # то же самое булевым массивом
comps = 0 # счётчик найденных компонент
c = 0 # номер текущей компоненты
v = -1 # вершина, в которой обход начался
stack = [] # фронт обхода (стек для DFS)
i = 0 # номер шага – двигает покадрово

# --- тело алгоритма ---
for start in range(n):
    if comp[start] != -1:
        continue
    comps += 1 # непокрашенная вершина => новая компонента
    c = comps
    stack = [start]
    comp[start] = c
    used[start] = True
    while stack:
        v = stack[-1]
        if v == -1:
            stack.pop()
            continue
        if used[v]:
            continue
        used[v] = True
        for u in adj[v]:
            if comp[u] == -1:
                stack.append(u)
    `
}

```



```

def union(a, b):
    global comps
    ra, rb = find(a), find(b)
    if ra == rb:
        return False # ребро внутри
    if rank[ra] < rank[rb]:
        ra, rb = rb, ra
    parent[rb] = ra
    if rank[ra] == rank[rb]:
        rank[ra] += 1
    comps -= 1
    return True

for e, (a, b) in enumerate(edges):
    u, v = a, b
    step = e + 1
    union(a, b)
print("компонент:", comps)
print("корни:", [find(x) for x in range(n)])`,
    variables: [
        { name: "parent", role: "DSU-массив: parent[v] = 0 ... n-1" },
        { name: "rank", role: "высота дерева DSU – union" },
        { name: "e / step", role: "номер обрабатываемого ребра" },
        { name: "u, v", role: "концы текущего ребра – подмножества" },
        { name: "comps", role: "сколько множеств осталось" }
    ],
},
"complexity-classes#hierarchy": {
    vizTitle: "Иерархия классов  $P \subseteq NP \subseteq PSPACE \subseteq EXP$ ",
    stepNote: "Шаг не нужен: схема подсвечивает тот овал, который является подмножеством",
    code: `cls = "NP" # имя класса: P, NP, PSPACE, EXP`
}

n = 10 # размер входа
ops = 0 # счётчик операций (нужен вк. в подсчёт сложности)

# --- тело алгоритма ---
# P: решается за полином – сортировка
a = [5, 2, 8, 1, 9, 3]

```

```

-----
    stepNote: "Шаг = один дизъюнкт: верификатор подстав.
    code: `clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -
cert = [1, 0, 1, 0]          # сертификат: x1=1, x2=0, x3
k = 0                        # номер проверяемого дизъюнк
ok = True                    # вердикт верификатора

# --- тело алгоритма ---
for k, cl in enumerate(clauses):
    hit = False
    for l in cl:
        val = cert[l - 1] if l > 0 else 1 - cert[-l - 1]
        if val == 1:
            hit = True
    ok = ok and hit
    print("дизъюнкт", k + 1, cl, "выполнен" if hit else
print("сертификат", "принят" if ok else "отвергнут")`,
    variables: [
        { name: "cert", role: "сертификат – назначение пер
        { name: "k", role: "номер дизъюнкта, который пров
        { name: "ok", role: "итоговый вердикт: сертификат
        { name: "clauses", role: "сама формула 3-SAT – 4
    ],
},
"complexity-classes#growth": {
    vizTitle: "Рост числа операций: полином или экспонен
    stepNote: "Шаг = одна точка (n, ops). Чем больше то
    code: `n = 1                                # размер входа
ops = 0                                         # сколько операций насчитал

# --- тело алгоритма ---
# Полиномиальный алгоритм: перебор всех пар –  $\Theta(n^2)$ .
# Точки (n, ops) сами ложатся на график вкладки «Рост».
for n in range(2, 11):
    ops = 0
    for i in range(n):
        for j in range(i, n):
            ops += 1
    print("n =", n, "ops =", ops)

```

```

        if rank[ra] == rank[rb]:
            rank[rb] += 1
        mst.append((w, a, b))
        total += w
print("вес остова:", total, "| рёбер:", len(mst))
print(mst)\,
    variables: [
        { name: "i", role: "номер текущего ребра – перематриваемое"},
        { name: "u, v, w", role: "концы и вес рассматриваемого ребра"},
        { name: "parent", role: "DSU-массив: цвета компонентов"},
        { name: "mst", role: "рёбра, уже взятые в остов"},
        { name: "total", role: "вес остова (51 на этом графе)"},
    ],
},
"mst#prim": {
    vizTitle: "Остовное дерево: Прим (рост из вершины E)",
    stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
    code: `import heapq

n = 9                                # вершин: A...I
adj = {"A": [("B", 2), ("D", 5)], "B": [("A", 2), ("C", 3), ("E", 4)],
       "C": [("B", 4), ("F", 9)], "D": [("A", 5), ("E", 3), ("G", 8)],
       "E": [("D", 3), ("B", 6), ("F", 7), ("H", 11)], "F": [("C", 9), ("E", 7), ("H", 10), ("I", 13)],
       "G": [("D", 8), ("H", 10)], "H": [("G", 10), ("E", 11), ("I", 12)],
       "I": [("H", 12), ("F", 13)]}}
start = "E"                          # из какой вершины растим дерево
heap = [(0, start, start)]           # (вес, вершина, откуда пришло ребро)
taken = []                          # вершины уже в остовае – красим в синий
mst = []                            # рёбра остова
total = 0                           # вес остова (= 51)
u, v, w = None, None, None          # текущее ребро

# --- тело алгоритма ---
while heap:
    w, x, frm = heapq.heappop(heap)   # самое лёгкое ребро
    if x in taken:                   # устаревшая запись
        continue
    taken.append(x)
    mst.append((w, frm, x))
    total += w
    u, v, w = x, None, None
    for y, w in adj[x]:
        if y not in taken:
            heap.append((w, y, x))
    heapq.heapify(heap)
print("остовное дерево: Прим", mst)
print("вес остова:", total)

```

```

-----
while len(mst) < n - 1:
    phase += 1
    cheapest = {}
    for (w, a, b) in edges:                # КАЖДАЯ компонента
        ra, rb = find(a), find(b)
        if ra == rb:
            continue
        u, v = a, b
        if ra not in cheapest or w < cheapest[ra][0]:
            cheapest[ra] = (w, a, b)
        if rb not in cheapest or w < cheapest[rb][0]:
            cheapest[rb] = (w, a, b)
    if not cheapest:
        break                               # граф несвязен
    for (w, a, b) in cheapest.values():
        ra, rb = find(a), find(b)
        if ra == rb:
            continue                         # две компоненты
        comp[ra] = rb
        mst.append((w, a, b))
        total += w
print("фаз:", phase, "| вес остова:", total)
print(mst)`
    variables: [
        { name: "comp", role: "DSU: корень компоненты верш",
        { name: "cheapest", role: "корень компоненты -> с",
        { name: "phase", role: "номер фазы; на этом графе",
        { name: "u, v, w", role: "ребро-кандидат, которое",
        { name: "mst / total", role: "взятые рёбра и вес c",
    ],
},
"sparse-table#2d-build": {
    vizTitle: "Разреженная таблица 2D: построение",
    stepNote: "Шаг = новый уровень k: блоки 2^k×2^k доп",
    code: `A = [[45, 12, 88, 34, 11, 76, 23, 90], [67,
        88, 29, 60], [25, 41, 16, 92, 9, 17, 56, 31], [5
        1, 42, 85, 14]]
r, c, k = 0, 0, 0

```

```

vizTitle: "Префиксные суммы 2D (прямоугольники)",
stepNote: "Демонстрация кликабельна вручную: наведи
code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):
    S.append([0] + [None] * m)
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1]

r1, c1, r2, c2 = 1, 1, 2, 2
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`,
variables: [
    { name: "i, j", role: "текущая ячейка таблицы S (
    { name: "S[i][j]", role: "сумма прямоугольника (0
    },
    { name: "r1, c1, r2, c2", role: "углы запроса; от
],
},
};

```

```

/** Безопасный доступ: страница без записи получает пус
export const getPageSync = (chapterId?: string | null, c
    (demoId && chapterId && PAGE_SYNC[`${chapterId}#${demo
    (chapterId && PAGE_SYNC[chapterId]) ||
    { variables: [] }];

```

/**

* Стартовое содержимое редактора для страницы.

*

* Формат: короткий комментарий-шапка + исполняемый реф

* Визуализация читает из его трассы знакомые имена (i,

* но точно так же примет пользовательский код с теми же

*/

```
-----
: `# «${chapterTitle}»\n# демо на странице нет — св

if (!sync.code) return head;
const init = extractInit(sync.code);
return `${head}\n${init}\n`;
}
```

=====

РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

=====

ФАЙЛ 26/111: src/data/tickets/ticket1_segtree.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 217 | РАЗМ

```
import { Chapter } from '../../types';

/**
 * Билет 1 — «Дерево отрезков с операциями на отрезках»
 *
 * Страница написана по канону остальных билетов (эталон
 * одна главная секция, строгое правило по центру, анализ
 * явная строка сложности, весь код спрятан в <details>
 *
 * Визуализация — src/components/SegmentTreeVisualizer.ts
 * построение / запросы / теория), синхронизация с комп
 * (переменные n, i, v, l, r, m) и её подрежимы #build
 */
export const ticket1Segtree: Chapter[] = [
  {
    id: "segment-trees",
    title: "1. Дерево отрезков с операциями на отрезках",
    type: "html",
    description:
      "Структура для запросов на отрезке и обновлений з
    category: "Продвинутые структуры",
```

```

    </div>
    <div class="bg-slate-900 border border-
      <p class="text-[10px] uppercase tra
      <p class="font-mono text-emerald-300
    </div>
  </div>

<div class="grid grid-cols-1 xl:grid-cols-2
  <div class="bg-slate-800 p-6 rounded-lg
    <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
    Аналогия 1: Корпоративная пре
  </div>
  <p class="text-slate-300 text-sm">Б
икам департаментов: «внутри всем +5 %». Нач
nt-mono text-emerald-300">lazy[v]</span>) и
свой баланс. Это и есть <b>отложенное обнов
  </div>

  <div class="bg-slate-900 p-6 rounded-lg
    <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
    Аналогия 2: Матрёшки-коробки
  </div>
  <p class="text-slate-300 text-sm">М
Нужно покрасить половину массива в синий? Н
ь спускаемся только тогда, когда нас спроси
(push).</p>
  </div>
</div>

<div class="bg-slate-800/70 p-5 rounded-lg
  <p class="text-sm font-bold text-white
  <p class="text-slate-300 text-sm mb-3">
на [L, R]» – всё это делается одним спуском
ss="font-mono text-emerald-300">tree[v] +=
</b>: перед любым спуском в детей вызываем
  <table class="w-full text-xs text-left

```



```
int m = (l + r) / 2;
add(2 * v, l, m, L, R, x);
add(2 * v + 1, m + 1, r, L, R, x);
tree[v] = tree[2 * v] + tree[2 * v + 1];
}

```

г накопиться. Забыл push – получил правильн

</div>
 </details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">

Код: итеративное дерево (2n, снизу)

```

</summary>
<div class="p-5 text-sm text-slate-300
    <pre class="bg-slate-950 p-4 rounded
        # для удобства
tree = [0] * (2 * n)
for i in range(n):                # листья: tree[n +
    tree[n + i] = a[i]
for v in range(n - 1, 0, -1):    # предки снизу вверх
    tree[v] = tree[2 * v] + tree[2 * v + 1]

def query(l, r):                  # сумма на полуинтервале
    res = 0
    l += n; r += n
    while l < r:
        if l & 1: res += tree[l]; l += 1    # l – правый
        if r & 1: r -= 1; res += tree[r]    # r – левый
        l //= 2; r //= 2
    return res

```

<p class="text-slate-400 text-xs">И
 ней заметно хитрее – на экзамене безопаснее

</div>
 </details>

<details class="bg-slate-900/40 rounded-lg l

```
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
  <div class="flex items-center mb-6 flex-wrap gap-3">
    <span class="bg-blue-600 text-white px-4 py-1 rounded">
      <h2 class="text-2xl sm:text-3xl font-bold text-white">
    </div>

    <div class="space-y-8">
      <div class="bg-slate-700/50 p-6 rounded-xl border">
        <h3 class="text-xl font-bold text-emerald-400">
          <div class="bg-slate-900 p-4 rounded-lg mb-4">
            <p class="text-lg text-emerald-300 italic">
              <p class="text-xl font-mono text-white">
            </div>

            <p class="text-slate-300 text-sm mb-4">Дейкстра
            ершины <span class="font-mono text-emerald-300">
              требование к графу – <b>неотрицательные веса
              ы, и на каждом шаге забирает в него ту непос
              n class="font-mono text-emerald-300">d[u]</span>
            </p>

            <p class="text-slate-300 text-sm mb-4">Почему
            ext-emerald-300">u</span> обязан выйти из о
            х</span>, а у неё <span class="font-mono text-emerald-300">
              бавляет</b> неотрицательные веса, значит кор
              ся механика: <b>извлечь минимум → зафиксировать
              ниже.</p>

            <div class="grid grid-cols-1 sm:grid-cols-3 gap-4">
              <div class="bg-slate-900 border border-slate-800 p-4">
                <p class="text-[10px] uppercase tracking-wider">
                <p class="font-mono text-emerald-300">
              </div>
              <div class="bg-slate-900 border border-slate-800 p-4">
                <p class="text-[10px] uppercase tracking-wider">
                <p class="font-mono text-emerald-300">
              </div>
              <div class="bg-slate-900 border border-slate-800 p-4">
                <p class="text-[10px] uppercase tracking-wider">
                <p class="font-mono text-emerald-300">
              </div>
            </div>
          </div>
        </h3>
      </div>
    </div>
  </div>
```

$(u \rightarrow v)$

: переменные `<span class="font`
`>`, `<span class="font-mono text-emerald-300"`
аут картинку напрямую.

```
<ol class="list-decimal pl-5 space-y-2 text-
  <li><b>Инициализация.</b> <span class="
text-emerald-300">∞</span>; в кучу кладём <
emerald-300">prev</span> пуст.</li>
  <li><b>Извлечь минимум.</b> Достаём пар
"font-mono text-emerald-300">du</span>. Если
xt-emerald-300">done</span> – пара устарела
  <li><b>Зафиксировать.</b> <span class="
меняется.</li>
  <li><b>Релаксировать рёбра.</b> Для каж
no text-emerald-300">u</span>: если <span c
="font-mono text-emerald-300">dist[v]</span>
<span class="font-mono text-emerald-300">(d
  <li><b>Повторять,</b> пока куча не пуст
∞</span>.</li>
</ol>
```

```
<div class="bg-slate-800/70 p-5 rounded-lg l
  <p class="text-sm font-bold text-white m
  <table class="w-full text-xs text-left l
    <thead>
      <tr class="text-slate-500 borde
        <th class="py-1.5 pr-2 font
        <th class="py-1.5 pr-2 font
        <th class="py-1.5 font-bold
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-slate
ld">Пиццерия А (0)</td><td class="py-2 font
        <tr class="border-b border-slate
ld">Парк С (2)</td><td class="py-2 font-mono
        <tr class="border-b border-slate
ld">Дом 1 В (3)</td><td class="py-2 font-mono
```

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-
<h3 class="text-xl font-bold text-indigo-400">

<p class="text-slate-300 text-sm mb-4">И
pan> извлекается из кучи, <span class="font
nt-mono text-emerald-300">s → u. Отс
00">dist[v] = δ(s, v) для всех дости
ext-emerald-300">dist[v] = ∞ для нед

<details class="bg-slate-900/40 rounded-lg l
<summary class="p-4 font-bold text-slate
-b border-slate-700/50">» Доказательство ин

<div class="p-5 text-sm text-slate-300
<p>Пусть <span class="font-mono tex
di
text-emerald-300">s → u и пусть <sp
не извлечена, а <span class="font-mono tex
nt-mono text-emerald-300">y не перва

<p>Так как <span class="font-mono t
pan class="font-mono text-emerald-300">(x, y
δ(s, x) + w(x, y) = δ(s, y). Путь д
font-mono text-emerald-300">u, веса
< dist[u].</p>

<p>Итого <span class="font-mono tex
text-emerald-300">u – противоречие с
0">u нет. ■</p>

<p class="text-slate-400 text-xs">Г
u)». С отрицательным ребром хвост п
о контрпример из карточки «Ограничения».</p>

</div>

</details>

<details class="bg-slate-900/40 rounded-lg l
<summary class="p-4 font-bold text-slate
-b border-slate-700/50">» Откуда берётся сл

<div class="p-5 text-sm text-slate-300

```

        </tr>
        <tr class="border-b border-slate-200">
        ><td class="py-2 pr-3 font-mono">0(V + E)</td><td class="py-2 pr-3 font-mono">0(E + V · C)</td><td class="py-2">ц
        </tbody>
    </table>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
    <summary class="p-4 font-bold text-slate-300">» Код: Дейкстра на
    <div class="p-5 text-sm text-slate-300">
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
INF = float("inf")

def dijkstra(graph, start):
    """graph[v] = [(сосед, вес), ...]; веса >= 0."""
    dist = {v: INF for v in graph}
    prev = {v: None for v in graph}
    dist[start] = 0
    done = set()
    pq = [(0, start)]  # (расстояние, v)

    while pq:
        du, u = heapq.heappop(pq)
        if u in done:  # устаревшая информация
            continue
        done.add(u)  # dist[u] окончательное
        for v, w in graph[u]:
            if du + w < dist[v]:  # релаксация
                dist[v] = du + w
                prev[v] = u
                heapq.heappush(pq, (dist[v], v))
    return dist, prev

def restore(prev, start, target):

```

```

vector<long long> d;
vector<int> p;

void dijkstra(int s) {
    d.assign(n, INF); p.assign(n, -1); d[s] = 0;
    priority_queue<pair<long long,int>,
                  vector<pair<long long,int>&,
                  greater<pair<long long,int>>>
    pq.push({0, s});
    while (!pq.empty()) {
        auto [du, u] = pq.top(); pq.pop();
        if (du > d[u]) continue; // устарев
        for (auto [v, w] : g[u]) {
            if (du + w < d[v]) {
                d[v] = du + w;
                p[v] = u;
                pq.push({d[v], v});
            }
        }
    }
}

vector<int> path(int s, int t) { // восст
    vector<int> res;
    if (d[t] == INF) return res; // недостижим
    for (int v = t; v != -1; v = p[v]) res.push_back(v)
    reverse(res.begin(), res.end());
    return res;
}
}

```

<p class="text-slate-400 text-xs">0
 уть из 10^5 рёбер веса 10^9 даёт 10^{14} – <span
 </div>
 </details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">» Код: $O(V^2)$ без кы
 <div class="p-5 text-sm text-slate-300

```
        <li><b>Ориентированность.</b> Демо-  
ом графе каждое ребро добавляется в оба спи  
        <li><b>Дейкстра  $\neq$  BFS.</b> BFS – ча  
        С разными весами FIFO ломается, нужна куча  
        <li><b>«Одна пара».</b> Алгоритм сч  
ий выход при извлечении цели (билет 17), но  
    </ul>  
</div>
```

```
<p class="text-slate-300 text-sm mt-4 mb-4">  
ов ценой скорости <span class="font-mono text-emerald-300"> $O(V^3)$ </span>; б  
после перевзвешивания потенциалов). Предст
```

```
    </div>  
  </div>  
</section>  
,  
{  
  id: "bellman-ford",  
  title: "15. Графы. Поиск кратчайшего пути. Форд-Беллман",  
  type: "html",  
  category: "Графы. Пути",  
  content: `  
<section id="bellman-ford-content" class="mb-12 scroll-m-12">  
  <div class="flex items-center mb-6 flex-wrap gap-3">  
    <span class="bg-blue-600 text-white px-4 py-1 rounded">15</span>  
    <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">Графы. Пути</h2>  
  </div>  
  
  <div class="space-y-8">  
    <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">  
      <h3 class="text-xl font-bold text-rose-400">Алгоритм Форда-Беллмана</h3>  
  
      <div class="bg-slate-900 p-4 rounded-lg mb-4">  
        <p class="text-lg text-rose-300 italic">Алгоритм Форда-Беллмана работает только для графов с  
        <p class="text-xl font-mono text-white">если в графе  
есть отрицательный цикл.</p>  
      </div>  
    </div>
```

ревне из V жителей просто не бывает. Порядо

```

    <div class="bg-slate-900 p-6 rounded-lg"
      <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
        Отрицательный цикл: временная п
      </div>
      <p class="text-slate-300 text-sm mb-4">
        меньше: <span class="font-mono text-emerald-300">
        и ответ « $-\infty$ ». Контрольная итерация – это де
      .</p>
    </div>
  </div>
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl border-rose-500 shadow-md"
  <h3 class="text-xl font-bold text-blue-400">Демонстрация
  <p class="text-slate-300 text-sm mb-4">Демонстрация (2 D, Лес E, Склад F) и <b>9</b> односторонних
  00">В → С = -2</span> (сплав по течению – б
  >, которая в обычном режиме весит <span cla
  no text-emerald-300">-6</span>. В компиляторе
  span> внутри <span class="font-mono text-em

```

```

<div class="bg-slate-800/70 p-5 rounded-lg border-rose-500 shadow-md"
  <p class="text-sm font-bold text-white">Таблица
  <table class="w-full text-xs text-left"
    <thead>
      <tr class="text-slate-500 border-b">
        <th class="py-1.5 pr-2 font-normal">Имя
        <th class="py-1.5 pr-2 font-normal">Возраст
        <th class="py-1.5 font-bold">Статус
      </tr>
    </thead>
    <tbody class="text-slate-300">
      <tr class="border-b border-slate-700">

```



```
</td></tr>
```

```
      <tr class="border-b border-slate-
"py-2 pr-2 font-mono">-10</td><td class="py-
nt-mono">-2</td></tr>
```

```
      <tr class="align-top"><td class="
12</td><td class="py-2 pr-2 font-mono">-9</
ont-bold">всё ещё уменьшается → отрицательны
</tbody>
```

```
</table>
```

```
<p class="text-slate-400 text-xs mt-3">
остижимые из цикла вершины (A, C, D, E, F).
х, до которых из него есть путь.</p>
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl bord
```

```
<h3 class="text-xl font-bold text-indigo-400
```

```
<details class="bg-slate-900/40 rounded-lg
```

```
  <summary class="p-4 font-bold text-slate-
-b border-slate-700/50">» Доказательство (и
```

```
    <div class="p-5 text-sm text-slate-300
```

```
      <p><b>Утверждение.</b> После <span c
ss="font-mono text-emerald-300">v</span> зн
```

```
      кратчайшего пути из <span class="font-mono
nt-mono text-emerald-300">i</span> рёбер.</p>
```

```
      <p><b>База.</b> <span class="font-m
"font-mono">s</span>, и <span class="font-m
```

```
      <p><b>Шаг.</b> Пусть утверждение вер
font-mono">P</span> из  $\leq$  <span class="font-m
```

```
      о – <span class="font-mono">(u, v)</span>,
– 1</span> рёбер и сам является кратчайшим
```

```
font-mono">i – 1</span> итераций <span clas
an>-й итерации ребро <span class="font-mono
```

```
   $\leq \text{dist}[u] + w(u, v) \leq |P|$ </span> ■</p>
```

```
      <p><b>Следствие 1.</b> Если отрицат
уть прост и содержит  $\leq$  <span class="font-mo
pan class="font-mono">dist</span> точны.</p>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border-1 border-slate-700/50">
  <h3 class="text-xl font-bold text-emerald-400">
    <details class="bg-slate-900/40 rounded-lg border-1 border-slate-700/50">
      <summary class="p-4 font-bold text-slate-300">
        -b border-slate-700/50">> Код: Форд–Беллман
      <div class="p-5 text-sm text-slate-300">
<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-emerald-400">
def bellman_ford(n, edges, s):
    """edges = [(u, v, w), ...] – ориентированные рёбра
    dist = [INF] * n
    prev = [-1] * n
    dist[s] = 0
    changed_vertex = -1

    for it in range(n):
        changed_vertex = -1
        for u, v, w in edges:
            if dist[u] != INF and dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                prev[v] = u
                changed_vertex = v
        if changed_vertex == -1:
            break

    if changed_vertex == -1:
        return dist, prev, None

    # changed_vertex улучшен на n-й итерации:
    # поднимаемся по prev на n шагов – гарантированно зациклимся
    y = changed_vertex
    for _ in range(n):
        y = prev[y]

    cycle = [y]
    v = prev[y]
```

```

int n; // число вершин
vector<Edge> edges; // плоский список рёбер
vector<long long> d;
vector<int> p;

// возвращает вершину, улучшенную на «лишней» итерации,
int bellman_ford(int s) {
    d.assign(n, INF); p.assign(n, -1); d[s] = 0;
    int x = -1;
    for (int it = 0; it < n; ++it) { // n-1
        x = -1;
        for (const auto &e : edges) {
            if (d[e.u] != INF && d[e.u] + e.w < d[e.v]) {
                d[e.v] = d[e.u] + e.w;
                p[e.v] = e.u;
                x = e.v;
            }
        }
        if (x == -1) break; // ранний
    }
    return x;
}

vector<int> negative_cycle(int x) {
    int y = x;
    for (int i = 0; i < n; ++i) y = p[y]; // захватили
    vector<int> cycle;
    for (int v = y;; v = p[v]) {
        cycle.push_back(v);
        if (v == y && cycle.size() > 1) break;
    }
    reverse(cycle.begin(), cycle.end());
    return cycle;
}

int main() {
    int s = 0;

```

```

return None # отрицательный
return dist</code></pre>

```

- Ранний выход. Если з
- рафах экономит кратно.
- SPFA (Shortest Path
- , у которых только что улучшился <span clas
- an>, но худший случай остаётся <span cla
- кас» из рёбер) SPFA ловит TL. Критерий цикла
- Порядок рёбер. Форма
- итерации, против течения – за <span class="
- ция даёт финальные
- DAG. Если граф ацикл
- 0(V + E), и

```

</ul>
</div>
</details>
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-amber-400

```

- ```

<div class="bg-rose-950/30 p-5 rounded-lg b
 <p class="text-sm font-bold text-rose-300
 <ul class="list-disc pl-5 space-y-2 tex
 V – 1, а не V. Рабочих и
ont-mono text-emerald-300">V-я – тол
отрицательным циклом <span class="font-mono
 Переполнение INF. <span <
INT_MAX превращается в отрицательно
запасом и long lon
 Неориентированный граф с мин
lass="font-mono">-3 – это цикл длины
рицательном цикле. Минусы в неориентированн
 Цикл есть, но не от старта.<
. Нужна проверка всего графа – добав
а, билет 17) и запустите от неё.
 «Ответ -∞». Если отрицат

```

(но не с отрицательными циклами) и пишется `<span class="font-mono">V × V</span>` размера `<span class="font-mono text-emerald-300">k</span>` восточных рёбер, `<span class="font-mono text-emerald-300">k</span>` в остальных клетках.

```
<div class="bg-slate-800 p-6 rounded-lg border border-emerald-500 shadow-md">
 <div class="absolute -top-3 left-4 bg-slate-800 border border-emerald-500 shadow-md">
 Суть фокуса (по шагам)
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Главный герой – внешний цикл k та-
 , если в качестве промежуточных k
 , ..., k?». Каждый шаг добавляет в ра-
 </p>
 <ul class="text-sm text-slate-300 space-y-2">
 Шаг k = -1 (старт): раз-
 Шаг k = 0: разрешаем пере-
 Шаг k = 1: разрешены пере-
 Шаг k = V - 1: разрешены пере-

</div>
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-emerald-500 shadow-md">
 <p class="text-[10px] uppercase tracking-wider">Шаг k = -1 (старт):</p>
 <p class="font-mono text-emerald-300">k</p>
 </div>
 <div class="bg-slate-900 border border-emerald-500 shadow-md">
 <p class="text-[10px] uppercase tracking-wider">Шаг k = 0:</p>
 <p class="font-mono text-emerald-300">k</p>
 </div>
 <div class="bg-slate-900 border border-emerald-500 shadow-md">
 <p class="text-[10px] uppercase tracking-wider">Шаг k = 1:</p>
 <p class="font-mono text-emerald-300">k</p>
 </div>
```

напрямую: измените `<span class="font-mono">`

```
<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 pr-2 font-normal">
 <th class="py-1.5 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-500">
 <td class="font-mono">0</td><td class="py-2">в вершине</td>
 <tr class="border-b border-slate-500">
 <td class="font-mono">4</td><td class="py-2">открыть</td>
 <tr class="border-b border-slate-500">
 <td class="font-mono">3</td><td class="py-2">2 – вх</td>
 <tr class="border-b border-slate-500">
 <td class="font-mono">2</td><td class="py-2">3 – вх</td>
 <tr class="border-b border-slate-500">
 <td class="font-mono">4</td><td class="py-2">через</td>
 <tr class="border-b border-slate-500">
 <td class="font-mono">0</td><td class="py-2">из 5 в</td>
 <tr class="align-top"><td class="font-mono">
 300">10</td><td class="py-2">ребро 6 → 1 за</td>
 </tbody>
 </table>
 <p class="text-slate-400 text-xs mt-3">
 mono text-emerald-300">0 → {0, 4, 7, 2, 6, 10}
 "font-mono">0 → 3 → 4 → 6 (2 + 4 + 10)
 всего в финальной матрице 11 таких пар. Диа</p>
 </div>
```

```
<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <table class="w-full text-xs text-left">
```

```

<p class="text-center font-mono text-
<p>Почему хватает одной матрицы.
n> и столбец k
+ d(k-1)[k][k], а <span class="font-
мое не меньше первого. Значит, читая <span k-й итерации
d[i][j] без
<p>Следствие: если в графе е
-mono">d[k][k] < 0 и рассуждение
</div>
</details>

```

```

<details class="bg-slate-900/40 rounded-lg
<summary class="p-4 font-bold text-slate
-b border-slate-700/50">» Эксперимент: пере
<div class="p-5 text-sm text-slate-300

```

```

<pre class="bg-slate-950 p-4 rounded overflow-x-auto te

```

```

M = [[0,4,INF,2,INF,INF,INF],
 [INF,0,3,INF,INF,3,INF],
 [INF,INF,0,INF,2,INF,INF],
 [INF,INF,INF,0,4,2,INF],
 [INF,INF,INF,INF,0,INF,1],
 [INF,INF,INF,INF,INF,0,5],
 [INF,1,INF,INF,INF,INF,0]]

```

```

def floyd(M, order="kij"):
 n = len(M)
 d = [row[:] for row in M]
 for a in range(n):
 for b in range(n):
 for c in range(n):
 k, i, j = {"kij": (a, b, c),
 "ijk": (c, a, b),
 "ikj": (c, a, b)}[order]
 if d[i][k] + d[k][j] < d[i][j]:
 d[i][j] = d[i][k] + d[k][j]
 return d

```

```

 for j in range(n):
 if d[k][j] == INF:
 continue
 if d[i][k] + d[k][j] < d[i][j]:
 d[i][j] = d[i][k] + d[k][j]
 nxt[i][j] = nxt[i][k]

negative = [i for i in range(n) if d[i][i] < 0]
return d, nxt, negative

def path(nxt, i, j):
 if nxt[i][j] == -1:
 return None # j недостижим
 p = [i]
 while p[-1] != j:
 p.append(nxt[p[-1]][j])
 return p

nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [
 ("S", "A", 4), ("S", "B", 5), ("B", "C", -2), ("A", "C", 1),
 ("C", "D", 3), ("C", "E", 4), ("D", "F", 2), ("E", "F", 1)
]
E = [(idx[u], idx[v], w) for u, v, w in E]

d, nxt, neg = floyd_warshall(len(nodes), E)
print([d[0][j] for j in range(len(nodes))]) # [0, 4, 5, 3, 2, 1]
print(path(nxt, idx["S"], idx["F"])) # [0, 2, 1]
print(neg) # [] — отрицательных циклов нет

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg p-4">

<summary class="p-4 font-bold text-slate-100 border-b border-slate-700/50">» Код: C++ (матрица Floyd-Warshall)

<div class="p-5 text-sm text-slate-300">

```

<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-100">
 t;

```

```

#include <algorithm>

```





```

 /span> Дейкстра на перевзвешенном графе. Тран-
 ции достижимость проверяется по DAG.</p>
 </div>
 </div>
</section>`
},
{
 id: "johnson-algo",
 title: "17. Графы. Поиск кратчайшего пути. Ускорени-
 type: "html",
 description:
 "Джонсон (перевзвешивание потенциалов), встречный
 category: "Графы. Пути",
 content: `
<section id="johnson-algo" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r-
 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-amber-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-amber-300 italic">
 <p class="text-xl font-mono text-white">
 е посещённых вершин (встречный поиск, ранний
 contraction hierarchies).</p>
 </div>

 <p class="text-slate-300 text-sm mb-4">Дейк-
 в лоб. На экзамене спрашивают, что сдела
 строки перевзвешивания» до промышленных у

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
 rder border-emerald-500 shadow-md">
 Аналогия 1: Такси с навигатором
```



кучу заменяют **деком**: ребро веса 0 к  
новременно живут максимум два уровня), поэто  
>. Обобщение – **алгоритм Дила (корзины)**  
 $E + V \cdot C$ ). Оба приёма – стандартный ответ на

**Форд**  
оторых расстояние только что уменьшилось  
ческого прогона, но асимптотика та же,  $O(V \cdot E)$   
пользуют осторожно, с эвристиками SLF/LLL).  
) уменьшает высоту и на практике быстрее, к

**Нави**  
в **предобработку**. Сначала – **сжатие**  
индексами  $0 \dots N$ , и уже по ним строят граф и се  
стояния от нескольких «ориентиров» до всех в  
етрии. **Contraction hierarchies** (так  
и запрос идёт двунаправленным поиском вверх  
вершине заранее приписывают «метки», и зап

**тро**, но не кратчайший. Переоценка = неверный  
именимой к отрицательным весам и даёт все п  
 $\text{in}_f + \text{min}_b \geq \mu$ . Остановка по встрече даёт  
ок сломается. Там нужен Dial.  
к практике меньше бесполезных релаксаций.

**py-0.5 rounded text-xs font-mono text-emera**

```

for s in range(n):
 d[s][s] = 0
 heap = [(0, s)]
 while heap:
 dist, v = heapq.heappop(heap)
 if dist > d[s][v]:
 continue
 for to, w in g[v]:
 if d[s][to] > dist + w:
 d[s][to] = dist + w
 heapq.heappush(heap, (d[s][to], to))
 for v in range(n):
 if d[s][v] < INF:
 d[s][v] = d[s][v] - h[s] + h[v] # optional
return d

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg p-4">

<summary class="p-4 font-bold text-slate-300 border-b border-slate-700/50">

Код: встречный Дейкстра, A\*, 0-1

</summary>

<div class="p-5 text-sm text-slate-300">

<pre class="bg-slate-950 p-4 rounded">

встречный Дейкстра: остановка по min\_f + min\_b

def bidirectional(g, rg, s, t): # rg – граф с обратными ребрами

INF = float("inf")

df = {s: 0}; db = {t: 0}

qf = [(0, s)]; qb = [(0, t)]

mu = INF

while qf and qb:

if qf[0][0] + qb[0][0] >= mu: # вот правый ответ

return mu

dist, v = heapq.heappop(qf)

if dist > df.get(v, INF): continue

for to, w in g[v]:

nd = dist + w

```

d = [INF] * n; d[s] = 0
dq = deque([s])
while dq:
 v = dq.popleft()
 for to, w in g[v]:
 if d[v] + w < d[to]:
 d[to] = d[v] + w
 (dq.appendleft(to) if w == 0 else dq.append(to))
return d

```

```

<details class="bg-slate-900/40 rounded-lg p-4 font-bold text-slate-300 border-slate-700/50">
 <summary>

```

Вопросы, которые любят задавать (и ответы на них)

</summary>

<div class="p-5 text-sm text-slate-300">

<p>• Почему после перевзвешивания весов не выполняется релаксация?</p>  
 есть условие отсутствия улучшающей релаксации.

<p>• Почему кратчайшие пути не меняются при изменении весов?</p>  
 иш) и одинаков для всех путей между этой парой вершин.

<p>• Что лучше для всех пар: Флойд-Уоршелл или BFS?</p>  
 g V) выигрывает на разреженных.</p>

<p>• Какая эвристика A\* допустима на графах с отрицательными весами?</p>  
 ии скорости можно делить на максимальную скорость.

<p>• Зачем нужен ранний выход в Дейкстре?</p>  
 смысла нет.</p>

<p>• Почему 0-1 BFS работает без кучи?</p>  
 же отсортирована.</p>

</div>

</details>

</div>

</div>

</section>

},

];

```

 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
он связан. MST – такое остовное дерево, у
</div>

```

```

<p class="text-slate-300 text-sm mb-4">Все
реза (cut property): разрежем вершины н
MST. Алгоритмы отличаются только тем, к
Прим – «выросшее дерево против остального гр

```

```

<p class="text-slate-300 text-sm mb-4">Второ
никогда не входит в MST. Именно его исполь
бразовавшемся цикле (рёбра-то мы берём по в

```

```

<div class="overflow-x-auto my-6">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3 font-b">
 <td class="py-2 pr-3">Рёбра
 <td class="py-2 pr-3 font-m">
 <td class="py-2 pr-3 font-m">
 <td class="py-2">рёбра даны
 </tr>
 <tr class="border-b border-slate-200">
 <td class="py-2 pr-3 font-b">
 <td class="py-2 pr-3">Растёр
 <td class="py-2 pr-3 font-m">
 <td class="py-2 pr-3 font-m">

```

Аналогия к билету 20 • Боровков

```

 </div>
 <p class="text-slate-300 text-sm">Самому чужому племени: к вечеру компонент $\log_2 V$ дней образует единую федерацию.
 </div>
</div>

```

```

<div class="bg-slate-900/40 rounded-lg border-slate-700/50">
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
 состояние от одной вершины до каждой; MST минимизирует вес куче другой: у Прима — вес ребра к дереву, у Крускала —
 тся.</p>
 <p class="text-slate-400 text-sm mb-1">
 вных весах деревьев может быть несколько, но MST единственный.
 <p class="text-slate-400 text-sm mb-1">
 дом вместо почти-константного find/union.</p>
 <p class="text-slate-400 text-sm mb-1">
 и Краскал построил минимальный остовный <code>find(u) == find(v)</code>
 <p class="text-slate-400 text-sm">• Краскал
 ин раз, DSU спасает от дубля (find(u) == find(v)).
 </div>

```

```

<p class="text-slate-300 text-sm mt-4">Сложность: $O(E \log V)$
 <p class="text-slate-300 text-sm">
 бесплатный); Прим с двоичной кучей <code>O(E \log V)</code>, на плотной
 <code>O(E \log V)</code>, на плотной
 <code>O(E \log V)</code>, на плотной
 <code>O(E \log V)</code>, на плотной
 <code>O(E \log V)</code>, на плотной
 </p>

```

```

<details class="bg-slate-900/40 rounded-lg border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 <b border="1" class="text-slate-300">

```

Код 18: Краскал + DSU (Скрыто)

```

</summary>

```



```
key[0] = 0
heap = [(0, 0)] # (вес ребра, вершина) ← вот
total = 0
```

```
while heap:
 w, v = heapq.heappop(heap)
 if used[v]:
 continue # устаревшая запись в куче
 used[v] = True
 total += w
 for to, weight in graph[v]:
 if not used[to] and weight < key[to]:
 key[to] = weight
 heapq.heappush(heap, (weight, to))
print(total)</pre>
```

<p class="text-slate-400 text-xs">С  
(накапливаем путь от старта), здесь <span  
а – другая задача.</p>  
</div>  
</details>

<details class="bg-slate-900/40 rounded-lg l  
<summary class="p-4 font-bold text-slate  
-b border-slate-700/50">

Код 20: Борувка (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300

<pre class="bg-slate-950 p-4 rounde  
= list(range(n))

```
def find(v):
 while parent[v] != v:
 parent[v] = parent[parent[v]]
 v = parent[v]
 return v
```

```
comps = n
total = 0
while comps > 1:
```

```
 <p>• Как найти второе по величине о
ребро на образовавшемся цикле (LCA + максим
 </div>
 </details>
 </div>
</div>
</section>`,
 },
];
```

---

ФАЙЛ 29/111: src/data/tickets/ticket21\_24.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 444 | РАЗМ

---

```
import { Chapter } from '../types';

export const tickets21to24: Chapter[] = [
 {
 id: "string-kmp",
 title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
 type: "html",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-blue-400 m
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic m
 <p class="text-xl font-mono text-white">
сом, оканчивающимся в позиции i.</p>
 </div>
```

```

<p>Тот же пример: abcabcd</p>
<ul class="list-disc pl-5 space-y-2">
 a: Префиксов нет. <code>
 ab: Из начала ("a")
>
 abc: Опять мимо. <code>
 abca: 0! Начало ("a"
</code>
 abcab: Начало ("ab")
/li>
 abcabc: Начало ("abc
e>
 abcabcd: «d» всё исп
последняя буква). Длина 3: abc проти
digo-300">pi[6] = 0</code>

</div>
</details>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код C++ (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
</div>
</details>

<h3 class="text-lg font-bold text-white mt-
<div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-xl font-mono text-white">

```



```

 </p>
 </div>
</div>

```

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

Пример вычисления (Скрыто)

```

 </summary>
 <div class="p-5 text-sm text-slate-300
 <p>Тот же пример: abcabcd</p>
 <ul class="list-disc pl-5 space-y-2
 <code>Z[0]</code> – не счита
 <code>Z[1]</code> (bcabcd
[1] = 0</code>
 <code>Z[2]</code> (cabcd

 <code>Z[3]</code> (abcd<
а дальше s[6]='d' против s[3]='a' – мимо.
 <code>Z[4], Z[5], Z[6]</code>

 </div>
</details>

```

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

Код C++ (Скрыто)

```

 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
</details>

```

```

 <div class="bg-slate-900 p-6 rounded-lg border-2"
 <div class="absolute -top-3 left-4 bg-rose-900
 -500 shadow-md">
 Аналогия 2: Матёшка (Терминальные)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 нашли и "he", потому что оно спрятано внутри
 </div>
 </div>

 <details class="bg-slate-900/40 rounded-lg border b
 <summary class="p-4 font-bold text-slate-400 ou
 r-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-
 <pre class="bg-slate-950 p-4 rounded overfl

int get_link(int v) {
 if (t[v].link == -1) {
 if (v == 0 || t[v].p == 0) t[v].link = 0;
 else t[v].link = go(get_link(t[v].p), t[v].pch)
 }
 return t[v].link;
}

int go(int v, char c) {
 if (t[v].go[c] == -1) {
 if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
 else t[v].go[c] = v == 0 ? 0 : go(get_link(v),
 }
 return t[v].go[c];
}</pre>
 </div>
 </details>
</div>
</section>`
},

```

```

 </div>
 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #4b4b9b; border-radius: 10px; box-shadow: 0 10px 30px #4b4b9b; position: relative; width: 100%; height: 100%;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-purple-500 shadow-md" style="border: 1px solid #8080ff; border-radius: 5px; padding: 5px; position: absolute; top: -3px; left: 4px; right: 4px; bottom: 4px;">
 Аналогия 2: Переводчик (сведения о сложности)
 </div>
 <p class="text-slate-300 text-sm">Сложность перевода (сведение за полином) и друг-англичанин. Отсюда главный инструмент: если к В сводится А, то А не стоит.</p>
 </div>
 </div>

```

```

<h3 class="text-lg font-bold text-white mt-3">Иерархия сложности
<p class="text-slate-300 text-sm mb-3">Известно, что сложность перевода в цепочке доказывается теоремой об иерархии сложности.
<div class="bg-slate-950 border border-slate-800 p-6 rounded-lg" style="background-color: #000080; color: white; border: 1px solid #000080; border-radius: 10px; padding: 10px;">
 <p class="font-mono text-sm sm:text-base">
 $P \subseteq NP \subseteq PSPACE \subseteq EXP \subseteq NEXP$

 NP-полнота (доказано)
 </p>
</div>

```

```

<div class="overflow-x-auto">
 <table class="w-full text-xs text-left" style="width: 100%; border-collapse: collapse;">
 <thead>
 <tr class="text-slate-500 border-b" style="border-bottom: 1px solid #4b4b9b;">
 <th class="py-2 pr-3 font-bold">Сложность
 <th class="py-2 pr-3 font-bold">Время
 <th class="py-2 font-bold">Примечания
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-800" style="border-bottom: 1px solid #4b4b9b;">
 <td class="py-2 pr-3">детерминированно за $O(n^k)$
 <td class="py-2 pr-3">удовольствия, максимальный поток
 </tr>
 <tr class="border-b border-slate-800" style="border-bottom: 1px solid #4b4b9b;">
 <td class="py-2 pr-3">сертификат проверяется за полиномиальное время
 </tr>
 </tbody>
 </table>

```

```

 ke.
 Параметризованная сложность
 n).
 Специальные классы входов
полиномиальными (независимое множество на д
 Готовые решатели: SAT/ILP
всего алгоритма.

</div>

```

```

<div class="bg-slate-900/40 rounded-lg border
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
NP: любая задача из P автоматически в NP.
 <p class="text-slate-400 text-sm mb-1">
становки – неразрешима, зато NP-трудна). NP
 <p class="text-slate-400 text-sm mb-1">
стной трудной A к B ($A \leq_p B$), а не н
 <p class="text-slate-400 text-sm mb-1">
 <p class="text-slate-400 text-sm mb-1">
дит по времени на $n = 40$. NP-полнота запрещ
 <p class="text-slate-400 text-sm">•
ит RSA. «Раз она в NP, значит NP-полная» –
</div>

```

```

<p class="text-slate-300 text-sm mt-4">с
ass="bg-slate-950 px-2 py-0.5 rounded text-
еребор сертификатов <code class="bg-slate-9
^poly(n))</code>; сведение обязано работать
0 border border-slate-800">0(poly(n))</code>

```

```

<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

Код: верификатор сертификата и пр

```
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```

```
<pre class="bg-slate-950 p-4 rounded
```



```

 if col[to] == -1:
 col[to] = col[v] ^ 1
 dq.append(to)
 elif col[to] == col[v]:
 return False
 return True</pre>

```

```

 <p class="text-slate-400 text-xs">0
товый ответ, но не находить его; для задачи
 </div>
</details>

```

```

<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">

```

```

 Вопросы, которые любят задавать (
</summary>

```

```

 <div class="p-5 text-sm text-slate-300
 <p>• Дайте определение NP через вер
нтны.</p>
 <p>• Что такое NP-полная задача и к
 </p>

```

```

 <p>• Почему первая NP-полная задача
 <p>• Если $P = NP$, что станет с крип
их задач исчезнет граница approximability)<
 <p>• Пример задачи в NP и co-NP, пр
– затем AKS доказал P ></p>
 <p>• Чем NP-трудная задача отличаетс
облема остановки)</p>
 </div>
</details>

```

```

 </div>
</div>
</section>`
 }
];

```

---

```

 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия: Списки = Контакты
 </div>
 <p class="text-slate-300 text-sm mb
т. Оптимально для почти всех задач.</p>
 </div>
</div>

{/* DFS vs BFS */}
<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-indigo-400">

 <div class="grid grid-cols-1 xl:grid-cols-2
 <!-- Аналогия DFS -->
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-indigo-500 shadow-md">
 ♂ DFS (Поиск в глубину) = Ж
 </div>
 <p class="text-slate-300 text-sm mb
 Что делает: Жадный алгор
ую вершину (например, минимальную по номеру
(рекурсивно) и пробует другой путь.
 </p>
 <ul class="text-xs text-indigo-300
 Где используется:
 Топологическая сортировка (
 Поиск мостов и точек сочлен
 Сильно связанные компоненты К
 Поиск циклов и просто провер

 </div>

 <!-- Аналогия BFS -->
 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
```

```
vis[start_node] = true;
```

```
while(!q.empty()) {
 int v = q.front(); q.pop();
 for (int u : adj[v]) {
 if (!vis[u]) {
 vis[u] = true;
 q.push(u);
 }
 }
}
```

```
</div>
</details>
</div>
```

```
</div>
</section>`,
```

```
{
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
```

```
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 7
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-800">Графы. Планарные. Покраска</h2>
 </div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-800">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.</p>
 <p class="text-xl font-mono text-white">...</p>
 </div>
 </div>
 <div class="grid grid-cols-1 gap-6">
```

ной запускаем DFS или BFS и присваиваем все  $comp[v] = c$ . Счётчик  $c$  увеличивается р...  
 Время  $O(V + E)$ , память  $O(V)$ . **Через**  
 каждое ребро делаем  $union(u, v)$ , в конце чи...  
 ту и после каждого добавления спрашиваю

```
<div class="grid grid-cols-1 xl:grid-cols-2" style="background-color: #f0f0f0; padding: 10px; border: 1px solid #ccc; border-radius: 10px; margin-bottom: 10px;">
 <div class="bg-slate-800 p-6 rounded-lg" style="background-color: #2d3748; color: white; padding: 10px; border: 1px solid #3d4d6d; border-radius: 10px; margin-bottom: 10px;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 2px solid #2ecc71; border-radius: 10px; padding: 5px;">
 Аналогия 1: Острова

```

```
<h3 class="text-lg font-bold text-white mt-10">Задача 3: Компоненты связности
<div class="overflow-x-auto" style="background-color: #f0f0f0; padding: 10px; border: 1px solid #ccc; border-radius: 10px; margin-bottom: 10px;">
 <table class="w-full text-xs text-left" style="width: 100%; border-collapse: collapse;">
 <thead>
 <tr class="text-slate-500 border-bottom" style="background-color: #e2e3e5; border: 1px solid #d1d3d4;">
 <th class="py-2 pr-3 font-bol...>ID
 <th class="py-2 pr-3 font-bol...>Parent
 <th class="py-2 pr-3 font-bol...>Size
 <th class="py-2 pr-3 font-bol...>Color

 <tbody>
 <tr>
 <td>1
 <td>0
 <td>1
 <td>Red

 <tr>
 <td>2
 <td>1
 <td>2
 <td>Blue

 <tr>
 <td>3
 <td>2
 <td>3
 <td>Green

 <tr>
 <td>4
 <td>3
 <td>4
 <td>Yellow

 <tr>
 <td>5
 <td>4
 <td>5
 <td>Purple

 <tr>
 <td>6
 <td>5
 <td>6
 <td>Orange

 <tr>
 <td>7
 <td>6
 <td>7
 <td>Pink

 <tr>
 <td>8
 <td>7
 <td>8
 <td>Brown

 <tr>
 <td>9
 <td>8
 <td>9
 <td>Grey


```

```

 py-0.5 rounded text-xs font-mono text-emerald-500
 union by rank) – <code class="bg-slate-950 px-2 py-0.5 rounded text-emerald-500 font-mono">
 (n))</code> на операцию, где α – обратная функция Эйлера.
 950 px-2 py-0.5 rounded text-xs font-mono text-emerald-500
 <code class="bg-slate-950 px-2 py-0.5 rounded text-emerald-500 font-mono">
 <details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-300">
 Код: обходы и DSU (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded text-emerald-500 font-mono">
 # компоненты обходом в глубину: comp[v] = номер компонента
 comp = [-1] * n
 c = 0
 for start in range(n):
 if comp[start] != -1:
 continue
 c += 1
 stack = [start]
 comp[start] = c
 while stack:
 v = stack.pop()
 for to in graph[v]:
 if comp[to] == -1:
 comp[to] = c
 stack.append(to)
 print("компонент:", c)

 # 2) То же BFS (удобно, когда нужен ещё и кратчайший «дальнейший» путь)
 from collections import deque
 def bfs_component(start, color):
 used[start] = True
 comp[start] = color
 dq = deque([start])
 while dq:
 v = dq.popleft()

```

```

 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Разложение задач по порядку выполнения",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 <p class="text-slate-300 text-sm mb-4">
 еская сортировка – это расписание, которое
 <p>
 </div>
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения: Циклы
 </div>
 <p class="text-slate-300 text-sm mb-4">
 получить опыт, нужна работа". Алгоритм выяв

```



```

 !

 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых расхреначивает",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
и fup[v] > tin[u].</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
rder border-emerald-500 shadow-md">
 Аналогия 1: Единственный мост
 </div>
 <p class="text-slate-300 text-sm mb-4">
 этот мост, экономикой обоих кусков придет
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`
 }
]

```



```
 </p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-rose-500 shadow-md">
 Аналогия: Главный перекресток
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь город, где два района соединяются, но её перекроют (удалят вершину) – районы будут хрупкостью системы.
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border border-emerald-500 shadow-md">
 Телеком: Центральный свитч
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У тебя несколько компьютеров в одной абелем (мостом). Сами свитчи – это точки со
 </p>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-700/50">
 <summary class="p-4 font-bold text-slate-400">
 Душный мод: Код и корень DFS (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300 space-y-2">
 <p class="text-blue-400 font-bold">Особ
 </p>
 Для стартовой вершины обхода правил (то выше неё прыгать некуда). Поэтому для не
```

```

 Как точка сочленения рушит
 Косарайю, свернув каждую SCC в одну супер-вершину
 (сделаем неориентированным), то любая вершина
 о и есть критическая SCC! Если удалить эту вершину,
 . Одно слабое звено изолирует целые касты SCC!

</div>
</div>
</section>`,
},
{
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-indigo-400">13. Графы. Эйлеров цикл
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-indigo-400">Эйлеров цикл
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">Эйлеров цикл — это замкнутый путь,
 <p class="text-xl font-mono text-white">который проходит по каждому ребру ровно один раз.
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4
 order border-emerald-500 shadow-md">
 Аналогия 1: Рисование не отрывая пера
 </div>
 <p class="text-slate-300 text-sm mb-4">Если граф имеет
 е сходится нечетное число линий, значит, из этой вершины
 а везде должно быть четное число (зашел-вышел)
 </div>
 </div>
 </div>
 </div>

```

```

 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
 { id: 6, x: 700, y: 250, label: "6" },
];

const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points (3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
];

// Adjacency list
const adj: number[][] = Array.from(
 { length: Object.keys(nodes).length },
 () => [],
);

edges.forEach((e) => {
 adj[e.u].push(e.v);
 adj[e.v].push(e.u);
});

interface StepInfo {
 desc: string;
 visited: Set<number>;
 ap: Set<number>;
 currentNode: number | null;
 tin: number[];
 low: number[];
}

export const ArticulationPointsViz: React.FC = () => {
 const [steps, setSteps] = useState<StepInfo[]>([]);

```

```

 if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], currentLow[to]);
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[${v}],
 v,
);
 } else {
 children++;
 dfs(to, v);
 currentLow[v] = Math.min(currentLow[v], currentLow[to]);

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}],
 v,
);

 if (currentLow[to] >= currentTin[v] && p !== ap.add(v);
 recordStep(
 `Найдена точка сочленения: вершина ${v}],
 v,
);
 }
 }
}

if (p === -1 && children > 1) {
 ap.add(v);
 recordStep(
 `Корень дерева DFS ${v} имеет >1 детей. Это
 v,
);
} else if (p === -1) {
 recordStep(
 `Корень дерева DFS ${v} имеет <=1 ребенка, он
 v,
);
}

```

```

 });

 return (
 <div className="bg-slate-900 rounded-xl border border-bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения
 </h3>

 <div className="flex items-center gap-2 bg-slate
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 aria-label="Сбросить обход"
 title="Сбросить обход"
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </div>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-3 g
 <div className="lg:col-span-2 relative h-96 lg:l

```

```

 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited
 ? "#34d399"
 : "#334155";
const r = isAp ? 24 : 20;

return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-all duration-100"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="14px"
 fontWeight="bold"
 className="select-none pointer-events-none"
 >
 {node.label}
 </text>

 {isVisited && (
 <>
 <text
 x={node.x}
 y={node.y - r - 15}
 textAnchor="middle"

```

```

<div className="space-y-4 flex-1 overflow-y-auto">
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 font-medium">
 Точка сочленения
 </h5>
 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-rose-500 border-1px border-slate-500">
 Точка сочленения
 </div>
 </div>
 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-rose-500 border-1px border-slate-500">
 Посещена
 </div>
 </div>
 <div className="flex items-center gap-2">
 <div className="w-3 h-3 rounded-full bg-rose-500 border-1px border-slate-500">
 Текущая
 </div>
 </div>
 <div className="flex items-center gap-2">
 <div className="w-8 h-1 bg-rose-500 border-1px border-slate-500">
 Мост
 </div>
 </div>
 </div>

 <div>
 <h5 className="text-xs text-slate-500 font-medium">
 ЛОГ ДЕЙСТВИЙ:
 </h5>
 <div className="space-y-2">
 {steps
 .slice(0, currentStepIndex + 1)
 .reverse()
 .map((step, idx) => (
 <div
 key={idx}
 className={`text-xs p-2 rounded $
 xt-slate-500`} `}
 >
 {step.desc}
 </div>
))}
 </div>
 </div>
</div>

```

```

 hasNegativeCycle?: boolean;
 activeCodeLine: number;
}

/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
 и листать вручную.";

export default function BellmanFordViz() {
 const [hasCycle, setHasCycle] = useState(false);
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState(false);

 const nodes: Node[] = [
 { id: 'S', x: 60, y: 150, name: 'База (S)' },
 { id: 'A', x: 160, y: 60, name: 'Застава (A)' },
 { id: 'B', x: 160, y: 240, name: 'Топь 1 (B)' },
 { id: 'C', x: 260, y: 150, name: 'Мост (C)' },
 { id: 'D', x: 350, y: 60, name: 'Топь 2 (D)' },
 { id: 'E', x: 350, y: 240, name: 'Лес (E)' },
 { id: 'F', x: 450, y: 150, name: 'Склад (F)' },
];

 const getEdges = (cycle: boolean): Edge[] => {
 const defaultEdges = [
 { u: 'S', v: 'A', w: 4 },
 { u: 'S', v: 'B', w: 5 },
 { u: 'B', v: 'C', w: -2 },
 { u: 'A', v: 'C', w: 1 },
 { u: 'C', v: 'D', w: 3 },
 { u: 'C', v: 'E', w: 4 },
 { u: 'D', v: 'F', w: 2 },
 { u: 'E', v: 'F', w: 1 },
];
 if (cycle) {
 return [...defaultEdges, { u: 'D', v: 'A', w: -6

```



```

 });

 const vCount = nodes.length;
 for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }

 simulationSteps.push({
 iteration: iter, checkingEdge: { u: edge.u, v: edge.v },
 distances: { ...dist }, previous: { ...prev },
 log: `Итерация ${iter}: Проверяем ${edge.u} → ${edge.v}
 updated ? ` Релаксация успешна! dist[${edge.v}] = ${uDist + edge.w}`,
 activeCodeLine: updated ? 6 : 5,
 });
 }

 if (!anyUpdate && iter < vCount - 1) {
 simulationSteps.push({
 iteration: iter, checkingEdge: null, distances: { ...dist },
 log: `Ранний выход после итерации ${iter}.`,
 activeCodeLine: 10,
 });
 break;
 }
 }

 if (simulationSteps[simulationSteps.length - 1].iteration === vCount) {

```

```

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveW = vizNumber(vars?.w);
const liveDist = vizRecord(vars?.dist);
const compilerLinked = liveI !== null || liveU !== null || liveV !== null || liveW !== null || liveDist !== null;
const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 iteration: liveI ?? 0,
 checkingEdge: liveU && liveV ? { u: liveU, v: liveV },
 distances: Object.fromEntries(nodes.map((node) => [node, liveDist[node]])),
 log: `Компилятор: i=${liveI ?? "-"}, ребро ${liveU} - ${liveV}, вес ${liveW}, расстояние ${liveDist}`
 }
 : baseStep;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
 <div className="flex flex-wrap items-center justify-center gap-3">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-blue-600 text-white rounded-lg">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-blue-300">Полный
 </p>
 </div>
 </div>
 </div>

 <div className="flex items-center gap-3">
 <div className="bg-slate-900 p-1 rounded-lg border border-slate-800">
 <button onClick={() => setHasCycle(false)}>

 </button>
 <button onClick={() => setHasCycle(true)}>

 </button>
 </div>
 </div>
 </div>

```

```

<svg className="w-full h-auto max-h-[500px]"
 <defs>
 <marker id="arrow-bf-normal" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#64748b" />
 <marker id="arrow-bf-active" markerWidth=
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 <marker id="arrow-bf-cycle" markerWidth=
 path d="M0,0 L6,3 L0,6 z" fill="#e11d48" />
 </defs>

 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === u);
 const vNode = nodes.find((n) => n.id === v);
 const isChecking = currentStep.checkingEdges;
 const isNeg = edge.w < 0;
 const isCyclePart = hasCycle && ((edge.u === 'D' || edge.v === 'D'));

 let strokeColor = '#475569';
 let marker = 'url(#arrow-bf-normal)';
 if (isChecking) { strokeColor = '#f59e0b'; }
 else if (currentStep.hasNegativeCycle && isCyclePart) { strokeColor = '#e11d48'; }

 return (
 <g key={idx}>
 <line x1={uNode.x} y1={uNode.y} x2={vNode.x} y2={vNode.y} stroke={strokeColor} strokeWidth={isChecking && isCyclePart ? 4 : 2} marker={marker} />
 <circle cx={({uNode.x + vNode.x} / 2) cy={({uNode.y + vNode.y} / 2)} fill={isChecking ? '#f59e0b' : isNeg ? '#f43f5e' : '#e11d48'} stroke="#f43f5e" stroke-width="2" />
 <text x={({uNode.x + vNode.x} / 2) y={({uNode.y + vNode.y} / 2)} text={isChecking ? 'Checking' : isNeg ? 'Neg' : 'Cycle'} font-size="11" font-weight="bold" />
 </g>
);
 })}

 {nodes.map((node) => {

```

```


)
 }}}}
 </div>
)}}
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10">
 <div>
 <h4 className="text-lg font-bold text-rose-950">
 Таблица расстояний
 </h4>
 <div className="grid grid-cols-4 gap-2 text-rose-950">
 {nodes.map(n => {
 const d = currentStep.distances[n.id];
 const isTgt = currentStep.checkingEdge === n.id;
 return (
 <div key={n.id} className={`p-2 rounded-lg
 ${isTgt ? 'border-slate-700/60 bg-slate-800/40' : ''}`>
 <div className="text-xs text-slate-400">
 {n.id}
 </div>
 <div className={`font-mono font-bold text-rose-950`}
 >
 {d}
 </div>
 </div>
);
 })}
 </div>
 </div>
 </div>
</div>

 {/* Код алгоритма */}
 <div className="w-full lg:w-1/2 bg-slate-900/60">
 <div>
 <h4 className="text-lg font-bold text-slate-400">
 Код алгоритма
 </h4>
 <div className="bg-slate-950/80 rounded-lg p-4">
 {codeLines.map(item => (
 <div key={item.line} className={`py-1.5`}>
 {item.code}
 </div>
))}
 </div>
 </div>
 </div>

```

```
const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [],
};
```

```
interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
 currentNode: number | null;
 logs: string;
}
```

```
const generateBfsSteps = (): Step[] => {
 const steps: Step[] = [];
 const queue: number[] = [];
 const visited = new Set<number>();
```

```
 // Init
 steps.push({
 activeLine: 1,
 queue: [],
 visited: [],
 currentNode: null,
 logs: "Инициализация: начинаем с корня (0)."
 });
```

```
 queue.push(0);
 visited.add(0);
 steps.push({
 activeLine: 2,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Добавляем стартовую вершину 0 в очередь и по"
 });
```

```
 while (queue.length > 0) {
 steps.push({
```

```

 }
 }
 } else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Соседей нет или все уже посещены.`
 });
 }
 }
}

steps.push({
 activeLine: 12,
 queue: [],
 visited: Array.from(visited),
 currentNode: null,
 logs: "Очередь пуста. Алгоритм завершен!"
});

return steps;
};

const STEPS = generateBfsSteps();

export default function BfsViz() {
 const [stepIdx, setStepIdx] = useState(0);
 const [isPlaying, setIsPlaying] = useState(false);

 const step = STEPS[stepIdx];

 const handleNext = useCallback(() => {
 setStepIdx(prev => Math.min(prev + 1, STEPS.length - 1));
 }, []);

 // @ts-expect-error зарезервировано под кнопку «назад»
 const handlePrev = useCallback(() => {

```

```

 onClick={handleNext}
 disabled={stepIdx >= STEPS.length - 1}
 className="p-2 text-emerald-400 hover:text-
ed: hover: bg-transparent"
 title="Шаг вперед"
 >
 <StepForward className="w-5 h-5" />
 </button>
 </div>
 </div>

 <div className="grid grid-cols-1 lg:grid-cols-2 g
 { /* Визуал графа */ }
 <div className="bg-slate-800 border border-slate
 50px]">
 <svg className="w-full h-full min-h-[350px]"
 { /* Рёбра */ }
 { EDGES.map((e, idx) => {
 const n1 = NODES.find(n => n.id === e.from)
 const n2 = NODES.find(n => n.id === e.to)
 const isVisited = step.visited.includes(n
 return (
 <line
 key={idx}
 x1={n1.x} y1={n1.y} x2={n2.x} y2={n2.
 className={`transition-all duration-5
 `)}
 >
 />
 </>
)
)
 }
)
 }
 { /* Вершины */ }
 { NODES.map(n => {
 const isCurrent = step.currentNode === n.
 const isInQueue = step.queue.includes(n.id)
 const isVisited = step.visited.includes(n

 let fill = '#1e293b'; // slate-800
 })
}

```

```

 </g>
);
 }}}
 </svg>
</div>

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
 <div className="bg-slate-900 border border-slate-800 p-4">
 <div className="bg-slate-800 text-slate-400 p-4 font-mono tracking-wider">
 bfs(graph, start)
 ⏏ {step}
 </div>
 <div className="p-4">
 {[
 { line: 1, code: 'queue = Queue()' },
 { line: 2, code: 'queue.push(start); visit' },
 { line: 3, code: '' },
 { line: 4, code: 'while not queue.isEmpty()' },
 { line: 5, code: ' node = queue.pop()' },
 { line: 6, code: ' for neighbor in graph[node]' },
 { line: 7, code: ' if neighbor not in visited:' },
 { line: 8, code: ' visited.add(neighbor)' },
 { line: 9, code: ' queue.push(neighbor)' },
]}.map((cl) => {
 const isActive = step.activeLine === cl.line;
 return (
 <div key={cl.line} className={`px-2 py-2 border-l-2 border-blue-400 -ml-0.5` : 'text-blue-400'}>

 {cl.code}
 </div>
);
 });
 </div>
 </div>
</div>

```



```
const imageMap: Record<string, { src: string; alt: string;
 dijkstra: {
 src: dijkstraImg,
 alt: 'Добрый Дейкстра',
 caption: ' Добрый — только позитив!',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',
 },
 'bellman-ford': {
 src: bellmanImg,
 alt: 'Фура по болоту',
 caption: ' Тяжёлая, но ей пофиг на ямы!',
 borderColor: 'border-rose-500/30',
 captionColor: 'text-rose-400',
 },
 floyd: {
 src: floydImg,
 alt: 'Все ко Всем Флойд',
 caption: ' Все связаны со всеми!',
 borderColor: 'border-emerald-500/30',
 captionColor: 'text-emerald-400',
 },
};
```

```
export default function ChapterImage({ vizType }: { vizType: string }) {
 const info = imageMap[vizType];
 if (!info) return null;

 return (
 <div>
 <div className={`rounded-2xl overflow-hidden border-2 border-${info.borderColor}>

 </div>
 <p className={`text-center text-xs mt-2 font-bold text-${info.captionColor}>
 {info.caption}
 </p>
 </div>
);
}
```

```


 {index + 1} / {total}

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-2 rounded-md
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 Вперёд
 <ChevronRight className="w-4 h-4" />
 </button>
 </div>
);
}

return (
 <nav className="grid grid-cols-1 sm:grid-cols-2 gap-4">
 {prev ? (
 <button
 onClick={() => onGo(prev.id)}
 className="group text-left p-4 rounded-xl bg-white border
 transition-all"
 >
 <div className="flex items-center gap-1.5 text-slate-600">
 <ChevronLeft className="w-3.5 h-3.5" /> Пре
 </div>
 <div className="text-sm font-bold text-slate-600">
 {prev.title}
 </div>
 </button>
) : (
 <div className="hidden sm:block" />
)}

 {next && (
 <button
 onClick={() => onGo(next.id)}

```

```

*/
export const ChapterQuiz: React.FC<Props> = ({ chapterId }) => {
 const list = quizzes[chapterId];
 const [idx, setIdx] = useState(0);
 const [picked, setPicked] = useState<number | null>(null);
 const [score, setScore] = useState(0);
 const [finished, setFinished] = useState(false);

 // другая тема – другой блиц с нуля
 useEffect(() => {
 setIdx(0);
 setPicked(null);
 setScore(0);
 setFinished(false);
 }, [chapterId]);

 if (!list || list.length === 0) return null;

 const q = list[idx];
 const answered = picked !== null;
 const correct = answered && picked === q.correctIndex;

 const pick = (i: number) => {
 if (answered) return;
 setPicked(i);
 if (i === q.correctIndex) setScore((s) => s + 1);
 };

 const next = () => {
 if (idx + 1 < list.length) {
 setIdx(idx + 1);
 setPicked(null);
 } else {
 setFinished(true);
 }
 };

 const restart = () => {

```

```

 </button>
 </div>
) : (
 <>
 <p className="text-sm font-semibold text-wh
 <div className="space-y-2">
 {q.options.map((option, i) => {
 const isRight = i === q.correctIndex;
 const chosen = picked === i;
 const tone = !answered
 ? "border-slate-600 bg-slate-800/70 t
 : isRight
 ? "border-emerald-500 bg-emerald-500
 : chosen
 ? "border-rose-500 bg-rose-500/15
 : "border-slate-700 bg-slate-800/
 return (
 <button
 key={i}
 type="button"
 onClick={() => pick(i)}
 disabled={answered}
 className={`w-full text-left flex i
 <div disabled:cursor-default ${tone}`}
 >
 {answered && isRight && <CheckCircle
 {answered && chosen && !isRight &&
 {option}
 </button>
 });
 })}
 </div>

 {answered && (
 <div role="status" className="mt-3 rounded
 <p className={`text-xs font-bold mb-1 $
 {correct ? "Верно" : "Неверно"}
 </p>

```

```

import { VizChapterContext } from "../data/vizStepBus";
import { VizLiveStatus } from "../VizLiveStatus";
import { ChapterNav } from "../ChapterNav";
import { ChapterQuiz } from "../ChapterQuiz";
import { downloadChapterHtml } from "../utils/exportHtml";
import { useTicketLinks } from "../utils/ticketLinks";
```

```
interface Props {
 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
 /** Перейти во вкладку компилятора – там уже подставлен код */
 onOpenCompiler?: () => void;
}
```

```
const isTouch = () => typeof window !== "undefined" && window.ontouchstart !== undefined;
```

```
export const ChapterView: React.FC<Props> = ({ chapter,
 const contentRef = useRef<HTMLDivElement>(null);
 const [anchor, setAnchor] = useState<HintAnchor | null>(null);
 const hideTimer = useRef<number | null>(null);
 const [saving, setSaving] = useState<"idle" | "work">("idle");
```

```
 useTermHints(contentRef, [chapter.id]);
```

```
 /**
```

```
 * Упоминания «билет 15» становятся переходами на стр. 15
```

```
 * Разметка идёт ПОСЛЕ терминов: подсказки глоссария и т.д.
```

```
 */
```

```
 useTicketLinks(contentRef, [chapter.id], { selfId: chapter.id });
```

```
 const cancelHide = useCallback(() => {
 if (hideTimer.current) {
 window.clearTimeout(hideTimer.current);
 hideTimer.current = null;
 }
 }, [hideTimer]);
```

```

}, [chapter.id]));

const saveHtml = useCallback(async () => {
 setSaving("work");
 try {
 await downloadChapterHtml(chapter);
 setSaving("done");
 window.setTimeout(() => setSaving("idle"), 2500);
 } catch {
 setSaving("idle");
 }
}, [chapter]);

const open = useCallback(
 (el: HTMLElement) => {
 const termId = el.dataset.termId;
 if (!termId) return;
 cancelHide();
 setAnchor({ termId, rect: el.getBoundingClientRect() });
 },
 [cancelHide]
);

const handlePointerOver = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
 if (el) open(el);
};

const handlePointerOut = (e: React.MouseEvent) => {
 if (isTouch()) return;
 const el = (e.target as HTMLElement).closest<HTMLElement>(".term");
 if (el) scheduleHide();
};

const handleClick = (e: React.MouseEvent) => {
 // ссылка на другой билет: переходим внутри приложения
 const link = (e.target as HTMLElement).closest<HTMLElement>(".term-link");
 if (link) {
 const href = link.getAttribute("href");
 if (href) {
 window.location.href = href;
 }
 }
};

```

```

 {saving === "done" ? "Файл сохранён" : "Скачать"}
 </button>
 <button
 onClick={() => window.print()}
 title="Распечатать или сохранить эту тему в PDF-формате. Вывод в формате PDF, без шапки, содержания и панели комментариев"
 className="flex items-center gap-1.5 text-slate-500 border border-slate-500 hover:text-white hover:border-indigo-500"
 >
 <Printer className="w-3.5 h-3.5" /> PDF этой темы
 </button>
 <a
 href={`?lite=1&topic=${encodeURIComponent(currentTopic)}`}
 target="_blank"
 rel="noreferrer"
 title="Текстовая версия темы без навигации, комментариев и панели комментариев"
 className="flex items-center gap-1.5 text-slate-500 border border-slate-500 hover:text-white hover:border-sky-500"
 >
 <FileText className="w-3.5 h-3.5" /> Текстовая версия

 </div>

 <div
 ref={contentRef}
 className="prose prose-invert max-w-none"
 onMouseOver={handlePointerOver}
 onMouseOut={handlePointerOut}
 onClick={handleClick}
 onFocus={handleFocus}
 dangerouslySetInnerHTML={{ __html: chapter.content }}
 />

 <p className="mt-6 flex items-start gap-2 text-slate-500 text-sm font-medium">
 <MousePointerClick className="w-4 h-4 shrink-0" />

```

```
 </VizChapterContext.Provider>
 </section>
 })

 {/* прочитал → посмотрел, как работает → провер
 <ChapterQuiz chapterId={chapter.id} />

 <ChapterNav prev={prev} next={next} index={inde
 </article>

 <TermPopover
 anchor={anchor}
 onClose={() => setAnchor(null)}
 onOpenSimulator={(id) => {
 setAnchor(null);
 onOpenSimulator(id);
 }}
 onCardEnter={cancelHide}
 onCardLeave={scheduleHide}
 />
</>
);
};
```

---

ФАЙЛ 38/111: src/components/ComplexityViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

---

```
import React, { useContext, useEffect, useMemo, useState
import { ChevronLeft, ChevronRight, Play, Pause, Rotate
import { VizChapterContext, emitVizDemo, useVizRuntime,
```

```
/**
```

```
 * Билет 24 – «Классы сложности, сведение задач».
```

```
 *
```

```
 * Четыре вкладки:
```



-----  
/\* ----- Сведение -----

```
const REDUCE_STEPS = [
 { title: "Дано: задача A, для которой быстрого алгоритма
 – то есть B NP-трудная.", boxes: ["A"], active: 0 },
 { title: "Берём вход x задачи A", note: "x – конкретный элемент
 "x"], active: 1 },
 { title: "Строим $f(x) = y$ за полиномиальное время", note: "Полиномиальное
 шинами из разных дизъюнктов, если литералы не противоречат.
 ve: 2 },
 { title: "Решаем B на входе y", note: "Гипотетический алгоритм B
 – но допустим, что он есть.", boxes: ["A", "x", "f"], active: 3 },
 { title: "Переводим ответ обратно: $A(x) = B(f(x))$ ", note: "Получили ответ
 $\Rightarrow$ формула выполнима. Полиномиальное время сохранилось.
 5 },
 { title: "Вывод: B не проще A", note: "Раз $A \leq_p B$ и A NP-полна.
 -полна.", boxes: ["A", "x", "f", "y", "B", "ans"], active: 6 },
];
```

```
const BOX_LABEL: Record<string, { t: string; c: string }> = {
 A: { t: "задача A\n(NP-полная)", c: "#f43f5e" },
 x: { t: "вход x\n|x| = n", c: "#64748b" },
 f: { t: "f: сведение\nO(poly(n))", c: "#f59e0b" },
 y: { t: "вход y\n|y| = poly(n)", c: "#64748b" },
 B: { t: "решатель B\n(гипотеза: быстрый)", c: "#6366f1" },
 ans: { t: "ответ\nA(x) = B(f(x))", c: "#10b981" },
};
```

-----  
/\* ----- Верификатор -----

```
/** 3-SAT формула: 4 дизъюнкта по 3 литерала, переменные 1..4
const CLAUSES: [number, number, number][] = [
 [1, 2, -3],
 [-1, 3, 4],
 [2, -3, -4],
 [-2, 1, 3],
];
/** Сертификат, который проверяем: x = (1, 0, 1, 0) – он удовлетворяет
```

```

 setStep(0);
 setPlaying(false);
 }, [tab]));

useEffect(() => {
 if (!playing) return;
 const t = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
 }, 1400);
 return () => window.clearInterval(t);
}, [playing, maxStep]);

/* живые переменные компилятора */
const liveStep = vars ? (vizNumber(vars.step) ?? vizNum
useEffect(() => {
 if (liveStep === null || !Number.isFinite(liveStep))
 setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);

const liveCls = vars ? vizString(vars.cls) : null;
const liveK = vars ? vizNumber(vars.k) : null;
const liveCert = vars ? vizArray(vars.cert) ?? vizArr
const liveOk = vars ? (vars.ok === undefined ? null :
const liveN = vars ? vizNumber(vars.n) : null;
const liveOps = vars ? (vizNumber(vars.ops) ?? vizNum

/** Копим точки (n, ops): код пользователя крутит цикл
useEffect(() => {
 if (liveN === null || liveOps === null) return;
 if (!Number.isFinite(liveN) || !Number.isFinite(live
 setSamples((prev) => {
 const last = prev[prev.length - 1];

```

```

const cls =
 slopeTail > slope + 0.8 || slopeTail > 4.5
 ? { name: "экспонента / суперполином", color: "#f59e0b",
 : slope <= 1.25
 ? { name: "≈ линейный, $O(n)$ ", color: "#10b981",
 : slope <= 1.8
 ? { name: "≈ $O(n \log n)$ ", color: "#06b6d4", hint: "log"
 : slope <= 2.4
 ? { name: "≈ квадратичный, $O(n^2)$ ", color: "#f59e0b",
 : slope <= 3.4
 ? { name: "≈ кубический, $O(n^3)$ ", color: "#fb9238",
 : { name: "полином высокой степени", color: "#fb9238"
 }
return { slope, slopeTail, ...cls };
}, [samples]);

```

```

const reduceFrame = REDUCE_STEPS[Math.min(step, REDUCE_STEPS.length - 1)];
const verifyIdx = Math.min(step, VERIFY_STEPS.length);
const verifyDone = verifyIdx >= VERIFY_STEPS.length;
const allOk = VERIFY_STEPS.every((s) => s.ok);

```

```

return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-2xl">
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-purple-600 text-white rounded">
 <Boxes className="w-5 h-5" />
 </div>
 <div>
 <h3 className="text-lg sm:text-xl font-bold">
 <p className="text-xs text-purple-300">P · I
 </h3>
 </div>
 </div>
 </div>

 <div className="flex flex-wrap gap-2 mb-4">
 {TABS.map((t) => (
 <button

```

```

 {on ? 3 : 1.5} strokeDasharray={on ? undefined : 1}
 <text x={c.x} y={c.y - c.ry + 18} textAnchor="middle">
 <text x={c.x} y={c.y - c.ry + 32} textAnchor="middle">
 </g>
);
 }}}
 /* примеры задач – внутри своих овалов */
 {CLASSES.map((c) =>
 c.tasks.map((t, i) => {
 <text key={t} x={c.x} y={c.y + (c.id - 1) * 10}
 textAnchor="middle" fill="#e2e8f0" fontSize="8">
 {t}
 </text>
 })
)}
 /* NP-полные = пересечение */
 <text x={250} y={352} textAnchor="middle">
 <text x={250} y={368} textAnchor="middle">
 рхии времени)</text>
 </svg>
</div>
<p className="text-xs text-slate-400 leading-4">
<p className="text-[11px] text-slate-500">
 Подсказка: выполни в компиляторе <code>class
</p>
</div>
})

```

```

/* ----- СВЕДЕНИЕ -----
{tab === "reduce" && {
 <div className="space-y-3">
 <div className="bg-slate-950 rounded-xl p-4 border">
 <div className="flex flex-wrap items-center">
 {reduceFrame.bboxes.map((b, i) => {
 const meta = BOX_LABEL[b];
 const active = i === reduceFrame.active
 return (
 <React.Fragment key={b + i}>

```



```

 const x = 44 + (Math.log(Math.max(1, s.n)) * 50);
 const y = 200 - (Math.log(Math.max(1, s.n)) * 50);
 return <circle key={`_${s.n}`} cx={Math.min(x, 100)} cy={Math.min(y, 100)}
strokeWidth={2} />;
 }}}
 {GROWTH_LINES.map((g, i) => (
 <g key={g.id + "l"}>
 <rect x={330 + (i % 3) * 58} y={16 + (i % 3) * 58} width={58} height={58} />
 <text x={343 + (i % 3) * 58} y={24 + (i % 3) * 58} text={g.name} />
 </g>
))}
 </svg>
 {samples.length === 0 && (
 <p className="text-xs text-slate-500 text-center">
 Пусто. Запусти в компиляторе код с переменными t-mono text-purple-300 ops</code> – например,
 </p>
)}
</div>
<div className={"rounded-xl border p-3.5 " + (samples.length > 0 ? "text-white" : "text-slate-500")}>
 <p className="text-sm font-bold text-white">Вердикт: {verdict ? `Похуже на ${verdict.name}` : "Неизвестно"}`</p>
 <p className="text-xs text-slate-400 leading-relaxed">
 {verdict ? `Наклон в лог-лог координатах ≈ ${verdict.slopeTail.toFixed(2)} – кривая загибается` : "Как только в коде появятся n и ops, они будут расти с n)."}
 </p>
</div>
</div>
)}

```

```

{/* живые переменные */}
<div className="mt-3 flex flex-wrap items-center">


```

```

const CODE_LINES: CodeLine[] = [
 { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
 { line: " visited[v] = true;", highlight: "normal" },
 { line: " tin[v] = low[v] = timer++;", highlight: "normal" },
 { line: " ", highlight: "normal" },
 { line: " for (int to : adj[v]) {", highlight: "normal" },
 { line: " if (to == p) continue;", highlight: "normal" },
 { line: " ", highlight: "normal" },
 { line: " if (visited[to]) {", highlight: "normal" },
 { line: " low[v] = min(low[v], tin[to]);", highlight: "normal" },
 { line: " } else {", highlight: "normal" },
 { line: " dfs(to, v);", highlight: "normal" },
 { line: " low[v] = min(low[v], low[to]);", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: " if (low[to] > tin[v]) {", highlight: "normal" },
 { line: " // ЭТО МОСТ!", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: "}", highlight: "normal" },
];
```

```
interface DfsStep {
 currentNode: number | null;
 visitedIndices: number[];
 tin: Record<number, number>;
 low: Record<number, number>;
 stack: number[];
 bridges: string[];
 log: string;
 activeEdge: [number, number] | null;
 backEdge: [number, number] | null;
 activeCodeLine: number[];
}
```

```
const GRAPH_NODES = [
 { id: 0, label: "A", x: 100, y: 120 },
 { id: 1, label: "B", x: 260, y: 120 },
```

```

 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1],
 activeEdge: null, backEdge: null,
 log: "У G нет соседей (кроме B). Возврат. low[G]=3 : ",
 activeCodeLine: [13,14,15]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,2],
 activeEdge: [1,2], backEdge: null,
 log: "Из B идём в C. tin[C]=4, low[C]=4.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:1}, stack: [0,1,2],
 activeEdge: null, backEdge: [2,0],
 log: "Видим соседа A (уже посещён)! Обратное ребро ",
 activeCodeLine: [7,8,9]
},
{
 currentNode: 3, visitedIndices: [0,1,6,2,3],
 tin: {0:1,1:2,6:3,2:4,3:5}, low: {0:1,1:2,6:3,2:1,3:3},
 activeEdge: [2,3], backEdge: null,
 log: "Идём C→D. tin[D]=5, low[D]=5.",
 activeCodeLine: [4,5,7,10,11]
},
{
 currentNode: 4, visitedIndices: [0,1,6,2,3,4],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6}, low: {0:1,1:2,6:3,2:1,3:3,4:2},
 activeEdge: [3,4], backEdge: null,
 log: "D→E. tin[E]=6, low[E]=6.",
 activeCodeLine: [4,5,7,10,11]
},
{

```



```

 log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,
 activeCodeLine: [11,13,16,17]
 },
 {
 currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6
 activeEdge: null, backEdge: null,
 log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,
 activeCodeLine: [11,13,16,17,18]
 },
 {
 currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
 tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6
 activeEdge: null, backEdge: null,
 log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм
 activeCodeLine: []
 },
];

```

```

export function DfsBridgesSimulator() {
 const [dfsIdx, setDfsIdx] = useState(0);
 useVizStepSync(dfsIdx, setDfsIdx, DFS_STEPS.length -
 const rawV = vizString(vars.v) ?? (vizNumber(vars.v
 const rawTo = vizString(vars.to) ?? (vizNumber(vars
 const toId = (value: string | null) => {
 if (value === null) return null;
 const byLabel = GRAPH_NODES.find((node) => node.l
 return byLabel ?? (Number.isFinite(Number(value))
 };
 const v = toId(rawV);
 const to = toId(rawTo);
 if (v === null) return null;
 const candidates = DFS_STEPS
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => step.currentNode === v && (
 return candidates.at(-1)?.index ?? null;
 });
 const [dfsAuto, setDfsAuto] = useState(false);

```

```

</button>
<button onClick={() => { setDfsAuto(false); s
 aria-label="Сбросить DFS" title="Сбросить D
 className="p-1.5 hover:bg-slate-700 rounded
 <RefreshCw className="h-3.5 w-3.5" />
</button>
</div>
</div>
<div className="grid grid-cols-1 md:grid-cols-5 g
 { /* Graph visualization */ }
 <div className="md:col-span-3 bg-slate-950 roun
 <div className="absolute inset-0 bg-[radial-g
 >
 <svg className="w-full h-[280px] z-10 relative
 {GRAPH_EDGES.map(e => {
 const u = GRAPH_NODES.find(n => n.id ===
 const v = GRAPH_NODES.find(n => n.id ===
 const bridge = step.bridges.includes(e.key
 const activeTree = step.activeEdge && (
 (step.activeEdge[0]===e.u && step.active
 (step.activeEdge[0]===e.v && step.active
));
 const activeBack = step.backEdge && (
 (step.backEdge[0]===e.u && step.backEdge
 (step.backEdge[0]===e.v && step.backEdge
));
 let sc = "#475569", sw = 2, dash = "none"
 if (bridge) { sc = "#eab308"; sw = 4; }
 else if (activeTree) { sc = "#10b981"; sw
 else if (activeBack) { sc = "#f43f5e"; sw
 else if (step.visitedIndices.includes(e.u
 return <line key={e.key} x1={u.x} y1={u.y
 stroke={sc} strokeWidth={sw} strokeDash
 className="transition-all duration-500"
 }}}
 {step.currentNode !== null && step.currentN
 const n = GRAPH_NODES.find(x => x.id ===

```

```

 { /* Code panel */ }
 <div className="bg-slate-950 rounded-lg border
 <div className="bg-slate-900 px-3 py-1.5 te
 >
 <div className="p-2 overflow-x-auto">
 <pre className="text-[10px] font-mono lea
 {CODE_LINES.map(({cl, i}) => {
 const hl = step.activeCodeLine.includ
 return <div key={i}
 className={`px-1 transition-all dur
 hl ? "bg-indigo-600/20 text-indig
 }`}>
 <span className="text-slate-600 mr-1
 {cl.line || ' '}
 </div>;
 })}
 </pre>
 </div>
 </div>

 { /* Stack */ }
 <div className="bg-slate-900/50 rounded-xl bo
 <div className="text-[10px] font-bold text-
 Стек DFS
 {step.s
 </div>
 <div className="flex flex-col-reverse gap-1
 {step.stack.length === 0
 ? <div className="text-[10px] text-slat
 : step.stack.map(({id, i}) => {
 <div key={id} className={`px-2 py-0
 i === step.stack.length-1
 ? "bg-emerald-600/20 text-emera
 : "bg-slate-900 text-slate-400
 }`}>Вершина {GRAPH_NODES.find(n=>n.
 <span className="text-slate-500 m
 </div>
 })
 })
 </div>
 </div>
 </div>
 </div>

```

```

 log: string;
 activeCodeLine: number;
}

/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
 ы листать вручную.";

export default function DijkstraViz() {
 const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
 { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
 { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
 { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
 { id: 'F', x: 450, y: 150, name: 'ТЦ (F)' },
 { id: 'G', x: 250, y: 150, name: 'Метро (G)' },
];

 const edges: Edge[] = [
 { u: 'A', v: 'B', w: 4 },
 { u: 'A', v: 'C', w: 2 },
 { u: 'C', v: 'B', w: 1 },
 { u: 'C', v: 'G', w: 3 },
 { u: 'C', v: 'E', w: 8 },
 { u: 'B', v: 'G', w: 4 },
 { u: 'B', v: 'D', w: 5 },
 { u: 'G', v: 'D', w: 1 },
 { u: 'G', v: 'E', w: 2 },
 { u: 'D', v: 'F', w: 3 },
 { u: 'E', v: 'F', w: 1 },
];

 const adjList: { [key: string]: { v: string; w: number } } = {};
 nodes.forEach(n => (adjList[n.id] = []));
 edges.forEach(e => {
 adjList[e.u].push({ v: e.v, w: e.w });
 });
}

```

```

 minD = dist[n.id];
 u = n.id;
 }
}

if (!u) break;

visited[u] = true;
simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Выбираем вершину ${u} с мин. непосещённой
 activeCodeLine: 5,
});

const neighbors = edges.filter(e => e.u === u);
for (const edge of neighbors) {
 const v = edge.v;
 if (visited[v]) continue;

 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Смотрим соседа ${v}. Текущее расстояние
 ${dist[u] + edge.w}.`,
 activeCodeLine: 8,
 });

 if (dist[u] + edge.w < dist[v]) {
 dist[v] = dist[u] + edge.w;
 prev[v] = u;
 simulationSteps.push({
 currentNode: u, checkingEdge: { u, v },
 distances: { ...dist }, visited: { ...visited },
 log: ` Улучшение (Релаксация)! Новое кратчайшее
 activeCodeLine: 9,
 });
 }
}

```

```

 currentNode: liveU,
 checkingEdge: liveU && liveV ? { u: liveU, v: liveV } : null,
 distances: Object.fromEntries(nodes.map((node) => [node, 0])),
 previous: Object.fromEntries(nodes.map((node) => [node, null])),
 visited: Object.fromEntries(nodes.map((node) => [node, false])),
 log: `Компилятор: u=${liveU ?? "-"}, v=${liveV ?? "-"}\n`
 }
 : baseStep;

```

```

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border-1px border-slate-700" >
 { /* Шапка симулятора */ }
 <div className="flex flex-wrap items-center justify-center" >
 <div className="flex items-center gap-3" >
 <div className="p-2.5 bg-indigo-600 text-white rounded-lg" >

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white" >
 <p className="text-sm text-indigo-300" >Полный код
 </p>
 </div>
 </div>

 <div className="flex flex-wrap items-center justify-center" >
 <button
 onClick={() => { setIsPlaying(false); setCompilerLinked(false) }}
 disabled={compilerLinked}
 title={compilerLinked ? HINT_TEXT : "Вернуть к началу"}
 className="flex items-center gap-1 bg-slate-700 text-white px-3 py-2 rounded"
 >
 Сброс
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 disabled={compilerLinked}
 title={compilerLinked ? HINT_TEXT : "Продолжить"}
 className={`flex items-center gap-1 px-4 py-2 rounded`}
 >
 Продолжить
 </button>
 </div>
 </div>

```

```

 <path d="M0,0 L6,3 L0,6 z" fill="#475568" />
 </marker>
 <marker id="arrow-dh-active" markerWidth="6" markerHeight="6" />
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e0b" />
 </marker>
 <marker id="arrow-dh-opt" markerWidth="6" markerHeight="6" />
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 </marker>
 </defs>
 {/* Рёбра */}
 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u);
 const vNode = nodes.find((n) => n.id === edge.v);
 const isChecking =
 currentStep.checkingEdge &&
 (currentStep.checkingEdge.u === edge.u &&
 currentStep.checkingEdge.v === edge.v);

 const isOptimal = currentStep.previous[edge.id] === 'optimal';

 let marker = 'url(#arrow-dh-normal)';
 if (isChecking) marker = 'url(#arrow-dh-active)';
 else if (isOptimal) marker = 'url(#arrow-dh-opt)';

 return (
 <g key={idx}>
 <line
 x1={uNode.x} y1={uNode.y}
 x2={vNode.x} y2={vNode.y}
 stroke={isChecking ? '#f59e0b' : isOptimal ? '#10b981' : '#475568'}
 strokeWidth={isChecking ? 5 : isOptimal ? 2 : 1}
 markerEnd={marker}
 className="transition-all duration-300"
 strokeDasharray={isChecking ? '4,4' : ''}
 />
 <circle
 cx={({uNode.x + vNode.x} / 2)} cy={({uNode.y + vNode.y} / 2)}
 r="12" fill="#1e293b"
 stroke={isChecking ? '#f59e0b' : isOptimal ? '#10b981' : '#475568'}
 strokeWidth={isChecking ? 2 : 1}
 />
 </g>
);
 })}
```

```

 </g>
);
 }}}
 </svg>
 <div className="w-full bg-slate-950/80 p-4 rounded-2xl">
 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">
 </div>
 </div>

 { /* Список смежности */ }
 <div className="w-full lg:w-1/3 bg-emerald-950/60 p-4 rounded-2xl">
 <h4 className="text-lg font-bold text-emerald-400">
 <div className="space-y-2.5 flex-1 overflow-y-hidden">
 {Object.keys(adjList).map((u) => {
 const isCurrentNode = currentStep.currentNode === u;
 return (
 <div key={u} className={`p-3 rounded-lg ${
 isCurrentNode ? 'bg-emerald-900/60 text-slate-300' : 'bg-slate-950/60 text-slate-300'
 }`} >
 <div className="flex items-center gap-2">

 {u}

 →
 </div>
 <div className="flex flex-wrap gap-2">
 {adjList[u].map((edge, idx) => {
 const isCheckingThis = currentStep.currentNode === edge.v;
 return (
 <span key={idx} className={`px-2 py-0.5 rounded ${
 isCheckingThis ? 'bg-amber-900/60 text-slate-300' : 'bg-slate-950/60 text-slate-300'
 }`} >
 {edge.v}


```



```

 <span className={`w-2 h-2 rounded
 {n.name}
 </td>
 <td className="py-2.5 px-3 font
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text
 {prev || '-'}
 </td>
 </tr>
);
 }}}
</tbody>
</table>
</div>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60
 <div>
 <h4 className="text-lg font-bold text-slate
 <div className="bg-slate-950/80 rounded-lg
 {codeLines.map((item) => {
 const isActive = currentStep.activeCode
 return (
 <div key={item.line} className={`py-1
 isActive ? 'bg-indigo-900/80 text
 }`}>
 <span className="text-slate-600 sel
 <span className="whitespace-pre fle
 {isActive && <span className="text-
 }
 </div>
);
 }}}
</div>
</div>

```

```

const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState<boolean>(false);
const [speed] = useState<number>(800);
const [activeTab, setActiveTab] = useState<'table' | 'graph'>('table');

useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',
 desc: `Вызов fibМемо(${n}). Проверяем наличие в кэше`,
 codeLine: 2,
 memoState: { ...memoMap }
 });

 if (n in memoMap) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'memo_hit',
 desc: `✓ Кэш-хит! fibМемо(${n}) уже посчитано`,
 codeLine: 2,
 memoState: { ...memoMap }
 });
 return memoMap[n];
 }

 if (n <= 2) {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'base_case',
 desc: `Базовый случай: n <= 2 (n=${n}). Возвращаем n`,
 codeLine: 2,
 memoState: { ...memoMap }
 });
 memoMap[n] = n;
 return n;
 }

 const n1 = simulateFib(n - 1);
 const n2 = simulateFib(n - 2);
 memoMap[n] = n1 + n2;
 return n1 + n2;
 }

 simulateFib(10);
}, [runtime]);

```

```

 codeLine: 6,
 memoState: { ...memoMap }
 });

 setSteps(generatedSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, [targetN]));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps, speed]);

const baseStep = steps[currentStepIndex] || null;
const vars = runtime?.variables;
const liveK = vizNumber(vars?.k) ?? vizNumber(vars?.n);
const liveMemo = vizRecord(vars?.memo);
const compilerLinked = liveK !== null || !!liveMemo;
const memoState = liveMemo
 ? Object.fromEntries(Object.entries(liveMemo).flatMap(
 ([key, value]) => {
 const number = vizNumber(value);
 return number === null ? [] : [[Number(key), number]];
 }
))
 : baseStep?.memoState || {};
const currentStep: DPStep | null = baseStep && compilerLinked
 ? {
 ...baseStep,
 n: liveK ?? baseStep.n,
 memoState,
 desc: `Компилятор: k=${liveK ?? "-"}; таблица м

```

```

 className="p-2 text-slate-400 hover:text-white"
 title="Сбросить"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === 0}
 className="p-2 text-slate-400 hover:text-white"
 title="Шаг назад"
 >
 <ChevronLeft className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(!isPlaying)}
 className={`flex items-center gap-1.5 px-2 py-2
 isPlaying
 ? 'bg-amber-500 text-slate-950 hover:bg-amber-600
 : 'bg-emerald-600 text-white hover:bg-emerald-700'
 }`
 >
 {isPlaying ? <Pause className="w-4 h-4" /> : <Play
 {isPlaying ? 'Пауза' : 'Пуск'}
 </button>
 <button
 onClick={() => { setIsPlaying(false); set
 disabled={currentStepIndex === steps.length - 1}
 className="p-2 text-slate-400 hover:text-white"
 title="Шаг вперед"
 >
 <ChevronRight className="w-5 h-5" />
 </button>
 </div>
 </div>
</div>

```

```

{/* DP Table View */}
<div className="mb-8 bg-slate-950 p-4 md:p-6 rounded-2xl"

```

```

 })
 </div>

 {/* Tabs */}
 <div className="flex items-center gap-2 border-b
 <button
 onClick={() => setActiveTab('table')}
 className={`flex items-center gap-2 px-6 py-3
 activeTab === 'table'
 ? 'border-emerald-500 text-emerald-400 bg
 : 'border-transparent text-slate-400 hove
 }` }
 >
 <Eye className="w-4 h-4" />
 Стен-бай-стен состояние
 </button>
 <button
 onClick={() => setActiveTab('code')}
 className={`flex items-center gap-2 px-6 py-3
 activeTab === 'code'
 ? 'border-emerald-500 text-emerald-400 bg
 : 'border-transparent text-slate-400 hove
 }` }
 >
 <Code className="w-4 h-4" />
 Интерактивный код
 </button>
 </div>

 {/* Tab Content */}
 <div className="min-h-[320px] flex flex-col justi
 {activeTab === 'table' && {
 <div className="bg-slate-950 p-6 rounded-2xl
 <h5 className="font-bold text-white text-sm

 </h5>
 <p className="text-sm text-slate-400">
 В классической рекурсии для $N=\{targetN\}$ п

```

```

 </div>
 </div>
 })
 </div>

 {/* Progress Bar */}
 <div className="mt-8 pt-6 border-t border-slate-800" >
 <div className="flex items-center justify-between" >
 Прогресс выполнения
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>
 <div className="h-2 bg-slate-950 rounded-full overflow-hidden" >
 <div
 className="h-full bg-gradient-to-r from-emerald-500 to-teal-500"
 style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
 ></div>
 </div>
 </div>
</div>
);
}

```

ФАЙЛ 42/111: src/components/EulerSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useContext, useEffect, useMemo, useState } from "react";
import { Undo, Terminal, Route } from "lucide-react";
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from "viz-react";

```

```

const C1_NODES = [
 { id: 0, label: "A", x: 190, y: 60 },
 { id: 1, label: "B", x: 280, y: 140 },
 { id: 2, label: "C", x: 100, y: 140 },
 { id: 3, label: "D", x: 280, y: 250 },
 { id: 4, label: "E", x: 100, y: 250 }
];

```

```

const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли");
};

const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы п");
 return;
 }
 const last = c1Path[c1Path.length - 1];
 const edge = C1_EDGES.find(e => (e.u === last && e.v === vId));
 if (!edge) {
 setC1Msg("Нет ребра между этими вершинами! Выбери");
 return;
 }
 const key = ekey(last, vId);
 if (c1VisitedEdges.includes(key)) {
 setC1Msg("Это ребро уже пройдено! Эйлер не прощает");
 setC1Done(true); return;
 }
 const nextEdges = [...c1VisitedEdges, key];
 const nextPath = [...c1Path, vId];
 setC1VisitedEdges(nextEdges);
 setC1Path(nextPath);

 if (nextEdges.length === C1_EDGES.length) {
 if (nextPath[0] === vId) {
 setC1Msg(" ЭЙЛЕРОВ ЦИКЛ! Все рёбра пройдены, ф");
 } else {
 setC1Msg(` ЭЙЛЕРОВ ПУТЬ! Все рёбра пройдены! С
 не цикл.`);
 }
 setC1Done(true);
 } else {
 setC1Msg(`Пройдено рёбер: ${nextEdges.length} / $`);
 }
};

```

```

 complete: keys.length >= C1_EDGES.length,
 });
}, [vars]));

const path = live && live.path.length ? live.path : c
const visitedKeys = live ? live.keys : c1VisitedEdges
const deg = live?.deg ?? DEG;
const odd = live?.odd ?? ODD;
const lastId = live ? live.last : path.length ? path[
const done = live ? live.complete : c1Done && visited
const msg = live
 ? live.complete
 ? `Компилятор прошёл все ${C1_EDGES.length} рёб
 шин ${odd.length} → это ${odd.length === 0 ? "ц
 : `Маршрут из кода: ${live.path.length ? live.pat
 ys.length}/${C1_EDGES.length}`
 : c1Msg;

const verdict =
 odd.length === 0
 ? { text: "0 нечётных → ЭЙЛЕРОВ ЦИКЛ есть (старт
 }
 : odd.length === 2
 ? { text: `2 нечётные (${odd.map((id) => C1_NODES
 в другой`, cls: "bg-amber-950/40 border-amber-
 : { text: `${odd.length} нечётных → эйлерова марш

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between"
 <h4 className="text-base font-bold text-white f
 все рёбра ровно по одному разу</h4>
 <button onClick={resetC1} className="flex items
 old border border-slate-700 transition-all">
 <Undo className="h-3.5 w-3.5" /> Сброс
 </button>
 </div>

```



```

 label}</text>
 <text x={n.x} y={n.y - 16} textAnchor="middle">deg {deg[n.id]}{isOdd ? " · нечёт" : ""}
 </g>;
 }}}
</svg>
<div className="absolute top-2 left-2 bg-slate-500 p-2">
 Ребёр: {visitedKeys.length}/{C1_EDGES.length}
</div>
</div>

<div className={`p-3 rounded-xl border text-sm ${
 done && visitedKeys.length === C1_EDGES.length
 ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
 : c1Done && !live
 ? "bg-rose-950/40 border-rose-500/30 text-rose-50"
 : "bg-slate-900/50 border-slate-700 text-slate-50"}`}>
 {msg}
</div>

{/* Критерий Эйлера */}
<div className={`p-3 rounded-xl border text-xs font-mono ${
 verdict ? "bg-emerald-950/40 border-emerald-500/30 text-emerald-50"
 : "bg-rose-950/40 border-rose-500/30 text-rose-50"}`}>
 Критерий Эйлера (для связного графа): {verdict}

 Степени: {C1_NODES.map((n) => `${n.label}=${deg[n.id]}`).join(" ")}
 две нечётны. Алгоритм Иера строит маршрут эйлерова графа.

</div>

<div className="flex flex-wrap items-center gap-2">

 /> из компилятора:
 {live ? (
 <>

 {live.path.map((id) => C1_NODES[id].label).join(" ")}


```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const runtime = useVizRuntime();
// k/i/j/d приходят напрямую; номер строки произвольн
const [isPlaying, setIsPlaying] = useState(false);

const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
];

const initialMatrix = [
 [0, 4, 999, 2, 999, 999, 999],
 [999, 0, 3, 999, 999, 3, 999],
 [999, 999, 0, 999, 2, 999, 999],
 [999, 999, 999, 0, 4, 2, 999],
 [999, 999, 999, 999, 0, 999, 1],
 [999, 999, 999, 999, 999, 0, 5],
 [999, 1, 999, 999, 999, 999, 0]
];

const adjList: { [key: number]: { v: number; w: numbe
 0: [{ v: 1, w: 4 }, { v: 3, w: 2 }],
 1: [{ v: 2, w: 3 }, { v: 5, w: 3 }],
 2: [{ v: 4, w: 2 }],
 3: [{ v: 4, w: 4 }, { v: 5, w: 2 }],
 4: [{ v: 6, w: 1 }],
 5: [{ v: 6, w: 5 }],
 6: [{ v: 1, w: 1 }],
};

const codeLines = [

```

```

 log: ` Улучшение пути [${i}]]→[${j}]] чере
 ∞'.`,
 activeCodeLine: 7,
 }));
 }
 }
}

simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r])
 log: ` Все пары отработаны. Алгоритм Флойда завер
 activeCodeLine: 8,
});

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, []));

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length -
 timer = setTimeout(() => setCurrentStepIndex((p)
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;

const vars = runtime?.variables;
const liveK = vizNumber(vars?.k);
const liveI = vizNumber(vars?.i);
const liveJ = vizNumber(vars?.j);
/**

```



```

</text>
</g>
);
}}
</svg>

```

```
<div className="w-full bg-slate-950/80 p-4 rounded-2xl">
 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">
 </div>
</div>
```

```

{ /* Список смежности */ }
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2 text-sm font-mono t
 {Object.keys(adjList).map((uStr) => {
 const u = parseInt(uStr);
 const isI = currentStep.i === u;
 const isK = currentStep.k === u;
 return (
 <div key={u} className={`flex flex-wrap
 isI ? 'bg-amber-900/40 border-ambe
 isK ? 'bg-indigo-900/40 border-ind
 `}>
 <span className="font-bold text-whi
e-400">→
 {adjList[u].map((edge, idx) => {
 const isUpd = currentStep.i === u
 const isVia = currentStep.k !== -
rentStep.j === edge.v);
 return (
 <span key={idx} className={`px-1
dow' : isVia ? 'bg-indigo-500 text-white bo
 {edge.v} ({edge.w})

)
 })}
 </div>
)
 })}
 </div>
 </div>
</div>

```

```

 let bg = 'bg-slate-900/40 text-slate-50'
 if (isUpd) bg = 'bg-emerald-900/80 text-emerald-50'
 else if (isVia) bg = 'bg-indigo-900/80 text-indigo-50'
 else if (currentStep.i === -1 && !currentStep.j)
 bg = 'bg-indigo-950/60 text-indigo-50'
 else if (i === j) bg = 'bg-slate-900/40 text-slate-50'
 const shown = val === null ? '.' : val
 return (<td key={j} className={`p-2 ${bg} text-sm`}>
 {shown}
 </td>
 </tr>
 </tbody>
</table>
</div>
</div>

{/* Код алгоритма */}
<div className="w-full lg:w-1/2 bg-slate-900/60 rounded-lg p-4">
 <div>
 <h4 className="text-lg font-bold text-slate-50">Алгоритм
 <div className="bg-slate-950/80 rounded-lg p-4">
 {codeLines.map(item => (
 <div key={item.line} className={`py-1.5 px-2 ${
 item.e === item.l ? 'bg-blue-900/80 text-amber-50' :
 'bg-slate-900/40 text-slate-50'
 }`}>
 {item.l}
 {item.e}
 </div>
))}
 </div>
 </div>
</div>
</div>
</div>
);
}

```

```

 { id: 1, label: "1", x: 175, y: 45 },
 { id: 2, label: "2", x: 205, y: 140 },
 { id: 3, label: "3", x: 105, y: 175 },
 { id: 4, label: "4", x: 330, y: 70 },
 { id: 5, label: "5", x: 420, y: 45 },
 { id: 6, label: "6", x: 380, y: 145 },
 { id: 7, label: "7", x: 130, y: 290 },
 { id: 8, label: "8", x: 230, y: 320 },
 { id: 9, label: "9", x: 330, y: 275 },
];

 const EDGES: [number, number][] = [
 [0, 1], [1, 2], [2, 3], [0, 3], [1, 3], // компонента
 [4, 5], [5, 6], [4, 6], // компонента
 [7, 8], [8, 9], // компонента
];

 const ADJ: number[][] = NODES.map(() => []);
 EDGES.forEach(([u, v]) => {
 ADJ[u].push(v);
 ADJ[v].push(u);
 });
 ADJ.forEach((a) => a.sort((x, y) => x - y));

 /** Палитра компонент. */
 const COLORS = ["#6366f1", "#10b981", "#f43f5e", "#f59e0b"];

 interface DfsFrame {
 title: string;
 note: string;
 comp: number[]; // 0 = не покрашена, иначе н
 cur: number | null; // текущая вершина
 frontier: number[]; // стек/очередь
 comps: number; // счётчик компонент
 edge: [number, number] | null;
 }

 /** Покадровая раскладка DFS-обхода: ровно тот порядок,
```

```

interface DsuFrame {
 title: string;
 note: string;
 parent: number[];
 comps: number;
 edge: [number, number] | null;
 merged: boolean;
}

function buildDsuFrames(): DsuFrame[] {
 const frames: DsuFrame[] = [];
 const parent = NODES.map((n) => n.id);
 const find = (v: number): number => (parent[v] === v ? v : find(parent[v]));
 let comps = NODES.length;
 const push = (title: string, note: string, edge: [number, number] | null, merged: boolean) => {
 frames.push({ title, note, parent: [...parent], comps, edge, merged });
 push("Старт: каждая вершина сама по себе", `parent[v] = v`, null, false);
 };
 EDGES.forEach(([u, v], i) => {
 const ru = find(u);
 const rv = find(v);
 if (ru === rv) {
 push(`Ребро ${i}: (${u}, ${v}) – корни совпали`, `union ничего не меняет, comps остаётся ${comps}`, null, false);
 return;
 }
 parent[rv] = ru;
 comps -= 1;
 push(`Ребро ${i}: union(${u}, ${v})`, `find(${u}) = ${ru}, find(${v}) = ${rv}, comps стало ${comps}`, [u, v], true);
 });
 push(`Готово: comps = ${comps}`, `Все ${EDGES.length} ребра обработаны, связности`, null, false);
 return frames;
}

const DFS FRAMES = buildDfsFrames();

```



```

 }
 return s + 1;
 });
}, 900);
return () => {
 if (timer.current) window.clearInterval(timer.cur
};
}, [playing, maxStep]));

const goTo = useCallback((n: number) => setStep(Math.r

/* — переменные компилятора, которые рисуются НАПРЯМ
const vars = runtime?.variables;
/** comp / color / used из кода пользователя → своя р
const liveComp = useMemo(() => {
 const arr = vars ? (vizArray(vars.comp) ?? vizArray
 if (!arr) return null;
 return arr.slice(0, NODES.length).map((v) => (typeo
}, [vars]);
/** parent / p из кода → DSU-множества по корням. */
const liveParent = useMemo(() => {
 const arr = vars ? (vizArray(vars.parent) ?? vizArr
 if (!arr || arr.length < NODES.length) return null;
 return arr.slice(0, NODES.length).map((v) => vizNum
}, [vars]);
const liveCur = vars ? (vizNumber(vars.v) ?? vizNumbe
const liveComps = vars ? (vizNumber(vars.comps) ?? vi
const liveFrontier = useMemo(() => {
 if (!vars) return null;
 const arr = vizArray(vars.stack) ?? vizArray(vars.d
 if (!arr) return null;
 return arr.map((v) => vizNumber(v)).filter((v): v i
}, [vars]);
/** Шаг из кода: i / e / step — двигает покадровую де
const liveStep = vars ? (vizNumber(vars.step) ?? vizN

useEffect(() => {
 if (liveStep === null || !Number.isFinite(liveStep))

```

```

 <Network className="w-5 h-5" />
 </div>
 <div>
 <h3 className="text-lg sm:text-xl font-bold">10
 <p className="text-xs text-emerald-300">10
 </div>
 </div>
 <div className="flex items-center bg-slate-950" >
 <button
 onClick={() => switchMode("dfs")}
 className={"flex items-center gap-1.5 px-3 py-3
 600 text-white shadow" : "text-slate-400 hover:text-white"}
 >
 <Layers className="w-3.5 h-3.5" /> 06ход (D
 </button>
 <button
 onClick={() => switchMode("dsu")}
 className={"flex items-center gap-1.5 px-3 py-3
 00 text-white shadow" : "text-slate-400 hover:text-white"}
 >
 <Network className="w-3.5 h-3.5" /> DSU (pë
 </button>
 </div>
 </div>

 {/* панель управления */}
 <div className="flex flex-wrap items-center gap-2" >
 <button onClick={() => { setStep(0); setPlaying(false);
 late-700 text-slate-300 hover:text-white transition-colors
 <RotateCcw className="w-4 h-4" />
 </button>
 <button onClick={() => { setPlaying(false); goToStep(0);
 g-slate-900 border border-slate-700 text-slate-300
 <ChevronLeft className="w-4 h-4" />
 </button>
 <button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-3"
 >

```

```

const color = c > 0 ? COLORS[(c - 1) % COLORS.length];
const isCur = cur === n.id;
const inFrontier = frontier.includes(n.id);
return (
 <g key={n.id} onClick={() => goTo(step)} >
 {isCur && <circle cx={n.x} cy={n.y} r={13} fill={color} />}
 {inFrontier && !isCur && <circle cx={n.x} cy={n.y} r={13} fill="white" />}
 </g>
 <circle cx={n.x} cy={n.y} r={13} fill={color} />
 <text x={n.x} y={n.y + 4} textAnchor="middle">{n.label}</text>
 {c > 0 && <text x={n.x} y={n.y - 19} textAnchor="middle">{c}</text>}
</g>
);
}}}
{/* стек/очередь и DSU-таблица справа */}
<g>
 <text x={470} y={24} textAnchor="end" fill="black">
 {mode === "dfs" ? "стек DFS" : "parent[]"}
 </text>
 {mode === "dfs"
 ? frontier.slice(-8).map((v, i) => (
 <g key={i}>
 <rect x={440} y={32 + i * 20} width={20} height={10} fill={color} />
 <text x={453} y={44 + i * 20} textAnchor="left">{v}</text>
 </g>
))
 : (parentNow ?? []).slice(0, 10).map((pv, i) => (
 const color = COLORS[(setOf(parentNow).get(pv) - 1) % COLORS.length];
 return (
 <g key={i}>
 <rect x={432} y={32 + i * 15} width={20} height={10} fill={color} />
 <text x={436} y={41.5 + i * 15} textAnchor="left">{pv}</text>
 </g>
);
))
);
}}}

```

---

ФАЙЛ 45/111: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from '...';
import { Play, Pause, SkipForward, Undo, RefreshCw } from '...';
```

```
type Node = { id: string; label: string; x: number; y: number };
type Edge = { u: string; v: string };
```

```
const nodes: Node[] = [
 { id: 'A', label: 'A', x: 200, y: 40 },
 { id: 'B', label: 'B', x: 100, y: 120 },
 { id: 'C', label: 'C', x: 300, y: 120 },
 { id: 'D', label: 'D', x: 50, y: 200 },
 { id: 'E', label: 'E', x: 150, y: 200 },
 { id: 'F', label: 'F', x: 250, y: 200 },
 { id: 'G', label: 'G', x: 350, y: 200 },
];
```

```
const edges: Edge[] = [
 { u: 'A', v: 'B' }, { u: 'A', v: 'C' },
 { u: 'B', v: 'D' }, { u: 'B', v: 'E' },
 { u: 'C', v: 'F' }, { u: 'C', v: 'G' },
];
```

```
const adj: Record<string, string[]> = {
 'A': ['B', 'C'],
 'B': ['A', 'D', 'E'],
 'C': ['A', 'F', 'G'],
 'D': ['B'],
 'E': ['B'],
 'F': ['C'],
 'G': ['C'],
};
```

```
function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Начинаем с узла ${start}`, path: Array.from(vis) });

 while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Добавляем узел ${v} в очередь`, path: [...Array.from(vis)] });
 q.shift();

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 vis.add(u);
 q.push(u);
 steps.push({ type: 'visit', node: u, desc: `Посещаем узел ${u}`, path: [...q], visitedNodes: [...Array.from(vis)] });
 }
 }
 }

 steps.push({ type: 'backtrack', node: '', desc: `Очищаем очередь и возвращаемся к началу`, path: [...Array.from(vis)] });

 return steps;
}

export function GraphTraversalViz() {
 const [mode, setMode] = useState<"dfs" | "bfs">("dfs");
 const [autoPlay, setAutoPlay] = useState(false);
 const [stepIdx, setStepIdx] = useState(0);
```



```

 </svg>
 </div>

 {/* Structure View (Stack / Queue) */}
 <div className="bg-slate-900 border border-
r">
 <div className={`absolute -top-3 left
 ${mode === 'dfs' ? 'border-
ld-300 border border-emerald-500'}`}>
 {mode === 'dfs' ? 'Стек вызовов' : ''}
 </div>

 <div className="flex flex-col gap-2 w-full" style={{
 {step.queueOrStack && step.queueOrStack.length > 0}
 }}>
 <div key={idx} className={`w-full
 ${mode === 'dfs' ? 'bg-slate-800
text-emerald-200'}
 ${idx === (mode === 'dfs' ? step.callStack.length : step.queueOrStack.length)
 ? 'border border-emerald-500' : ''}
 `}>
 {mode === 'dfs' ? 'Узел: {item}' : ''}
 {idx === (mode === 'dfs' ? step.callStack.length : step.queueOrStack.length)
 ? {item}
 : ''}
 </div>
 </div>
 </div>
 </div>
 </div>

 <div className="flex flex-col sm:flex-row gap-2" style={{
 {/* Controls */}
 <div className="flex bg-slate-900 border border-slate-800 p-2" style={{width: '100%'}}>
 <button onClick={() => {setAutoPlay(!autoPlay)}}
 {"Назад" title="Назад" className="p-2 text-slate-400 hover:text-slate-200"}
 </div>
 </div>
 </div>

```

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
import { useVizRuntime, vizArray, vizNumber } from '../viz';

interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}

// Max-heap helpers for Patients
function siftUp(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 let idx = a.length - 1;
 while (idx > 0) {
 const parent = Math.floor((idx - 1) / 2);
 if (a[parent].priority < a[idx].priority) {
 [a[parent], a[idx]] = [a[idx], a[parent]];
 idx = parent;
 } else break;
 }
 return a;
}

function siftDown(arr: Patient[]): Patient[] {
 const a = arr.map(x => ({ ...x }));
 const n = a.length;
 let idx = 0;
 while (true) {
 let largest = idx;
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < n && a[l].priority > a[largest].priority) largest = l;
 if (r < n && a[r].priority > a[largest].priority) largest = r;
 if (largest !== idx) {
 [a[idx], a[largest]] = [a[largest], a[idx]];
 idx = largest;
 } else break;
 }
 return a;
}
```



```

 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шме
 { name: 'Оля (Порез)', symptom: ' Глубокий порез',
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emo
];
 start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animS
 });
 return arr;
})());

```

```

let uid = 300;

```

```

export const HeapViz: React.FC = () => {
 const [heap, setHeap] = useState<Patient[]>([]);
 const runtime = useVizRuntime();
 const liveHeap = vizArray(runtime?.variables?.h) ?? v
 const liveI = vizNumber(runtime?.variables?.i);
 const visualHeap: Patient[] = liveHeap
 ? liveHeap.map((value, index) => {
 const tuple = vizArray(value);
 const priority = vizNumber(tuple?.[0]) ?? vizNum
 return {
 id: `python-${index}`,
 name: tuple ? String(tuple[1] ?? value) : Str
 emoji: " ",
 symptom: `индекс ${index}`,
 priority,
 animState: "idle",
 };
 })
 : heap;
 const [customName, setCustomName] = useState('');
 const [customPriority, setCustomPriority] = useState(0);
 const [log, setLog] = useState<string>('');
 const [shakeEmpty, setShake] = useState(false);
 const [busy, setBusy] = useState(false);
 const [healingPatient, setHealing] = useState<Patient>({

```

```
}, [busy, heap]));
```

```
const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() * 10)]
 const randomShift = Math.floor(Math.random() * 10)
 const prio = Math.max(10, Math.min(99, preset.priority + randomShift))
 insertPatient(preset.name, prio);
};
```

```
const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
};
```

```
// --- EXTRACT MAX: Направить самого тяжелого в реанимацию
const extractMax = useCallback(() => {
 if (busy || heap.length === 0) {
 setShake(true);
 addLog('⚠ Нет пациентов для экстренной госпитализации');
 setTimeout(() => setShake(false), 400);
 return;
 }
 setBusy(true);
 const topPatient = heap[0];
 setHealing(topPatient);
```

```
 addLog(`    СРОЧНО! Забираем самого тяжелого: ${topPatient.name}`);
```

```
 // Step 1: Root lights up as winner
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, status: 'winning' } : x));
```

```
 setTimeout(() => {
 // Step 2: Root exits the tree, tree reorganizes
 setHeap(prev => prev.map((x, i) => i === 0 ? { ...x, status: 'healed' } : x));

 setTimeout(() => {
```

```

 res.push(
 <line key={`el${idx}`}
 x1={`${p.x}%`} y1={`${p.y + 4}%`}
 x2={`${lp.x}%`} y2={`${lp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 if (r < visualHeap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={`${p.x}%`} y1={`${p.y + 4}%`}
 x2={`${rp.x}%`} y2={`${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 return res;
 });

 return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-emerald-950/40 border border-e
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-emerald-400 te
 ty Queue)</h3>
 <p className="text-xs text-slate-300 mt-1 lea
 В реанимации не важно, кто пришел раньше, а
 остояние (максимальный приоритет).
 Куча (Heap) автоматически перестраивает пац
 оказывался на самом верху дерева (в корне).
 </p>
 </div>
 </div>
 </div>
)

```

```

 disabled={busy || !customName.trim() || heap.length === 0}
 className="bg-teal-600 hover:bg-teal-500 disabled:opacity-50 px-4 rounded-xl shadow-lg active:scale-95 transition-all"
 >
 Везти
 </button>
 </form>

 {/* Забрать самого тяжелого */}
 <button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-medium rounded-xl shadow-lg transition-all cursor-pointer flex items-center justify-center"
 >
 В реанимацию (EXTRACT MAX)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800 text-white font-medium cursor-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

 {/* ИНТЕРАКТИВНОЕ ОТДЕЛЕНИЕ */}
 <div className="grid grid-cols-1 lg:grid-cols-12 gap-4" >

 {/* ЛЕВАЯ КОЛОНКА: ДЕРЕВО ПАЦИЕНТОВ */}
 <div className={`lg:col-span-8 bg-slate-900 rounded-2xl shadow-2xl p-4 min-h-[380px] overflow-hidden ${shakeEmpty ? 'animate-pulse' : ''}`} >
 <div className="absolute top-4 left-4 text-slate-300 font-medium" >
 Сортировка по тяжести (Двоичная куча / Max-Heap)
 </div>

```

```

 <div className="text-emerald-400">
 </div>
 </div>
</div>
);
 }}}
</div>

 <div className="w-full h-3 bg-gradient-to-r f
</div>

 { /* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */ }
 <div className="lg:col-span-4 bg-slate-900 round
 <div className="absolute top-4 left-4 text-[1
 Операционная койка
 </div>

 <div className="flex-1 flex flex-col justify-
 {healingPatient ? (
 <div className="flex flex-col items-cente
 <div className="relative">
 <span className="text-6xl block anim-l
 <span className="absolute -top-4 -right
 <span className="absolute -bottom-2 -
 </div>
 <div>
 <h4 className="text-white font-black
 Пациент: {healingPatient.name
 </h4>
 <p className="text-emerald-400 text-x
 Состояние: Стабилизация ({healingPa
 </p>
 <div className="flex justify-center g
 <span className="w-2 h-2 rounded-fu
 <Heart className="w-4 h-4 text-rose
 </div>
 </div>
 </div>
 </div>

```

```
/** Общие примитивы для мини-визуализаций подсказок. */
export const W = 260;
export const H = 130;

/**
 * Общий множитель скорости. Кадры были слишком короткими
 * заметить, что именно переехало, и анимация читалась
 */
export const SPEED = 2.1;

/** Длительность переезда узлов. Должна быть заметно меньше
export const MOVE_MS = 800;
export const EASE = "cubic-bezier(.34,.01,.2,1)";
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;

/** Наведение на карточку ставит анимацию на паузу — мож
export const DemoPauseContext = createContext(false);

export function useLoop(steps: number, ms = 900) {
 const [step, setStep] = useState(0);
 const paused = useContext(DemoPauseContext);
 const period = Math.round(ms * SPEED);

 useEffect(() => {
 if (steps <= 1 || paused) return;
 const t = setInterval(() => setStep((s) => (s + 1) % steps), period);
 return () => clearInterval(t);
 }, [steps, period, paused]);

 return step;
}

export const C = {
 idle: "#334155",
 idleStroke: "#475569",
 text: "#e2e8f0",
 dim: "#94a3b8",
```

```

 {label}
 </text>
 })
 </g>
);

export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 strokeWidth={width}
 strokeDasharray={dashed ? "4 3" : undefined}
 style={{ transition: MOVE }}
 />
);

```

---

ФАЙЛ 48/111: src/components/hints/demosExtra.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```

import React from "react";
import { C, Edge, MOVE, Node, Svg, useLoop } from "../demo";

/** Флойд: внешний цикл по «разрешённой» промежуточной
export const FloydDemo: React.FC = () => {
 const step = useLoop(3, 1200);
 // i -> j напрямую 9; через k=1 получаем 3+4=7

```

```

 [{"D", "E"}],
 [{"F", "G"}, {"G", "H"}, {"F", "H"}],
];
 const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
 };
 const colors = [C.active, C.warn, C.done];
 const step = useLoop(comps.length + 1, 900);
 const compOf = (v: string) => comps.findIndex((e) => e === v);

 return (
 <Svg caption={step === 0 ? "Граф распался на острова" : ""}>
 {comps.flatMap((edges, ci) =>
 edges.map(([u, v], i) => (
 <Edge key={`_${ci}-${i}`} a={pos[u]} b={pos[v]} color={colors[ci]} />
))
)}
 {Object.entries(pos).map(([k, [x, y]]) => {
 const ci = compOf(k);
 return <Node key={k} x={x} y={y} r={12} label={k} color={colors[ci]} />
 })}
 </Svg>
);
};

```

/\*\* Что такое граф: вершины, рёбра, направление и вес.

```

export const GraphBasicsDemo: React.FC = () => {
 const step = useLoop(3, 1300);
 const pos: Record<string, [number, number]> = { A: [50, 50], B: [150, 50], C: [250, 50],
 const edges: [string, string, number][] = [{"A", "B", 1}, {"B", "C", 1}, {"C", "A", 1}];
 return (
 <Svg
 caption={
 step === 0
 ? "Вершины – объекты (города, слова, состояния)"
 : step === 1

```



```

const startX = (260 - raw.length * (boxW + 4)) / 2;

return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты – огромные и с дырами"
 : step === 1
 ? "Сортируем уникальные значения"
 : "Каждое число заменяем его номером → плотн
 }
 >
 {raw.map((v, i) => {
 <g key={i}>
 <rect x={startX + i * (boxW + 4)} y={20} width
 <text x={startX + i * (boxW + 4) + boxW / 2}
 {v}
 </text>
 </g>
 })}

 {step >= 1 &&
 sorted.map((v, i) => {
 <g key={v}>
 <rect x={30 + i * 52} y={62} width={46} hei
 <text x={53 + i * 52} y={77} textAnchor="mi
 {v}
 </text>
 </g>
 })}

 {step >= 2 &&
 raw.map((v, i) => {
 <g key={`c${i}`}>
 <rect x={startX + i * (boxW + 4)} y={98} wi
 <text x={startX + i * (boxW + 4) + boxW / 2
 {sorted.indexOf(v)}
 </text>

```

```

 {id}
 </text>
 </g>
)))
</Svg>
);
};

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => {
 <SplayFrames
 captions={["р – корень, х под ним", "Один поворот: "]}
 frames={[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"
]}
 />
};

/** Zig-Zig – х и р с одной стороны, крутим сначала верш
export const ZigZigDemo: React.FC = () => {
 <SplayFrames
 captions={["Линия g → p → x («бамбук»)", "Поворот В
 frames={[
 { pos: { g: [176, 24], p: [126, 70], x: [76, 112]
 { pos: { p: [130, 26], x: [78, 78], g: [186, 78]
 { pos: { x: [102, 24], p: [152, 70], g: [202, 112]
]}
 />
};

/** Zig-Zag – х и р с разных сторон, крутим сначала ниж
export const ZigZagDemo: React.FC = () => {
 <SplayFrames
 captions={["Зигзаг: g влево, х вправо", "Поворот НИ
 frames={[
 { pos: { g: [176, 24], p: [110, 70], x: [152, 112]
 { pos: { g: [176, 24], x: [110, 70], p: [64, 112]

```

```
};
```

```
export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={head ? `Идём вглубь: ${visited.join(" ")}` : ""}
 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(iu - iv) <= 1;
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]} color={onPath ? "red" : "gray"} />
 })}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={k === head ? "red" : "gray"} />
))}
 </Svg>
);
};
```

```
export const ToposortDemo: React.FC = () => {
 const nodes = [
 { id: "труссы", x: 52, y: 32 }, { id: "штаны", x: 148, y: 32 },
 { id: "носки", x: 52, y: 98 }, { id: "боты", x: 148, y: 98 }
];
 const edges: [string, string][] = [
 ["труссы", "штаны"],
 ["труссы", "носки"],
 ["штаны", "боты"],
 ["носки", "боты"]
];
 const order = ["труссы", "носки", "штаны", "боты"];
 const step = useLoop(order.length + 1, 800);
 const placed = order.slice(0, step);
 const pos = Object.fromEntries(nodes.map((n) => [n.id, {x: n.x, y: n.y}]));
 return (
 <Svg caption={`Порядок: ${placed.join(" → ") || "..."} />
 {edges.map(([u, v], i) => (
 <Edge key={i} a={pos[u]} b={pos[v]} color={placed.includes(v) ? "red" : "gray"} />
))}
 {nodes.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.id} fill={placed.includes(n.id) ? "red" : "gray"} />
))}
 </Svg>
);
};
```

```

const step = useLoop(2, 1300);
const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
};
const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D", "A"]
];
const cut = step === 1;
return (
 <Svg caption={cut ? "Убрали C-D → граф развалился на 2 компонента" : ""}
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} color={i % 2 === 0 ? "red" : "blue"} />
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={cut ? "red" : "blue"} />
))}
 </Svg>
);
};

```

```

export const ArticulationDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [36, 40], B: [36, 96], C: [130, 68], D: [224, 40]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["A", "D"], ["B", "C"], ["B", "D"], ["C", "D"]
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали вершину C → две отдельные компоненты" : ""}
 {edges.map(([u, v], i) => (cut && (u === "C" || v === "C") ? null : <Edge key={i} a={pos[u]} b={pos[v]} color={i % 2 === 0 ? "red" : "blue"} />))}
 {Object.entries(pos).map(([k, [x, y]]) => (
 cut && k === "C" ? null : (
 <Node key={k} x={x} y={y} label={k} fill={k === "C" ? "red" : "blue"} />
)
))}
 </Svg>
);
};

```

```
const edges: [string, string][] = [["A", "B"], ["B",
const step = useLoop(2, 1400);
const scc = ["A", "B", "C"];
const inScc = (u: string, v: string) => step === 1 &&
return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
 {edges.map(([u, v], i) => (
 <Edge key={i} a={pos[u]} b={pos[v]} color={inScc
)})}
 {Object.entries(pos).map(([k, [x, y]]) => (
 <Node key={k} x={x} y={y} label={k} fill={step
)})}
 </Svg>
);
};
```

ФАЙЛ 50/111: `src/components/hints/demosStruct.tsx`  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```
import React from "react";
import { C, Edge, Node, Svg, useLoop } from "../demoKit"

export const HeapDemo: React.FC = () => {
 const states = [[4, 8, 9, 12, 2], [4, 2, 9, 12, 8], ...];
 const step = useLoop(states.length, 1000);
 const heap = states[step];
 const pos: [number, number][] = [[130, 24], [80, 68], ...];
 const moving = step === 0 ? 4 : step === 1 ? 1 : 0;
 return (
 <Svg caption="Новый элемент всплывает наверх, пока не достигнет вершины">
 <Edge a={pos[0]} b={pos[1]} /><Edge a={pos[0]} b={pos[2]} />
 <Edge a={pos[1]} b={pos[3]} /><Edge a={pos[1]} b={pos[4]} />
 {heap.map((v, i) => (
 <Node key={i} x={pos[i][0]} y={pos[i][1]} label={v}>{v}</Node>
))}
 </Svg>
)
}
```

```

 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} height={20}>
 <text x={53 + i * 54} y={71} textAnchor="middle">{v}</text>
 </g>
)}
 <line x1={24} y1={71} x2={10} y2={71} stroke={C.d} />
</Svg>
);
};

```

```

export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3], [0, 0, 2, 3]];
 const step = useLoop(parents.length, 950);
 const p = parents[step];
 const pos: [number, number][] = [[50, 95], [105, 95], [155, 95], [205, 95]];
 const rootColor = ["#6366f1", "#10b981", "#f59e0b", "#ff7f0e"];
 const root = (i: number): number => (p[i] === i ? i : root(p[i]));
 return (
 <Svg caption="find = «кто твой староста», union = 0" >
 {p.map((par, i) =>
 par !== i ? (
 <path key={i} d={`M ${pos[i][0]} ${pos[i][1]} L ${pos[root(i)][0]} ${pos[root(i)][1]}`
 fill="none" stroke={rootColor[root(i)]} stroke-width={2} />
) : null
)}
 {pos.map(([x, y], i) => <Node key={i} x={x} y={y} />)}
 </Svg>
);
};

```

```

const TRIE_NODES = [
 { id: 0, x: 130, y: 22, ch: "*", parent: null as number },
 { id: 1, x: 72, y: 62, ch: "a", parent: 0 },
 { id: 2, x: 188, y: 62, ch: "b", parent: 0 },
 { id: 3, x: 42, y: 104, ch: "b", parent: 1 },
 { id: 4, x: 102, y: 104, ch: "c", parent: 1 },
 { id: 5, x: 188, y: 104, ch: "c", parent: 2 },

```

```

return (
 <Svg caption="Суффиксная ссылка: «куда откатиться,
 {TRIE_NODES.filter((n) => n.parent !== null).map(
 const p = TRIE_NODES[n.parent as number];
 return <Edge key={n.id} a={[p.x, p.y]} b={[n.x,
]}}
 {links.slice(0, step).map(([from, to], i) => {
 const a = TRIE_NODES[from];
 const b = TRIE_NODES[to];
 return (
 <path key={i} d={`M ${a.x} ${a.y} Q ${(a.x +
 fill="none" stroke={C.warn} strokeWidth={1.
)};
)}}
 {TRIE_NODES.map((n) => (
 <Node key={n.id} x={n.x} y={n.y} label={n.ch} f
)}}
 </Svg>
);
};

```

```

export const SegmentTreeDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const nodes = [
 { x: 130, y: 24, w: 226, label: "[0..7]", lvl: 0 },
 { x: 72, y: 60, w: 110, label: "[0..3]", lvl: 1 },
 { x: 188, y: 60, w: 110, label: "[4..7]", lvl: 1 },
 { x: 44, y: 96, w: 52, label: "[0..1]", lvl: 2 },
 { x: 100, y: 96, w: 52, label: "[2..3]", lvl: 2 },
 { x: 160, y: 96, w: 52, label: "[4..5]", lvl: 2 },
 { x: 216, y: 96, w: 52, label: "[6..7]", lvl: 2 },
];
 return (
 <Svg caption="Запрос собирается из $O(\log n)$ готовых
 {nodes.map((n, i) => (
 <g key={i}>
 <rect x={n.x - n.w / 2} y={n.y - 10} width={n
 fill={n.lvl === step ? C.active : C.idle} s

```

```
export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы дл
 {Array.from({ length: 8 }).map((_, i) => {
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={18} />
 <text x={25 + i * 30} y={29} textAnchor="middle" />
 </g>
 })}
 {Array.from({ length: count }).map((_, i) => {
 <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 18} width={26} height={18}
 fill={C.active} opacity={i === 0 ? 1 : 0.4} stroke={C.done} />
 })}
 </Svg>
);
};
```

```
export const AmortizedDemo: React.FC = () => {
 const costs = [1, 1, 1, 8, 1, 1, 1, 1];
 const step = useLoop(costs.length + 1, 550);
 const total = costs.slice(0, step).reduce((a, b) => a + b, 0);
 const avg = step ? (total / step).toFixed(2) : "-";
 return (
 <Svg caption={`Сделано ${step} операций, суммарно $
 {costs.map((c, i) => {
 <rect key={i} x={14 + i * 30} y={104 - c * 10} width={26} height={18}
 fill={i < step ? (c > 2 ? C.bad : C.done) : C.done} stroke={C.done} />
 })}
 <line x1={8} y1={106} x2={252} y2={106} stroke={C.done} />
 <text x={130} y={20} textAnchor="middle" fontSize={14} />
 </Svg>
);
};
```



```

 $\pi[i]$ — длина самого длинного «префикс = суффикс»
 </text>
 </Svg>
);
};

export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;
 return (
 <g key={idx}>
 <rect x={40 + c * 38} y={22 + r * 32} width={38} height={32}
 fill={idx === step - 1 ? C.active : fill}
 {filled && (
 <text x={57 + c * 38} y={36 + r * 32} text={C.dp[idx]}
 fontSize={10} fontWeight="700" fill="white" />
)}
 </g>
);
)}
)}
 </Svg>
);
};

<text x={130} y={122} textAnchor="middle" fontSize={14}
 каждая клетка считается один раз и переиспользуется
</text>
</Svg>
);
};

```

```

 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
 graph: GraphBasicsDemo,
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
 };

/** Список работающих мини-демо — его проверяет `npm run`
export const MINI_DEMO_KINDS = Object.keys(REGISTRY);

export const MiniDemo: React.FC<{ kind: DemoKind }> = (
 const [paused, setPaused] = useState(false);
 const Cmp = REGISTRY[kind];
 if (!Cmp) return null;
 return (
 <DemoPauseContext.Provider value={paused}>
 <div
 className="bg-slate-950/70 rounded-lg border bo
 onMouseEnter={() => setPaused(true)}
 onMouseLeave={() => setPaused(false)}
 >
 <Cmp />
 </div>
 </DemoPauseContext.Provider>
);
};

```

```

 ded-2xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py-4">
 <div className="min-w-0 flex-1">
 <h3 className="font-bold text-white text-sm">
 {viz.hint} && <p className="text-[11px] text-white">
 <VizLiveStatus demoTitle={viz.title} className="text-white">
 </div>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hover:bg-slate-700"
 aria-label="Заккрыть симулятор"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="overflow-y-auto overscroll-contain">
 {/* Тот же id главы: развёрнутая демонстрация */}
 <VizChapterContext.Provider value={vizId}>
 <Component />
 </VizChapterContext.Provider>
 </div>
 </div>
 </div>,
 document.body
);
 };
};

```

ФАЙЛ 53/111: src/components/hints/TermPopover.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

```

import React, { useEffect, useLayoutEffect, useRef, useState } from "react";
import { createPortal } from "react-dom";
import { Play, X } from "lucide-react";
import { GLOSSARY_BY_ID } from "../../data/glossary";
import { MiniDemo } from "../MiniDemo";

```

```

 setPos({ top: Math.max(8, Math.min(top, vh - height
 }, [anchor]));

useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onClose, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onClose, true);
 };
}, [anchor, onClose]);

if (!anchor) return null;
const entry = GLOSSARY_BY_ID.get(anchor.termId);
if (!entry) return null;
const viz = getViz(entry.simulator);
const simulatorId = entry.simulator;

return createPortal(
 <div
 ref={cardRef}
 onMouseEnter={onCardEnter}
 onMouseLeave={onCardLeave}
 className="fixed z-[100] term-popover"
 style={{
 top: pos ? pos.top : -9999,
 left: pos ? pos.left : -9999,
 width: Math.min(CARD_W, typeof window !== "undefined" ? window.innerWidth : 1000),
 visibility: pos ? "visible" : "hidden",
 }}
 role="dialog"
 >
 <div className="bg-slate-900 border border-indigo-500 p-4"
 <div className="flex items-start gap-2 px-3 pt-3"

```

```
 </div>,
 document.body
);
};
```

---

ФАЙЛ 54/111: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizSteps';

export function JohnsonViz() {
 const [step, setStep] = useState(0);
 const MAX_STEP = 7;
 useVizStepSync(step, setStep, MAX_STEP, (vars) => {
 const edge = `${vizString(vars.u) ?? ""}${vizString(vars.v) ?? ""}`;
 return edge === "AB" ? 4 : edge === "BC" ? 5 : 0;
 });

 const nodes = [
 { id: 'S', x: 200, y: 150, label: 'S' },
 { id: 'A', x: 200, y: 50, label: 'A' },
 { id: 'B', x: 320, y: 220, label: 'B' },
 { id: 'C', x: 80, y: 220, label: 'C' },
];

 const edges = [
 { id: 'SA', u: 'S', v: 'A', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SB', u: 'S', v: 'B', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'SC', u: 'S', v: 'C', weight: 0, labelDx: 0, labelDy: 0 },
 { id: 'AB', u: 'A', v: 'B', weight: -2, newWeight: 2, labelDx: 0, labelDy: 0 },
 { id: 'BC', u: 'B', v: 'C', weight: 1, newWeight: 1, labelDx: 0, labelDy: 0 },
 { id: 'CA', u: 'C', v: 'A', weight: 2, newWeight: 2, labelDx: 0, labelDy: 0 },
];
}
```

```

 return "stroke-slate-500";
 };

 const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#a
 if (e.id === 'AB' && step > 4) return "url(#arr
 if (e.id === 'BC' && step === 5) return "url(#a
 if (e.id === 'BC' && step > 5) return "url(#arr
 if (e.id === 'CA' && step === 6) return "url(#a
 if (e.id === 'CA' && step > 6) return "url(#arr

 if (step >= 4) return "url(#arrow-slate-dim)";
 return "url(#arrow-slate)";
 };

 const getEdgeContent = (e: any) => {
 if (e.u === 'S') return e.weight;

 if (step < 4) return e.weight;

 if (e.id === 'AB') {
 if (step === 4) return `${e.weight} + (${0}
 return e.newWeight;
 }
 if (e.id === 'BC') {
 if (step < 5) return e.weight;
 if (step === 5) return `${e.weight} + (${-2}
 return e.newWeight;
 }
 if (e.id === 'CA') {
 if (step < 6) return e.weight;
 if (step === 6) return `${e.weight} + (${-1}

```

Запускаем медленный, но мощный алгоритм на всех вершинах:

```


 </div>
);
case 3: return (
 <div>
 Шаг 3: Магическая формула.

 Всем ребрам графа назначаем новый вес.
 <div className="text-center font-monospace">
 Это повышает или понижает вес ребра?
 </div>
 </div>
);
case 4: return (
 <div>
 Шаг 3.1: Пересчет ребра A → B.
 Старый вес:
 Вычисляем: $-2 + 0 - (-2) =$
 </div>
);
case 5: return (
 <div>
 Шаг 3.2: Пересчет ребра B → C.
 Старый вес: 1.

 Вычисляем: $1 + (-2) - (-1) =$
 </div>
);
case 6: return (
 <div>
 Шаг 3.3: Пересчет ребра C → A.
 Старый вес: 2.

 Вычисляем: $2 + (-1) - 0 =$
 </div>
);
case 7: return (
 <div className="text-emerald-300">
 Готово! Фиктивную вершину S мы
 Все новые веса ребер в графе стали:
 </div>
);

```

```

 <path d="M 0 0 L 10 5 L 10 10" />
 </marker>
 </defs>

```

```

 { /* Edges */
 {edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);

 const isS = e.u === 'S';
 if (isS && (step === 0 || step === 1)) return null;

 const midX = (u.x + v.x) / 2;
 const midY = (u.y + v.y) / 2;

 const edgeColorClass = getEdgeColorClass(e);
 const textHigh = isEdgeTextHigh(e);
 const textEmer = isEdgeTextEmer(e);

 return (
 <g key={i}>
 <line
 x1={u.x} y1={u.y}
 x2={v.x} y2={v.y}
 className={`stroke-${edgeColorClass}`}
 markerEnd={getMarkerEnd(e)}
 />

 <text
 x={midX} y={midY}
 textAnchor="middle"
 dominantBaseline="bottom"
 className={`text-${edgeColorClass}`}
 style={{textHigh, textEmer}}
 {isS ? 'font-weight: bold;' : ''}
 >{getEdgeContent(e)}
 </text>
 </g>
);
 })}

```





```

 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];
```

```
const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];
```

```
interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
 desc: string;
 visited: Set<number>;
 currentNode: number | null;
 order: number[];
 transposed: boolean;
 sccMap: Record<number, number>; // node -> sccId
 currentSccId: number;
 activeEdges: GEdge[];
}
```

```
export const KosarajuViz: React.FC = () => {
 const [steps, setSteps] = useState<AlgoStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode === v));
 return candidates.at(-1)?.index ?? null;
});
const [isPlaying, setIsPlaying] = useState(false);
```

```

const adj: number[][] = Array.from({ length: 5 }, (
initialEdges.forEach((e) => adj[e.u].push(e.v)));

// DFS 1: post-order list
const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение
 u,
 false,
 -1,
);

 for (const to of adj[u]) {
 if (!visited.has(to)) {
 dfs1(to);
 }
 }

 order.push(u);
 recordStep(
 "dfs1",
 `DFS 1: вершина ${u} полностью обработана. Запи
 u,
 false,
 -1,
);
};

// Run DFS1 on all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 dfs1(i);
 }
}

const topoOrder = [...order].reverse();

```

```

 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в
 u,
 true,
 sccId,
);
 }
 }

 recordStep(
 "finished",
 `Алгоритм успешно завершен! Найдено ${sccId} компонент
 null,
 true,
 sccId,
);

 setSteps(generatedSteps);
 setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
 phase: "init",
 desc: "Загрузка...",
 visited: new Set(),
 currentNode: null,

```

```

<div className="flex items-center gap-2 bg-slate-500" >
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-2 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 aria-label="Сбросить обход"
 title="Сбросить обход"
 className="flex items-center justify-center gap-2"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

```

```

<div className="grid grid-cols-1 lg:grid-cols-12" >
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] relative" >
 <svg className="w-full h-full max-w-[650px]" >
 {/* Markers for directed arrows */}
 <defs>
 <marker
 id="arrow"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"

```

```

const x2 = step.transposed ? u.x : v.x;
const y2 = step.transposed ? u.y : v.y;

const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
let stroke = "#475569";
let markerId = "arrow";

if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f46e5";
 markerId = "arrow-transposed";
} else {
 if (
 step.currentNode === edge.u ||
 step.currentNode === edge.v
) {
 stroke = "#10b981";
 markerId = "arrow-active";
 }
}

// Adjust slightly for bidirectional link
const isBidi =
 (edge.u === 3 && edge.v === 4) ||
 (edge.u === 4 && edge.v === 3);
const dy = isBidi ? (edge.u === 3 ? -10 : 10) : 0;

return (
 <line
 key={`edge-${idx}`}
 x1={x1}
 y1={y1 + dy}
 x2={x2}
 y2={y2 + dy}
 stroke={stroke}
 strokeWidth="2.5"
 />

```

```

 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && {
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 SCC #{sccId}
 </text>
 }}
 </g>
);
 }}}
</svg>

{step.transposed && {
 <div className="absolute top-4 left-4 bg-in
 1 rounded">
 Граф транспонирован (Ребра обратные)
 </div>
 }}
</div>

{/* Logs & Stacks */}
<div className="lg:col-span-4 bg-slate-950 p-5
00 overflow-y-auto">
 <h4 className="text-xs font-bold text-slate-4
 Порядок обхода и стек
 </h4>

```

```

 .reverse()
 .slice(0, 5)
 .map(({s, idx}) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 rounded`
 >
 {s.desc}
 </div>
)))
 </div>
 </div>
 </div>

 <div className="mt-4 pt-3 border-t border-slate-200">

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1.5">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev) => prev - 1)}
 className="bg-slate-900 border border-slate-700 px-2 py-1"
 >
 Назад
 </button>
 <button
 disabled={currentStepIdx >= steps.length - 1}
 onClick={() => setCurrentStepIdx((prev) => prev + 1)}
 className="bg-slate-900 border border-slate-700 px-2 py-1"
 >
 Вперед
 </button>
 </div>
 </div>
</div>
</div>
</div>

```



```

// 9 Vertices layout
const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
 { id: '0-1', u: 0, v: 1, weight: 2, status: 'pending' },
 { id: '3-4', u: 3, v: 4, weight: 3, status: 'pending' },
 { id: '1-2', u: 1, v: 2, weight: 4, status: 'pending' },
 { id: '0-3', u: 0, v: 3, weight: 5, status: 'pending' },
 { id: '1-4', u: 1, v: 4, weight: 6, status: 'pending' },
 { id: '4-5', u: 4, v: 5, weight: 7, status: 'pending' },
 { id: '3-6', u: 3, v: 6, weight: 8, status: 'pending' },
 { id: '2-5', u: 2, v: 5, weight: 9, status: 'pending' },
 { id: '6-7', u: 6, v: 7, weight: 10, status: 'pending' },
 { id: '4-7', u: 4, v: 7, weight: 11, status: 'pending' },
 { id: '7-8', u: 7, v: 8, weight: 12, status: 'pending' },
 { id: '5-8', u: 5, v: 8, weight: 13, status: 'pending' },
];

export const KruskalSimulator: React.FC = () => {
 const [activeAlgo, setActiveAlgo] = useState<'kruskal'>
 const [vertices, setVertices] = useState<Vertex[]>(in
 const [edges, setEdges] = useState<Edge[]>(initialEdg
 const [stepIndex, setStepIndex] = useState<number>(0)
 const [heapQueue, setHeapQueue] = useState<string[]>(
 const [activeCheatTab, setActiveCheatTab] = useState<

 const [logs, setLogs] = useState<StepLog[]>([

```

```
// Step 4: Include D-E
() => {
 setVertices(prev => prev.map(v => v.id === 3 || v
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в осто́ве',
 description: 'Вершины D и E окрашены в синий цвет',
 type: 'success'
 }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',
 description: 'Оно соединяет красную вершину B и
 type: 'info'
 }, ...prev]));
},
// Step 6: Include B-C
() => {
 setVertices(prev => prev.map(v => v.id === 2 ? {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро B-C в осто́ве',
 description: 'Вершина C перекрашена в красный цвет',
 type: 'success'
 }, ...prev]));
},
// Step 7: Inspect A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Берем ребро A-D (вес 5)',
 description: 'ВНИМАНИЕ! Ребро соединяет КРАСНУЮ
 ю — мы перекрашиваем всё в один цвет!"',
 type: 'warning'
 }, ...prev]));
}
```

```
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',
 description: 'Оно соединяет фиолетовую вершину
 type: 'info'
 }, ...prev]));
},
// Step 14: Include D-G
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-G в осто́ве',
 description: 'Вершина G окрашена в фиолетовый.'
 type: 'success'
 }, ...prev]));
},
// Step 15: Inspect C-F (9) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '2-5' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Берем ребро C-F (вес 9)',
 description: 'Оно соединяет вершины C и F, кото
 type: 'warning'
 }, ...prev]));
},
```

```

 () => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
 },
 // Step 21: Inspect H-I (12)
 () => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю',
 type: 'info'
 }, ...prev]));
 },
 // Step 22: Include H-I
 () => {
 setVertices(prev => prev.map(v => v.id === 8 ? {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Успех: MST построено Краскалом!',
 description: 'Все 9 вершин окрашены в единый фиолетовый цвет.
 + 4 + 5 + 7 + 8 + 10 + 12 = 51.',
 type: 'success'
 }, ...prev]));
 },
];

```

// --- PRIM STEPS ---

const primSteps = [

// Step 1: Start at E

() => {

setVertices(prev => prev.map(v => v.id === 4 ? {

setHeapQueue(['D-E (3)', 'B-E (6)', 'E-F (7)', 'E-G (8)'],

setEdges(prev => prev.map(e => ['3-4', '1-4', '4-5', '4-6', '4-7', '4-8', '4-9']),

setLogs(prev => [{



```

 description: 'Плесень из центра Токио (Е) тянет',
 type: 'info'
 }, ...prev]);
},
// Step 13: Include E-F, add F's edges
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. Р
 type: 'success'
 }, ...prev]);
},
// Step 14: Pop D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 8: Куча выдает ребро D-G (вес 8)',
 description: 'Плесень расширяется вниз к G.',
 type: 'info'
 }, ...prev]);
},
// Step 15: Include D-G, add G's edges
() => {
 setVertices(prev => prev.map(v => v.id === 6 ? {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 eap' } : e));
 setHeapQueue(['C-F (9 - цикл)', 'G-H (10)', 'E-H
 setLogs(prev => [{
 title: 'Успех: Вершина G захвачена',
 description: 'В кучу добавлено G-H(10).',
 type: 'success'
 }, ...prev]);
},
// Step 16: Pop C-F (9) - Cycle!
```



```

 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red' }
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue' }
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы',
 description: 'Все выбранные дороги построены радом с {D, E, F, G, H, I}.',
 type: 'success'
 }, ...prev]);
 },
 // Step 3: Five-Year Plan 2 (Inspecting bridge between two villages)
 () => {
 // Now both kolkhozes look at all outgoing roads
 // Outgoing roads between {A, B, C} and {D, E, F, G, H, I}.
 // A-D (5), B-E (6), C-F (9).
 // Cheapest is A-D (5).
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 ...e, status: 'included', color: 'red'
 } : e));
 setLogs(prev => [{
 title: 'Вторая пятилетка: Колхозы выбирают мосты',
 description: 'Теперь Красный и Синий колхозы одобрят мосты A-D с весом 5. Остальные мосты B-E(6) и C-F(9) будут отклонены.',
 type: 'warning'
 }, ...prev]);
 },
 // Step 4: Concurrently merge into single Kolkhoz
 () => {
 setVertices(prev => prev.map(v => ({ ...v, color: 'blue' })));
 setEdges(prev => prev.map(e =>
 e.status === 'included' || e.id === '0-3' ? { ...e, color: 'red' } : e));
 setLogs(prev => [{
 title: 'Объединение завершено: Один гигантский колхоз'
 }]);
 }
);

```



```

 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем стартовый (Heap) и захватываем соседей!',
 type: 'info',
 }]);
}
return [{
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 деревень объединиться в колхоз.',
 type: 'info',
}]);
};

const handleReset = (algo: 'kruskal' | 'prim' | 'boruvka') => {
 setActiveAlgo(algo);
 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(0);
 setHeapQueue([]);
 setLogs(introLog(algo));
};

/* ----- связь с Python-компилятором ----- */
const chapterId = useContext(VizChapterContext);
const runtime = useVizRuntime();
const vars = runtime?.variables;

// активная вкладка = подрежим страницы: код в панели
useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, activeAlgo);
}, [chapterId, activeAlgo]);

/** Перемотка на кадр n: сброс и повтор первых n шагов */
const goToStep = useCallback((n: number) => {
 const target = Math.max(0, Math.min(currentSteps.length, n));
 setVertices(initialVertices);
 setEdges(initialEdges);
 setStepIndex(target);
 setHeapQueue([]);
 setLogs(introLog(activeAlgo));
}, []);

```

```

 if (u === null || v === null) return null;
 return { u, v, w: nums && nums.length ? Number(nums[0]) : null };
 }
 if (Array.isArray(item)) {
 const letters = item.filter((x) => typeof x === 'string');
 const nums = item.filter((x) => typeof x === 'number');
 if (letters.length >= 2) {
 const u = idOf(letters[0]); const v = idOf(letters[1]);
 if (u === null || v === null) return null;
 return { u, v, w: nums.length ? nums[0] : null };
 }
 if (nums.length >= 2) {
 const u = idOf(nums[nums.length - 2]); const v = idOf(nums[nums.length - 1]);
 if (u === null || v === null) return null;
 return { u, v, w: nums.length > 2 ? nums[0] : null };
 }
 }
 return null;
};

```

```

const PALETTE = ['red', 'blue', 'purple', 'mold'];
/** Цвет по корне компоненты: одинаковый корень → одинаковый цвет */
const colorOfComp = (root: string | undefined): string | undefined => {
 if (!root) return 'none';
 let h = 0;
 for (let i = 0; i < root.length; i++) h = (h * 31 + root.charCodeAt(i)) % PALETTE.length;
 return PALETTE[h];
};

```

```

/** Всё, что код пользователя рассказал про граф. */
const live = useMemo(() => {
 if (!vars) return null;
 const parentRec = vizRecord(vars.parent) ?? vizRecord(0);
 const parentArr = vizArray(vars.parent) ?? vizArray(0);
 /** Метка множества вершины, как её записал код (будет использоваться для подсчёта) */
 const rawComp = new Map<number, string>();
 /** То же, но только если метка – вершина графа: по метке можно найти вершину */
 const parentIdx = new Map<number, number>();

```

```

 compOfVertex.set(id, String.fromCharCode(65 + cur
));
 const takenRaw = vizArray(vars.mst) ?? vizArray(var
 const takenEdges = (takenRaw ?? []).map(parseEdge).
 const takenVertices = (vizArray(vars.taken) ?? vizA
 .map((x) => idOf(typeof x === 'string' ? x : vizN
 .filter((x): x is number => x !== null);
 const curU = idOf(vizString(vars.u) ?? vizNumber(va
 const curV = idOf(vizString(vars.v) ?? vizNumber(va
 const curW = vizNumber(vars.w);
 const heapRaw = vizArray(vars.heap) ?? vizArray(var
 const heapEdges = (heapRaw ?? []).map(parseEdge).fi
 const cheapestRaw = vizRecord(vars.cheapest) ?? viz
 const cheapestEdges = cheapestRaw ? Object.values(c
 l } => e !== null) : [];
 const total = vizNumber(vars.total) ?? vizNumber(va
 const any = compOfVertex.size > 0 || takenEdges.leng
 (curU !== null && curV !== null) || heapEdges.leng
 return any ? { compOfVertex, takenEdges, takenVerti
 // eslint-disable-next-line react-hooks/exhaustive-c
}, [vars]);

```

/\*\* Рёбра, которые рисуем: код пользователя важнее ка

```

const effEdges: Edge[] = useMemo(() => {
 if (!live) return edges;
 const key = (a: number, b: number) => `${Math.min(a
 const takenKey = new Set(live.takenEdges.map((e) =>
 const heapKey = new Set(live.heapEdges.map((e) => k
 const cheapKey = new Set(live.cheapestEdges.map((e)
 return edges.map((e) => {
 const k = key(e.u, e.v);
 const isCur = (live.curU === e.u && live.curV ===
 if (isCur) return { ...e, status: 'inspecting' as
 if (takenKey.has(k)) return { ...e, status: 'incl
 if (heapKey.has(k)) return { ...e, status: 'in_he
 if (cheapKey.has(k)) return { ...e, status: 'insp
 return { ...e, status: 'pending' as const };
 });

```

```

 if (color === 'mold') return '#10b981';
 }
 return '#475569';
};

return (
 <div className="space-y-12">

 {/* 3-in-1 MST Simulator Box */}
 <div className="bg-slate-900/90 border border-slate-800" >
 <div className="flex flex-col lg:flex-row justify-between" >
 <div>
 <div className="flex items-center gap-2 mb-2" >

 Сунер-Интерактив 3 в 1

 <h3 className="text-2xl font-bold text-white" >
 MST
 </h3>
 </div>
 <p className="text-slate-400 text-sm" >
 Наблюдай за работой Краскала (Раскраска),
 </p>
 </div>

 {/* Algorithm Tabs */}
 <div className="flex flex-wrap items-center gap-2" >
 <div className="flex items-center bg-slate-800/50" >
 <button
 onClick={() => handleReset('kruskal')}
 className={
 "flex items-center gap-1.5 px-3 py-2"
 +
 (activeAlgo === 'kruskal'
 ? 'bg-indigo-600 text-white shadow-md'
 : 'text-slate-400 hover:text-slate-300')
 }
 >
 <GitGraph className="w-3.5 h-3.5" />
 </button>
 </div>
 </div>
 </div>
 </div>
 </div>
);

```

```

 <button
 onClick={handleNextStep}
 disabled={stepIndex >= currentSteps.length}
 className={
 "flex items-center justify-center gap-2
initial cursor-pointer " +
 (stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border
: activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500
: activeAlgo === 'prim'
 ? "bg-emerald-600 hover:bg-emerald-500
: "bg-rose-600 hover:bg-rose-500 text
)
 >
 <SkipForward className="w-4 h-4" />
 {stepIndex === 0 ? 'Начать' : stepIndex}
 </button>
 </div>
 </div>

 /* Что компилятор сообщил графу прямо сейчас */
 <div className="mb-6 flex flex-wrap items-center" >

 <Terminal className="w-3.5 h-3.5" /> из комп

 {live ? (
 <>
 {live.curU !== null && live.curV !== null && (

 ребро {String.fromCharCode(65 + live.curU)}

) : ''}
 </div>
) : ''}
 {live.takenEdges.length > 0 && (

 в осто́ве: {live.takenEdges.map((e) =>

```

```

<div className="grid grid-cols-1 lg:grid-cols-3
 {/* Graph Canvas */}
 <div className="lg:col-span-2 bg-slate-950 ro
 active min-h-[480px] overflow-hidden">

 {/* Legend */}
 <div className="absolute top-4 left-4 bg-sl
 -center gap-3 text-xs">
 <div className="flex items-center gap-1.5
 <div className="w-3 h-3 rounded-full bg
 </div>
 {activeAlgo === 'kruskal' ? (
 <
 <div className="flex items-center gap
 <div className="w-3 h-3 rounded-ful
 </div>
 <div className="flex items-center gap
 <div className="w-3 h-3 rounded-ful
 </div>
 <div className="flex items-center gap
 <div className="w-3 h-3 rounded-ful
 </div>
 </>
) : activeAlgo === 'prim' ? (
 <
 <div className="flex items-center gap
 <div className="w-3 h-1 bg-sky-600
 </div>
 <div className="flex items-center gap
 <div className="w-3 h-3 rounded-ful
 </div>
 </>
) : (
 <
 <div className="flex items-center gap
 <div className="w-3 h-3 rounded-ful
 </div>

```

```

 <text
 x={{u.x + v.x} / 2 + "%"}
 y={{u.y + v.y} / 2 + "%"}
 fill={isInspecting ? '#f59e0b' :
: '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
 }}}
</svg>

{/* HTML Overlay for Vertices */}
<div className="absolute inset-0 w-full h-f
 {effVertices.map(vertex => {
 <div
 key={vertex.id}
 style={{ top: vertex.y + "%", left: v
 className={"absolute -translate-x-1/2
font-bold text-base border-2 transition-all
id)}}
 >
 {vertex.label}
 {vertex.id === 4 && activeAlgo === 'p
 <span className="text-[8px] block -
 >
 </div>
)}}
</div>
</div>

```

```

 </div>
 </div>
 })

 <div className="flex-1 p-4 overflow-y-auto"
 {logs.map((log, idx) => {
 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-a
 {log.type === 'success'
 ? 'bg-emerald-950/40 border-emera
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-5
 : log.type === 'error'
 ? 'bg-rose-950/40 border-rose-500
 : 'bg-slate-900/60 border-slate-7
 }
 >
 <div className="flex items-center gap
 {log.type === 'success' && <CheckCi
 {log.type === 'warning' && <HelpCir
 {log.type === 'error' && <XCircle c
 {log.type === 'info' && <Play class
 <h5 className="font-bold text-sm le
 </div>
 <p className="text-xs text-slate-300
 </div>
 })})
 </div>
 </div>
 </div>
</div>

```

```

/* Cheat Sheet: Complexity & Code for all 3 algo
<div className="bg-slate-900/80 border border-sla
 <div className="flex flex-col sm:flex-row items
 <div className="flex items-center gap-2.5">

```



```

 </div>
 <div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs for
 <Award className="w-3.5 h-3.5 text-in
 </p>
 <p className="text-2xl font-black text-r
 <p className="text-slate-400 text-xs mt
 </div>
 <div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs for
ожении:</p>
 <p className="text-xs text-slate-300 le
ди и красим вершины в один цвет, бракуя цик
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-1
 <p className="text-slate-400 text-xs for
 <Code className="w-3.5 h-3.5 text-ind
 </p>
 <pre className="text-xs text-emerald-40
80 flex-1">

```

```
{`# 1. Инициализируем DSU
```

```
def find(i):
 if parent[i] == i: return i
 parent[i] = find(parent[i])
 return parent[i]
```

```
def union(i, j):
 r_i, r_j = find(i), find(j)
 if r_i != r_j:
 parent[r_i] = r_j
 return True
 return False
```

```
2. Жадный выбор ребер
edges.sort() # O(E log E)
mst = []
```

```

<pre className="text-xs text-emerald-400" style="font-size: 0.8em; flex: 1">
{`import heapq

def prim(graph, start):
 visited = [False] * len(graph)
 heap = [(0, start, -1)] # (weight, node, parent)
 mst = []

 while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

 for next_w, v in graph[u]:
 if not visited[v]:
 heapq.heappush(heap, (next_w, v, u))
 return mst`}
</pre>
</div>
</div>
</div>
})

{activeCheatTab === 'boruvka' && (
 <div className="grid grid-cols-1 lg:grid-cols-2" style="background-color: #f0f0f0; padding: 10px; border-radius: 10px; margin-bottom: 10px;">
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-3xl" style="background-color: #000000; color: #ffffff; padding: 10px; border-radius: 15px; margin-bottom: 10px;">
 <p className="text-slate-400 text-xs font-mono">
 <Clock className="w-3.5 h-3.5 text-rose-500" style="display: block; margin: 0 auto 10px auto; width: 1.5em; height: 1.5em; border: 1px solid #f0f0f0; border-radius: 50%; position: relative; transform: rotate(45deg); transform-origin: center; box-shadow: 0 0 0 2px #f0f0f0;"/>
 <p className="text-2xl font-black text-rose-500">Clock</p>
 <p className="text-slate-400 text-xs font-mono">
 mst = prim(graph, 0)
 </p>
 </p>
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl" style="background-color: #000000; color: #ffffff; padding: 10px; border-radius: 15px; margin-bottom: 10px;">
 <p className="text-slate-400 text-xs font-mono">
 mst = prim(graph, 0)
 </p>
 </div>
 </div>
 </div>
)

```

```
 }}
 </div>

 </div>
);
};
```

---

ФАЙЛ 57/111: src/components/LiteGuide.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useMemo } from "react";
import { chapters, chapterTopics } from "../data/content";
import { getViz } from "../vizRegistry";
import { PAGE_SYNC } from "../data/vizSync";
import { linkifyTicketsHtml } from "../utils/ticketLink";
import { quizzes } from "../data/quizzes";
```

```
/**
```

```
 * «Лёгкая» текстовая версия пособия.
```

```
 *
```

```
 * Открывается по адресу `?lite=1` (весь список тем) или
```

```
 * (одна тема). Здесь НЕТ навигации, сайдбара, интерактивных
```

```
 * панели компилятора – только сам текст тем, чтобы:
```

```
 * 1) любую страницу можно было распечатать/сохранить
```

```
 * 2) внешний парсер (ридер, «версия для слабовидящих»
```

```
 * LLM-индексатор) получал чистый текст без служебных
```

```
 *
```

```
 * Все спойлеры <details> раскрыты, визуализация описания
```

```
 * показывает, какие у неё вкладки и какие переменные к
```

```
 */
```

```
const LIGHT_CSS = `
```

```
 :root { color-scheme: light; }
```

```
 body { background: #ffffff !important; color: #111827;
```

```
 .lite-wrap { max-width: 46rem; margin: 0 auto; padding: 10px 0;
```

```

 .replace(/<button[\s\S]*?</button>/gi, "")
 .replace(/<svg[\s\S]*?</svg>/gi, "")
 // интерактивные подсказки терминов оставляем как есть
 .replace(/\\sdata-term-id="[^"]*" /g, "")
 .replace(/class="term-hint"[^"]*" /g, "")
 // ссылки на другие билеты должны вести в текстовую
 .replace(/href="\\?topic=/g, 'href="?lite=1&topic=')
 .replace(/class="ticket-link"/g, "");
}

```

```

interface Props {
 /** id одной темы; undefined – все темы подряд. */
 topicId?: string;
}

```

```

export const LiteGuide: React.FC<Props> = ({ topicId }) => {
 const list = useMemo(() => {
 if (!topicId) return chapters;
 const one = chapters.filter((c) => c.id === topicId);
 return one.length ? one : chapters;
 }, [topicId]);
}

```

```

return (
 <div className="lite-wrap">
 <style dangerouslySetInnerHTML={{ __html: LIGHT_GUIDE }} />

 <h1>Универсальное пособие: алгоритмы и структуры</h1>
 <p className="lite-meta">
 Билеты 1–24 • текстовая версия (без навигации,
 {topicId ? `одна тема: ${list[0]?.title} ?? topicId` : ""}
 полная интерактивная версия
 </p>

```

```

 {!topicId && (
 <nav className="lite-toc">
 <h3>Содержание</h3>

 {list.map((c) => (

```

```

 <p>
 {demo ? `Вкладка «${demo}»`
 {sync.vizTitle ? ` – ${sync.vizTitle}`
 {sync.stepNote ? <> Шаг демонстрации
 </p>
 {sync.variables.length > 0 && (

 {sync.variables.map((v) => (
 <li key={v.name}>
 <code>{v.name}</code> – {v.value}
 {v.range ? ` (${v.range})`

))}

)}
 </div>
);
}})
<p className="lite-meta" style={{ margin: 0; padding: 0; }}>
 В интерактивной версии эта демонстрация
 переменных перерисовывают её напрямую
</p>
</div>
)}

{quizzes[chapter.id] ?? []}.length > 0 &&
 <div className="lite-quiz">
 <h3>Блиц: вопросы по теме</h3>

 {quizzes[chapter.id] ?? []}.map((q, i) => (
 <li key={i}>
 <p>
 {q.question}
 </p>

 {q.options.map((option, j) => (
 <li key={j}>
 {j === q.correctIndex ? Правильно : }

```

```
const fmt = (value: DebugValue | undefined): string =>
 if (value === undefined || value === null) return "-";
 if (typeof value === "number") {
 if (value === Number.POSITIVE_INFINITY) return "∞";
 if (value === Number.NEGATIVE_INFINITY) return "-∞";
 return String(value);
 }
 if (typeof value === "string") {
 const t = value.trim().toLowerCase();
 if (t === "inf" || t === "infinity") return "∞";
 if (t === "-inf" || t === "-infinity") return "-∞";
 return value;
 }
 return String(value);
};
```

/\*\*

\* Таблица «ваших» расстояний, когда имена вершин в коде  
 \*  
 \* Схема демонстрации нарисована на фиксированном наборе  
 \* S...F у Форда–Беллмана). Если в коде пользователя верши  
 \* {0...6, v1...v9}, картинка физически не может их показат  
 \* просто пропадали, и казалось, что визуализация живёт  
 \* выводятся рядом, а подсказка объясняет, как связать  
 \*/

```
export function LiveDistTable({ dist, prev, done, demoIds }) {
 if (!dist) return null;
```

```
 const keys = Object.keys(dist);
 if (keys.length === 0) return null;
```

```
 const demoSet = new Set(demoIds);
 const unknown = keys.filter((key) => !demoSet.has(key));
 // Все вершины совпали со схемой – таблица не нужна,
 if (unknown.length === 0) return null;
```

```
 const doneSet = new Set((done ?? []).map((value) => S
```

```

 +{keys.length - shown.length} ещё

) : null}
 </div>
 </div>
);
}
```

---

ФАЙЛ 59/111: src/components/MnemonicCards.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import dijkstraImg from '../assets/dijkstra-kind.jpg';
import bellmanImg from '../assets/bellman-ford-truck.jpg';
import floydImg from '../assets/floyd-network.jpg';

const cards = [
 {
 img: dijkstraImg,
 alt: 'Добрый робот Дейкстра',
 title: ' Добрый (Дейкстра)',
 desc: 'Быстрый, жадный, но работает только с',
 highlight: 'положительными',
 highlightColor: 'text-emerald-400',
 rest: 'весами. Подсунь отрицательное ребро – сломает',
 complexity: 'O((V+E) log V)',
 border: 'border-blue-500/30 hover:border-blue-400/60',
 titleColor: 'text-blue-400',
 badge: 'text-blue-400/70 bg-blue-950/40',
 },
 {
 img: bellmanImg,
 alt: 'Фура по болоту Форд-Беллман',
 title: ' Фура по Болоту (Ф-Б)',
```

```
 <div className={`mt-3 text-xs font-mono px-3`} >
 </div>
 </div>
)})
 </div>
);
}
```

---

ФАЙЛ 60/111: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useEffect, useState } from "react";
import { List, X } from "lucide-react";
import { Sidebar } from "../Sidebar";

interface Props {
 selectedChapterId: string;
 setSelectedChapterId: (id: string) => void;
 /** Заголовок текущей темы — показываем в кнопке, что */
 currentTitle: string;
 index: number;
 total: number;
}

/**
 * Мобильное содержание: липкая панель снизу-сверху + в
 * На десктопе не рендерится (там обычный сайдбар).
 */
export const MobileToc: React.FC<Props> = ({ selectedChapterId,
 const [open, setOpen] = useState(false);

 useEffect(() => {
 document.body.style.overflow = open ? "hidden" : "";
 return () => {
 document.body.style.overflow = "";
 };
 });
}
```



```
 className="p-2 -mr-2 text-slate-400 hover:cursor-pointer"
 aria-label="Заккрыть содержание"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 <div className="flex-1 min-h-0">
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
</div>
 </>
);
};
```

---

ФАЙЛ 61/111: src/components/Navbar.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useState } from 'react';
import { Sparkles, FileDown, FileText, Code2, Loader2, } from 'lucide-react';
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';

interface NavbarProps {
 /** Открыта ли боковая панель компилятора. */
 compilerOpen: boolean;
 onToggleCompiler: () => void;
}

/**
```

```

 </div>
</div>

<div className="flex items-center w-full lg:w-auto" >

 <BookOpen className="w-4 h-4" /> Учебное пособие

 <button
 id="tab-compiler-btn"
 onClick={onToggleCompiler}
 aria-pressed={compilerOpen}
 title="Боковая панель Python-компилятора (настройка страниц)"
 aria-label={compilerOpen ? "Скрыть панель Python-компилятора" : "Показать панель Python-компилятора"}
 className={`shrink-0 flex-1 lg:flex-none flex items-center justify-center transition-all duration-200 whitespace-nowrap ${
 compilerOpen
 ? 'bg-emerald-600 text-white shadow-lg'
 : 'text-slate-400 hover:text-white'
 }`}>
 >
 <Terminal className="w-4 h-4" /> Python
 </button>
 { /*
 Выгрузки: на телефоне они уезжали за край экрана, поэтому
 кнопку нельзя было ни найти, ни нажать («кнопка не видна на
 узком экране это одно меню «Скачать», а на широком экране
 кнопки в ряд.
 */}
 <details id="download-menu" className="relative" >
 <summary
 className="lg:hidden flex items-center justify-center gap-2 border-indigo-500/40 cursor-pointer select-none"
 title="PDF пособия, PDF с кодом проекта, PDF с исходным кодом"
 >
 <FileDown className="w-4 h-4" /> Скачать
 </summary>
 </details>

```

```
 <FileText className="w-4 h-4" /> TXT

 <button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center justify-center
 er:bg-amber-600/20 border border-amber-500/
 title="Скачать всё пособие одним автономным
 >
 {busy ? <Loader2 className="w-4 h-4 animate
 </button>
 </div>
 </details>
</div>
</div>
</header>
);
};
```

---

ФАЙЛ 62/111: src/components/PlanarColoringViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useCallback, useContext, useEffect, use
import { ChevronLeft, ChevronRight, Play, Pause, Rotate
import { PlanarityDemo } from "./PlanarityDemo";
import { VizChapterContext, emitVizDemo, useVizRuntime,
```

```
/**
```

```
 * Билет 7 – «Графы. Планарные. Покраска».
```

```
 *
```

```
 * Четыре вкладки:
```

```
 * 1. «Планарность» – K5 и K3,3 с подсвеченными пересече
```

```
 * 2. «Жадная покраска» – $\chi \leq \Delta + 1$: вершины красятся
```

```
 * достаётся наименьший цвет, не занятый соседями;
```

```

const g: Graph = { key, label: "Двудольный (6 вершин)" };
g.adj = buildAdj(g);
return g;
}

const edges: [number, number][] = [[0, 1], [0, 2], [1, 3], [1, 4], [2, 5], [3, 6], [4, 6], [5, 7], [6, 7]];
const pos = [
 { x: 70, y: 60 }, { x: 175, y: 40 }, { x: 150, y: 185 },
 { x: 250, y: 185 }, { x: 365, y: 110 }, { x: 350, y: 60 }
];

const g: Graph = { key, label: "Основной (8 вершин, Δ = 3)" };
g.adj = buildAdj(g);
return g;
}

function buildAdj(g: Graph): number[][] {
 const adj: number[][] = g.edges.reduce((acc: number[][], [u, v]) => {
 acc[u].push(v);
 acc[v].push(u);
 return acc;
 }, Array.from({ length: g.n }, () => [] as number[]));
 adj.forEach(a => a.sort((x, y) => x - y));
 return adj;
}

interface Frame {
 title: string;
 note: string;
 color: number[]; // 0 = не покрашена
 ecolord?: number[]; // 0 = не покрашено
 v: number | null;
 u: number | null;
 e: number | null;
 used?: number[];
 bad?: [number, number] | null;
}

/** Жадная покраска вершин: $\chi \leq \Delta + 1$. */

```

```

const color = new Array(g.n).fill(0);
const queue: number[] = [];
let conflict: [number, number] | null = null;
frames.push({
 title: "Старт: пробуем покрасить в 2 цвета",
 note: "BFS красит вершину в цвет, противоположный р-
 двудольный ($\chi > 2$).",
 color: [...color], v: null, u: null, e: null,
});
for (let start = 0; start < g.n && !conflict; start++)
 if (color[start] !== 0) continue;
 color[start] = 1;
 queue.push(start);
 frames.push({
 title: `Запускаем BFS из ${g.names[start]}: цвет
 note: `Вершина ${g.names[start]} ещё не покрашена
 и 2.`,
 color: [...color], v: start, u: null, e: null,
 });
while (queue.length && !conflict) {
 const v = queue.shift() as number;
 for (const u of g.adj[v]) {
 if (color[u] === 0) {
 color[u] = 3 - color[v];
 queue.push(u);
 frames.push({
 title: `${g.names[v]} → ${g.names[u]}: цвет
 note: `Сосед не покрашен: color[${g.names[u]
 x}).join(", ") || "пуста"}].`,
 color: [...color], v, u, e: null,
 });
 } else if (color[u] === color[v]) {
 conflict = [v, u];
 frames.push({
 title: `Конфликт: ${g.names[v]} и ${g.names
 note: `Ребро (${g.names[v]}, ${g.names[u]})
 покрасить: $\chi \geq 3$.`,
 color: [...color], v, u, e: null, bad: conf

```

```

frames.push({
 title: `Рёбро $\{g.names[a]\}, \{g.names[b]\}$: цве
 note: `В вершине $\{g.names[a]\}$ заняты цвета $\{g.names[b]\}$. Наименьший св
 да.`
 color: new Array(g.n).fill(0), ecolord: [...ecolor]
});
});
const chiE = Math.max(...ecolor);
frames.push({
 title: `Готово:  $\chi' = \chiE$  при  $\Delta = \Delta$ `,
 note:
 g.key === "bip"
 ? `Двудольный граф: $\chi' = \Delta = \Delta$ — это теор
 сс 1 по Визингу.`
 : ` $\chi' = \chiE$. Теорема Визинга: $\Delta \leq \chi' \leq \Delta + 1$
 $\chi' = \Delta + 1$ — классом 2 (например, K3 и любой
 color: new Array(g.n).fill(0), ecolord: [...ecolor],
});
return frames;
}

```

```

const TABS: { id: Tab; label: string; icon: React.React
 { id: "planar", label: "1 · Планарность: K5 и K3,3",
 { id: "greedy", label: "2 · Покраска вершин ($\chi \leq \Delta + 1$)",
 { id: "bipartite", label: "3 · Двудольность (2 краски)",
 { id: "edges", label: "4 · Покраска рёбер (Визинг, Кё
];

```

```

export const PlanarColoringViz: React.FC = () => {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const [tab, setTab] = useState<Tab>("greedy");
 const [graphKey, setGraphKey] = useState<GraphKey>("m
 const [step, setStep] = useState(0);
 const [playing, setPlaying] = useState(false);

```

```

const liveColor = vars ? vizArray(vars.color) ?? vizA
const liveEcolor = vars ? vizArray(vars.ecolor) ?? vi
const liveV = vars ? vizNumber(vars.v) : null;
const liveU = vars ? vizNumber(vars.u) : null;
const liveE = vars ? vizNumber(vars.e) : null;
const liveChi = vars ? vizNumber(vars.chi) ?? vizNumb
const linked = Boolean(liveColor || liveEcolor || (li

const frame = frames[Math.min(step, maxStep)] ?? ({ t
const nodeColor = (id: number): number => {
 if (liveColor && typeof liveColor[id] === "number")
 return frame.color?.[id] ?? 0;
};
const edgeColor = (i: number): number => {
 if (liveEcolor && typeof liveEcolor[i] === "number"
 return frame.ecolor?.[i] ?? 0;
};
const curV = liveV !== null && Number.isFinite(liveV)
const curU = liveU !== null && Number.isFinite(liveU)
const curE = liveE !== null && Number.isFinite(liveE)
const delta = Math.max(...graph.adj.map((a) => a.leng
const usedColors = Array.from(new Set(graph.pos.map((
const usedEcolors = Array.from(new Set(graph.edges.ma

const go = useCallback((d: number) => {
 setPlaying(false);
 setStep((s) => Math.max(0, Math.min(maxStep, s + d)
}, [maxStep]);

if (tab === "planar") {
 return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded
 <Header />
 <Tabs tab={tab} setTab={setTab} />
 <PlanarityDemo />
 <p className="text-[11px] text-slate-500 mt-2">
 Формула Эйлера $V - E + F = 2$ и оценка $E \leq 3V$

```

```

 <span className="bg-slate-950 border border-s
 {tab === "edges"
 ? <span className="bg-slate-950 border bord
 rs}
 : <span className="bg-slate-950 border bord
 }}

 </div>

 <div className="grid grid-cols-1 lg:grid-cols-5 g
 <div className="lg:col-span-3 bg-slate-950 round
 <svg viewBox="0 0 480 300" className="w-full l
 {graph.edges.map(([a, b], i) => {
 const ec = edgeColor(i);
 const isCur = i === curE;
 const isBad = Boolean(frame.bad && ((frame
)))
 return (
 <g key={i}>
 <line
 x1={graph.pos[a].x} y1={graph.pos[a
 stroke={isBad ? "#f43f5e" : ec} > 0
 strokeWidth={isCur ? 6 : isBad ? 5
 strokeDasharray={ec === 0 && !isBad
 className="transition-all duration-
 opacity={isCur || isBad ? 1 : 0.85}
 />
 {tab === "edges" && ec > 0 && (
 <text x={{graph.pos[a].x + graph.po
 ={{colorOf(ec)}} fontSize="10" fontWeight="bo
 }}
 </g>
);
 }}}
 {graph.pos.map((p, id) => {
 const c = nodeColor(id);
 const isCur = id === curV;
 const isNbr = id === curU;

```



```

 /DFS за $O(V + E)$. Распознавание 3-раскрашива
 {tab === "edges" && <>Рёберная покраска: <b
 Δ (Кёниг). Задача « $\chi' = \Delta?$ » NP-полна даже
 </div>
 </div>
</div>

<div className="mt-3 flex flex-wrap items-center" >
 <span className="text-[10px] uppercase tracking
 /> из компилятора:
 {vars && linked ? (
 ["v", "u", "e", "color", "ecolor", "chi"].map
 vars[name] === undefined ? null : (
 <span key={name} className="text-[11px] fo
 y-1">
 {name} = {JSON.stringify(vars[name]).sl

)
) : (

 открой панель Python: массив <code classNam
 igo-300">ecolor</code> для рёбер) и вершина

)}
</div>
</div>
);
};

```

```

const Header: React.FC = () => {
 <div className="flex flex-wrap items-center justify-b
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-fuchsia-600 text-white r
 <Palette className="w-5 h-5" />
 </div>
 <div>
 <h3 className="text-lg sm:text-xl font-bold tex

```

```

<div className="absolute top-2 left-2 bg-slate-
 00/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
<svg className="w-full h-full absolute inset-0"
 {(() => {
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.f
 function intersect(a:number,b:number,c:nu
 if (a===c||a===d||b===c||b===d) return
 const p1=findPt(a),p2=findPt(b),p3=find
 function ccw(ax:number,ay:number,bx:nu
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)
 }
 const crosses: number[] = [];
 for(let i=0;i<edges.length;i++) {
 for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1]
 if (!crosses.includes(i)) crosses.p
 if (!crosses.includes(j)) crosses.p
 }
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {

```

```

 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 if (!p1||!p2||!p3||!p4) return false;
 function ccw(ax:number,ay:number,bx:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
 }
 const crosses:number[]=[];
 for(let i=0;i<edges.length;i++) for(let j=0;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x} y1={p1.y} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width=2 />);
 }
 const labels = ["\u04141", "\u04142", "\u04143"];
 for(const p of pts) {
 circles.push(<g key={`c${p.id}`}>
 <circle cx={p.x} cy={p.y} r={8} fill="#4f46e5" />
 <text x={p.x} y={p.y+3} textAnchor="middle">{p.id}</text>
 </g>);
 }
 return <>{lines}{circles}</>;
 })()
</svg>
<div className="absolute bottom-2 left-2 right-2 top-2" style={z-20}>
 V=6, E=9 {'>'} 2V-4 = 8 (двудольный) {'-->'}
</div>
</div>

```

```

 SkipForward,
 Terminal,
 Unlink,
 Variable,
 X,
} from "lucide-react";
import { buildInitTemplate, buildSyncTemplate, getPageS
import { clearVizState, emitVizState, onVizDemo } from
import {
 baseVarName,
 buildDebugRunner,
 changedVars,
 DEBUG_STEP_CAP,
 orderVars,
 type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "./Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
const PLAYBACK_SPEED_KEY = "pythonCompilerPlaybackSpeed
const PLAYBACK_SPEEDS = [0.25, 0.5, 1, 1.5, 2, 4] as co

interface PythonCompilerProps {
 /** Страница, с которой синхронизируется редактор. */
 chapterId: string;
 chapterTitle: string;
 /** Размер панели (px) управляется родителем и сохран
 width?: number;
 height?: number | null;
 onWidthChange?: (w: number) => void;
 onHeightChange?: (h: number) => void;
 /** Перейти к странице пособия (посмотреть визуализац
 /** Кнопка «открыть страницу в пособии» – не нужна, е
 onOpenGuide?: () => void;
 /** Скрыть панель. */
 onClose: () => void;

```

```
function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}
```

```
/**
```

```
 * Предел вывода в панель «Результат».
```

```
 *
```

```
 * Трасса ограничена по ЧИСЛУ шагов (DEBUG_STEP_CAP), не
```

```
 * напечатанной строки: print("x" * 10_000_000) или print
```

```
 * – это один шаг, после которого весь текст уезжал в <
```

```
 * Обрезаем по границе строки и честно пишем, сколько э
```

```
 */
```

```
const OUTPUT_CHAR_CAP = 20_000;
```

```
function clipOutput(text: string): string {
 if (text.length <= OUTPUT_CHAR_CAP) return text;
 const cut = text.slice(0, OUTPUT_CHAR_CAP);
 const lastBreak = cut.lastIndexOf("\n");
 const head = lastBreak > OUTPUT_CHAR_CAP / 2 ? cut.sl
 return `${head}\n... вывод обрезан: показаны первые ${h
}
```

```
/*
```

```
/* Шаблон синхронизации: короткая шапка + минимальный
```

```
/* скелет, шаг-в-шаг с подсветкой визуализации (см. da
```

```
/*
```

```
/*
```

```
/* Шпаргалка Python: команды сортировки, циклы, структ
```

```
/* «Вставить» кладёт сниппет в редактор под курсор.
```

```
/*
```

```
interface Snippet {
```

```
 label: string;
```

```
 tip: string;
```

```
 code: string;
```

```

],
},
{
 group: "Структуры",
 items: [
 {
 label: "Стек (список)",
 tip: "LIFO, как рекурсия DFS: append = push, pop = push",
 code: 'st = []\nst.append("A") # push\nst.append("A")\nst.pop() # pop\nprint(st)\n',
 },
 {
 label: "Очередь (deque)",
 tip: "FIFO для BFS: popleft() за O(1). list.pop(0) за O(n).",
 code: 'from collections import deque\nq = deque()\nq.append("A")\nq.append("B")\nq.popleft()\nprint(q)\n',
 },
 {
 label: "Куча (heapq)",
 tip: "Мин-куча: (вес, вершина) – как очередь приоритетов",
 code: 'import heapq\nh = []\nheapq.heappush(h, (1, "A"))\nheapq.heappush(h, (2, "B"))\nheapq.heappush(h, (3, "C"))\nheapq.heappop(h)\nprint(h)\n',
 },
 {
 label: "Матрица n × m",
 tip: "Таблица расстояний: dist[i][j], бесконечность float('inf')",
 code: 'n, m = 3, 4\ndist = [[float("inf")] * m for _ in range(n)]\n',
 },
 {
 label: "Множество visited",
 tip: "Посещённые вершины: in/add за O(1).",
 code: 'visited = set()\nvisited.add("A")\nprint(visited)\n',
 },
],
},
{
 group: "Вывод",
 items: [
 {

```

```

const initTemplate = useMemo(() => buildInitTemplate(

const [code, setCode] = useState(() => {
 const saved = savedEditors.get(chapterId);
 if (!saved) return initTemplate;
 if (saved.code === saved.base || saved.code.trim()
 return saved.code;
});
/** «Глаз» – переключатель: эталонный код демо ↔ иници
const [refOn, setRefOn] = useState(() => {
 const saved = savedEditors.get(chapterId);
 return !!saved && saved.code.trim() !== "" && saved
});
const [stdin, setStdin] = useState("");
const [output, setOutput] = useState("Код выполнится .
const [running, setRunning] = useState(false);
const [runtimeReady, setRuntimeReady] = useState(false);
const [runtimeMessage, setRuntimeMessage] = useState(
/** Принудительный повтор (глаз / Ctrl+Enter), даже е
const [traceRequest, setTraceRequest] = useState(0);
const [copied, setCopied] = useState(false);
const [stripTab, setStripTab] = useState<"vars" | "hi
const [outputOpen, setOutputOpen] = useState(true);
const editorRef = useRef<HTMLTextAreaElement>(null);
/**
 * Панель жива? Заккрытие (крестик, «глаз» в шапке, см
 * компонент, но `runPythonAsync` к тому моменту може
 * этого флага доехавший до конца прогон опубликовал с
 * после закрытия: панель исчезла, а визуализация про
 * кода, которого на экране больше нет.
 */
const aliveRef = useRef(true);
useEffect(() => {
 aliveRef.current = true;
 return () => {
 aliveRef.current = false;
 };
}, []);

```

```

 setPlaying(false);
 setEditHold(null);
 setDebug(null); // трасса ссылается на строки старого
 setRefOn(false);
 if (code === prev.base || code === prev.full || code === prev.strip) {
 setCode(initTemplate);
 setDesyncedFrom(null);
 } else {
 setDesyncedFrom(chapterId);
 }
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, [initTemplate, template]);

const resync = useCallback(() => {
 prevTplRef.current = { base: initTemplate, full: template, strip: strip };
 setCode(initTemplate);
 setRefOn(false);
 setDesyncedFrom(null);
 setStripTab("vars");
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 setTraceRequest((n) => n + 1);
}, [initTemplate, template]);

const copyCode = useCallback(async () => {
 try {
 await navigator.clipboard.writeText(code);
 setCopied(true);
 setTimeout(() => setCopied(false), 1600);
 } catch {
 setRuntimeMessage("Не получилось скопировать — буфер переполнен");
 }
}, [code]);

```

/\* — Пошаговый отладчик

/\*\* Связь больше не зависит от эталонных строк: совпадения



```

 });
 }, []));

const publishDebug = useCallback(
 (result: DebugResult, idx: number, source: string) => {
 if (!aliveRef.current) return; // панель закрыта
 if (!hasViz || result.steps.length === 0) return;
 const snapshot = snapshotAt(result, idx, source);
 emitVizState(chapterId, {
 line: snapshot.line,
 func: snapshot.func,
 variables: snapshot.variables,
 changed: snapshot.changed,
 });
 },
 [chapterId, hasViz, snapshotAt]
);

// Единственная точка публикации: любое перемещение т
// визуализации согласованный снимок, без side-effect
useEffect(() => {
 if (debug) publishDebug(debug.result, debug.idx, de
}, [debug, publishDebug]);

const goDebug = useCallback(
 (nextIdx: number) => {
 setDebug((d) => {
 if (!d || d.result.steps.length === 0) return d
 const idx = Math.max(0, Math.min(d.result.steps
 return { ...d, idx };
 });
 },
 []
);

const stepBy = useCallback(
 (delta: number) => {
 setDebug((d) => {

```

```

clearVizState(chapterId);
setOutputOpen(false);

// Пустой редактор: Pyodide не гоняем, панель в реж
// связь с визуализацией снимаем — демо показывает
if (source.trim() === "") {
 setDebug({ result: { steps: [], error: "", trunca
 setOutput("В редакторе пусто — выполнять нечего.\n
 «глаз».");
 setRuntimeMessage("Пустой код: связь с визуализаци
 setOutputOpen(true);
 setRunning(false);
 return;
}

setRuntimeMessage(runtimeReady ? "Обновляю трассу..."

const stdout: string[] = [];
const stderr: string[] = [];
const inputLines = input.split(/\r?\n/);

try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 runtime.setStdout({ batched: (message) => stdout.p
 runtime.setStderr({ batched: (message) => stderr.p
 runtime.setStdin({ stdin: () => (inputLines.length

 const raw = await runtime.runPythonAsync(buildDeb
 // Панель могли закрыть, пока Python считал: выхо
 if (!aliveRef.current) return;
 const result = JSON.parse(String(raw)) as DebugRe
 const printed = clipOutput([...stdout, ...stderr]

 if (result.steps.length === 0) {
 // Шагов нет (только комментарии, синтаксическа
 // заменяется пустым листингом, а визуализация
 // показывает собственный встроенный сценарий (

```

```

 } finally {
 setRunning(false);
 }
}, [chapterId, code, stdin, traceRequest, running, run

// Код выполняется сам: debounce даёт спокойно допечать
// изменился во время загрузки Pyodide, после завершения
useEffect(() => {
 if (running || editHold === code) return;
 if (debug?.src === code && debug.input === stdin &&
 const timer = window.setTimeout(() => void debugRun
 return () => window.clearTimeout(timer);
}, [code, stdin, traceRequest, running, runtimeReady,

// Стрелки ← → в режиме отладки листают ШАГИ (а не стр
useEffect(() => {
 if (!debug || debug.result.steps.length === 0) return;
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target)
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying(false);
 stepBy(1);
 }
 if (e.key === "ArrowLeft") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying(false);
 stepBy(-1);
 }
 }
 if (e.key === " ") {
 e.stopPropagation();
 e.preventDefault();
 setPlaying((v) => !v);
 }
}

```

```

/** Одна и та же кнопка запуска всегда видна: из реда
 * проигрывает её, в отладчике переключает play/pause
const runOrTogglePlayback = useCallback(() => {
 if (running) return;
 if (!debug || debug.result.steps.length === 0) {
 void debugRun(true);
 return;
 }
 if (playing) {
 setPlaying(false);
 return;
 }
 if (debug.idx >= debug.result.steps.length - 1) step
 setPlaying(true);
}, [debug, debugRun, playing, running, stepTo]);

/** Вставка сниппета из шпаргалки в позицию курсора.
const insertSnippet = useCallback(({snippet: string} =
 const ta = editorRef.current;
 setCode((current) => {
 if (!ta) return current + (current.endsWith("\n")
 const start = ta.selectionStart ?? current.length
 const end = ta.selectionEnd ?? start;
 const padded = (start > 0 && !current.slice(0, st
 const next = current.slice(0, start) + padded + c
 requestAnimationFrame(() => {
 ta.focus();
 ta.selectionStart = ta.selectionEnd = start + p
 });
 return next;
 });
}, []);

return (
 <aside
 aria-label="Python-компилятор"
 className="fixed z-40 inset-x-0 bottom-0 h-[58dvh

```

```

 })

 { /* Левый край меняет ширину на десктопе. */
 {onWidthChange && (
 <div
 role="separator"
 aria-orientation="vertical"
 aria-label="Потяните, чтобы изменить ширину п
 title="Потяните, чтобы изменить ширину панели
 className="hidden lg:block absolute top-0 bot
 onPointerDown={(e) => {
 const el = e.currentTarget;
 el.setPointerCapture(e.pointerId);
 const startX = e.clientX;
 const startW = width ?? 560;
 const onMove = (ev: PointerEvent) => {
 const next = Math.round(startW + (startX
 onWidthChange(Math.max(380, Math.min(next
 });
 const onUp = () => {
 el.removeEventListener("pointermove", onM
 el.removeEventListener("pointerup", onUp
 };
 el.addEventListener("pointermove", onMove a
 el.addEventListener("pointerup", onUp as Ev
 })
 >
 <div className="absolute left-1/2 top-1/2 -tr
 digo-500 transition-colors" />
 </div>
 })
 } /* — Заголовок панели —————
 <div className="shrink-0 flex flex-wrap items-cen
 <span className={`w-2 h-2 rounded-full shrink-0
 <Terminal className="w-4 h-4 text-emerald-400 s
 <span className="text-[13px] font-bold text-whi
 <span className="hidden sm:inline text-[10px] t

```

```

 aria-label="Сбросить к шаблону страницы"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip side="bottom" content="Скрыть панель"
 <button
 type="button"
 onClick={onClose}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Закрыть компилятор"
 >
 <X className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip
 side="bottom"
 content={
 refOn
 ? "Убрать эталонный код: редактор станет
 собственному встроенному демо. Кнопка ↶ вернет
 : "Вставить полный эталонный код этого
 ,
 будет заменён."
 }
 >
 <button
 type="button"
 onClick={() => {
 const next = !refOn;
 setRefOn(next);
 // «Убрать эталон» – значит убрать код
 // выполняется, поэтому демонстрация бо
 // значений и честно показывает встроенн
 // странице скажет об этом явно).
 setCode(next ? template : "");
 setDesyncedFrom(null);
 setPlaying(false);
 setEditHold(null);
 }}
 >

```

```

/* — Мини-вкладки: переменные страницы / шпаргалка
<div className="shrink-0 border-b border-slate-800" >
 <div className="flex items-center gap-1 px-2 pt-2" >
 <button
 type="button"
 onClick={() => setStripTab("vars")}
 className={`inline-flex items-center gap-1.5
 stripTab === "vars" ? "bg-slate-800 text-white" : ""
 }` >
 >
 <Variable className="w-3.5 h-3.5" /> Переменные
 </button>
 <button
 type="button"
 onClick={() => setStripTab("hints")}
 className={`inline-flex items-center gap-1.5
 stripTab === "hints" ? "bg-slate-800 text-white" : ""
 }` >
 >
 <HelpCircle className="w-3.5 h-3.5" /> Подсказки
 </button>
 <div className="ml-auto pr-1" >
 <Tooltip
 side="bottom"
 content={
 hasViz
 ? "Связь идёт по именам ниже: реальный код
 а не сравнивается."
 : "На выбранной странице нет интерактивных элементов"
 } >
 >
 <span
 tabIndex={0}
 className={`cursor-help inline-flex items-center
 hasViz ? "text-emerald-400" : "text-slate-400"
 }` >
 >
 <Link2 className="w-3 h-3" />

```

```

 >
 <Unlink className="w-3.5 h-3.5 shrink
 Страница сменилась – подставить её пер
 </button>
 </Tooltip>
 })
 {hasViz && {
 <div className="flex flex-wrap items-centr
 {sync.variables.map((v) => {
 <Tooltip
 key={v.name}
 side="bottom"
 content={

 <span className="block font-bol
 <code className="font-mono te

 {v.role
 {v.range && <span className="bl

 }
 }
 >
 <code
 tabIndex={0}
 className="cursor-help rounded-md
font-bold text-emerald-300 hover:bg-emerald
 >
 {v.name}
 </code>
 </Tooltip>
 }}}
 {sync.stepNote && {
 <Tooltip side="bottom" content={`War
 <span
 tabIndex={0}
 className="cursor-help inline-fle
text-[10px] text-slate-400 hover:text-white
 >

```



```

 /* Заголовок редактора: сразу видно, что выпол
<div className="shrink-0 flex items-center gap-1
 <Pencil className="w-3.5 h-3.5 text-emerald-4
 <span className="text-[10px] font-bold upperc
 Редактор • выполняется этот текст

 {state && (
 <span
 className="inline-flex items-center gap-1
 ont-bold text-amber-300 shrink-0"
 title="Трасса ниже построена по предыду
 >
 <Loader2 className="w-2.5 h-2.5 animate-s

)}
 <span className="ml-auto text-[10px] text-sla
 {code.trim() === "" ? "пусто – ничего не вы

</div>

<textarea
 ref={editorRef}
 value={code}
 onChange={(event) => {
 setEditHold(null);
 setRefOn(false);
 setCode(event.target.value);
 }}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && even
 event.preventDefault();
 void debugRun();
 }
 event.stopPropagation();
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="flex-[2] min-h-[110px] w-full resi

```

```

abled={!curDebug?.step || debug.idx >= debug.result.steps.length - 1};
e hover:bg-slate-800 disabled:opacity-40 transition-colors duration-200
 <ChevronRight className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip content="В конец трассы">
 <button type="button" aria-label="В конец трассы" disabled={!curDebug?.step}
result.steps.length - 1); }> disabled={!curDebug?.step}
d text-slate-400 hover:text-white hover:bg-slate-800 transition-colors duration-200
 <SkipForward className="w-4 h-4" />
 </button>
</Tooltip>

 {curDebug?.step ? (
 <>
 шаг {debug.idx + 1}/{debug.result.steps.length}
 {curDebug.step.func !== "<module>" ? (
 </>
) : (
 "шагов нет — исправьте ошибку и начните трассировку"
)}

 {debug.result.truncated && (
 <Tooltip content={`Выполнение остановлено: ${debug.result.truncated} шагов`}

 лимит

 </Tooltip>
)}
 <Tooltip content={hasViz ? "Визуализация: старт трассы. Эталонный текст не обязателен — пишите, что хотите, но не используйте
нет имён, связанных с демонстрацией; Python" : ""}>
 <span
 tabIndex={0}
 className={`cursor-help inline-flex items-center gap-1 border-emerald-500/40 bg-emerald-500/10 text-emerald-500`}
 >

```

```

{debug.src.split("\n").map(({line, i}) => {
 const ln = i + 1;
 const active = ln === curDebug?.step?.line;
 return (
 <div
 key={i}
 data-line={ln}
 className={`flex whitespace-pre px-2
 active ? "bg-emerald-500/15 text-emerald-500" : ""
 }`
 >
 <span
 className={`w-7 shrink-0 select-none
 active ? "text-emerald-300" : "text-emerald-500"
 }`
 >
 {active ? "▶" : ln}

 {line || " "}
 </div>
);
})}
</div>

```

```

{/* панель переменных – «Variables», как в VS Code */}
<div className="shrink-0 max-h-[34%] overflow-hidden" >
 <div className="flex items-center gap-1.5" >
 <Variable className="w-3 h-3 text-indigo-500" />
 Переменные на этом шаге

 </div>
 {debug.result.error && (
 <pre className="mt-1 whitespace-pre-wrap" />
)}
 {debugOrdered.length === 0 && !debug.result.error && (
 <div className="mt-1 text-[11px] text-slate-500" />
)}
</div>

```

```

 </label>
 <input
 id="python-stdin"
 value={stdin}
 onChange={(event) => {
 setEditHold(null);
 setStdin(event.target.value);
 }}
 spellCheck={false}
 placeholder="строки ввода через ↵"
 className="w-full bg-transparent font-mono"
 />
 </div>
</div>

{/* — Вывод ————— */}
<div className="shrink-0 border-t border-slate-800" >
 <button
 type="button"
 onClick={() => setOutputOpen((o) => !o)}
 className="flex items-center justify-between gap-2"
 >

 <Terminal className="w-3.5 h-3.5 text-emerald-500" />

 {runtimeReady && <CheckCircle2 className="w-3 h-3 text-emerald-500" />}
 {outputOpen ? "Закрыть" : "Открыть"}

 </button>
 {outputOpen && (
 <pre
 data-testid="compiler-output"
 className="flex-1 min-h-[70px] overflow-auto text-slate-300"
 >
 {output}
 </pre>
)}
</div>

```

```

 { name: 'Бизнесмен', emoji: ' ', color: 'from-slate-500 to-emerald-500',
 tartan: ' ',
 },
];
```

```
let counter = 3;
```

```
export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: ' ', color: 'from-slate-500 to-emerald-500',
 bubbleText: 'голода после матана!', animState: 'idle' },
 { id: 'q2', name: 'Кодер Семён', emoji: ' ', color: 'from-slate-500 to-emerald-500',
 bubbleText: 'ищу ИИ за порцию борща!', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: ' ', color: 'from-slate-500 to-emerald-500',
 bubbleText: 'ез сметаны, плиз.', animState: 'idle' },
]);
```

```
 const runtime = useVizRuntime();
 const liveQ = vizArray(runtime?.variables?.q);
 const liveV = vizString(runtime?.variables?.v);
 const displayQueue: Customer[] = liveQ
 ? liveQ.map((value, index) => ({
 id: `python-${index}`,
 name: String(value),
 emoji: " ",
 color: "from-emerald-500/20 to-emerald-600/20 bubbleText: `q[${index}] из Python`,
 animState: "idle",
 }))
 : queue;
```

```
 const [servedHistory, setServedHistory] = useState<string[]>([]);
 const [isServing, setIsServing] = useState(false);
 const [shakeEmpty, setShake] = useState(false);
 const [log, setLog] = useState<string>('');
```

```
 const addLog = (msg: string) => setLog(prev => [msg, ...prev]);
```

```
 // ENQUEUE: Встать в конец очереди (хвост / Tail)
```

```

addLog(`  DEQUEUE: Повар наливает суп первому – ${headGuest.name}`);

// Step 1: Head guest gets eating animation
setQueue(prev => prev.map((c, idx) => idx === 0 ? {
 ...c, animState: 'in'
} : c));

setTimeout(() => {
 // Step 2: Guest leaves with happy face, queue shifts
 setQueue(prev => prev.map((c, idx) => idx === 0 ? {
 ...c, emoji: '😄'
 } : c));

 setServedHistory(prev => [`${headGuest.emoji} ${headGuest.name} поел и ушел сытым.`]);

 setTimeout(() => {
 setQueue(prev => prev.slice(1));
 setIsServing(false);
 addLog(`  ${headGuest.name} поел и ушел сытым.`, 400);
 }, 900);
}, [queue, isServing]);

const reset = () => {
 counter = 3;
 setQueue([
 { id: 'r1', name: 'Студент Вася', emoji: '😞', color: 'red', col: 1, animState: 'idle', text: 'У меня нет денег, я голода после матана!', animState: 'in' },
 { id: 'r2', name: 'Кодер Семён', emoji: '😞', color: 'blue', col: 2, animState: 'idle', text: 'Напишу ИИ за порцию борща!', animState: 'in' },
 { id: 'r3', name: 'Кот Борис', emoji: '😞', color: 'green', col: 3, animState: 'idle', text: 'Дайте котика без сметаны, плиз.', animState: 'in' },
]);
 setServedHistory([]);
 setIsServing(false);
 setLog([' Столовая сброшена. Снова голодная толпа!']);
 setTimeout(() => {
 setQueue(prev => prev.map(c => ({ ...c, animState: 'idle' })), 500);
 });
};

```

```

 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800
 r-pointer border border-slate-700"
 >
 Сброс
 </button>
 </div>

```

```

 { /* ИНТЕРАКТИВНАЯ КУХНЯ */ }
 <div className="grid grid-cols-1 md:grid-cols-12" >

```

```

 { /* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */ }
 <div className={`md:col-span-9 bg-slate-900 rounded-2xl
 -[300px] overflow-hidden ${shakeEmpty ? 'animate-shake' : ''}`} >
 <div className="absolute top-4 left-4 text-[1.2em] font-mono" >
 Очередь голодных гостей (Буфер FIFO / BFS)
 {liveQ && {liveQ.length}}
 </div>

```

```

 <div className="flex-1 flex flex-col justify-between" >
 <div className="flex items-end justify-between" >

```

```

 { /* ПОВАРИХА И КОТЕЛ С СУПОМ */ }
 <div className="flex flex-col items-center" >
 <div className="relative" >
 <div className="absolute -top-10 left-1/2 -translate-x-1/2" >
 <div className="w-1.5 h-6 bg-slate-400 rounded" >
 <div className="w-2 h-4 bg-slate-400 rounded" >

```

```

 <div className="absolute -bottom-2 text-3xl rotate-45"></div>
 </div>

 { /* Персонаж */ }
 <div className={`
 relative
 w-14 h-14 rounded-2xl border-1px
 ${customer.color}
 ${isHead ? 'ring-4 ring-amber-500' : ''}
 `}>
 {customer.name}

 { /* Индексы HEAD/TAIL */ }
 {(isHead || isTail) && (
 <span className={`
 absolute -bottom-2 text-3xl
 ${isHead ? 'bg-amber-500' : ''}
 `}>
 {isHead && isTail ? 'h+t' : isHead ? 'h' : 't'}

)}
 </div>
 </div>
 </div>
 </div>

 { /* Пол кухни */ }
 <div className="w-full h-3 bg-gradient-to-r from-slate-900 to-slate-800"></div>

 { /* ПРАВАЯ КОЛОНКА: СЫТЫЕ И ДОВОЛЬНЫЕ */ }
 <div className="md:col-span-3 bg-slate-900 rounded-2xl">

```



```
 <div className="bg-slate-900 border border-slate-800" >
 <div className="text-[10px] font-mono text-slate-100" >
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between`}>
 >
 {idx === 0 ? [{entry}] : {entry}}
 </div>
)
)}
 </div>
 </div>
);
};
```

---

ФАЙЛ 66/111: src/components/SalmonAutomatonWidget.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useEffect, useContext, useMemo } from 'react';
import { Terminal } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from 'viz';
import { Play, RotateCcw, Plus, Volume2, Info } from 'lucide-react';
```

```
interface TrieNode {
 id: number;
 char: string;
 parent: number;
 children: { [key: string]: number };
 fail: number;
 term_link: number;
 is_terminal: boolean;
 words: string[];
 depth: number;
 x?: number;
```

```

 if (!liveWordsSig) return;
 const parsed = (JSON.parse(liveWordsSig) as unknown
 .map((w) => (typeof w === "string" ? w : typeof w
 .filter((w) => Boolean(w))
 .map((w) => w.toUpperCase().replace(/[^A-ZА-Я]/g,
 .filter((w) => w.length > 0)
 .slice(0, 8);
 if (!parsed.length) return;
 if (parsed.join("|") === words.join("|")) return;
 setWords(parsed);
 buildInitialTrie(parsed);
 // eslint-disable-next-line react-hooks/exhaustive-
 }, [liveWordsSig]));

// состояние автомата и шаг BFS – подсвечиваем вершины
useEffect(() => {
 if (liveState !== null && liveState >= 0 && nodes[l
 if (liveStep !== null && liveStep >= 0) {
 // simulationStep – грубый статус: 0 = бор готов,
 const total = Object.keys(nodes).length;
 const failReady = liveFail ? Object.keys(liveFail
 setSimulationStep(liveStep <= 0 ? 0 : failReady ?
 }
 if (liveQueue) {
 const q = liveQueue.map((x) => (typeof x === "num
 efined);
 if (q.length) setBfsQueue(q);
 }
 // eslint-disable-next-line react-hooks/exhaustive-
}, [vars]);

const compilerLinked = Boolean(liveWordsSig || liveSt

// Регулярное моргание рыбы
useEffect(() => {
 const blinkInterval = setInterval(() => {
 setIsBlinking(true);
 setTimeout(() => setIsBlinking(false), 200);

```

```

 }
 if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
 }
 });

 let currentX = 0;
 const paddingX = 70;
 const paddingY = 80;

 const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {
 layoutDFS(v, depth + 1);
 if (leftX === -1) leftX = newNodes[v].x!;
 rightX = newNodes[v].x!;
 });
 newNodes[u].x = leftX === -1 ? 40 + currentX * paddingX : (leftX + rightX) / 2;
 if (leftX === -1) currentX++;
 }
 };

 layoutDFS(0, 0);

 setNodes(newNodes);
 setSimulationStep(0);
 setBfsQueue([]);
 setCurrentNode(null);
 setSpeechText(

```

```

 rootChildren.forEach(({childId}) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на графике

 уже суффикса просто нет. Жми «Следующий шаг»!';
);
 };

// Один шаг BFS
const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полный

 граф. Теперь можно полюбоваться таблицей переходов!';
);
 return;
 }

 const r = bfsQueue[0];
 const newQueue = bfsQueue.slice(1);
 const updatedNodes = { ...nodes };
 const rNode = updatedNodes[r];
 const childrenIds = Object.values(rNode.children);

 let logs = `Рассматриваем вершину #${r} (${rNode.children.length} детей)`;

 for (const [char, u] of Object.entries(rNode.children)) {
 newQueue.push(u);
 let v = rNode.fail;
 while (v !== 0 && updatedNodes[v].children[char] != null) {
 v = updatedNodes[v].fail;
 }
 updatedNodes[rNode.fail] = { children: {} };
 updatedNodes[rNode.fail].children[char] = u;
 logs += `Добавляем ребро ${r} -> ${u} с суффиксом ${char}`;
 }

 setBfsQueue(newQueue);
 setSimulationStep(1);
 setCurrentNode(r);
 setSpeechText(logs);
};

```

```

 </div>
 </div>
 <div className="flex items-center gap-2">
 <button
 onClick={() => buildInitialTrie(words)}
 className="flex items-center gap-1 bg-slate
 transition-colors"
 title="Сбросить автомат"
 >
 <RotateCcw className="w-4 h-4" /> Сбросить
 </button>
 </div>
</div>

{/* Верхняя панель: Говорящий лосось и диалоговое
<div className="grid grid-cols-1 md:grid-cols-3 g
 {/* Аватар Лосося (Сеня) с анимацией моргания и
 <div className="flex flex-col items-center just
 <div className="w-40 h-40 relative flex items
 full border border-blue-500/30 shadow-inner
 {/* SVG Лосося */}
 <svg
 viewBox="0 0 200 200"
 className={`w-36 h-36 transition-transform
 isTalking ? 'scale-105 drop-shadow-[0_0_
]`}
 >
 {/* Тело лосося */}
 <ellipse cx="100" cy="100" rx="70" ry="45"
 <path d="M 40 85 Q 100 55 160 85 Q 100 115
 40 85"
 fill="#be123c"
 className="origin-[35px_100px] animate-
 />

```

```


 </div>

 {/* Пузырь с текстом (Баббл) */}
 <div className="md:col-span-2 bg-slate-800 border-2 border-t-transparent min-h-[140px]">
 {/* Треугольник баббла */}
 <div className="absolute -left-3 top-1/2 -transform rotate(-90deg) border-b-[12px] border-b-transparent border-l-[12px] border-l-transparent border-r-[12px] border-r-transparent border-t-[12px] border-t-transparent" />

 <div className="flex items-center space-x-2 text-white">
 <Volume2 className={`w-4 h-4 ${isTalking ? 'animate-pulse' : ''}`} />
 Прямое вещание из аквариума:
 </div>

 <p className="text-slate-200 text-base leading-relaxed">
 {speechText}
 </p>

 <div className="mt-4 pt-3 border-t border-slate-700">
 <div className="text-xs text-slate-400 flex justify-between">
 <Info className="w-3.5 h-3.5 text-blue-400" />
 Статус: {simulationStep === 0 ? 'Бор готов к старту!' : 'Бор занят!'}
 </div>

 <div className="flex gap-2">
 {simulationStep === 0 && (
 <button
 onClick={startBFS}
 className="bg-emerald-600 hover:bg-emerald-500 text-white px-3 py-1 shadow-lg shadow-emerald-900/40 transition" />
 <Play className="w-4 h-4" /> Запустить
 </button>
)}
 {simulationStep === 1 && (
 <button
 onClick={stopBFS}
 className="bg-rose-600 hover:bg-rose-500 text-white px-3 py-1 shadow-lg shadow-rose-900/40 transition" />
 <Stop className="w-4 h-4" /> Остановить
 </button>
)}
 </div>
 </div>
 </div>

```

```

return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`}
 <defs>
 <marker id="arrowhead" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 <marker id="fail-arrow" markerWidth="6"
 <polygon points="0 0, 6 3, 0 6" fill=
 </marker>
 </defs>

 {/* Draw Edges */}
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x ||

 const isCurrent = currentNode === node.

 return (
 <g key={`edge-${node.id}`}>
 <line
 x1={parent.x}
 y1={parent.y + 16}
 x2={node.x}
 y2={node.y - 16}
 stroke={isCurrent ? '#60a5fa' : '#c0c0c0'}
 strokeWidth="2"
 markerEnd="url(#arrowhead)"
 />
 <text
 x={(parent.x + node.x) / 2 - 10}
 y={(parent.y + node.y) / 2}
 fill="#94a3b8"
 fontSize="12"
 fontWeight="bold"
 >
 {node.char}

```

```

let fill = '#1e293b';
let stroke = '#475569';

if (isCurrent) {
 fill = '#2563eb';
 stroke = '#60a5fa';
} else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
} else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
}

return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 />
 <text
 x={node.x}
 y={node.y + 4}
 fill="white"
 fontSize={isRoot ? "10" : "12"}
 fontWeight="bold"
 textAnchor="middle"
 >
 {isRoot ? 'R' : node.char}
 </text>
 {!isRoot && (
 <text
 x={node.x + 12}

```



```

 }}}
 </div>
</div>

<form onSubmit={handleAddWord} className="flex flex-col gap-2" >
 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.value)}
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-slate-700 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-600 transition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
</form>
</div>

{/* Таблица Вершин и суффиксных ссылок */}
<div className="lg:col-span-2 bg-slate-900/50 p-4" >
 <h5 className="text-white font-bold mb-3 flex items-center" >

 Таблица переходов и fail-

 Всего вершин: {Object.keys(nodes).length}

 </h5>

 <div className="max-h-[220px] overflow-y-auto" >
 <table className="w-full text-left border-collapse" >
 <thead>

```

```

 #{node.id}
 {node.id === 0 && <span className=
</td>
<td className="py-2.5 px-3">
 <span className="bg-slate-800 b
 {node.char}

</td>
<td className="py-2.5 px-3 text-s
 {node.id === 0 ? '-' : `#${node
</td>
<td className="py-2.5 px-3">
 {node.id === 0 ? {
 <span className="text-slate-5
 } : {
 <span className="bg-indigo-95
 #{node.fail}

 }}
</td>
<td className="py-2.5 px-3">
 {node.id === 0 || node.term_lin
 <span className="text-slate-5
 } : {
 <span className="bg-rose-950 l
0 transition-colors cursor-help" title={`Ск
 #{node.term_link}

 }}
</td>
<td className="py-2.5 px-3 text-s
 {Object.keys(node.children).len
 ? 'ø'
 : Object.entries(node.children
 .map([c, id]) => `${c}→#
 .join(', '))}
</td>
<td className="py-2.5 px-3">

```

```

 nt-mono text-cyan-300">state</code>/<code c

 })
</div>
</div>
);
};
```

ФАЙЛ 67/111: src/components/SegmentTreeVisualizer.tsx  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useState, useEffect, useCallback, useMe
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRi
import { useVizRuntime, vizNumber } from '../data/vizSt
```

```
// ===== TYPES =====
interface Step {
 id: number;
 desc: string;
 codeLine: number;
 treeState: Record<number, number>; // node inde
 highlightNodes: number[]; // current
 mergeChildren?: [number, number]; // pair be
 arrayHighlight?: [number, number]; // [L, R]
 result?: number; // partial
}
```

```
// ===== PURE LOGIC: build tree =====
function buildTreeFull(arr: number[]): Record<number, n
 const n = arr.length;
 const tree: Record<number, number> = {};
 function go(v: number, l: number, r: number) {
 if (l === r) { tree[v] = arr[l]; return; }
 const m = (l + r) >> 1;
 go(2 * v, l, m);
```

```

const m = (l + r) >> 1;
steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}
 { ...tree }, highlightNodes: [v], arrayHighlight:
go(2 * v, l, m);
steps.push({ id: id++, desc: `Вернулись в build(${v}
 lightNodes: [v], arrayHighlight: [l, r] }));
go(2 * v + 1, m + 1, r);
tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
steps.push({ id: id++, desc: `Объединяем: tree[${v}]
 tate: { ...tree }, highlightNodes: [v], mergeChild
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: ` Дерево построено! Корень
 Highlight: [0, n - 1] }));
return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], ql: number, qr: number) {
 const n = arr.length;
 const tree = buildTreeFull(arr);
 const steps: Step[] = [];
 let id = 0;

 function go(v: number, l: number, r: number, ql: number, qr: number) {
 if (ql > r || qr < l) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]
 odeLine: 2, treeState: tree, highlightNodes: [v]
 return 0;
 }
 if (ql <= l && r <= qr) {
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]
] = ${tree[v]}`, codeLine: 4, treeState: tree, highlightNodes: [v]
 return tree[v];
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `query(${v}, [${l}..${r}]
 codeLine: 6, treeState: tree, highlightNodes: [v]

```

```

 }
 if (r & 1) {
 r--;
 res += tree[r] ?? 0;
 steps.push({ id: id++, desc: `r=${r + 1} нечётный`, treeState: tree, highlightNodes: [r], arrayHighlight: [qL, qR], result: res });
 highlights.push(r);
 }
 l >>= 1;
 r >>= 1;
 if (l < r) {
 steps.push({ id: id++, desc: `Поднимаемся: l = ${l}`, treeState: tree, highlightNodes: [l], arrayHighlight: [qL, qR], result: res });
 }
}
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}])`, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

```

// ===== TREE RENDERING HELPER =====

```
interface VNode { v: number; l: number; r: number; val: number; left: VNode; right: VNode; }
```

```

function buildVisualTree(treeState: Record<number, number>): VNode {
 if (l === r) return { v, l, r, val: treeState[v] ?? null, left: null, right: null };
 const m = (l + r) >> 1;
 return { v, l, r, val: treeState[v] ?? null, left: buildVisualTree(treeState, l, m), right: buildVisualTree(treeState, m + 1, r) };
}

```

// ===== CODE SNIPPETS =====

```

const BUILD_CODE = [
 "build(v, l, r) {",
 " if (l === r) { tree[v] = A[l]; return; }",
 " // -----",
 " // -----",
 " mid = (l + r) >> 1;",
 " build(2*v, l, mid);",
 "}"
]

```

```

 const t = setTimeout(() => setIdx(i => i + 1), speed);
 return () => clearTimeout(t);
 }, [playing, idx, steps, speed]);

const step = steps[idx] || steps[0];
if (!step) return null;

const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500 border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-emerald-500 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-900' }, amber: { btn: 'bg-amber-600 hover:bg-amber-500 border-amber-500 bg-amber-600', childRing: 'ring-rose-500' }, [color]: { btn: 'bg-slate-800 hover:bg-slate-700 border-slate-700 bg-slate-800', childRing: 'ring-slate-900' } };

return { step, idx, setIdx, playing, setPlaying, speed };
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr: number[] }) {
 | 'emerald' | 'amber'; arr: number[] }) {
 const p = Player({ steps, color });
 if (!p) return null;
 const { step, idx, setIdx, playing, setPlaying, speed } = usePlayer(p);
 const n = arr.length;

 const vTree = useMemo(() => buildVisualTree(step.tree, n));

 const renderNode = useCallback((nd: VNode): React.ReactNode => {
 const isHigh = step.highlightNodes.includes(nd.v);
 const isChild = step.mergeChildren?.includes(nd.v);
 const has = nd.val !== null;

 let cls = 'bg-slate-800 border-slate-700 text-slate-200';
 if (isHigh) cls = `${clr.ring} text-white ring-2 shadow-2xl`;
 else if (isChild) cls = `${clr.childRing} text-amber-500`;
 else if (has) cls = `${clr.filled} text-slate-200`;
 });

```

```

 er-500'} -translate-y-0.5` : 'bg-slate-900
 <span className="text-[7px] text-slate-500
 <span className="text-[11px] font-bold font
 </div>
 });
 })}
</div>

{/* Tree */}
<div className="flex-1 p-2 overflow-x-auto bg-slate-900
 <div className="scale-[0.85] origin-top min-w-max
</div>

{/* Code */}
<pre className="px-3 py-2 text-[9px] md:text-[10px] font-mono
 x">
 {code.map((line, i) => {
 const ln = i + 1;
 const active = step.codeLine === ln;
 return (
 <div key={ln} className={`px-2 py-0.5 rounded
 2 border-indigo-500 text-indigo-200` : color === 'emerald' ? 'text-emerald-300' : 'text-slate-300'}>

 </div>
);
 });
 })}
</pre>

{/* Description + result */}
<div className="px-3 py-2.5 bg-slate-900/60 border-slate-500 border-1
 px]">

 {step.desc}
 {step.result !== undefined && (

 = {step.result}
)}

```

```

// =====
export function SegmentTreeVisualizer() {
 const runtime = useVizRuntime();
 const liveI = vizNumber(runtime?.variables?.i);
 const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
 const [customInput, setCustomInput] = useState('5, 8, 3, 12');
 const [qL, setQL] = useState(1);
 const [qR, setQR] = useState(4);
 const [view, setView] = useState<'build' | 'queries'>('build');

 // Clamp query range when array changes
 useEffect(() => {
 setQL(prev => Math.min(prev, arr.length - 1));
 setQR(prev => Math.min(prev, arr.length - 1));
 }, [arr]);

 const handleApply = (e: React.FormEvent) => {
 e.preventDefault();
 const parsed = customInput.split(',').map(s => parseInt(s));
 if (parsed.length >= 2 && parsed.length <= 16) {
 setArr(parsed);
 }
 };

 const randomize = () => {
 const size = arr.length;
 const a = Array.from({ length: size }, () => Math.floor(Math.random() * 100));
 setArr(a);
 setCustomInput(a.join(', '));
 };

 const buildSteps = useMemo(() => genBuildSteps(arr), [arr]);
 const queryTDSteps = useMemo(() => genQueryTopDownSteps(arr, qL, qR), [arr, qL, qR]);
 const queryBUSteps = useMemo(() => genQueryBottomUpSteps(arr, qL, qR), [arr, qL, qR]);

 const totalSum = arr.reduce((a, b) => a + b, 0);

 return (

```



```

<div className="flex items-center gap-1">

 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qL}
 onChange={e => setQL(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 />

</div>
<div className="flex items-center gap-1">

 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qR}
 onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-indigo-500 focus:border-indigo-500"
 />

</div>
<div className="text-[10px] text-slate-500">
 Запрос: $\text{sum}(A[\{qL\}..\{qR\}]) = \{arr.\text{slice}(qL, qR + 1).reduce((sum, val) => sum + val, 0)}\}$
</div>
</div>
</div>
</div>

{/* ---- TABS ---- */}
<div className="flex flex-wrap items-center gap-1">
 <button onClick={() => setView('build')} className={`border ${view === 'build' ? 'border-indigo-500' : 'border-slate-200'}`}>
 Построить
 </button>
</div>

```

```

 key={`qbu-${arr.join(' - ')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) — Снизу-вверх`}
 icon="↑"
 steps={queryBUSteps}
 code={QUERY_BU_CODE}
 color="amber"
 arr={arr}
 />
 </div>
)}

```

```

{view === 'theory' && (
 <div className="space-y-5">
 <div className="bg-slate-950 p-5 rounded-xl border">
 <h4 className="text-base font-bold text-white">Почему
 во разбилось именно так для N={arr.length}?
 <p className="text-sm text-slate-300 leading-relaxed">
 Каждый узел берёт <code className="bg-slate-900 px-1">
 к получает <code className="bg-slate-900 px-1">
 -slate-900 px-1 rounded font-mono text-emerald-400">
 евая часть забирает на 1 больше (округление вправо)
 </p>
 <div className="bg-slate-900 p-4 rounded-lg">
 <div className="text-slate-400">Корень: L = 1
 <div className="text-slate-300">mid = (0 + 1) >> 1
 <div className="text-indigo-300">→ Левый: L = 0
 <div className="text-emerald-300">→ Правый: R = 1
 </div>
 <div className="text-slate-500 mt-2">Всего узлов: 3
 </div>
 </div>
)

```

```

 <div className="grid grid-cols-1 xl:grid-cols-2">
 <div className="bg-slate-950 p-5 rounded-xl">
 <div className="absolute -top-3 left-4 bg-white text-slate-900">

```



```

 aria-label="Очистить поиск"
 >
 <X className="w-3.5 h-3.5" />
 </button>
 })
 </div>
})
</div>

<div className="flex-1 overflow-y-auto px-3 pb-4" >
 {groups.length === 0 && <p className="text-sm text-slate-500">
 Нет результатов
 </p> ||
 {groups.map(({ section, items }) => {
 <div key={section.id}>
 <div className="flex items-baseline gap-1.5">

 {section.from} > 0 && {

 {section.to}

 </div>
 <div className="space-y-1">
 {items.map((chapter) => {
 const isSelected = chapter.id === selectedChapterId
 const hasViz = Boolean(VIZ_REGISTRY[chapter.id])
 // «18–20. Графы. Основное дерево...» → «18–20. Графы»
 const num = chapter.title.match(/^(\d+)(?:\s+)?/)[1]
 const label = num ? chapter.title.slice(0, num) : chapter.title
 return (
 <button
 key={chapter.id}
 onClick={() => {
 setSelectedChapterId(chapter.id);
 onNavigate?.();
 window.scrollTo({ top: 0, behavior: 'smooth' });
 }}
 title={chapter.title}
 aria-current={isSelected ? "page" : undefined}
 className={`w-full text-left rounded-md ${isSelected ? 'bg-slate-100' : ''}`}
 >
 {label}
 </button>
)
 })}
 </div>
 </div>
 </div>
 })}

```

```
 <b className="text-slate-200">Порядок чтения
 номерам билетов 1 → 24, каждая опирается на п
 Следующая тема» продолжают этот же порядок.
 </p>
 <p className="leading-relaxed">
 Подчёркнутые термины в тексте раскрываются по
 симулятора.
 </p>
 <p className="text-slate-400 italic">Стрелки ←
 </div>
 </aside>
);
};
```

---

ФАЙЛ 69/111: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar

interface Plate {
 id: string;
 name: string;
 icon: string;
}

interface Person {
 id: string;
 name: string;
 icon: string;
}

interface Hypothesis {
 id: string;
 text: string;
```

```

 { name: 'Миска с остатками салата', icon: ' ' },
 { name: 'Тарелка от тако', icon: ' ' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setStack(prev => [...prev, newPlate]);
 setPopMessage(` Положили поверх стопки: ${newPlate.name} `);
};

const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠ Стопка пуста! Все тарелки вымыты");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(` Вымыта верхняя тарелка (LIFO): ${topPlate.name} `);
};

const resetStack = () => {
 setStack([
 { id: '1', name: 'Тарелка из-под пасты', icon: '🍝' },
 { id: '2', name: 'Тарелка из-под пиццы', icon: '🍕' },
 { id: '3', name: 'Блюдец от торта', icon: '🍰' },
]);
 setPopMessage(" Стопка тарелок восстановлена.");
};

// Queue handlers
const addPerson = () => {
 const presets = [
 { name: 'Любитель борща', icon: '🍲' },
 { name: 'Курьер доставки', icon: '🚚' },
 { name: 'Усталый админ', icon: '🧑‍💻' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPerson = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setQueue(prev => [...prev, newPerson]);
 setAddMessage(` Добавили в очередь: ${newPerson.name} `);
};

const removePerson = () => {
 if (queue.length === 0) {
 setAddMessage("🚫 Очередь пуста!");
 return;
 }
 const topPerson = queue[queue.length - 1];
 setQueue(prev => prev.slice(0, prev.length - 1));
 setAddMessage(` Удален из очереди: ${topPerson.name} `);
};

const resetQueue = () => {
 setQueue([
 { id: '1', name: 'Любитель борща', icon: '🍲' },
 { id: '2', name: 'Курьер доставки', icon: '🚚' },
 { id: '3', name: 'Усталый админ', icon: '🧑‍💻' },
]);
 setAddMessage(" Очередь восстановлена.");
};

```

```

 probability: Number(newCustomProb)
 };
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(` Добавлена гипотеза с вероятностью
 `);
};

const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability)
 maxIdx = i;
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(` Выбран самый "тяжелый" кандидат:
 `);
};

const resetHeap = () => {
 setHeap([
 { id: '1', text: 'Кандидат: "Привет, я искусственный интеллект"' },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный хлеб"' },
 { id: '3', text: 'Кандидат: "Привет, я испанский язык"' },
]);
 setHeapMessage(" Гипотезы восстановлены.");
};

// Sorted heap for display (Max-Heap property view)
const sortedHeap = [...heap].sort((a, b) => b.probability - a.probability);

return (
 <div className="bg-slate-900/90 border border-slate-500 p-10">

```

```

 }` }
 >
 <div className="flex items-center gap-3">
 <AlignLeft className={`w-5 h-5 ${activeTab === 'fifo' ? 'bg-indigo-500' : 'bg-slate-800'} border-indigo-500 border-slate-700 text-white text-sm font-bold`}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">FIFO
 <div className="text-xs opacity-80">FIFO
 </div>
 </div>

 BFS

 </button>

 <button
 onClick={() => setActiveTab('heap')}
 className={`flex items-center justify-between gap-2 p-2 rounded-md
 ${activeTab === 'heap' ? 'bg-emerald-600/20 border-emerald-500 text-emerald-500' : 'bg-slate-800/60 border-slate-700/60 text-white'}
 `}
 >
 <div className="flex items-center gap-3">
 <Award className={`w-5 h-5 ${activeTab === 'heap' ? 'bg-emerald-500' : 'bg-slate-800'} border-emerald-500 border-slate-700 text-white text-sm font-bold`}>
 <div className="text-left">
 <div className="font-bold text-sm text-white">Приоритетная
 <div className="text-xs opacity-80">Приоритетная
 </div>
 </div>

 Beam Search

 </button>
 </div>

 { /* STACK TAB */ }
 { activeTab === 'stack' && (
 <div className="space-y-6">

```



```

<div className="bg-blue-950/60 border border-
imate-fade-in">
 <span className="inline-block w-2 h-2 round
 {popMessage}
 </div>
})

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
 >
 {stack.length === 0 ? (
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Стор
 <p className="text-xs mt-1">Добавьте гр
 </div>
) : (
 <div className="w-full max-w-sm flex flex
 <div className="absolute top-0 text-xs
full flex items-center gap-1">
 ВЕРШИНА СТЕКА (TOP)
 <ArrowDown className="w-3.5 h-3.5 anim
 </div>

 {/* Reversed map so top of stack is vis
 {stack.slice().reverse().map((plate, ind
 const isTop = index === 0;
 return (
 <div
 key={plate.id}
 className={`w-full flex items-cen
 isTop
 ? 'bg-gradient-to-r from-blue
le-102'
 : 'bg-slate-900/90 border-sla
 }`}
 >
 <div className="flex items-center

```

```

 <button
 onClick={addPerson}
 className="flex-1 md:flex-none flex item
-2.5 rounded-xl text-sm font-bold shadow-lg
 >
 <Plus className="w-4 h-4" /> Встать в о
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex item
er border-indigo-500/40 px-4 py-2.5 rounded
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 </div>
 </div>

 {dequeueMessage && (
 <div className="bg-indigo-950/60 border bor
p-3 animate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {dequeueMessage}
 </div>
)}

 {/* Visualization */}
 <div className="bg-slate-950/60 p-6 rounded-2
overflow-x-auto">
 {queue.length === 0 ? (
 <div className="text-center py-12 text-sl

```

```

 <span className={`text-[10px] font-
 isFirst ? 'bg-indigo-500/30 text-
 }`}>
 {isFirst ? 'Получает сун' : `В

 </div>
);
 }}}

 <div className="flex flex-col items-cen
 -600 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />
 <span className="text-xs font-mono up
 </div>
</div>
)}
<div className="mt-6 text-xs text-slate-500
 ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • И
</div>
</div>
</div>
)}

{/* HEAP TAB */}
{activeTab === 'heap' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Приоритетная очередь / Куча</sp
 <span className="text-xs font-mono bg-e
 x-Heap
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Здесь не важно, кто пришел первым. Важн
 </p>
 </div>
 }
 }
}

```



```
 </div>

 <div className="flex items-center"
 {/* Custom progress bar */>
 <div className="hidden sm:block"
 <div
 className={`h-full rounded-
 style={{ width: `${percent}%`
 ></div>
 </div>
 <span className={`font-mono font-
 isTop ? 'text-emerald-400 tex
 }`}>
 {percent}%

 </div>
 </div>
);
 }
 </div>
 <div className="mt-6 text-xs text-slate-500"
 НЕ ВАЖНО, КТО ПРИШЕЛ ПЕРВЫМ • ВАЖЕН САМЫЙ
 </div>
</div>
</div>
)}
</div>
);
};
```

---

ФАЙЛ 70/111: src/components/SplayPlayground.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useContext, useEffect, useState } from
```

```

<div className="flex flex-wrap gap-2 mb-5">
 {[
 { id: "cases" as Tab, label: "1 · Механика: Z" },
 { id: "big" as Tab, label: "2 · Большое дерево" },
 { id: "play" as Tab, label: "3 · Собери сам" }
]}.map((t) => (
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-medium
 ${tab === t.id ? "bg-indigo-600 text-white" : ""}
 `}
 >
 {t.label}
 </button>
))
</div>

{tab === "cases" && <SplayRotationsViz />}
{tab === "big" && <SplayWalkViz />}
{tab === "play" && <SplayRotationSandbox />}

{tab === "play" && (
 <p className="text-xs text-slate-500 mt-3">
 Порядок поворотов в песочнице сверяется с прав
 </p>
)}
</div>
);
};

```

```
export default SplayPlayground;
```

```

 name: "Змейка",
 hintCase: "Zig-Zag",
 target: 3,
 build: () => {
 const n3 = leaf(3);
 const n2: TNode = { key: 2, left: leaf(1), right:
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
},
{
 name: "Один уровень",
 hintCase: "Zig",
 target: 2,
 build: () => ({ key: 4, left: { key: 2, left: leaf(
},
{
 name: "Смешанный",
 hintCase: "Zig-Zag, затем Zig-Zig",
 target: 5,
 build: () => {
 const n5 = leaf(5);
 const n6: TNode = { key: 6, left: n5, right: leaf
 const n4: TNode = { key: 4, left: leaf(3), right:
 return { key: 8, left: n4, right: leaf(9) };
 },
},
};

```

/\* ----- операции над деревом -----

```

export const findPath = (root: TNode | null, key: number) => {
 const path: TNode[] = [];
 let cur = root;
 while (cur) {
 path.push(cur);
 if (key === cur.key) return path;
 cur = key < cur.key ? cur.left : cur.right;
 }
 return path;
}

```

```

const pLeft = g.left === p;
if (xLeft === pLeft) return { node: p.key, kind: "Zig"
return { node: x.key, kind: "Zig-Zag (сначала нижняя
}

```

```

/* ----- раскладка -----

```

```

interface Placed {
 key: number;
 x: number;
 y: number;
 depth: number;
}

```

```

function layout(root: TNode | null): { nodes: Placed[];
 const nodes: Placed[] = [];
 const edges: [number, number][] = [];
 let order = 0;

```

```

 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 nodes.push({ key: n.key, x: order++, y: depth, depth: depth });
 if (n.left) edges.push([n.key, n.left.key]);
 if (n.right) edges.push([n.key, n.right.key]);
 walk(n.right, depth + 1);
 };
 walk(root, 0);

```

```

 const cols = Math.max(1, order);
 const rows = Math.max(1, Math.max(...nodes.map((n) => n.depth)));
 const colW = 46;
 const rowH = 62;
 return {
 nodes: nodes.map((n) => ({ ...n, x: n.x * colW + colW / 2, y: n.y * rowH + rowH / 2 })),
 edges,
 width: cols * colW,
 height: rows * rowH + 20,
 };
}

```



```

 setHistory((h) => [...h, clone(tree) as TNode]);
 setMoves((m) => [...m, { node: key, correct: want?.correct }]);
 setTree((t) => rotateUp(clone(t) as TNode, key));
 });

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);
 setHistory((h) => h.slice(0, -1));
 setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build(), wrong));

return (
 <div className="w-full bg-slate-950 rounded-2xl border border-slate-800">
 <div className="flex items-start justify-between p-4">
 <div>
 <h4 className="text-sm sm:text-base font-bold text-slate-400">
 <p className="text-[12px] text-slate-400 leading-4">
 Клик по узлу = один поворот с его родителем

 поворотов выбираешь сам, разбор покажем в конце

 </p>
 </div>
 <button
 onClick={() => reset((presetIdx + 1) % PRESETS.length)}
 className="flex items-center gap-1.5 px-3 py-3 text-xs font-bold transition-colors shrink-0"
 >
 <Dices className="w-3.5 h-3.5" /> Другое дерево
 </button>
 </div>
 <div className="flex gap-2 mb-3 overflow-x-auto no-scrollbar">
 {PRESETS.map((p, i) => (
 <button
 key={i}
 className="flex items-center gap-1.5 px-3 py-3 text-xs font-medium transition-colors shrink-0"
 onClick={() => reset(i)}
 >
 {p.name}
 </button>
))}
 </div>
 </div>
 </div>
);

```

```

const isTarget = n.key === target;
const clickable = !solved && n.depth > 0;
return (
 <g
 key={n.key}
 onClick={() => clickable && clickNode(n)}
 style={{
 transform: `translate(${n.x}px, ${n.y}px)`,
 transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
 cursor: clickable ? "pointer" : "default",
 }}
 >
 {isTarget && (
 <circle r={22} fill="none" stroke="#6366f1" stroke-width={2} />
)}
 <circle
 r={16}
 fill={isTarget ? "#6366f1" : path.hasMoved ? "#a5b4fc" : "#475568"}
 stroke={isTarget ? "#a5b4fc" : "#475568"}
 stroke-width={2}
 />
 <text textAnchor="middle" dominantBaseline="middle">
 {n.key}
 </text>
 </g>
);
}}
</svg>
</div>

```

```

<div className="flex flex-wrap items-center gap-2">
 <button
 onClick={undo}
 disabled={history.length === 0}
 className="flex items-center gap-1.5 px-3 py-2 text-sm font-medium text-white disabled:opacity-40 text-xs font-bold"
 >
 <Undo2 className="w-3.5 h-3.5" /> ОТМЕНИТЬ
 </button>

```

```

 }` }
 >
 <div className="flex items-center gap-2 font-
 {wrong === 0 ? <CheckCircle2 className="w-4
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.leng
 : `Узел ${target} в корне за ${moves.leng
 </div>
 {wrong > 0 && (
 <ul className="list-disc pl-5 space-y-0.5">
 {moves.map(
 (m, i) =>
 !m.correct && (
 <li key={i}>
 ход {i + 1}: крутили <span classN
 {m.expected}

)
)}

)}
 {wrong === 0 && moves.length > minimal && (
 <div className="opacity-80">Порядок верный,
)}
 </div>
)}
</div>
);
};

```

ФАЙЛ 72/111: src/components/SplayRotationsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРО

```

import React, { useEffect, useMemo, useRef, useState }
import { Gauge, Pause, Play, RotateCcw, Terminal } from

```

```

 /** Номера узлов: роль → число. */
 keys: Partial<Record<NodeId, number>>;
 /** Обход слева направо – одинаковый на всех кадрах.
 inorder: string[];
 /** Что означают поддеревья A..D. */
 ranges: string;
 frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
 pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
 edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
 key: "zig",
 label: "Zig",
 when: "родитель x – уже корень, деда нет",
 rotations: "1 поворот",
 keys: { x: 20, p: 40 },
 inorder: ["A", "20", "B", "40", "C"],
 ranges: "A < 20 · B между 20 и 40 · C > 40",
 frames: [
 {
 ...ZIG_BEFORE,
 kind: "start",
 title: "Было: 20 висит под корнем 40",
 text: "Нам нужен ключ 20, а он на глубине 1. Деду
 ms: RESULT_MS,
 },
 {
 ...ZIG_BEFORE,

```

```
frames: [
 {
 ...ZZ_S0,
 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый сл
 зглаживает.",
 ms: RESULT_MS,
 },
 {
 ...ZZ_S0,
 kind: "aim",
 title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
 text: "Главная хитрость этого случая: первым крути
 focus: ["g", "p"],
 ms: AIM_MS,
 },
 {
 ...ZZ_S1,
 kind: "result",
 title: "Поворот 1: 40 поднялось над 60",
 text: "40 встало в корень, 60 опустилось к нему пр
 убине 1, а не 2.",
 moved: ["p", "g", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZZ_S1,
 kind: "aim",
 title: "Прицел 2: ребро 40 – 20",
 text: "А теперь обычный одиночный поворот – тот ж
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [100, 45], A: [45, 118], p: [188, 118],
 edges: [["x", "A"], ["x", "p"], ["p", "B"], ["p",
 kind: "result",
```

```

 {
 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
 text: "Дед (60) стоит на месте и в этом повороте",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 ...ZG_S1,
 kind: "result",
 title: "Поворот 1: змейка выпрямилась",
 text: "40 заняло место 20 и забрало его к себе ле",
 moved: ["x", "p", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZG_S1,
 kind: "aim",
 title: "Прицел 2: ребро 60 – 40",
 text: "Остался одиночный поворот вокруг деда – и",
 focus: ["g", "x"],
 ms: AIM_MS,
 },
 {
 pos: { x: [180, 50], p: [90, 130], g: [275, 130],
 edges: [["x", "p"], ["x", "g"], ["p", "A"], ["p",
 kind: "result",
 title: "Стало: 20 и 60 – два сына сорока",
 text: "Дерево стало идеально симметричным: все че
 очку». ",
 moved: ["x", "p", "g", "C"],
 ms: RESULT_MS,
 },
],
};

```

```
const CASES: Case[] = [ZIG, ZIGZIG, ZIGZAG];
```

```

 return (
 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fill={color}
 {len > pad * 2 + 6 && (
 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}
 stroke="#a5b4fc"
 strokeWidth={1.5}
 strokeDasharray="4 3"
 markerEnd="url(#splay-arrow)"
 />
)}
 </g>
);
)}
 }

 {edges.map(([a, b], i) => {
 const pa = pos[a];
 const pb = pos[b];
 if (!pa || !pb) return null;
 const hot = Boolean(focus && focus.includes(a) || focus.includes(b));
 return (
 <line
 key={`-${a}-${b}-${i}`}
 x1={pa[0]}
 y1={pa[1]}
 x2={pb[0]}
 y2={pb[1]}
 stroke={hot ? "#fbbf24" : "#475569"}
 strokeWidth={hot ? 5 : 2}
 opacity={focus && !hot ? 0.25 : 1}
 style={{ transition: "all 1200ms cubic-bezier(0.16, 1, 0.3, 0.9)" }}
 />
);
 })}

```

```
};
```

```
/** Имя случая из кода пользователя → вкладка демонстрация
const CASE_BY_NAME: Record<string, number> = {
 zig: 0, "zig-zig": 1, zigzig: 1, "zig zig": 1, "zig-zigzig": 1,
};
```

```
export const SplayRotationsViz: React.FC = () => {
 const [caseIdx, setCaseIdx] = useState(0);
 const [frame, setFrame] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [slow, setSlow] = useState(false);
```

```
/* — связь с Python-компилятором: case переключает в
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveCase = vars ? vizString(vars.case) ?? vizString('') : '';
const liveStep = vars ? vizNumber(vars.step) ?? vizNumber(0) : 0;
const liveX = vars ? vizNumber(vars.xk) ?? vizNumber(0) : 0;
const liveP = vars ? vizNumber(vars.pk) ?? vizNumber(0) : 0;
const liveG = vars ? vizNumber(vars.gk) ?? vizNumber(0) : 0;
const lastSig = useRef<string | null>(null);
```

```
useEffect(() => {
 if (!liveCase && liveStep === null) return;
 const norm = (liveCase ?? "").trim().toLowerCase();
 const idx = CASE_BY_NAME[norm] ?? caseIdx;
 const sig = `${idx}:${liveStep ?? ""}`;
 if (sig === lastSig.current) return;
 const first = lastSig.current === null;
 lastSig.current = sig;
 setPlaying(false);
 if (first || idx !== caseIdx) {
 setCaseIdx(idx); // новый случай — начинаем его
 setFrame(0);
 } else {
 // тот же случай, следующий шаг кода — пролистываем
 setFrame((f) => Math.min(CASES[idx].frames.length, f + 1));
 }
}, [liveCase, liveStep, caseIdx]);
```



```

<div className="mb-3 text-xs sm:text-[13px] text-
 Кор,
 ({active.rotat
</div>

```

/\* шаг + пояснение стоят НАД картинкой: сначала

```

<div className="mb-3 rounded-xl border border-sla
 <div className="flex items-center gap-2 mb-1 fl
 <span
 className={`text-[10px] font-bold uppercase
 current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 }`
 >
 {current.kind === "aim" ? "прицел" : current

 <span className="text-sm font-bold text-white
 Шаг {idx + 1} из {total}. {current.title}

 </div>
 <p className="text-[13px] text-slate-400 leading
</div>

```

```

<div className="bg-slate-900 rounded-xl border bo
 <Tree frame={current} prev={prevPos} keys={acti

```

/\* обход слева направо – одинаковый на всех ка

```

<div className="mt-2 pt-2 border-t border-slate
 <div className="flex items-center gap-1.5 fle
 Обход
 {active.inorder.map((t, i) => (
 <span
 key={i}
 className={`px-1.5 py-0.5 rounded font-r
 /\d/.test(t) ? "bg-emerald-500/15 tex

```

```

 setPlaying(false);
 setFrame((f) => (f + 1) % total);
 }}
 className="px-4 py-2 rounded-lg bg-indigo-600
 w-indigo-900/40"
 >
 {last ? "⏮ Сначала" : "Дальше →"}
</button>
<button
 onClick={() => setPlaying((p) => !p)}
 className="flex items-center gap-1.5 px-3 py-1
 ext-xs font-bold transition-colors"
 >
 {playing ? <Pause className="w-3.5 h-3.5" />
 {playing ? "Стоп" : "Авто"}}
</button>
<button
 onClick={() => setSlow((s) => !s)}
 title="Автопрокрутка ещё медленнее"
 className={`flex items-center gap-1.5 px-3 py-1
 slow
 ? "bg-emerald-600/20 border-emerald-500 text-emerald-500"
 : "bg-slate-800 border-slate-700 text-slate-200"}
 >
 <Gauge className="w-3.5 h-3.5" /> {slow ? "0.5x" : "1x"}
</button>
<button
 onClick={() => {
 setFrame(0);
 setPlaying(false);
 }}
 className="flex items-center gap-1.5 px-3 py-1
 ext-xs font-bold transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Сброс
</button>

```

```
 }}
 </div>

 <p className="mt-2 text-[11px] text-slate-500">
 Листай кнопкой «Дальше» в своём темпе – кадры не
 </p>
</div>
);
};
```

---

ФАЙЛ 73/111: src/components/SplayWalkViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useEffect, useMemo, useRef, useState } from 'react';

/**
 * Splay на БОЛЬШОМ дереве – по урокам treap-прохода.
 *
 * Главный принцип: НИКАКИХ «хоп». Каждый одиночный поворот
 * aim – прицел на тройку g–p–x, назван случай;
 * cut – отстёгиваем среднее поддереве β (пунктир, поворот);
 * rot – поворот: зелёным подсвечены рёбра, которых не было;
 * Рядом с логом всегда висит врезка «Механика одного поворота»
 * На каждом кадре – живая автопроверка in-order (сортировка)
 *
 * Чистые функции (splaySteps, rotateUp, bstInsert, collect)
 */

export interface SN {
 key: number;
 left: SN | null;
 right: SN | null;
 parent: SN | null;
}
```

```

export function rotateUp(child: SN): void {
 const p = child.parent;
 if (!p) return;
 const g = p.parent;
 if (p.left === child) {
 p.left = child.right;
 if (child.right) child.right.parent = p;
 child.right = p;
 p.parent = child;
 } else {
 p.right = child.left;
 if (child.left) child.left.parent = p;
 child.left = p;
 p.parent = child;
 }
 child.parent = g;
 if (!g) return;
 if (g.left === p) g.left = child;
 else g.right = child;
}

/** Рёбра в виде "p→c" — для diff между кадрами. */
export function collectEdges(t: SN | null): string[] {
 const es: string[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push(`${n.key}→${n.left.key}`);
 if (n.right) es.push(`${n.key}→${n.right.key}`);
 walk(n.left);
 walk(n.right);
 };
 walk(t);
 return es.sort();
}

const diffEdges = (before: string[], after: string[]):
 const rm = before.filter((e) => !after.includes(e));
 const ad = after.filter((e) => !before.includes(e));

```

```

const pathKeys: number[] = [];
let found = false;
let last: SN = root;
while (cur) {
 last = cur;
 pathKeys.push(cur.key);
 if (key === cur.key) {
 found = true;
 break;
 }
 cur = key < cur.key ? cur.left : cur.right;
}
const x = found ? cur : last;
steps.push({
 kind: "search",
 root: cloneWithParents(root)!,
 pathKeys: [...pathKeys],
 note: isInsert
 ? `Вставили ${key} обычным BST-спуском. Теперь по
 : found
 ? `Спуск по ключу ${key}: путь ${pathKeys.join(
 : `Ключа ${key} нет. Спуск упёрся в ${last.key}
 rotCount: 0,
});

// --- подъём ---
let rotCount = 0;
let target: SN = x as SN;
const fixRoot = () => {
 while (root && root.parent) root = root.parent;
};
/** Один поворот = три кадра: aim → cut → rot. */
const emitRotation = (child: SN, label: string, aimEdge: string) => {
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge,
 caseLabel: label,
 });
 // ...

```

```

 "Zig",
 [p.key, target.key],
 `Zig: у ${target.key} нет деда — хватит одного
);
 continue;
}
const pIsLeft = g.left === p;
const xIsLeft = p.left === target;
if (pIsLeft === xIsLeft) {
 const side = pIsLeft ? "оба слева" : "оба справа"
 emitRotation(
 p,
 "Zig-Zig",
 [g.key, p.key],
 `Zig-Zig (${side}: линия ${g.key}—${p.key}—${ta
 ${g.key}, а ${target.key} пока не трогаем.`
);
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zig",
 note: `Теперь обычный зиг: ребро ${target.parent
 rotCount,
 });
 emitRotation(
 target,
 "Zig-Zig",
 [target.parent.key, target.key],
 `Доводим: ${target.key} над ${target.parent.key
);
} else {
 const side = `${g.key} → ${pIsLeft ? "влево" : "в
 emitRotation(
 target,
 "Zig-Zag",
 [p.key, target.key],
 `Zig-Zag (змейка: ${side}). Порядок ОБРАТНЫЙ: п

```

```

};
const heightOf = (t: SN | null): number => (t ? 1 + Math.max(
const inorderKeys = (t: SN | null): number[] => (t ? [...
 ...

export const INITIAL_KEYS = [50, 40, 35, 32, 30, 28, 12, 10, 8, 6, 4, 2];

const buildInitial = (): SN => {
 let t: SN | null = null;
 for (const k of INITIAL_KEYS) t = bstInsert(t, k);
 return t!;
};

const CODE_LINES = [
 { line: 1, text: "while x has parent:" },
 { line: 2, text: " p = parent(x); g = parent(p)" },
 { line: 3, text: " if g is None: # Z" },
 { line: 4, text: " rotate(x over p)" },
 { line: 5, text: " elif p,g,x на одной линии: # B" },
 { line: 6, text: " rotate(p over g) # B" },
 { line: 7, text: " rotate(x over p)" },
 { line: 8, text: " else: # Z" },
 { line: 9, text: " rotate(x over p) # H" },
 { line: 10, text: " rotate(x over g)" },
];

/* ----- врезка «механика одного поворота» -----

const MechanismInset: React.FC<{ step: SplayStep | null }> = ({ step }) => {
 const isCut = step?.kind === "cut";
 const isRot = step?.kind === "rot";
 return (
 <div className="bg-slate-950/80 rounded-lg border border-slate-700 p-4">
 <h4 className="text-xs font-bold text-slate-300 mb-2">Механика одного поворота
 <p className="text-[10px] text-slate-500 mb-2">
 Как вправление вывиха: одна кость ходит вокруг другой
 </p>
 <svg viewBox="0 0 340 190" className="w-full">
 <g transform="translate(10,6)">

```

```

 x1="45" y1="86" x2="63" y2="126"
 stroke={isRot ? "#10b981" : "#64748b"}
 strokeWidth={isRot ? 3 : 2}
 />
 <circle cx="70" cy="46" r="15" fill="#059669"
 <text x="70" y="50" textAnchor="middle" fill=
 <circle cx="45" cy="86" r="15" fill="#1e293b"
 <text x="45" y="90" textAnchor="middle" fill=
 <circle cx="27" cy="126" r="13" fill="#0f172a"
 <text x="27" y="130" textAnchor="middle" fill=
 <circle
 cx="63" cy="126" r="13"
 fill={isRot ? "#78350f" : "#0f172a"}
 stroke={isRot ? "#10b981" : "#64748b"}
 strokeWidth={isRot ? 2.5 : 1.5}
 />
 <text x="63" y="130" textAnchor="middle" fill=
 <circle cx="95" cy="126" r="13" fill="#0f172a"
 <text x="95" y="130" textAnchor="middle" fill=
 <text x="112" y="92" fill={isRot ? "#10b981"
 {isRot ? "↓ β теперь при p" : "β при p"}
 </text>
 <text x="30" y="160" fill="#64748b" fontSize=
 </g>
</svg>
<p className="text-[10px] text-slate-500 leading-
 Обход слева-направо α x β p γ не меняется никог
 числа поворотов. Единственный, кто меняет хозяин
 </p>
</div>
);
};

```

/\* ----- рендер -----

```

const VIEW_W = 760;
const SLOT_W = 88;
const LEVEL_H = 76;

```



```

 return;
 }
 timer.current = setTimeout(() => setIdx((v) => v +
 return () => {
 if (timer.current) clearTimeout(timer.current);
 });
}, [playing, idx, steps]);

const step = steps ? steps[Math.min(idx, steps.length) - 1] : null;
const viewRoot = step ? step.root : baseRef.current;
const pos = useMemo(() => layout(viewRoot), [viewRoot]);
const nodes = useMemo(() => [...pos.entries()].map(([key, node]) => node), [pos]);
const edges = useMemo(() => {
 const es: { a: number; b: number }[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push({ a: n.key, b: n.left.key });
 if (n.right) es.push({ a: n.key, b: n.right.key });
 walk(n.left);
 walk(n.right);
 };
 walk(viewRoot);
 return es;
}, [viewRoot, treeVers, idx]);

const autoW = Math.max(VIEW_W, ...nodes.map((n) => n.w));
const autoH = Math.max(300, ...nodes.map((n) => n.py));

const dBefore = useMemo(() => (steps ? depthOf(steps[steps.length - 1].root) : 0), [steps]);
const dNow = steps ? depthOf(viewRoot, steps[steps.length - 1].root) : 0;
const hBefore = useMemo(() => (steps ? heightOf(steps[steps.length - 1].root) : 0), [steps]);
const hNow = heightOf(viewRoot);

// переехавшие рёбра: разница с предыдущим кадром
const prevEdgeSet = useMemo(() => {
 if (!steps || !step || idx <= 0) return null;
 return new Set(collectEdges(steps[idx - 1].root));
}, [steps, step, idx]);

```

```

 return "#64748b";
 };

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border"
 {/* Wanka */}
 <div className="flex flex-wrap items-center justify-center"
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white"

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-indigo-300">Клик
 </div>
 </div>
 <div className="flex flex-wrap items-center gap-3">
 <input
 value={input}
 onChange={(e) => setInput(e.target.value)}
 onKeyDown={(e) => {
 if (e.key === "Enter") {
 const v = parseInt(input, 10);
 if (!Number.isNaN(v)) {
 const has = inorderKeys(baseRef.current);
 if (has) runKey(v);
 else doInsert();
 setInput("");
 }
 }
 }}
 placeholder="ключ + Enter"
 className="w-28 bg-slate-950 border border-indigo-500"
 />
 <button
 onClick={() => {
 baseRef.current = buildInitial();
 }}
 />
 </div>
 </div>
 </div>
);
}

```

```

<button onClick={() => setPlaying(!playing)}
 er-600" : "bg-emerald-600"}`}>
 {playing ? " Пауза" : "▶ Пуск"}
</button>
<button onClick={() => { setPlaying(false); s
 lassName="px-3 py-1.5 rounded-lg text-xs fo
 War →
</button>

 War {idx + 1} из {steps.length} • поворотов

{step?.caseLabel && (
 <span className="px-2.5 py-1 rounded-full t
 00/60">
 {step.caseLabel}

)}
</div>
)}

{/* Дерево */}
<div className="bg-indigo-950/20 rounded-xl p-4 b
 {!steps && (
 <div className="absolute top-3 left-3 bg-indi
 ont-bold">
 Кликни по любому узлу – splay покадрово
 </div>
)}
<svg viewBox={`0 0 ${autoW} ${autoH}`} className
 {edges.map((e) => {
 const a = pos.get(e.a);
 const b = pos.get(e.b);
 if (!a || !b) return null;
 const isAim = step?.aimEdge && step.aimEdge
 const isCutEdge =
 step?.kind === "cut" && step.cutEdge && s
 .cutEdge[1];
 const isMoved = !isAim && !isCutEdge && pre

```

```

 />
 <text y={4} fill="#ffffff" fontSize="13
 >
 {n.key}
 </text>
 </g>
);
 }}}
</svg>

{/* Лог + механика */}
<div className="w-full flex flex-col lg:flex-rot
 <div className="flex-1 bg-slate-950/80 p-4 ro
 <p className="font-semibold text-amber-400
 {step?.kind === "aim"
 ? " Прицел:"
 : step?.kind === "cut"
 ? "∞ Отстёгиваем среднее поддерево:"
 : step?.kind === "rot"
 ? " Поворот:"
 : step?.kind === "search"
 ? " Спуск:"
 : "Статус:"}
 </p>
 <p className="min-h-[40px] flex items-cente
 {step ? step.note : "Клигни по узлу или в
 ке справа – механика одного поворота."}
 </p>
 {steps && step && step.kind !== "search" &&
 <p className={`text-xs mt-2 ${checkOk ? "
 {checkOk ? " " : " "} проверка кадра: o
 {inorderKeys(viewRoot).length}
 {movedCount > 0 ? ` · переехало рёбер:
 </p>
 }}
 </div>
 <div className="w-full lg:w-[300px] shrink-0"
 <MechanismInset step={step} />

```

```

 const za1 = step?.caseLabel === "Zig-Zag"
 const za2 = step?.caseLabel === "Zig-Zag"
 const active = zig || zz1 || zz2 || za1 || za2
 return (
 <div key={l.line} className={`py-1 px-2 border-400" : "text-slate-400"}`>

 {l.text}

 </div>
);
 }
 }
 </div>
</div>
</div>
);
};

export default SplayWalkViz;

```

ФАЙЛ 74/111: src/components/StackViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useCallback } from 'react';
import { ArrowUp, Sparkles } from 'lucide-react';
import { useVizRuntime, vizArray } from '../data/vizSteps';

interface Plate {
 id: string;
 name: string;
 emoji: string;
 color: string;
 isDirty: boolean;
 animState: 'in' | 'idle' | 'washing' | 'clean';
}

```

```

const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>()

const addLog = (msg: string) => setLog(prev => [msg, ...prev])

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут упасть');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }

 const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
 counter++;

 const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,
 animState: 'in',
 };

 setDirtyStack(prev => [...prev, newPlate]);
 addLog(` PUSH: Положили ${newPlate.emoji} сверху стопки`);

 setTimeout(() => {
 setDirtyStack(prev => prev.map(p => p.id !== newPlate.id ? p : { ...p, animState: 'out' }));
 }, 500);
}, [dirtyStack, isWashing]);

// POP: Помыть верхнюю тарелку (LIFO)
const popPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length === 0) {
 setShake(true);
 return;
 }

 const topPlate = dirtyStack[dirtyStack.length - 1];
 addLog(` POP: Помыли тарелку ${topPlate.emoji} (LIFO)`);
 setDirtyStack(prev => prev.slice(0, -1));
}, [dirtyStack, isWashing]);

```

```

 mState: 'in' },
 });
 setCleanRack([]);
 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
 });

 const topIndex = displayStack.length - 1;

 return (
 <div className="flex flex-col gap-6">
 { /* МНЕМОНИКА / ПРАВИЛО */ }
 <div className="bg-blue-950/40 border border-blue-950/40">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl">ПРАВИЛО</h3>
 <p className="text-xs text-slate-300 mt-1 leading-4">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именно самую верхнюю.
 Попробуй вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>

 { /* УПРАВЛЕНИЕ */ }
 <div className="flex items-center gap-3 flex-wrap">
 <button
 onClick={pushPlate}
 disabled={isWashing}
 className="flex-1 min-w-[150px] bg-gradient-to-r from-blue-950/40 to-blue-950/0
 text-white font-medium text-sm font-sans text-slate-950 disabled:cursor-not-allowed text-white
 rounded-md transition-all cursor-pointer flex items-center justify-center">
 Положить грязную (PUSH)
 </button>

 <button

```

```

 <div className="absolute w-6 h-6 rounded-1
"></div>
 <div className="absolute w-4 h-4 rounded-1
"></div>
 <div className="absolute w-5 h-5 rounded-1
"></div>
 <div className="absolute text-4xl anim-cr
</div>
 }}

{displayStack.length === 0 ? (
 <div className="flex-1 flex flex-col items-
 <span className="text-6xl mb-3 animate-pu
 <p className="text-white font-black text-
 <p className="text-slate-500 text-xs mt-1
 </div>
) : (
 <div className={`flex-1 flex flex-col justifi

 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-
 style={{ bottom: `${24 + topIndex * 44}
 >
 <span className="text-xs font-mono text
er gap-1">
 <ArrowUp className="w-3.5 h-3.5" />
 ВЕРШИНА (TOP)

 <span className="text-blue-400 animate-p
 </div>

 {/* Тарелки (отрисовываем в обратном поряд
 {displayStack.slice().reverse().map(({plate
 const idx = displayStack.length - 1 - r
 const isTop = idx === topIndex;
 return (
 <div

```



```

<div className="flex-1 flex flex-col justify-between"
 {cleanRack.length === 0 ? (
 <div className="flex-1 flex flex-col items-center"

 <p className="text-xs font-bold font-mono">
 <p className="text-[10px] mt-1">Здесь 6
 </p>
 </div>
) : (
 <div className="flex flex-col gap-2 w-full"
 {cleanRack.map(plate => (
 <div
 key={plate.id}
 className="w-full h-8 rounded-full flex
items-center justify-center gap-2 opacity-100"
 >
 {plate.name}

 </div>
 </div>
)
)}
 </div>
)}
</div>

{/* Столешница сушилки */}
<div className="w-full h-4 bg-gradient-to-r from-slate-900 to-slate-800"
 </div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-800 p-4"
 <div className="text-[10px] font-mono text-slate-100"
 {log.map((entry, idx) => (
 <div
 key={idx}
 className={`text-xs font-mono flex items-center justify-between`}
 >
 {idx === 0 ?

```

```

 if (s[i] === s[j]) {
 steps.push({ i, j, pi: [...pi], highlight:
 ска увеличивается.` });
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight:
 овпадений нет.` });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записывае

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true
 }
 }
 return { s, steps, patternLength: pattern.length };
}

```

```

function generateZSteps(pattern: string, text: string)
 if (!pattern) pattern = " ";
 const s = pattern + "#" + text;
 const n = s.length;
 const z = new Array(n).fill(0);
 const steps: any[] = [];

 let l = 0, r = 0;
 steps.push({ i: 0, l, r, z: [...z], desc: `Начало.

 for (let i = 1; i < n; i++) {
 steps.push({ i, l, r, z: [...z], desc: `Шаг i=$

 if (i <= r) {
 steps.push({ i, l, r, z: [...z], desc: `Вни
 !` });
 z[i] = Math.min(r - i + 1, z[i - l]);
 steps.push({ i, l, r, z: [...z], desc: `Пре

```

```

const safePattern = pattern || " ";
const safeText = text || " ";

const { s: demoString, steps, patternLength: demoPa
 if (mode === "kmp") return generateKmpSteps(safeP
 else return generateZSteps(safePattern, safeText
}, [mode, safePattern, safeText]);
const runtime = useVizRuntime();
const runtimeString = vizString(runtime?.variables?
const s = runtimeString ?? demoString;
const patternLength = runtimeString ? runtimeString

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [s, mode]);

// Timer loop for autoPlay
useEffect(() => {
 if (autoPlay) {
 timer.current = setInterval(() => {
 setStepIdx(prev => {
 if (prev >= steps.length - 1) { set
 return prev + 1;
 });
 }, 1200);
 }
 return () => { if (timer.current) clearInterval
}, [autoPlay, steps.length]);

const playPause = () => setAutoPlay(!autoPlay);
const nextStep = () => { setAutoPlay(false); setSte
const prevStep = () => { setAutoPlay(false); setSte

```

```

 <div className="flex items-center gap-2"
 <div className="bg-slate-800 p-1 flex items-center"
 {pattern}
 <input value={pattern} onChange={onChangePattern} type="text" />
 </div>
 <div className="bg-slate-800 p-1 flex items-center"
 {text}
 <input value={text} onChange={onChangeText} type="text" />
 </div>
 </div>
 </div>
 </div>

 <div className="bg-slate-950 rounded-xl border-[2px] flex items-center justify-start"
 <div className="flex gap-1.5 px-4 h-full"
 {s.split("").map((char, index) => {
 const isPattern = index < pattern.length;
 const isSeparator = index === pattern.length;

 let borderColor = "border-slate-900/40";
 let bgColor = "bg-slate-900";
 let textColor = isSeparator ? "text-white" : "text-slate-400";

 // KMP Logic
 if (mode === "kmp") {
 if (step.highlight?.include(char)) {
 borderColor = "border-amber-500";
 bgColor = step.match === char ? "bg-amber-500" : "bg-slate-900/40";
 }
 }

 // Z Logic
 if (mode === "z") {
 if (index >= step.l && index <= step.r) {
 borderColor = "border-amber-500";
 bgColor = step.match === char ? "bg-amber-500" : "bg-slate-900/40";
 }
 }
 })}
 </div>
 </div>
 </div>
)

```

```

z-20">
 <span className=
 </div>
 })
 {mode == "kmp" && step
 <div className="abs
 <span className=
 </div>
 })
 {mode == "kmp" && step
 <div className="abs

20">
 <span className=
 </div>
 })

 {/* Z fingers */}
 {mode == "z" && step.i
 <div className="abs

e z-20">
 <span className=
 </div>
 })
 {mode == "z" && step.z
 <div className="abs
 <span className=
 </div>
 })
 {mode == "z" && step.z
 <div className="abs
 <span className=
 </div>
 })

 {/* Found flash effect
 {step.found && mode ==
 <div className="abs

ulse z-0 bg-emerald-500/20 shadow-[0 0 15px

```

```

 <div className="text-[10px] text-slate-400" data-bbox="274 100 1000 145">
 {steps.length}</div>
 <div className="text-sm text-slate-400" data-bbox="407 148 1000 193">
 {step.desc}
 </div>
 </div>
</div>

{/* Why this matters card */}
<div className="bg-slate-800 p-4 rounded-xl" data-bbox="274 315 1000 335">

 <p className="text-xs text-slate-400" data-bbox="341 361 1000 381">
 Обрати внимание на хитрость с решёткой.
 Мы пишем Шаблон + # + Текст.
 Когда значени {mode === 'kmp' ? "проблема" : "не найдено"}
 о было найдено!
 Пройди шаги вручную, посмотри, как работает алгоритм
 для пропуска работы (в Z).
 </p>
</div>

{/* Code Sneak Peek */}
<div className="bg-slate-900 border border-slate-800" data-bbox="274 623 1000 643">
 <div className="bg-slate-800 px-4 py-2 border border-slate-400 uppercase" data-bbox="323 646 1000 686">
 Код C++ ({mode === 'kmp' ? 'КМП' : 'ЗКМП'})
 </div>
 <pre className="p-4 text-xs font-mono text-slate-400" data-bbox="323 741 1000 761">
{mode === 'kmp' ? `vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}` : `int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - l]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
 if (i + z[i] <= r) l = i, r = i + z[i];
}`}
 </pre>

```

```

return (
 <span
 className={`relative inline-flex ${className} ?? ""
 onMouseEnter={() => setOpen(true)}
 onMouseLeave={() => setOpen(false)}
 onFocus={() => setOpen(true)}
 onBlur={() => setOpen(false)}
 >
 {children}
 <span
 role="tooltip"
 className={`pointer-events-none absolute left-1/2
 bg-slate-900 px-3 py-2 text-left text-[11px]
 l duration-150 ${
 side === "top" ? "bottom-full mb-2" : "top-full
 } ${open ? "visible translate-y-0 opacity-100"
 >
 {content}
 <span
 className={`absolute left-1/2 h-2 w-2 -transl
 side === "top" ? "-bottom-[5px] border-b bo
 }`}
 />

);
};

```

ФАЙЛ 77/111: src/components/TopologicalSortViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect } from "react";
import { useVizStepSync, vizArray, vizNumber } from "...
import {
 Play,

```

```

 currentNode: number | null;
 topoList: number[];
 dfsStack: number[];
}

export const TopologicalSortViz: React.FC = () => {
 const [steps, setSteps] = useState<TStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode !== v));
 return candidates.at(-1)?.index ?? null;
});
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const listSteps: TStep[] = [];
 const visited = new Set<number>();
 const fullyProcessed = new Set<number>();
 const topoList: number[] = [];
 const dfsStack: number[] = [];

 const recordStep = (desc: string, currentNode: number) => {
 listSteps.push({
 desc,
 visited: new Set(visited),
 fullyProcessed: new Set(fullyProcessed),
 currentNode,
 topoList: [...topoList],
 dfsStack: [...dfsStack],
 });
 };
});

```



```
// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 recordStep(
 `Запускаем новый обход DFS от нетронутой верши
 null,
);
 dfs(i);
 }
}

recordStep(
 "Топологическая сортировка завершена! Полученный
 null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []);

const step = steps[currentStepIdx] || {
 desc: "Запуск...",
 visited: new Set(),
 fullyProcessed: new Set(),
 currentNode: null,
 topoList: [],
 dfsStack: [],
};

useEffect(() => {
 let timer: NodeJS.Timeout;
 if (isPlaying && currentStepIdx < steps.length - 1)
 timer = setInterval(() => {
 setCurrentStepIdx((prev) => prev + 1);
 }, 1600);
} else {
 setIsPlaying(false);
}
```

```

 <button
 onClick={reset}
 aria-label="Сбросить сортировку"
 title="Сбросить сортировку"
 className="flex items-center justify-center
 ors"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
 { /* Playfield SVG */ }
 <div className="lg:col-span-8 bg-[#0B1120] relative" >
 <svg className="w-full h-full max-w-[650px]" >
 <defs>
 <marker
 id="dag-arrow"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#4" />
 </marker>
 <marker
 id="dag-arrow-active"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#8

```

```

 if (isCurrent) {
 fill = "#312e81";
 stroke = "#818cf8";
 textColor = "text-white font-bold";
 } else if (isProcessed) {
 fill = "#022c22";
 stroke = "#059669";
 textColor = "text-emerald-300";
 } else if (isVisited) {
 fill = "#3730a3";
 stroke = "#6366f1";
 textColor = "text-indigo-200";
 }

 return (
 <g key={node.id}>
 <rect
 x={node.x - 55}
 y={node.y - 22}
 width="110"
 height="44"
 rx="8"
 fill={fill}
 stroke={stroke}
 strokeWidth="2"
 className="transition-all duration-500"
 />
 <text
 x={node.x}
 y={node.y - 4}
 textAnchor="middle"
 fill="white"
 fontSize="11px"
 fontWeight="bold"
 className="pointer-events-none"
 >
 {{node.label}} {node.name.split(" "
 </text>
)
)

```

```

 <span className="text-slate-600 text-
 ожидаем обработку узлов...

) : (
 getTopoListString()
 })
</div>
</div>

{/* Color key */}
<div className="space-y-1">
 <span className="text-[10px] text-slate-5
 ВЕРШИНЫ В DFS:

 <div className="flex items-center gap-2 t
 <span className="w-3.5 h-3.5 rounded bg
 <span className="text-slate-400 text-[1
 Серые (зашли)

 </div>
 <div className="flex items-center gap-2 t
 <span className="w-3.5 h-3.5 rounded bg
 <span className="text-slate-400 text-[1
 Черные (готово)

 </div>
</div>
</div>

<div className="mt-4 pt-3 border-t border-sla

 Шаг {currentStepIdx + 1} из {steps.length}

 <div className="flex gap-1">
 <button
 disabled={currentStepIdx === 0}
 onClick={() => setCurrentStepIdx((prev)
 className="bg-slate-900 border border-s

```

## Универсальное пособие • полный исходный код

```

* layers – миф «дети в массиве по $2^k/2^{k+1}$ »: работает
* search – «отсортирую бинарным поиском»: поиск не сортирует
* сортировка платит $\geq n \cdot \log n$ сравнений, сортировка
*
* Канонический набор: все x и все y различны $\Rightarrow$ дерево
* Узел идентифицируется парой $id = "x:y"$.
*/
```

```
export interface Pair {
 key: number;
 grade: number;
}
type NodeId = string;
const pairId = (key: number, grade: number): NodeId =>
```

```
interface TNode {
 id: NodeId;
 key: number;
 grade: number;
 left: TNode | null;
 right: TNode | null;
}
```

```
export const POINTS: Pair[] = [
 { key: 45, grade: 8 },
 { key: 3, grade: 6 },
 { key: 6, grade: 5 },
 { key: 2, grade: 4 },
 { key: 9, grade: 3 },
 { key: 7, grade: 2 },
 { key: 12, grade: 0 },
 { key: 1, grade: -4 },
];
```

/\*\* Порядок ввода (как в черновике) – намеренно НЕ отсортирован

```
export const INPUT_ORDER: Pair[] = [
 { key: 3, grade: 5 },
 { key: 45, grade: 8 },
```

```

 const sub = go(t.right);
 trace.push({ atId: t.id, goLeft: true, note: `(${t.key}
 о правое поддерево.` });
 t.right = sub.l;
 return { l: t, r: sub.r };
 }
 const sub = go(t.left);
 trace.push({ atId: t.id, goLeft: false, note: `(${t.key}
 о левое поддерево.` });
 t.left = sub.r;
 return { l: sub.l, r: t };
};
const res = go(root);
return { l: res.l, r: res.r, trace };
}

export interface MergeResult {
 root: TNode | null;
 trace: { aId: NodeId | null; bId: NodeId | null; chosenId: NodeId | null; }
}
export function mergeTree(a: TNode | null, b: TNode | null): MergeResult {
 const trace: MergeResult["trace"] = [];
 const go = (a: TNode | null, b: TNode | null): TNode | null {
 if (!a || !b) {
 trace.push({ aId: a?.id ?? null, bId: b?.id ?? null, chosenId: null,
 note: `Объединение в одно целое.` });
 return a ?? b;
 }
 if (a.grade > b.grade) {
 trace.push({ aId: a.id, bId: b.id, chosenId: a.id,
 note: `Выбор ${a.key} с его правым поддеревом с (${b.key}, ${b.grade})` });
 a.right = go(a.right, b);
 return a;
 }
 trace.push({ aId: a.id, bId: b.id, chosenId: b.id,
 note: `Выбор ${b.key} с его левым поддеревом.` });
 b.left = go(a, b.left);
 return b;
 }
}

```

```

 if (n.key <= min || n.key > max) bst = false;
 if (parentGrade !== null && n.grade > parentGrade) {
 walk(n.left, min, n.key, n.grade);
 if (prev !== null && n.key < prev) bst = false;
 prev = n.key;
 walk(n.right, n.key, max, n.grade);
 };
 walk(root, -Infinity, Infinity, null);
 return { bst, heap };
}

```

/\* ----- раскладка сцены дерева -----

```

const layoutTree = (root: TNode | null) => {
 const pos = new Map<NodeId, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 44 });
 slot += 1;
 walk(n.right, depth + 1);
 };
 walk(root, 0);
 return pos;
};

```

```

const treeEdges = (root: TNode | null): { from: NodeId; to: NodeId }[] => {
 const es: { from: NodeId; to: NodeId }[] = [];
 const walk = (n: TNode | null) => {
 if (!n) return;
 if (n.left) { es.push({ from: n.id, to: n.left.id }); }
 if (n.right) { es.push({ from: n.id, to: n.right.id }); }
 };
 walk(root);
 return es;
};

```

```

 }}}
 {nodes.map((n) => {
 const p = pairParts(n.id);
 const pulse = pulseId === n.id;
 return (
 <g key={`tn-${n.id}`} transform={`translate
 <circle r={pulse ? 24 : 20} fill={nodeColo
 } strokeWidth={pulse ? 4 : 2} className="tr
 <text y={-2} fill="#fff" fontSize="13" fo
 <text y={13} fill="#cbd5e1" fontSize="10"
 </g>
);
 })}
 </svg>
 {subtitle && <p className="text-xs text-slate-400
</div>
);
};

const pairParts = (id: NodeId): [number, number] => {
 const [a, b] = id.split(":").map(Number);
 return [a, b];
};

/* ----- плоскость (клетчатая бумага) -----

const PL_W = 780;
const PL_H = 300;
const PX = (key: number) => 42 + (key - 1) * ((PL_W - 6
const PY = (grade: number) => 24 + (8 - grade) * ((PL_H

interface PlaneProps {
 points: Pair[];
 drawnEdges: { a: Pair; b: Pair }[];
 pending: Pair | null;
 interactive?: boolean;
 selected?: Pair | null;
 cursor?: { x: number; y: number } | null;

```



```

 return <line key={`pe-${i}`} x1={a.x} y1={a.y} :
 2} strokeDasharray={isBad ? "4,3" : "none"}
 }}}

```

```

 {/* временная линия игры */}
 {interactive && selected && cursor && (
 <line
 x1={PX(selected.key)} y1={PY(selected.grade)}
 stroke="#818cf8" strokeWidth={2.5} strokeDash
 />
)}

```

```

 {points.map((p) => {
 const done = drawnEdges.some((e) => (e.a.key ===
 const isSel = selected && selected.key === p.key
 return (
 <g
 key={`pp-${p.key}:${p.grade}`}
 className="transition-all duration-500"
 onPointerDown={interactive && onPointDown ?
 onPointerUp={interactive && onPointUp ? () :
 style={interactive ? { cursor: "pointer" }
 >
 <circle cx={PX(p.key)} cy={PY(p.grade)} r={
 stroke={isSel ? "#a5b4fc" : done ? "#38bdf8"
 <text x={PX(p.key)} y={PY(p.grade) + 3.5} f
 ">{p.key}</text>
 </g>
);
)}
)}

```

```

 {pending && (
 <g className="transition-all duration-500">
 <circle cx={PX(pending.key)} cy={PY(pending.g
 >
 <text x={PX(pending.key)} y={PY(pending.grade
 ly="monospace">{pending.key}</text>
 </g>
)}

```

```

/** Проверка BST леса: у узла ≤1 ребёнка слева и ≤1 справа.
export function forestIsBst(edges: GEdge[], all: Pair[]) {
 const kids = new Map<string, Pair[]>();
 for (const e of edges) kids.set(gpid(e.a), [...(kids.get(gpid(e.b)) || []), e]);
 const hasParent = new Set(edges.map((e) => gpid(e.b)));
 const roots = all.filter((p) => !hasParent.has(gpid(p.a)));
 const ok = (n: Pair | undefined, lo: number, hi: number) => {
 if (!n) return true;
 if (n.key <= lo || n.key > hi) return false;
 const ch = (kids.get(gpid(n.key)) ?? []).slice().sort((a, b) => a.key - b.key);
 const left = ch.filter((c) => c.key < n.key);
 const right = ch.filter((c) => c.key > n.key);
 if (left.length > 1 || right.length > 1) return false;
 return ok(left[0], lo, n.key) && ok(right[0], n.key, hi);
 };
 return roots.every((r) => ok(r, -Infinity, Infinity));
}

```

```

/** Полный вердикт по текущим линиям. */
export function gameVerdict(edges: GEdge[], all: Pair[]) {
 const heap = edges.every((e) => e.a.grade > e.b.grade);
 const oneParent = new Set(edges.map((e) => gpid(e.b)));
 const acyclic = !edges.some((e) => reaches(edges, e.a, e.b));
 const bst = forestIsBst(edges, all);
 const roots = all.filter((p) => !edges.some((e) => gpid(e.a) === p.key));
 const complete = edges.length === all.length - 1 && roots.length === 1;
 return { heap, oneParent, acyclic, bst, complete, win: heap && complete };
}

```

```

/** Попытка хода: почему нельзя – или ОК. */
export function tryConnect(edges: GEdge[], a: Pair, b: Pair) {
 if (sameP(a, b)) return { ok: false, reason: "Это одна линия" };
 if (a.grade === b.grade) {
 return { ok: false, reason: "«!» равны – «кто выше?»" };
 }
 const parent = a.grade > b.grade ? a : b;
 const child = a.grade > b.grade ? b : a;
 if (parentOf(edges, gpid(child))) {
 return { ok: false, reason: "Уже есть родитель" };
 }
 return { ok: true };
}

```

```
};
```

```
const DEMO_RAW = "45 8, 3 6, 6 5, 2 4, 9 3, 7 2, 12 0,
```

```
/* ----- игра «соедини мышкой»: чистая логика
```

```
/** Ребро игры: a = родитель (выше по y), b = ребёнок.
```

```
export interface GameEdge {
```

```
 a: Pair;
```

```
 b: Pair;
```

```
}
```

```
const pid = (p: Pair) => `${p.key}:${p.grade}`;
```

```
export function canConnect({
```

```
 edges: GameEdge[],
```

```
 x: Pair,
```

```
 y: Pair,
```

```
): { ok: boolean; edge?: GameEdge; reason?: string } {
```

```
 if (x.key === y.key && x.grade === y.grade) return {
```

```
 for (const e of edges) {
```

```
 if ((pid(e.a) === pid(x) && pid(e.b) === pid(y)) ||
```

```
 return { ok: false, reason: "такая линия уже пров
```

```
 }
```

```
 if (x.grade === y.grade) return { ok: false, reason:
```

```
 " };
```

```
 const [parent, child] = x.grade > y.grade ? [x, y] :
```

```
 if (edges.some((e) => pid(e.b) === pid(child)))
```

```
 return { ok: false, reason: `y точки (${child.key});
```

```
 // цикл: родитель уже сидит в компоненте ребёнка (реб
```

```
 const childComp = new Set<string>([pid(child)]);
```

```
 let grew = true;
```

```
 while (grew) {
```

```
 grew = false;
```

```
 for (const e of edges) {
```

```
 if (childComp.has(pid(e.b)) && !childComp.has(pid
```

```
 childComp.add(pid(e.a));
```

```
 grew = true;
```

```
 }
```

```

// BST: дети по x, inorder не убывает; корень один; э
const children = new Map<string, Pair[]>();
for (const e of edges) children.set(pid(e.a), [...(ch
const roots = points.filter((p) => !parentCount.has(p
let bstOk = roots.length === 1;
if (bstOk) {
 const seen = new Set<string>();
 const inorder: number[] = [];
 const walk = (p: Pair): void => {
 if (seen.has(pid(p))) {
 bstOk = false; // цикл
 return;
 }
 seen.add(pid(p));
 const kids = children.get(pid(p)) ?? [];
 if (kids.length > 2) {
 bstOk = false;
 return;
 }
 const sorted = [...kids].sort((u, v) => u.key - v
 let left: Pair | null = null;
 let right: Pair | null = null;
 if (sorted.length === 2) {
 if (sorted[0].key === sorted[1].key || sorted[0
 bstOk = false; // двое детей с равным x или с
 return;
 }
 left = sorted[0];
 right = sorted[1];
 } else if (sorted.length === 1) {
 // одиночный ребёнок: сторона важна – определяет
 if (sorted[0].key <= p.key) left = sorted[0];
 else right = sorted[0];
 }
 if (left) walk(left);
 inorder.push(p.key);
 if (right) walk(right);
 };
};

```

```

}

const BuildMode: React.FC = () => {
 const [raw, setRaw] = useState(DEMO_RAW);
 const live = useTreapLive();

 // набор точек из Python-кода пересобирает плоскость
 useEffect(() => {
 if (!live.pairsText || live.pairsText === raw) return;
 setRaw(live.pairsText);
 setPhase("connect");
 setOrder("y-desc");
 setSortIdx(3);
 setConnIdx(10000); // все рёбра проведены – ср
 setPlaying(false);
 // eslint-disable-next-line react-hooks/exhaustive-
 }, [live.pairsText]);
 const [phase, setPhase] = useState<BuildPhase>("sort");
 const [algo, setAlgo] = useState<SortAlgo>("none");
 const [order, setOrder] = useState<ConnectOrder>("y-d
 const [sortIdx, setSortIdx] = useState(0);
 const [connIdx, setConnIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [sortCost, setSortCost] = useState<string | null>

 const parsed = useMemo(() => parsePairs(raw), [raw]);
 const userPairs = parsed.errors.length ? [] : parsed.p

 const n = userPairs.length;

 const userTree = useMemo(() => (userPairs.length >= 1
 const canonEdges = useMemo(() => {
 const es: { a: Pair; b: Pair }[] = [];
 const walk = (node: TNode | null) => {
 if (!node) return;
 if (node.left) { es.push({ a: { key: node.key, gr
 left); }
 if (node.right) { es.push({ a: { key: node.key, g

```

```

const [cursor, setCursor] = useState<{ x: number; y: number }>(null);
const [gameMsg, setGameMsg] = useState<string | null>(null);
const [badEdge, setBadEdge] = useState<GameEdge | null>(null);
const [hintUsed, setHintUsed] = useState(false);

const resetGame = () => {
 setGameEdges([]);
 setSel(null);
 setCursor(null);
 setGameMsg(null);
 setBadEdge(null);
 setHintUsed(false);
};

const startGame = () => {
 resetGame();
 setPhase("game");
 setPlaying(false);
};

const verdict = useMemo(() => checkGameState(userPair, gameEdges));
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree(gameEdges) : null));

const attemptConnect = (x: Pair, y: Pair) => {
 const r = canConnect(gameEdges, x, y);
 if (r.ok && r.edge) {
 setGameEdges((prev) => [...prev, r.edge!]);
 setGameMsg(r.reason ?? null);
 setBadEdge(null);
 } else {
 setGameMsg(r.reason ?? "так нельзя");
 setBadEdge({ a: x, b: y });
 window.setTimeout(() => setBadEdge(null), 900);
 }
};

const pointDown = (p: Pair) => {
 if (phase !== "game") return;
 if (!sel) {

```

```

<div className="flex flex-wrap items-center justify-center" >
 <h4 className="text-sm font-bold text-slate-300" >Демо-набор
 <button onClick={() => { setRaw(DEMO_RAW); setRows(2); }}
 py-1 rounded-lg text-xs font-bold bg-slate-500 text-white
 </button>
</div>
<textarea
 value={raw}
 onChange={(e) => { setRaw(e.target.value); setRows(2); }}
 className="w-full bg-slate-950 border border-slate-500"
 placeholder="3 5, 45 8, 1 -4 ..."
/>
{parsed.errors.length > 0 && (
 <div className="mt-2 text-xs text-rose-300 bg-slate-950 border border-slate-500 p-2" >
 {parsed.errors.map((e, i) => <div key={i}>{e}</div>)}
 </div>
)}
{parsed.errors.length === 0 && (parsed.equalX || parsed.equalY) && (
 <div className="mt-2 text-xs text-amber-300 bg-slate-950 border border-slate-500 p-2" >
 {parsed.equalY && <div>⚡ Есть равные y (сортировка ста
 кто раньше в списке</div>
 {parsed.equalX && <div>⚡ Есть равные x (сортировка ста
 де.key → left).</div>}
 </div>
)}
</div>

{/* Координатная плоскость – НАВЕРХУ */}
<div className="bg-emerald-950/20 rounded-xl p-4" >
 <div className="flex flex-wrap items-center justify-center" >
 <h4 className="text-base font-bold text-emerald-400" >Координатная плоскость
 <div className="flex items-center gap-2" >
 {phase === "sort" ? (
 <div>
 Сортировка по

```

```

 </>
 })
 </div>
 </div>
 <Plane
 points={userPairs}
 drawnEdges={phase === "game" ? gameEdges : ph
 pending={null}
 interactive={phase === "game"}
 selected={sel}
 cursor={cursor}
 badEdge={badEdge}
 onPointDown={pointDown}
 onPointUp={pointUp}
 onMove={planeMove}
 />
 <div className="mt-2 text-xs text-slate-300 bg-
 {phase === "sort" ? (
 algo === "none" ? (
 <>Ввод приходит как попало ({n} па
 ть.</>
) : sortIdx < 3 ? (
 algo === "quicksort" ? (
 <>⚡ quicksort: счётчик сравнений: <b cl
 {comparisons} ($n \cdot \log_2 n$).</>
) : (
 <> counting: точки в корзины по значени
 , потому что у – маленькие целые.</>
)
) : (
 <>Отсортировано ({sortCost ?? "способ не
 Игру </>
)
) : phase === "game" ? (
 gameWon ? (
 <> Готово! {hintUsed ? "Решение подсказа
 ево: линии вниз по y, обход по x отсортиров
) : gameMsg ? (

```



```

 <button onClick={resetGame} disabled={!gameI
ate-700 text-white disabled:opacity-30"> С
 <button onClick={showSolution} disabled={ga
te disabled:opacity-30"> Показать решение<
 </div>
 })
</div>

{/* Победное дерево в игре */}
{phase === "game" && gameTree && (
 <div className="bg-emerald-950/20 rounded-xl p-
 <h4 className="text-sm font-bold text-emerald
 <TreeView root={gameTree} subtitle="Слева-нап
ществует." />
 </div>
)}

{/* Дерево */}
{order === "y-desc" && userTree && (
 <div className="bg-slate-900/40 rounded-xl p-4
 <h4 className="text-sm font-bold text-slate-3
 <TreeView root={userTree} subtitle="Слева-нап
 </div>
)}
</div>
);
};

/* ===== РЕЖИМ SPLIT ===== */

const SPLIT_KEYS = [2, 3, 6, 7, 9, 12, 45];

const SplitMode: React.FC = () => {
 const [x0, setX0] = useState(7);
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();
 const fullTree = useMemo(() => buildTree(POINTS), [])
 const split = useMemo(() => splitTree(buildTree(POINT

```

```

 <div className="bg-slate-900/40 rounded-xl p-4 bo
 <h4 className="text-sm font-bold text-slate-300
 , розовые → R
 <TreeView
 root={fullTree}
 nodeColor={{id} => (inL.has(id) ? "#1e3a8a" :
 nodeStroke={{id} => (inL.has(id) ? "#60a5fa"
 subtitle={done ? "Каждый узел покрашен ровно
 />
 </div>

 <div className="text-xs text-slate-300 bg-slate-900/40
 {steps[idx]?.note}
 </div>

 {done && (
 <div className="grid grid-cols-1 md:grid-cols-2
 <div className="bg-sky-950/20 rounded-xl p-3
 <h5 className="text-sm font-bold text-sky-300
 <TreeView root={lTree} width={480} height={
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3
 <h5 className="text-sm font-bold text-rose-
 <TreeView root={rTree} width={480} height={
 </div>
 </div>
 </div>
)}

 <div className="text-xs text-slate-400 bg-slate-900/40
 Как это работает на самом деле: идём с
 деревом), отдаётся вместе с веткой – д
 </div>
 </div>
);
};

/* ===== РЕЖИМ MERGE ===== */

```

```

 <div className="text-xs text-slate-300 bg-slate-900" >
 {cur?.chosenId ? (
 <>⊗ {cur.note}</>
) : (
 <>Готово: {done ? "деревья склеены обратно в 1" : ""}</>
)}
 </div>

 {done && (
 <div className="bg-emerald-950/20 rounded-xl p-4" >
 <h5 className="text-sm font-bold text-emerald-400" >
 <TreeView root={merged.root} subtitle="merge({
 </div>
)}
)}

 <div className="text-xs text-slate-400 bg-slate-900" >
 Грамотно про «слияние»: у корней сравним
 в (то, что сохраняет порядок x) отправляется ,
 </div>
 </div>
);
};

/* ===== РЕЖИМ ERASE ===== */

const EraseMode: React.FC = () => {
 const [key, setKey] = useState(9);
 const [stepIdx, setStepIdx] = useState(0);
 const live = useTreapLive();
 const fullTree = useMemo(() => buildTree(POINTS), [])
 const res = useMemo(() => eraseNode(buildTree(POINTS)))
 const steps = res.trace;

 // key из кода выбирает удаляемую точку, step листает
 useEffect(() => {
 if (live.key === null || !POINTS.some((p) => p.key === key))

```

```

 nodeColor={{id} => (pathSet.has(id) ? "#783"
 nodeStroke={{id} => (pathSet.has(id) ? "#f5
 subtitle="Куча сама умеет удалять только вер
 />
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4
 <h4 className="text-sm font-bold text-slate-3
 <TreeView root={res.root} subtitle="Валидность
 <ul className="mt-2 space-y-1 text-sm">
 обход слева-направо по-прежнему сортир
 каждое ребро вниз по у (куча целая) –

 </div>
)}

<div className="text-xs text-slate-300 bg-slate-9
 {steps[idx]?.note}
</div>

<div className="text-xs text-slate-400 bg-slate-9
 Ответ на «убрать можно только самый верхний
 ягивает merge двух детей. Удаляется любая<
</div>
</div>
);
};

/* ===== РЕЖИМ LAYERS (2k/2k+1) =====

const LayersMode: React.FC = () => {
 const [pick, setPick] = useState<"heap" | "treap">("h
 // Полное дерево из 7 узлов (куча): индексы 1..7
 const heapArr = [0, 98, 45, 90, 32, 21, 76, 12];
 const full = useMemo(() => buildTree(POINTS), []);
 // Честная слотовая разметка treap: индексы как в куче
 const slots = useMemo(() => {
 const arr: (string | null)[] = Array(16).fill(null)

```

```

 <h4 className="text-sm font-bold text-rose-300" >
 <TreeView root={full} subtitle="Попробуй «пос
 " />
 <div className="flex gap-2 overflow-x-auto mt-2" >
 {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13].map((i) => {
 const id = slots[i];
 return (
 <div key={i} className={`px-2.5 py-2 rounded border-slate-700` : `bg-rose-950/40 border-rose-950`} >
 <div className="text-slate-500 text-sm" >
 {id ? <div className="text-white font-weight-bold" >
 </div>
 : </div>
 </div>
 </div>
);
 })}
 </div>
 <p className="text-xs text-rose-300 mt-2">Смотри, что левая ветка ушла на 4 уровня и заняла два уровня указателями (left/right), а не арифметикой.</p>
 </div>
)}

<div className="text-xs text-slate-400 bg-slate-900 p-2" >
 Грабля «дети в массиве по 2k/2k+1»: формула рекурсивным: слои дырявые, и «адрес» ребёнка посчитан
</div>
</div>
);
};

/* ===== РЕЖИМ SEARCH (поиск ≠ сортировка) ===== */

const SearchMode: React.FC = () => {
 // Подобрано так, что бинарный поиск ГАРАНТИРОВАННО работает
 // mid (индекс 3) = 9 > 7 → уходит влево, а цель лежит вправо
 const shuffled: Pair[] = [
 { key: 12, grade: 0 },
 { key: 3, grade: 6 },
];
};

```

```

 </button>
 })
 {stage === 2 && (
 <button onClick={() => setBidx(Math.min(bsSteps.length - 1.5 rounded-lg text-xs font-bold bg-indigo-500)}>
 Шаг поиска →
 </button>
)}
 <button onClick={() => { setStage(0); setBidx(0) }}>
 Сброс</button>
</div>

<div className="flex gap-1.5 overflow-x-auto">
 {(stage === 2 ? sortedByKey : shuffled).map((p, i) => {
 const highlight = stage === 1 && i === wrongMid;
 const inBs = stage === 2 && (() => { const s = sortedByKey.get(p.key); return s && s.bidx === bidx; })();
 const isMid = stage === 2 && (() => { const s = sortedByKey.get(p.key); return s && s.bidx === bidx; })();
 const isFound = stage === 2 && bidx >= bsSteps.length - 1;
 return (
 <div key={` ${p.key}: ${p.grade} - ${i} `} className={`${highlight ? "border-rose-500 border-2" : "border-slate-200 border-1"} p-2 rounded-r-amber-500 bg-amber-950/30`}>
 <div className="text-white font-bold">{p.key}</div>
 <div className="text-slate-400 text-[10px]">{p.grade}</div>
 </div>
);
 })}
</div>

<div className="text-xs text-slate-300 bg-slate-900 p-4">
 {stage === 0 && <>Массив пар не отсортирован в порядке</></>
 {stage === 1 && <> Бинарный поиск посмотрел на левую
 половину... в которой лежала цель. На следующем шаге
 переставил: массив как был в беспорядке, так и остался.
 {stage === 2 && bidx < bsSteps.length - 1 && <>
 {stage === 2 && bidx >= bsSteps.length - 1 && <>
 сравнения. Но чтобы поиск вообще заработал,

```

```

 setEdges([...edges, { a: parent, b: child }]);
 setMsg(`  Линия  (${parent.key}; ${parent.grade})`);
 setSel(null);
 } else {
 setMsg("x " + res.reason);
 setSel(null);
 }
};

```

```
const win = v.win;
```

```

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-2">
 <button onClick={() => { setEdges(edges.slice(0, 3)); }}>
 rounded-lg text-xs font-bold bg-slate-700 tex
 <button onClick={() => { setEdges([]); setSel(null); }}>
 3 py-1.5 rounded-lg text-xs font-bold bg-slate-700
 <button onClick={() => { setEdges(CANON_GAME.map(e => ({
 }); }} className="px-3 py-1.5 rounded-lg text-xs font-bol
 линий:
 </div>

 <div className="bg-emerald-950/20 rounded-xl p-4">
 <svg viewBox={`0 0 ${PL_W} ${PL_H + 20}`} class="w-full h-full">
 <defs>
 <pattern id="gm-grid" width="14" height="14">
 <path d="M 14 0 L 0 0 0 14" fill="none" stroke="white" stroke-width="1">
 </pattern>
 </defs>
 <rect x="34" y="16" width={PL_W - 46} height={PL_H - 16}>
 <line x1="34" y1={PY(0)} x2={PL_W - 12} y2={PY(0)}>
 <line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={PY(1)}>

 {edges.map((e, i) => (
 <line
 key={`ge-${i}`}
 x1={PX(e.a.key)} y1={PY(e.a.grade)}

```

```

 {win && {
 <p className="mt-3 text-sm font-bold text-eme
 ения просто не существует.</p>
 }}
 </div>

 <div className="text-xs text-slate-300 bg-slate-9
</div>
);
};

/* ===== СВЯЗЬ С КОМПИЛЯТОРОМ =====

/**
 * Живые переменные Python-кода страницы treap:
 * points / pairs – набор пар (x; y): по нему ПЕРЕСТРАИ
 * mid – разрез для вкладки Split;
 * key – удаляемая точка для вкладки Erase;
 * step / i – номер шага (листаёт кадры Split / E
 * height, n – высота дерева и число узлов (показ
 */
const useTreapLive = () => {
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

 const pairsText = useMemo(() => {
 if (!vars) return null;
 const rawPts = vizArray(vars.points) ?? vizArray(va
 if (!rawPts || rawPts.length === 0) return null;
 const parts: string[] = [];
 rawPts.forEach((it) => {
 if (Array.isArray(it) && it.length >= 2) {
 const x = vizNumber(it[0]);
 const y = vizNumber(it[1]);
 if (x !== null && y !== null) parts.push(`${x}
 } else if (typeof it === "string") {
 const m = it.match(/-?\d+(?:\.\d+)?/g);
 if (m && m.length >= 2) parts.push(`${m[0]} ${m

```



```

<div className="bg-slate-800/90 p-6 rounded-2xl border"
 <div className="flex flex-wrap items-center justify"
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white"

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white"
 <p className="text-sm text-emerald-300">Пар
 </p>
 </div>
 </div>
 </div>
</div>

```

```

<div className="flex flex-wrap gap-2 mb-5">
 {TABS.map((t) => {
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-
 900/60 text-slate-400 hover:text-white border
 >
 {t.label}
 </button>
)})
 </div>

```

```

{/* что сейчас сообщил компилятор */}
<div className="mb-4 flex flex-wrap items-center"
 <span className="text-[10px] uppercase tracking"
 <Terminal className="w-3.5 h-3.5" /> из компи

 {live.vars ? (
 <>
 {live.pairsText && <span className="text-[1
 d px-2 py-1">points = {live.pairsText}
 {live.n !== null && <span className="text-[
 py-1">n = {live.n}}
 {live.height !== null && <span className="t

```

```

/** Название демонстрации – подписывается в режиме vs
demoTitle?: string;
className?: string;
}

/** Имена, по которым визуализации обычно читают матрицу
const MATRIX_NAMES = ["d", "dist", "dp", "D", "S", "P",

/**
* Строка состояния связи «компилятор → демонстрация» д
*
* Раньше компонент молча проигрывал собственный встроен
* редакторе Python был пустой код, код с другими именам
* размерности, – выглядело так, будто компилятор испол
* картинка живёт отдельной жизнью. Теперь источник дан
* «данные из компилятора (i, j, dist 5x5)», «компилятор
* пока нет» (отладчик стоит до первой строки) или «встр
*/
export function VizLiveStatus({ demoTitle, className =
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const names = vars ? Object.keys(vars).filter((name) :
 const linked = names.length > 0;

 const matrixSource = pickLiveVar(vars, MATRIX_NAMES);
 const matrix = liveMatrix(matrixSource?.value);

 const detail = matrix
 ? `${matrixSource?.name} ${matrix.sizeLabel}`
 : names.length
 ? names.slice(0, 6).join(", ") + (names.length >
 : undefined;

 return (
 <VizSourceBadge
 linked={linked}
 awaiting={!linked && !!runtime}
 line={runtime?.line}

```

```

import { TopologicalSortViz } from "../TopologicalSortViz";
import { WaterfallAnimationWidget } from "../WaterfallAnimation";
import ChapterImage from "../ChapterImage";
import { Simulator } from "../Simulator";
import { PrefixSumViz } from "../visualizers/PrefixSumViz";
import { SparseTableViz } from "../visualizers/SparseTableViz";
import { TreapBuildViz } from "../TreapBuildViz";
```

```
export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}
```

```
/**
```

```
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающей подсказки
 * и для модального окна «Открыть симулятор».
 */
```

```
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 hint: "Кладите и снимайте тарелки — это ровно то, что нужно",
 Component: StackViz,
 },
 "queue-bfs": {
 title: "Очередь и обход в ширину",
 hint: "Очередь слева, живой BFS по графу справа.",
 Component: () => (
 <div className="space-y-10">
 <QueueViz />
 <BfsViz />
 </div>
),
 },
 "heap-beam-search": {
```

```

"bcc-kosaraju": { title: "Компоненты сильной связности",
"graph-articulation": { title: "Точки сочленения", Component: GraphArticulationViz,
"bridges-code": { title: "Мосты: DFS + tin/low", Component: BridgesCodeViz,
"euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: EulerPathVsCycleViz,
"planarity-euler-formula": {
 title: "Планарность и покраска: $K_5/K_{3,3}$, $\chi \leq \Delta+1$, д
 hint: "Четыре вкладки: непланарные K_5 и $K_{3,3}$, жадна
 ашего кода красит вершины напрямую.",
 Component: PlanarColoringViz,
},
dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируе
 Component: () => (
 <>
 <ChapterImage vizType="dijkstra" />
 <DijkstraViz />
 </>
),
},
"bellman-ford": {
 title: "Форд–Беллман",
 Component: () => (
 <>
 <ChapterImage vizType="bellman-ford" />
 <BellmanFordViz />
 </>
),
},
floyd: {
 title: "Флойд–Уоршелл",
 Component: () => (
 <>
 <ChapterImage vizType="floyd" />
 <FloydViz />
 </>
),
},

```

Универсальное пособие • полный исходный код

-----  
export const getViz = (id?: string): VizEntry | undefined

-----  
ФАЙЛ 81/111: src/components/VizSourceBadge.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

-----  
import { Cpu, Film } from "lucide-react";  
import { Tooltip } from "../Tooltip";

interface VizSourceBadgeProps {

/\*\*

\* true — картинку рисуют переменные из выполненного кода

\* false — компонент показывает собственную встроенную картинку

\*/

linked: boolean;

/\*\*

\* Компилятор уже выполнил код и прислал шаг, но переключатель ещё не нажали

\* (отладчик стоит ДО первой строки или в коде другие условия)

\* подпись должна объяснять, почему картинка ещё не инициализирована

\*/

awaiting?: boolean;

/\*\* Строка пользовательского кода, на которой стоит отладчик

line?: number;

/\*\* Короткая подпись данных компилятора: «матрица 5×5»

detail?: string;

/\*\* Как называется встроенная демонстрация (для подписи)

demoTitle?: string;

/\*\* Размеры демо-сетки и данных пользователя не совпадают

mismatch?: string | null;

className?: string;

}

/\*\*

\* Явный индикатор источника данных визуализации.

\*

\* Без него демонстрация молча проигрывала собственный код

```

) : null}

 {mismatch ? (
 <Tooltip side="bottom" content={mismatch}>
 <span
 tabIndex={0}
 className="cursor-help rounded-full border
0"
 >
 размерность не совпадает

 </Tooltip>
) : null}
 </div>
);
}

```

ФАЙЛ 82/111: src/components/WaterfallAnimationWidget.tsx  
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useContext, useEffect, useMemo, useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime,

```

```

// Структура заранее подготовленного красивого бора для
// Словарь: ["HE", "SHE", "HIS", "HERS"]

```

```

interface WNode {
 id: number;
 label: string; // путь от корня
 char: string;
 x: number; // Координаты для SVG водопада
 y: number;
 depth: number;
 fail: number;
 term link: number;
}

```

```

/* — связь с Python-компилятором (PAGE_SYNC["aho-cor"]
 text — сканируемый текст, state — вершина автомата
 matches — найденные вхождения (индекс конца, слово)
const chapterId = useContext(VizChapterContext);
const runtime = useVizRuntime();
const vars = runtime?.variables;

useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "aho");
}, [chapterId]);

const liveText = vars ? vizString(vars.text) ?? vizString("") : null;
const liveState = vars ? vizNumber(vars.state) ?? vizNumber(0) : null;
const liveI = vars ? vizNumber(vars.i) ?? vizNumber(0) : null;
const liveMode = vars ? vizString(vars.mode) : null;
const liveMatchesRaw = vars ? vizArray(vars.matches) : null;
const liveMatchesSig = useMemo(() => {liveMatchesRaw ? liveMatchesRaw.join(",") : ""});
const liveFail = vars ? vars.fail : undefined;

useEffect(() => {
 if (!vars) return;
 if (liveMode && /search|поиск/i.test(liveMode)) setLiveMode("search");
 else if (liveMode && /train|обуч|bfs/i.test(liveMode)) setLiveMode("train");
 if (liveText) {
 const clean = liveText.toUpperCase().replace(/[^A-Za-z0-9_]/g, "");
 if (clean && clean !== textToScan) {
 setTextToScan(clean);
 setCurrentIndex(0);
 setCurrentNode(0);
 setFoundMatches([]);
 setIsSearchDone(false);
 setJumpPath(null);
 setActiveMode("search");
 }
 }
}, [liveText, liveMode]);

if (liveState !== null && liveState >= 0 && WATERFAILL) setLiveState(liveState);
if (liveI !== null && liveI >= 0) setCurrentIndex(liveI);

```

```

const handleTextChange = (e: React.ChangeEvent<HTMLInp
 const clean = e.target.value.toUpperCase().replace(
 setTextToScan(clean);
 resetSearchSimulation();
};

const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 setIsSearchDone(false);
 setSearchLog('Симуляция сброшена. Лосось вернулся н
 setJumpPath(null);
 setFoundMatches([]);
 setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось дово
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;
 let logMsg = `[Символ '${char}']`;

 let jumpType: 'normal' | 'fail' | null = null;
 void jumpType;
 let originalCurr = curr;

 if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];
 logMsg += `У пусла #${curr} (${WATERFALL_NODES[cu
 _NODES[nextNode].label}).`;
 setJumpPath({ from: curr, to: nextNode, type: 'no

```



```

 setActiveSearchCodeLine(4);
 }

 setCurrentNode(curr);
 setCurrentIndex(currentIndex + 1);
 setSearchLog(logMsg);
 if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMat
 }
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
 {
 targetNodeId: 0,
 title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
 desc: 'Начало алгоритма BFS. Корень (ROOT) не обр
 рние вершины сразу инициализируются с fail = RO
 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'ø',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди
 просто не существует. fail[H] автоматически нап
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 7,
 title: 'Шаг 2 (Depth 1): Вершина #7 (S)',

```

```

 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 3,
 title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
 desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
 е подходят. fail[HER] = ROOT.',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'R',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 6,
 title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
 desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель
 но ветка "S" → #7 (S) совпадает! fail[HIS] = #7',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 9,
 title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С
 desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плывет в #1 (H) и ищет ветку "E". У вершины :
 codeLine: 4,
 scoutPos: 1,
 inspectedParentFail: 1,
 checkTargetChar: 'E',
 checkingBranchesFrom: 1,
 },
 {

```

```

 ? 'bg-indigo-600 text-white shadow-lg s
 : 'text-slate-400 hover:text-white'
 }` }
 >
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корн
 </button>
 <button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p
 activeMode === 'search'
 ? 'bg-blue-600 text-white shadow-lg sha
 : 'text-slate-400 hover:text-white'
 }` }
 >
 <Play className="w-4 h-4" />
 Режим 2: Поиск в тексте
 </button>
</div>
</div>

```

```

{ /* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */ }
{ activeMode === 'training' ? {
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3
 { /* Управление шагами обучения */ }
 <div className="bg-slate-900/90 p-5 rounded-x
 <div>
 <h5 className="text-indigo-300 font-bold
 Полный обход в ширину (В
 </h5>
 <p className="text-xs text-slate-300 mb-3
 Показаны все шаги очереди BFS (начиная с
 ар 8 (SHE → HE), где наглядно виде
 </p>
 <div className="space-y-1.5 mb-6 overflow
 { TRAINING_STEPS_INFO.map((step, idx) =>
 <button

```

```

>
<div>
 <h5 className="text-white font-bold mb-3">

 Код BFS построения fail

 Вершина: #{targetTrainingNode.id} ({t

 </h5>
 <div className="bg-slate-950 p-4 rounded-1
 <div className={`p-1.5 rounded flex item
-l-4 border-indigo-500 text-indigo-300 font
 1. let u = trie[r][char]; // Те
 {currentTrainInfo.codeLine === 1 && <
 тут}
 </div>
 <div className={`p-1.5 rounded flex item
-l-4 border-indigo-400 text-indigo-200 font
 2. let v = fail[r]; // Подъем к
ainInfo.inspectedParentFail].label}}
 {currentTrainInfo.codeLine === 2 && <
 к родителю}
 </div>
 <div className={`p-1.5 rounded flex item
l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
 т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex item
r-l-4 border-emerald-500 text-emerald-300 f
 4. if (trie[v][char] !== undefin
 /span>
 {currentTrainInfo.codeLine === 4 && <
 ан IF!}
 </div>
 <div className={`p-1.5 rounded flex item

```

```

 />
 </div>

 <div>
 <div className="mb-4 flex flex-wrap gap-1" >

 {textToScan.split('').map((char, idx) =>
 <span
 key={idx}
 className={`w-7 h-8 flex items-center justify-center
 ${idx === currentIndex
 ? 'bg-blue-600 text-white shadow-md'
 : idx < currentIndex
 ? 'bg-emerald-950/60 text-emerald-500'
 : 'bg-slate-800 text-slate-500'}
 }`}
 >
 {char}

)}
 </div>
 <div>
 {currentIndex >= textToScan.length && (

 ГOTOBO ✓

)}
 </div>
 </div>

 <button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.length === 0}
 className={`w-full py-2.5 rounded-lg font-medium
 ${isSearchDone || textToScan.length === 0
 ? 'bg-slate-700 text-slate-500 cursor-not-allowed'
 : 'bg-gradient-to-r from-blue-600 to-emerald-900/40 active:scale-[0.98]'}
 }`}
 >
 {currentIndex === 0 ? 'Начать план' : 'Продолжить'}
 </button>
 </div>
</div>

```

```

 <div>
 <h5 className="text-slate-300 font-bold m-0">
 <AlertCircle className="w-3.5 h-3.5 text-slate-300"/>
 </h5>
 <div className="bg-slate-800 p-3.5 rounded-2xl items-center">
 {searchLog}
 </div>
 </div>
 </div>
 </div>
)}

```

```

 {/* SVG Анимация водопада */}
 <div className="bg-gradient-to-b from-blue-950 via-blue-900 to-transparent pointer-events-none hidden shadow-2xl">
 {/* Водопадные фоновые эффекты */}
 <div className="absolute inset-0 opacity-10 bg-gradient-to-b from-blue-900 to-transparent pointer-events-none"></div>
 <div className="absolute top-0 left-1/2 -translate-x-1/2">
 <div className="w-8 bg-blue-400 h-full animate-pulse"></div>
 <div className="w-12 bg-blue-300 h-full animate-pulse"></div>
 <div className="w-6 bg-blue-500 h-full animate-pulse"></div>
 </div>

 <div className="flex items-center justify-between p-4">
 <h5 className="text-white font-bold text-base">
 Ветвящийся синий водопад (Борис)
 </h5>
 <div className="flex flex-wrap gap-4 text-xs">

 </div>
 </div>
 </div>
 </div>

```

```

 // Подсветка в режиме поиска
 let isHighlighted = activeMode === 'search' || activeMode === 'normal';

 // Подсветка в режиме обучения (когда и
 let isTrainingExamined = activeMode === 'training' || activeMode === 'normal';
 let isMatchBranch = isTrainingExamined && !isHighlighted;

 return (
 <g key={`edge-${fromNode.id}-${toNode.id}`}
 {/* Линия течения воды */}
 <line
 x1={fromNode.x}
 y1={fromNode.y}
 x2={toNode.x}
 y2={toNode.y}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth={isHighlighted ? 2 : 1}
 className={isHighlighted ? 'highlighted' : ''}
 />
 {/* Символ перехода */}
 <circle cx={{fromNode.x + toNode.x} / 2} cy={{fromNode.y + toNode.y} / 2}
 r={10} fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5a') : 'white'}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth={isHighlighted ? 2 : 1}
 />
 <text
 x={{(fromNode.x + toNode.x) / 2}}
 y={{(fromNode.y + toNode.y) / 2} + 15}
 fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5a') : 'white'}
 fontSize="12"
 fontWeight="bold"
 textAnchor="middle"
 />
 {char}
 </text>

 {/* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В РЕЖИМЕ ОБУЧЕНИЯ */}
 {isTrainingExamined && (
 <g transform={`translate(${(fromNode.x + toNode.x) / 2}, ${
 (fromNode.y + toNode.y) / 2})`}
 <rect
 x={-10} y={-10} width={20} height={20}
 fill={isMatchBranch ? '#f43f5a' : '#10b981'}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth={isHighlighted ? 2 : 1}
 />
 </g>
)}
)
)
}

```

```

 stroke={isHighlighted ? '#f43f5e' : '#2563eb'}
 strokeWidth={isHighlighted || isJustEstablished ? 2 : 1}
 strokeDasharray="6,6"
 className={isHighlighted || isJustEstablished ? 'highlighted' : ''}
 opacity={isHighlighted || isJustEstablished ? 1 : 0.5}
 />
 {isJustEstablished && (
 <text x={cx} y={cy - 10} fill="#10b981">
 fail[{node.label}] = {targetNode.label}
 </text>
)}
 </g>
);
}
}
}

```

```

 { /* Рендер вершин (бассейнов водопада) */ }
 {Object.values(WATERFALL_NODES).map((node) => {
 const isCurrent = activeFishPosNode.id === node.id;
 const isTargetChild = activeMode === 'target' && node.id === targetNode.id;
 const hasWords = node.words.length > 0;

```

```

 return (
 <g key={`node-${node.id}`} className={isCurrent ? 'current' : ''}>
 { /* Внешнее свечение для активной вершины */ }
 {isCurrent && (
 <circle cx={node.x} cy={node.y} r={10} fill="none" stroke="#f43f5e" />
)}
 {isTargetChild && (
 <circle cx={node.x} cy={node.y} r={10} fill="none" stroke="#2563eb" />
)}

```

```

 { /* Основной круг вершины */ }
 <circle
 cx={node.x}
 cy={node.y}
 r={isCurrent || isTargetChild ? 24 : 18}
 fill={isCurrent ? '#2563eb' : isTargetChild ? '#f43f5e' : '#fff'}

```



```

 }}}

 { /* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */
 <g>
 { /* Круг-подложка под рыбу */
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b5'
 stroke="#ffffff"
 strokeWidth="2"
 style={{ transition: 'cx 0.8s cubic-bez
 className="shadow-lg"
 />
 { /* Сама рыба */
 <text
 x={activeFishPosNode.x + 15}
 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezi
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' :
 </text>
 </g>

 </svg>
</div>
</div>

{ /* Список найденных совпадений (только в режиме
{activeMode === 'search' && {
 <div className="mt-6 bg-slate-900/50 p-5 rounded
 <h5 className="text-white font-bold mb-3 text
 Улов лосося (Найденные слова
 </h5>

```

```


 })
 {liveMatchesSig && <span className="text-[1
d px-2 py-1">matches: {foundMatches.map((m)
{liveFail !== undefined && <span className=
ed-md px-2 py-1">fail = {JSON.stringify(liv
</>
) : (

 открой панель Python и запусти Ахо–Корасик:
 "font-mono text-cyan-300">state</code>, <co
an-300">matches</code> поведут лосося по эт

)}
 </div>
</div>
);
};

```

## РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 83/111: src/components/visualizers/PrefixSumViz.ts  
 КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 466 | РАЗМЕР: 1000

```

import React, { useContext, useEffect, useMemo, useState } from 'react'
import {
 emitVizDemo,
 liveMatrix,
 pickLiveVar,
 useVizRuntime,
 vizArray,
 vizNumber,
 VizChapterContext,

```

```

 [
 ["1d", "1. Одномерные суммы"],
 ["2d", "2. Двумерные суммы"],
] as const
).map(({key, label}) => {
 <button
 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm
 mode === key ? "bg-indigo-600 text-white"
 }`
 >
 {label}
 </button>
)))
</div>
{mode === "1d" ? <Prefix1D /> : <Prefix2D />}
</div>
);
};

```

```

/* ----- 1D -----
const Prefix1D: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[v]],
 []
);
};

```

```

/** Что сейчас под курсором: префикс P[i] или элемент
const [hover, setHover] = useState<{ kind: "pref" | "el" }>({ kind: "pref" });
const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 });
/** Снимок i/P из автоматически выполненного Python-кода
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveP = vizArray(vars?.P);
const hasLiveP = !!vars && Object.prototype.hasOwnProperty.call(vars, "P");
const compilerLinked = liveI !== null || hasLiveP;

```

```

 key={i}
 onMouseEnter={() => setHover({ kind:
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange({r} => (i < r
 className={`${cellBase} cursor-pointe
 isDrivenAdd
 ? "bg-amber-500 text-black ring-2
 : inCover
 ? "bg-indigo-500 text-white rin
 : inRange
 ? "bg-indigo-900/70 text-indi
 : "bg-slate-800 text-slate-30
 }`}
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
);
 }}}
</div>

```

```

{/* массив P */}
<div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" &&
 const isL = i === range.l;
 const isR = i === range.r + 1;
 const liveValue = liveP?.[i];
 const blank = hasLiveP && (liveValue ===
 const masked = !hasLiveP && shownDriven !
 const isDrivenCur = shownDriven !== null
 const shownValue = hasLiveP ? liveValue :
 return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind:
 onMouseLeave={() => setHover(null)}

```

```

 <b className="text-emerald-400">P[0] = 0
 отрезка, начинающегося с нуля.
 </p>
) : (
 <p className="text-slate-300">
 <b className="text-emerald-400">
 P[{hover.i}] = {pref[hover.i]}
 {" "}
 – это сумма всего, что набежало от начала

 a[0..{hover.i - 1}] = {A1.slice(0, hove

 </p>
)
) : hover?.kind === "a" ? (
 <p className="text-slate-300">
 <b className="text-indigo-400">
 a[{hover.i}] = {A1[hover.i]}
 {" "}
 входит во все префиксы, начиная с{" "}

 подвинуть границу запроса.

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b classN
 у <b className="text-indigo-400">a – ку
 </p>
)
 })
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву
 </div>
 <p className="font-mono text-sm sm:text-base te
 sum(a[{range.l}..{range.r}]) = P[{range.r + 1

```

```

* размере матрицы часть ячеек становилась «пустыми»,
* шаги компилятора не совпадали с картинкой.
*/

```

```

const livePrefSource = pickLiveVar(vars, ["S", "P", "g"]);
const liveS = liveMatrix(livePrefSource?.value);
const hasLiveS = liveS !== null;

```

```

const liveArrSource = pickLiveVar(vars, ["a", "A", "g"]);
const liveA = liveMatrix(liveArrSource?.value);

```

```

const prefRows = liveS?.rows ?? P.length;
const prefCols = liveS?.cols ?? (P[0]?.length ?? 0);
const grid: (number | null)[][] = liveS
 ? Array.from({ length: prefRows }, (_, i) =>
 Array.from({ length: prefCols }, (_, j) => liveS[i][j]
)
 : P;

```

```

const srcRows = liveA?.rows ?? n;
const srcCols = liveA?.cols ?? m;
const srcGrid: (number | null)[][] = liveA
 ? Array.from({ length: srcRows }, (_, i) => Array.from(
 { length: srcCols }, (_, j) => liveA[i][j]
)
 : A2;

```

```

const cellAt = (i: number, j: number): number | null {
 if (i >= 0 && j >= 0 && i < prefRows && j < prefCols) {
 return grid[i][j];
 }
 return null;
}

```

```

const sizeMismatch =
 hasLiveS && liveS && (liveS.rows !== P.length || liveS.cols !== P[0].length)
 ? `Демо построено на матрице ${n}x${m} (сетка преобразована в массив).

 Если матрица имеет размер ${liveS.sizeLabel}. Ниже нарисована матрица`
 : null;

```

```

useEffect(() => {
 const r1 = vizNumber(vars?.r1);
 const c1 = vizNumber(vars?.c1);
 const r2 = vizNumber(vars?.r2);
 const c2 = vizNumber(vars?.c2);

```

```

 /* Исходная матрица */
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 Исходная матрица {liveA ? `${liveArrSource?`
 чтобы задать прямоугольник
 </div>
 <div className="inline-block">
 {srcGrid.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const inRect = i >= r1 && i <= r2 &&
 return {
 <div
 key={j}
 onClick={() =>
 setRect((r) =>
 i < r.r1 || j < r.c1
 ? { r1: i, c1: j, r2: Math.r
 : { ...r, r2: i, c2: j }
)
 }
 className={` ${cellBase} cursor-po
 inRect ? "bg-indigo-500 text-wh
 }`}
 >
 {v === null ? "." : v}
 </div>
)};
 })))
 </div>
 })))
 </div>
 </div>

```

```

 /* Матрица префиксных сумм */
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 Префиксные суммы {liveS ? `${livePrefSource

```

```

 {corner}

 })
 </div>
);
 }}}
</div>
 }}}
</div>
</div>
</div>

<div className="min-h-[54px] bg-slate-900 border border-indigo-500"
 {hover ? {
 <p className="text-slate-300">
 <b className="text-amber-400">
 {liveS ? livePrefSource?.name ?? "S" : "P"}
 {" "}
 — сумма всего прямоугольника от левого верхнего угла
 </p>
 } : {
 <p className="text-slate-500">Наведите курсор на элемент
 }}
</div>

<div className="bg-slate-900 border border-indigo-500"
 <p className="font-mono text-sm sm:text-base text-slate-300">
 сумма = A + <span
 C + <span
 ={" "}
 {
 </p>
 <p className="font-mono text-[11px] text-slate-300">
 прямоугольник ({r1}, {c1}) ... ({r2}, {c2}) · c
 </p>
 <p className="text-[11px] text-slate-500 mt-1">
 Вычли два «хвоста» и вернули дважды вычитенный
 </p>

```



```

for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j) - 1][j - 1]);
 }
}

```

// Data for 2D Matrix (Sparse Table 2D)

```

const val2D = [
 [45, 12, 88, 34, 11, 76, 23, 90],
 [67, 19, 44, 55, 33, 21, 65, 87],
 [14, 51, 99, 13, 22, 64, 43, 76],
 [89, 32, 54, 71, 15, 88, 29, 60],
 [25, 41, 16, 92, 9, 17, 56, 31],
 [59, 18, 77, 24, 61, 82, 35, 12],
 [73, 8, 38, 85, 47, 95, 19, 58],
 [39, 81, 62, 28, 51, 42, 85, 14]
];

```

```

const ST2D: number[][][][] = Array.from({length: 8}, () =>
 Array.from({length: 8}, () =>
 Array.from({length: 4}, () =>
 Array(4).fill(0)
)
)
);

```

```

for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
}

```

```

for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
 for (let r = 0; r + (1 << kx) <= 8; r++) {
 for (let c = 0; c + (1 << ky) <= 8; c++) {
 if (kx > 0) {

```

```

 className={`px-3 sm:px-4 py-2 rounded-lg text
 digo-600 text-white" : "bg-slate-800 text-s
 >
 2. Сворачивание 2D (Построение)
 </button>
 <button
 onClick={() => setTab("2d_query")}
 className={`px-3 sm:px-4 py-2 rounded-lg text
 se-600 text-white" : "bg-slate-800 text-sla
 >
 3. Запрос 2D (Формула)
 </button>
</div>

<AnimatePresence mode="wait">
 {tab === "1d" ? (
 <Viz1D key="1d" />
) : tab === "2d_build" ? (
 <Viz2DBuild key="2d_build" />
) : (
 <Viz2DQuery key="2d_query" />
)}
</AnimatePresence>
</div>
);
};

const Viz1D: React.FC = () => {
 const [step, setStep] = useState(0);
 const [hover, setHover] = useState<{ i: number; j: number}>();
 const runtime = useVizRuntime();
 const runtimeVars = runtime?.variables;
 const liveI = vizNumber(runtimeVars?.i);
 const liveJ = vizNumber(runtimeVars?.j);
 const liveSource = vizArray(runtimeVars?.a);
 const liveRows = vizArray(runtimeVars?.st);
 const hasLiveSource = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'a');
 const hasLiveTable = !!runtimeVars && Object.prototype.hasOwnProperty.call(runtimeVars, 'st');

```

```

return (
 <div className="flex flex-col gap-5">
 <div className="flex items-center justify-between">
 <div>
 <h3 className="text-lg font-bold text-indigo-600">
 Построение Sparse Table (Min)
 </h3>
 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + 2^{j-1}][j-1])$
 </p>
 {dimMismatch ? (
 <p className="mt-2 max-w-[560px] rounded-lg border border-slate-200 p-2">
 {dimMismatch}
 </p>
) : null}
 </div>
 <div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1))}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
 disabled={step === 0}
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1))}
 className="p-2 bg-indigo-600 hover:bg-indigo-500 disabled:opacity-50"
 disabled={step === maxStep}
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:opacity-50"
 >
 <RotateCcw size={20} />
 </button>
 </div>
 </div>
 </div>
)

```

```

 длины 2^{hover.j} = {1 << hover.j}:{ " " }
 <span className="font-mono text-slate-400" style={
 min({arr1D.slice(covered.from, covered.to)}

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое число
 </p>
)}
 </div>
</div>

<div className="overflow-x-auto pb-2">
 <div className="inline-block min-w-full bg-slate-100">
 <div className="grid grid-cols-[auto_repeat(4,1fr)] gap-2">
 { /* Headers */ }
 <div className="text-slate-500 font-mono text-sm">
 i \ j
 </div>
 <div className="text-center font-bold text-slate-500">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^2 (L=4)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^3 (L=8)
 </div>

 { /* Matrix */ }
 </div>
 </div>
</div>

```

```

const displayValue =
 shownValue === undefined || shownValue === null
 ? "."
 : typeof shownValue === "object"
 ? JSON.stringify(shownValue)
 : String(shownValue);

return (
 <div key={j} className="relative flex items-center justify-center"
 {!isValid ? (
 <div className="w-[clamp(30px,9vw,110px)] h-[clamp(11px,3vw,17px)]"
) : (
 <motion.div
 onMouseEnter={() => (compilerLink ? setCompilerLink(compilerLink) : null)}
 onMouseLeave={() => setCompilerLink(null)}
 onClick={() => (compilerLink ? compilerLink.click() : null)}
 >
 {displayValue}
 </motion.div>
)}
 >
 <div
 initial={{ opacity: 0, scale: 1 }}
 animate={{
 // При st = [] каркас уже сфокусирован
 opacity: compilerLink ? 1 : 0,
 scale: compilerLink || isValid ? 1 : 0.8,
 }}
 className={`w-[clamp(30px,9vw,110px)] h-[clamp(11px,3vw,17px)] font-bold transition-colors duration-200
 hover && hover.i === i && hover.s === s && hover.t === t && hover.v === v && hover.h === h
 ? "bg-sky-500 text-white"
 : isActive
 ? "bg-indigo-500 text-white"
 : isSrc1
 ? "bg-emerald-500 text-white"
 : isSrc2
 ? "bg-amber-500 text-white"
 : isVisible
 ? "bg-slate-800 text-white"
 : compilerLink
 ? "bg-slate-950 text-white"
 : "bg-transparent text-slate-500"
 }
 >
 {displayValue}
 </div>
 </div>
)

```

```

 { /* Formula helper text */ }
 <div className="bg-slate-900 border border-slate-
 { compilerLinked && liveTarget ? (
 <p className="text-base text-slate-300">

 i =
 – активна ячейка <span className="font-mono
 {(hasLiveTable || hasLiveSource) && <> = <sp
 ?. [liveTarget.j] ?? (liveTarget.j === 0 ? l
 </p>
) : step > 0 && step <= maxStep ? (
 (() => {
 const c = computeSteps[step - 1];
 const half = 1 << (c.j - 1);
 return (
 <p className="text-lg text-slate-300">

 ST[{c.i}][{c.j}]
 {" "}
 = min(
 <span className="text-emerald-400 font-l
 ST[{c.i}][{c.j - 1}]

 ,
 <span className="text-amber-400 font-bo
 ST[{c.i + half}][{c.j - 1}]

) → min(
 {st1D

 {st1D[c.i + half][c.j - 1]}

) = {" "}
 <span className="text-indigo-400 font-b
 {st1D[c.i][c.j]}

 </p>
);

```

```

const compilerLinked = codeCell !== null || liveK !== null
const shownK = pinnedK !== null
 ? pinnedK
 : liveK !== null
 ? Math.max(0, Math.min(3, Math.trunc(liveK)))
 : hasLiveSource
 ? 0
 : k;

/** 2D-демонстрация нарисована на фиксированной сетке
const liveSource2D = liveMatrix(vars?.A);
const dimMismatch2D = liveSource2D
 ? liveSource2D.rows !== 8 || liveSource2D.cols !== 8
 ? `Демо 2D построено на матрице 8x8, а в вашем коде
 не её не отрисовываются, значения вашей матрицы
 : null
 : null;

// Уровень и клетка берутся прямо из k/r/c; номер строки
useEffect(() => {
 setHover(null);
 setLocked(null);
}, [k]);

const L = 1 << shownK;
const activeCell = codeCell || locked || hover;

return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 scroll-smooth">
 <h3 className="text-xl font-bold text-indigo-400">Матрица
 <p className="text-sm text-slate-400 mb-6 text-slate-500">
 Для наглядности показываем только <b className="text-rose-300">

 иц!
 </p>
 {dimMismatch2D ? (

```

```

<span className={`text-[10px] font-
rder-indigo-800/40 text-indigo-300` : `bg-s
</h4>
<div className="max-w-full overflow-
 <div className={`grid gap-[2px] p-
go-900/40 shadow-[0_0_25px_rgba(99,102,241,
eColumns: `repeat(${stepSize}, max-content)
 {Array.from({length: stepSize * st
 const r = Math.floor(idx / step
 const c = idx % stepSize;

 const liveValue = liveTable?.[`
 // Базовый слой (блоки 1x1) – э
 // одного элемента равен ему сам
 // показываем из A сразу, как A
 // цикла построения st2. Уровни
 const sourceValue = step === 0
 const shownValue = compilerLink
 const isBlank = shownValue ===
 let highlightClass = isBlank
 ? `bg-slate-950 text-slate-600
 : isCurr
 ? `bg-slate-800/80 text-sla
 : `bg-slate-800/50 text-sla
 let zIndex = `z-0`;

 const isActive = !!activeCell &
ll.c + L) && ((r - activeCell.r) % (1 << st

 if (isActive) {
 if (isCurr) {
 // Значения в ячейке ещё не
 // контур «сюда смотрит код
 // залитая пустая клетка вы
 highlightClass = isBlank
 ? `bg-indigo-500/10 text-
 == c ? `ring-[3px] ring-white ring-offset-2
 : `bg-indigo-500 text-whi

```



```

 title={hasLiveTable ? `st2[
 >
 {isBlank ? "." : String(shor
 </div>
 });
 }}}
 </div>
</div>
</div>
</React.Fragment>
)
 }}}
</div>
</div>

<div className="flex-1 min-w-0 flex flex-col gap-
 <div className="bg-slate-900 p-4 sm:p-6 rounded
 <h3 className="text-lg font-bold text-slate-3
 {shownK === 0 ? (
 <div className="text-slate-400 text-sm lea
 {liveTableEmpty ? (
 <div className="rounded-xl border bord
 <p className="font-bold text-amber-3
 <p className="mt-2 text-xs text-slate
 Слева на вкладке «Оригинал (1x1)»
 блоки 1x1 равны её элементам. Пуст
 циклам:
 </p>
 <div className="mt-3 rounded-lg bg-[

 </div>
 </div>
) : (
 <>
 <p className="mb-4">При <span classN
драты имеют размер <span className="font-bo
 <div className="bg-[#0a0fle] rounded
 /

```

```

ded border border-rose-500/20"><Lock size={
 </p>
 <p className="text-indigo-300 borde
 <span className="font-bold text-w
 }}[shownK]][shownK]

 </p>

 <div className="space-y-2 pt-3 pl-2
 {(() => {
 const prevL = 1 << (shownK -
 return (
 <>
 <div className="flex ite
 <span className="flex
shadow-[0_0_8px_#f43f5e]"></div> ST[{activeC
 <span className="text
tiveCell.r][activeCell.c][shownK-1][shownK-
 </div>
 <div className="flex ite
 <span className="flex
shadow-[0_0_8px_#3b82f6]"></div> ST[{activeC
 <span className="text
tiveCell.r][activeCell.c + prevL][shownK-1]
 </div>
 <div className="flex ite
 <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{a
 <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
 </div>
 <div className="flex ite
 <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{activ
 <span className="text
[activeCell.r + prevL][activeCell.c + prevL
 </div>
 </>
 </>
 }
)
}

```

```

const nc2 = vizNumber(vars?.c2);
if (nr1 !== null) setR1(Math.max(0, Math.min(7, M
if (nc1 !== null) setC1(Math.max(0, Math.min(7, M
if (nr2 !== null) setR2(Math.max(0, Math.min(7, M
if (nc2 !== null) setC2(Math.max(0, Math.min(7, M
}, [runtime]));

const queryVars = runtime?.variables;
const liveCoords = ["r1", "c1", "r2", "c2"].map((na
const queryMismatch = liveCoords.some((v) => v !== r
 ? `Координаты запроса в вашем коде выходят за дем
] ?? "-"}, r2=${liveCoords[2] ?? "-"}, c2=${live
 : null;

const H = Math.max(1, r2 - r1 + 1);
const W = Math.max(1, c2 - c1 + 1);
const kx = Math.floor(Math.log2(H));
const ky = Math.floor(Math.log2(W));
const Lx = 1 << kx;
const Ly = 1 << ky;

const matRows = 8 - Lx + 1;
const matCols = 8 - Ly + 1;

return (
 <div className="flex flex-col xl:flex-row gap-6
 <div className="flex-1 min-w-0 bg-slate-900
 <h3 className="text-xl font-bold text-ro
 <p className="text-slate-400 text-[13px]
 еpx, r2, c2 - правый низ).</p>
 {queryMismatch ? (
 <p className="mb-3 rounded-lg border b
 {queryMismatch}
 </p>
) : null}

 <div className="flex flex-col items-cent
 <div className="grid grid-cols-[repea

```

```

 <div className="absolute pointer-e
00 shadow-[0_0_10px_#f59e0b]" style={{ top:
00}% + 8px)`, width: `calc(${(Ly / 8) * 100
</div>

```

```

 <div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset_0_2px_10px_rgba(
 <label className="flex items-cen
 <span className="text-slate-
 <div className="flex gap-2">
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 </div>
 </label>
 <label className="flex items-cen
 <span className="text-slate-
 <div className="flex gap-2">
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 </div>
 </label>
 </div>
 </div>
</div>

```

```

<div className="flex-1 bg-slate-900 p-6 rou
 <h3 className="text-xl font-bold text-em
 <p className="text-sm font-sans text-sla
 Мы знаем, что максимальная степень дв
rounded text-white">{Lx}. Аналогично
te">{Ly}.
 </p>
 <p className="text-sm font-sans text-sla
 Чтобы покрыть весь прямоугольник, мы

```

```

 {'}'}));
</div>

<div className="w-full flex justify-center"
 <div className="flex flex-col items-center"
 <h4 className="text-slate-400 font-bold"
 Откуда берутся эти 4 числа: матрица
border-slate-700 shadow-sm">ST[][][kx]][ky]
 </h4>
 <div className="grid gap-[2px] p-2.5"
Columns: `repeat(${matCols}, 1fr)`}}>
 {Array.from({length: matRows * matCols}, (v, idx) => {
 const r = Math.floor(idx / matCols);
 const c = idx % matCols;
 let isTL = r === r1 && c === c1;
 let isTR = r === r1 && c === c2;
 let isBL = r === r2 - Lx + 1 && c === c1;
 let isBR = r === r2 - Lx + 1 && c === c2;

 let color = "bg-slate-800 text-white";
 let txt = ST2D[r][c][kx][ky];

 let badge = null;
 if (isTL) { color = "bg-rose-500 text-white";
0 border-2"; badge = "TL"; }
 else if (isTR) { color = "bg-rose-500 text-white";
ue-300 border-2"; badge = "TR"; }
 else if (isBL) { color = "bg-emerald-500 text-white";
-emerald-300 border-2"; badge = "BL"; }
 else if (isBR) { color = "bg-emerald-500 text-white";
mber-300 border-2"; badge = "BR"; }

 return (
 <div key={idx} className={color}
tion-all duration-300 relative shrink-0 ${c}
 {txt}
 {badge && <div className="border-2 border-slate-700 text-white px-1 font-bold rounded

```

```

 </div>
 </main>
</div>

<script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal').cla
 ? 'px-3 py-1 rounded text-sm font-bold l
 : 'px-3 py-1 rounded text-sm font-bold

 document.getElementById('btn-mode-wtf').cla
 ? 'px-3 py-1 rounded text-sm font-bold l
 : 'px-3 py-1 rounded text-sm font-bold

 const aside = document.querySelector('aside')
 const headerMain = document.querySelector('h1')
 const questionsContainer = document.getElementById('questions')
 const wtfContainer = document.getElementById('wtf')

 if (mode === 'wtf') {
 document.body.classList.add('wtf-mode')
 if (aside) aside.style.display = 'none'
 if (headerMain) headerMain.style.display = 'none'
 if (questionsContainer) questionsContainer.style.display = 'none'
 if (wtfContainer) wtfContainer.classList.remove('hidden')

 // Remove the URL hash to avoid scrolling
 history.pushState("", document.title, window.location.pathname + window.location.search)
 window.scrollTo(0, 0);
 } else {
 document.body.classList.remove('wtf-mode')
 if (aside) aside.style.display = '';
 if (headerMain) headerMain.style.display = '';
 if (questionsContainer) questionsContainer.style.display = '';
 if (wtfContainer) wtfContainer.classList.add('hidden')
 }
 }

```

```

function toggleDrawer() {
 const isClosed = mobileDrawer.classList.contains('closed')
 if (isClosed) {
 mobileDrawer.classList.remove('-translateX-100%')
 if (drawerOverlay) {
 drawerOverlay.classList.remove('opacity-0')
 // Timeout allows display:block to be applied
 setTimeout(() => drawerOverlay.classList.remove('opacity-0'), 0)
 }
 document.body.style.overflow = "hidden"
 } else {
 mobileDrawer.classList.add('-translateX-100%')
 if (drawerOverlay) {
 drawerOverlay.classList.add('opacity-100')
 setTimeout(() => drawerOverlay.classList.add('opacity-100'), 0)
 }
 document.body.style.overflow = "";
 }
}

if (mobileMenuBtn && mobileDrawer) {
 mobileMenuBtn.addEventListener("click", () => toggleDrawer())
}
if (closeDrawerBtn && mobileDrawer) {
 closeDrawerBtn.addEventListener("click", () => toggleDrawer())
}
if (drawerOverlay) {
 drawerOverlay.addEventListener("click", () => toggleDrawer())
}

// Close on link click
document.querySelectorAll("#mobile-drawer a").forEach(link => {
 link.addEventListener("click", () => {
 if (mobileDrawer && !mobileDrawer.classList.contains('closed')) {
 toggleDrawer()
 }
 })
})
});
});

```

```
<html lang="ru" class="scroll-smooth">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,
 <title>Пособие по алгебре: От пиццы к предикатам</t
 <script src="https://cdn.tailwindcss.com"></script>
 <script src="https://polyfill.io/v3/polyfill.min.js
 <script id="MathJax-script" async src="https://cdn.
 <script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']]
 displayMath: [['$$', '$$'], ['\\[', '\\
 }
 };
 </script>
 <style>
 @import url('https://fonts.googleapis.com/css2?
 @reference "./src/index.css";
 .math-font { font-family: 'Fira Code', monospac
 .uno-card {
 width: 70px; height: 105px; border-radius:
 box-shadow: 0 4px 8px rgba(0,0,0,0.5); posi
 align-items: center; justify-content: cente
 color: white; overflow: hidden; flex-shrink
 text-shadow: 2px 2px 0 #000, -1px -1px 0 #0
 font-family: 'Arial Black', Impact, sans-se
 }
 .uno-card::before, .uno-card::after {
 content: attr(data-val); position: absolute
 text-shadow: 1px 1px 0 #000, -1px -1px 0 #0
 }
 .uno-card::before { top: 2px; left: 4px; }
 .uno-card::after { bottom: 2px; right: 4px; tra
 .uno-card.mini { width: 45px !important; height
 .uno-card.mini::before, .uno-card.mini::after {

 /* Global formula scroll settings */
```



```

 .uno-equation .text-2xl { font-size: 1.2rem }
 .uno-equation.gap-4 { gap: 0.5rem !important }

 /* Allow horizontal scroll for equations in
 .formula-scroll { flex-wrap: nowrap !important;
 px; }
 }
 .uno-inner {
 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
 }
 .uno-val { position: relative; z-index: 2; }
 .card-red { background-color: #e52521; } .card-
 .card-blue { background-color: #004dcf; } .card-
 .card-green { background-color: #339e35; } .card-
 .card-yellow { background-color: #ffc400; } .ca
 .card-black { background-color: #222; } .card-b
 .card-black::before, .card-black::after { text-
 .info-block { @apply bg-slate-800 border-l-4
 .info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
 ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
 king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
 r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 fo

/* THEME OVERRIDES */
body.theme-light {
 background-color: #f8fafc !important;
 color: #0f172a !important;
}
body.theme-light .bg-slate-900, body.theme-light
body.theme-light .bg-slate-800 { background-col

```

```

 animation: creepy-blink 4s infinite;
 }
 .creepy-eye-alt {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
 }
 .creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
 }
</style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

 <div class="sticky top-0 z-50 w-full bg-slate-950/90
 center items-center gap-4 sm:gap-8 transition-colors">
 <div class="flex items-center gap-3">
 Тема
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setMode('normal')" id="btn-normal"
 transition-colors">Список тем</button>
 <button onclick="setMode('wtf')" id="btn-wtf"
 transition-colors">WTF</button>
 </div>
 </div>
 <div class="flex items-center gap-3">
 Тема
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setTheme('dark')" id="btn-dark"
 transition-colors">Темная</button>
 <button onclick="setTheme('light')" id="btn-light"
 transition-colors">Светлая</button>
 <button onclick="setTheme('notebook')" id="btn-notebook"
 transition-colors">Тетрадь</button>
 </div>
 </div>
 </div>

```

```

 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
 </g>
 <g id="lego-rem">
 <rect x="0" y="0" width="40" height="10"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
 </g>
 <!-- МЯСОРУБКА -->
 <g id="meat-grinder">
 <path d="М 10 40 L 15 20 Q 25 15 35 20
 <rect x="5" y="40" width="40" height="2"
 <!-- Ручка -->
 <path d="М 45 50 Q 60 50 60 30 L 75 30"
 <circle cx="75" cy="30" r="4" fill="#333"
 <!-- Выход -->
 <path d="М 5 45 L -5 45 L -5 55 L 5 55
 <circle cx="25" cy="50" r="15" fill="#f
 </g>
 <g id="lego-green">
 <rect x="0" y="10" width="20" height="1"
 <rect x="3" y="5" width="14" height="5"
 </g>
 <g id="lego-white">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="0" y1="2" x2="40" y2="2" stro
 <text x="20" y="14" fill="#475569" font
 </g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->
<button id="mobile-menu-btn" class="lg:hidden fixed
 0/50 z-50 hover:bg-blue-500 transition-transform
 <svg xmlns="http://www.w3.org/2000/svg" class="

```

```

 <a href="#section-1-3" class="b
 <a href="#section-1-4" class="b
</div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-2" class:
er-indigo-500 pl-3 font-bold text-indigo-300
 2. Комплексные числа

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#section-2-1" class="b
 <a href="#section-2-2" class="b
 <a href="#section-2-3" class="b
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-3" class:
er-teal-500 pl-3 font-bold text-teal-300">
 3. Тригонометрическая форма

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#section-3-1" class="b
 <a href="#section-3-2" class="b
 <a href="#section-3-3" class="b
 <a href="#section-3-4" class="b
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-4" class:
er-amber-500 pl-3 font-bold text-amber-300">
```

```

 <a href="#q6-3" class="block ho
 <a href="#q6-4" class="block ho
 <a href="#q6-5" class="block ho
 <a href="#q6-6" class="block ho
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-7" class=
er-pink-500 pl-3 font-bold text-pink-300">
 7. Взаимно простые числ

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q7-1" class="bloc
 <a href="#q7-2" class="bloc
 <a href="#q7-3" class="bloc
 <a href="#q7-4" class="bloc
 <a href="#q7-5" class="bloc
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-8" class=
er-violet-500 pl-3 font-bold text-violet-300">
 8. Сравнимость и Поля В

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q8-1" class="bloc
 <a href="#q8-2" class="bloc
 <a href="#q8-3" class="bloc
 <a href="#q8-4" class="bloc
 <a href="#q8-5" class="bloc
 <a href="#q8-6" class="bloc
 <a href="#q8-7" class="bloc
```

```
der-cyan-500 pl-3 font-bold text-cyan-400">
 11. НОД и НОК многочлен

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q11-1" class="blo
 <a href="#q11-2" class="blo
нственности НОД
 <a href="#q11-3" class="blo
ПИРАЛЬ)
 <a href="#q11-4" class="blo
вращается!)
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-12" clas
der-blue-500 pl-3 font-bold text-blue-400">
 12. Взаимно простые мно

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q12-1" class="blo
 <a href="#q12-2" class="blo
 <a href="#q12-3" class="blo
еление)
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-13" clas
der-emerald-500 pl-3 font-bold text-emerald
 13. Корни многочленов.

 </summary>
 <div class="pl-6 space-y-2 text
```

```
 <details class="group">
 <summary class="list-none cursor-pointer text-red-500 pl-3 font-bold text-red-400">
 16. Неприводимые многоч.

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>

 <div class="mt-4 mb-2 text-xs font-l">


```

```

 let visibleId = null;
 entries.forEach(entry => {
 if (entry.isIntersecting) {
 visibleId = entry.target.id;

 // Highlight active
 document.querySelector(
 `a[href="#${visibleId}"]`
).style.fontWeight = 'bold';
 }
 });

 // Open the parent
 const correspondingLi = document.querySelector(
 `li[id="${visibleId}"]`
);
 if (correspondingLi) {
 const details = correspondingLi.querySelector(
 'details'
);
 if (details && !details.open) {
 details.open = true;
 document.querySelector(
 `a[href="#${visibleId}"]`
).style.fontWeight = 'normal';
 }
 }
 });
});

}, {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: [0, 0.5]
});

sections.forEach(section => {
 observer.observe(section);
});
});

```



```

textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';

```

```

const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';

```

```

const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#333';
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
 if (navigator.clipboard) {
 navigator.clipboard.writeText(textarea.value);
 copyBtn.innerText = 'Скопировано';
 } else {
 textarea.select();
 document.execCommand('copy');
 copyBtn.innerText = 'Скопировано';
 }
 setTimeout(() => copyBtn.innerText = 'Копировать', 2000);
};

```

```

const closeBtn = document.createElement('button');
closeBtn.innerText = 'Закрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = '#333';
closeBtn.style.color = '#fff';
closeBtn.style.border = 'none';
closeBtn.style.borderRadius = '5px';

```

```

 fallbackModal(text, 'Markdown')
 return;
 }
 const blob = new Blob([text], {
 const url = URL.createObjectURL
 const link = document.createElement('a');
 link.href = url;
 link.download = 'vyisshaya_algebra.pdf';
 document.body.appendChild(link);
 link.click();
 document.body.removeChild(link);
}

function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate-500');

 document.getElementById('theme-dark').checked = theme === 'dark';
 document.getElementById('theme-light').checked = theme === 'light';
 document.getElementById('theme-auto').checked = theme === 'auto';

 let oldStyle = document.getElementById('theme-style');
 if (oldStyle) oldStyle.remove();
 const style = document.createElement('style');
 style.id = 'theme-style';

 if (theme === 'dark') {
 body.classList.add('bg-slate-500');
 document.getElementById('theme-dark').checked = true;
 } else if (theme === 'light') {
 body.classList.add('bg-white');
 document.getElementById('theme-light').checked = true;
 style.innerHTML = `
 body.bg-white section header {
 body.bg-white main { color: #000000; }
 `;
 }
}

```

```

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuch
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre
body.bg-white .text-ind

`;
} else if (theme === 'terminal')
 body.classList.add('bg-black')
 document.getElementById('theme')
 style.innerHTML = `
 body.bg-black .bg-slate-950 {
 background-color: #22c55e;
 }
 body.bg-black section header,
 body.bg-black button {
 color: #4ade80 !important;
 border-color: #22c55e !important;
 font-family: monospace;
 }
 body.bg-black svg path,
 stroke: #22c55e !important;
 fill: transparent !important;
 `;
}

document.head.appendChild(style)
</script>

```

```

 терминала"> </button>
 </div>
 </div>
 </div>

</div>
</aside>

<!-- Основной контент -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16
-base min-w-0">
 <header class="text-center mb-12 border-b border-slate-700">
 <h1 class="text-4xl md:text-5xl font-bl
4">
 ВЫСШАЯ АЛГЕБРА
 </h1>
 <p class="text-slate-400 italic text-lg">
 Высшая алгебра: теория и практика

 <!-- Print Table of Contents -->
 <div class="hidden print:block mt-8 text-center">
 <h2 class="font-bold text-xl mb-4">Содержание
 <ul class="text-base space-y-2">
 1. Алгебра
 2. Коммутативная алгебра
 3. Топология
 4. Статистика
 5. Группы
 6. Дифференциальная геометрия
 7. Алгебраические структуры
 8. Квантовая механика
 9. Комплексные функции
 10. Теория вероятностей
 11. Теория чисел
 12. Теория игр
 13. Теория информации
 14. Теория оптимизации
 15. Теория управления

```

```

 <div class="font-bold text-slate-500">

 <!-- Row 2: Базовые операции / Евклид / НОД и НОК

-4 border-yellow-500 hover:bg-slate-700 transition-colors duration-200
 <div class="text-xs text-yellow-500">
 <div class="font-bold text-slate-500">

-1-4 border-orange-500 hover:bg-slate-700 transition-colors duration-200
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-500">

-4 border-cyan-500 hover:bg-slate-700 transition-colors duration-200
 <div class="text-xs text-cyan-500">
 <div class="font-bold text-slate-500">

 <!-- Row 3: НОД и НОК / Алгоритм Евклида

-4 border-yellow-500 hover:bg-slate-700 transition-colors duration-200
 <div class="text-xs text-yellow-500">
 <div class="font-bold text-slate-500">

-1-4 border-orange-500 hover:bg-slate-700 transition-colors duration-200
 <div class="text-xs text-orange-500">
 <div class="font-bold text-slate-500">

 <!-- Empty 3rd row cell -->
 <div class="hidden md:block"></div>

 <!-- Row 4: Факторизация / Структурная теория

-4 border-yellow-500 hover:bg-slate-700 transition-colors duration-200
 <div class="text-xs text-yellow-500">

```

```
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

<div class="hidden md:block"></div>

<!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
<div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл

 <a href="#question-17" onclick="setl
-l-4 border-teal-500 hover:bg-slate-700 tra
 <div class="text-xs text-teal-5
 <div class="font-bold text-slate

 <a href="#question-18" onclick="setl
-l-4 border-blue-500 hover:bg-slate-700 tra
 <div class="text-xs text-blue-5
 <div class="font-bold text-slate

 <a href="#question-19" onclick="setl
-l-4 border-emerald-500 hover:bg-slate-700
 <div class="text-xs text-emeral
 <div class="font-bold text-slate

 <a href="#question-20" onclick="setl
-l-4 border-rose-500 hover:bg-slate-700 tra
 <div class="text-xs text-rose-5
 <div class="font-bold text-slate

 <a href="#question-21" onclick="setl
-l-4 border-indigo-500 hover:bg-slate-700 t
 <div class="text-xs text-indigo
 <div class="font-bold text-slate

 <a href="#question-22" onclick="setl
-l-4 border-fuchsia-500 hover:bg-slate-700
```

```

<!-- Прямое -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a2b3c; color: white;">
 <h3 class="text-xl text-blue-400">Прямое
 <p class="text-sm text-slate-400">Прямое
 <ul class="list-disc pl-5 sp-2">

 k$).

 ово.

</div>

<!-- От противного -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a2b3c; color: white;">
 <h3 class="text-xl text-fuchsia-400">От противного
 <p class="text-sm text-slate-400">От противного
 <ul class="list-disc pl-5 sp-2">

 i>

 т.д.).

 что так и задумано: "Ой, противоречие, теорема не верна".

</div>

<!-- Единственность -->
<div class="bg-slate-800/80 p-6" style="background-color: #1a2b3c; color: white;">
 <h3 class="text-xl text-emerald-400">Единственность
 <p class="text-sm text-slate-400">Единственность
 <ul class="list-disc pl-5 sp-2">

 , r_2$).

 счет ограничений (размер остатка) докажи, что


```

```
 <p class="text-xs font-
со следующим. Если в конце 0 – пациент мерт
</div>
```

```
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
c. Пока получается 0 – корень жив. Как то
корня минус один.</p>
</div>
```

```
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
ициентов (они делятся на p), кроме старше
зложить.</p>
</div>
```

```
 <div class="bg-slate-800 p-
 <h4 class="text-red-400
 <p class="text-sm text-
 <p class="text-xs font-
. Чаще всего после этого Фейс-контроль p
</div>
</div>
```

```
 <h2 class="text-2xl font-bold b
(Правила экстрасенса)</h2>
 <p class="text-sm text-slate-40
отношения), а ты их не помнишь. Юзай базовые
нейность) для делимости.</p>
```

```
 <ul class="space-y-4 pb-10">
 <li class="bg-slate-800/50 p-

```



```
 </section>
 </div>
```

```
 <div id="questions-container">
```

```
 <!-- ===== ВОПРОС 1 =====
```

---

## РАЗДЕЛ: УТИЛИТЫ

---

---

ФАЙЛ 87/111: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
```

```
export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 88/111: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 341 | РАЗМЕР: 15.5 KB

---

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";
import { annotateTicketLinks } from "../ticketLinks";
```

```
/**
```

```
 * Выгрузка главы (или всего пособия) в самостоятельный
```

```
 *
```

```

.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid #0f172a; }
.export-gloss h2 { color: #fff; font-size: 1.25rem; font-weight: bold; }
.export-term { background: #0f172a; border: 1px solid #0f172a; padding: 10px; }
.export-term h3 { color: #c7d2fe; font-size: .95rem; font-weight: bold; }
.export-term p { margin: 0 0 .3rem; font-size: .85rem; }
.export-term .formal { color: #94a3b8; font-size: .8rem; font-weight: bold; }
.export-term .cx { color: #6ee7b7; font-size: .78rem; font-weight: bold; }
.export-foot { margin-top: 3rem; border-top: 1px solid #0f172a; }
body { font-size: 15px; line-height: 1.6; }
.export-shell p, .export-shell li { line-height: 1.65; }
.export-shell pre { font-size: 12px; line-height: 1.5; }
.export-figs { margin: 0 0 1.6rem; }
.export-figs-title { color: #e2e8f0; font-size: .95rem; font-weight: bold; }
.export-fig { margin: 0 0 .9rem; }
.export-fig img { width: 100%; border: 1px solid #1e293b; }
.export-fig figcaption { color: #94a3b8; font-size: .75rem; }
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important; }
 pre { white-space: pre-wrap; }
}
`;

```

```

const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">");

/**
 * Размечает термины и превращает их в якорные ссылки на
 * Возвращает готовый HTML главы и список найденных терминов
 */
function prepareBody(html: string, opts: { selfId?: string;
 body: string;
 termIds: string[];
}) {
 const host = document.createElement("div");

```

```

// <details> в файле лучше раскрыть: иначе код и разб
host.querySelectorAll("details").forEach((d) => d.set

return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[], book = false) {
 if (!termIds.length) return "";
 const items = termIds
 .map((id) => {
 const e = GLOSSARY_BY_ID.get(id);
 if (!e) return "";
 return `<div class="export-term" id="term-${escape
<h3>${escapeHtml(e.title)}</h3>
<p>${escapeHtml(e.short)}</p>
${e.formal ? `<p class="formal">${escapeHtml(e.formal
${e.complexity ? `<p class="cx">Сложность: ${escapeHt
</div>`;
 })
 .join("\n");

 return `<section class="export-gloss">
<h2>${book ? "Словарик терминов пособия" : "Словарик
<p style="font-size:.8rem;color:#94a3b8;margin:0 0 1r
 ны сюда.</p>
 ${items}
</section>`;
}

function wrap(opts: { title: string; subtitle: string;
 const stamp = new Date().toLocaleString("ru-RU");
 return `<!doctype html>
<html lang="ru">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, init
<title>${escapeHtml(opts.title)}</title>
<style>${opts.css}</style>

```

```

const src = img.getAttribute("src") ?? "";
if (!src || src.startsWith("data:")) return;
try {
 const res = await fetch(src);
 if (!res.ok) return;
 const blob = await res.blob();
 if (blob.size > 4_000_000) return;
 const data = await new Promise<string>((resolve, reject) => {
 const fr = new FileReader();
 fr.onload = () => resolve(String(fr.result));
 fr.onerror = () => reject(fr.error);
 fr.readAsDataURL(blob);
 });
 img.setAttribute("src", data);
} catch {
 /* картинка недоступна — глава останется со ссылкой */
}
});
return doc.body.innerHTML;
}

/**
 * Кадры визуализаций: книга встраивает снимки демонстраций,
 * файл показывал, что крутилось в приложении. Снимки делались
 * (capture-viz-shots.mjs) и публикует в /export/viz-shots.
 * без них (офлайн-сборка, локальный дев) книга остаётся
 * нормально и молча пропускается.
 */
async function vizFigures(chapterId: string): Promise<string> {
 try {
 const base = import.meta.env.BASE_URL || "/";
 const manifestRes = await fetch(`${base}export/viz-figures/${chapterId}.json`);
 if (!manifestRes.ok) return "";
 const manifest = (await manifestRes.json()) as { files: string[] };
 const mine = manifest.filter((m) => m.file === `${chapterId}.mjs`);
 const figures: string[] = [];
 for (const m of mine.slice(0, 4)) {
 const res = await fetch(`${base}${m.file}`);
 if (!res.ok) continue;
 const blob = await res.blob();
 const dataUrl = await new Promise<string>((resolve, reject) => {
 const fr = new FileReader();
 fr.onload = () => resolve(String(fr.result));
 fr.onerror = () => reject(fr.error);
 fr.readAsDataURL(blob);
 });
 figures.push(dataUrl);
 }
 return figures.join("\n");
 } catch {
 /* ошибка загрузки — глава останется со ссылкой */
 }
}

```

```

 const base = id.replace(/^[^a-z0-9-]+/gi, "-").toLowerCase()
 return `${num ? String(num).padStart(2, "0") + "-":
}
```

```
/** Собрать автономный HTML одной главы. */
```

```
export async function buildChapterHtml(chapter: Chapter)
 const css = await collectCss();
 const { body: rawBody, termIds } = prepareBody(chapter);
 const body = (await vizFigures(chapter.id)) + (await ...);
 return wrap({
 title: chapter.title,
 subtitle: chapter.category || "Универсальное пособие",
 css,
 body,
 gloss: glossarySection(termIds),
 origin: window.location.origin,
 });
}
```

```
/** Выгрузить одну главу. */
```

```
export async function downloadChapterHtml(chapter: Chapter)
 triggerDownload(await buildChapterHtml(chapter), safe)
}
```

```
/** Собрать книгу: всё пособие одним файлом со сквозным
```

```
export async function buildBookHtml(chapters: Chapter[])
 const css = await collectCss();
 const allTerms: string[] = [];
 const bodies: string[] = [];
```

```
 const toc = chapters
```

```
 .map((c) => `-

```

```
 for (const c of chapters) {
```

```
 const { body: rawBody, termIds } = prepareBody(c);
 // картинки глав встраиваются сразу: книга одна, ас
 // снимки демонстраций подтягиваются из /export/viz
```

```

import { chapterTopics } from "../data/content";

/**
 * Перекрёстные ссылки между билетами.
 *
 * В тексте страниц десятки упоминаний вида «...берите Фор
 * или «сердце Краскала (билет 18)». Раньше это был про
 * номер, но попасть на него не мог — ссылка была «битой
 * в никуда. Здесь такие упоминания превращаются в насто
 *
 * • в приложении — `

```

```

 inCode = Math.max(0, inCode + (closing ? -1 : 1));
 } else if (tag === "a") {
 inLink = Math.max(0, inLink + (closing ? -1 : 1));
 }
 return part;
}
if (inCode > 0 || inLink > 0) return part;
return part.replace(TICKET_RE, (whole, a: string,
 const from = Number(a);
 const to = b ? Number(b) : from;
 const id = chapterForRange(from, to);
 if (!id || id === opts.selfId) return whole;
 return `<a class="ticket-link" data-goto="${id}
 });
});
.join("");
}

/**
 * DOM-версия — для интерактивной страницы. Проходит то же самое
 * (как `annotateTerms`), поэтому атрибуты и разметка не меняются.
 * Возвращает число созданных ссылок — его проверяют DOM-версии.
 */
export function annotateTicketLinks(
 root: HTMLElement,
 opts: { selfId?: string; lite?: boolean } = {}
): number {
 if (typeof document === "undefined") return 0;
 const walker = document.createTreeWalker(root, SHOW_TEXT,
 acceptNode(node) {
 const parent = (node as Text).parentElement;
 if (!parent) return FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return FILTER_REJECT;
 const text = node.nodeValue ?? "";
 if (/билет/i.test(text)) return FILTER_REJECT;
 return FILTER_ACCEPT;
 },
);
 return 0;
});

```

```

export function useTicketLinks(
 ref: React.RefObject<HTMLElement | null>,
 deps: unknown[],
 opts: { selfId?: string; lite?: boolean } = {}
) {
 useEffect(() => {
 const el = ref.current;
 if (el) annotateTicketLinks(el, opts);
 // eslint-disable-next-line react-hooks/exhaustive-deps
 }, deps);
}
```

## =====

## РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

## =====

-----  
ФАЙЛ 90/111: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМЕР: 1.5 КБ

```

/**
 * Аудит покрытия глав пособия: у каких тем нет «объяснений»
 * (блок «Аналогия») и/или 2D-визуализации (интерактивной)
 *
 * node scripts/audit-coverage.mjs # отчёт
 * node scripts/audit-coverage.mjs --md # markdown
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "
const TMP = path.join(ROOT, "tmp", "audit");

fs.mkdirSync(TMP, { recursive: true });
```



```

const orphanViz = [...vizIds].filter((id) => !chapters.

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) =>

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
 console.log("| # | Глава | Аналогия | 2D-виз. | SVG |
 console.log("| --- | --- | --- | --- | --- | --- |");
 for (const r of rows) {
 console.log(
 `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy
 ge)} |`
);
 }
} else {
 console.log(`Глав в пособии: ${rows.length}\n`);
 const dump = (title, list) => {
 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(` * ${r.title} [$
 console.log();
 };
 dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
 dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
 dump("НЕТ 2D (аналогия есть)", gapViz);
 console.log(`Пропущенные номера билетов 1-24: ${missi
 console.log(`Визуализаторы без главы: ${orphanViz.join
}

```

```

const perSurface = new Map();
let highlights = 0;
const ids = new Set();

for (const m of text.matchAll(re)) {
 const surface = m[1].toLowerCase();
 const id = map.get(surface);
 if (!id) continue;
 const ut = perTerm.get(id) ?? 0;
 const us = perSurface.get(surface) ?? 0;
 if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
 perTerm.set(id, ut + 1);
 perSurface.set(surface, us + 1);
 highlights += 1;
 ids.add(id);
 used.add(id);
}

if (ids.size === 0) bad.push(c.title);
console.log(
 String(ids.size).padStart(3), "|", String(highlights).padStart(3),
 c.title.slice(0, 46).padEnd(47), [...ids].slice(0, 10).join(", ")
);
}
console.log("\nГлав без единой подсказки:", bad.length, "шт.");
console.log("Терминов в глоссарии:", GLOSSARY.length, "шт.");
console.log("Не встретились:", GLOSSARY.filter((g) => !g).length, "шт.");

```

---

ФАЙЛ 92/111: scripts/export/build-guide-pdf.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 677 | РАЗМЕР: 10.5 КБ

---

/\*\*

\* Компактный PDF пособия: public/export/guide\_lite.pdf

\*

\* текст всех тем (без служебной разметки)

```
const CANDIDATES = {
 regular: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
 "/usr/share/fonts/dejavu/DejaVuSans.ttf",
 "/usr/local/share/fonts/DejaVuSans.ttf",
 "/System/Library/Fonts/Supplemental/Arial Unicode.ttf",
 "C:/Windows/Fonts/arial.ttf",
],
 bold: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
 "/usr/share/fonts/dejavu/DejaVuSans-Bold.ttf",
 "/usr/local/share/fonts/DejaVuSans-Bold.ttf",
 "C:/Windows/Fonts/arialbd.ttf",
],
 oblique: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSans-Oblique.ttf",
 "/usr/share/fonts/dejavu/DejaVuSans-Oblique.ttf",
],
 mono: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
 "/usr/share/fonts/dejavu/DejaVuSansMono.ttf",
 "/usr/local/share/fonts/DejaVuSansMono.ttf",
 "C:/Windows/Fonts/consola.ttf",
],
};

const pick = (list) => list.find((f) => fs.existsSync(f));

/**
 * Убираем символы, которых нет в DejaVu (эмодзи и пиктограммы)
 * на их месте появляется «пустой» глиф. Стрелки, математические
 * геометрические фигуры DejaVu знает — их оставляем.
 */
const UNSUPPORTED = /[\u{1F000}-\u{1FAFF}\u{2700}-\u{2700}]/gu;

/** Русское склонение счётных существительных: 1 страница, 2 страницы */
const plural = (n, one, few, many) => {
```

```

 return import("data:text/javascript;base64," + Buffer
}
```

```
/* ----- снимки визуализаций -----
```

```
function shotsFor(chapterId) {
 if (!fs.existsSync(SHOTS_DIR)) return [];
 return fs
 .readdirSync(SHOTS_DIR)
 .filter((f) => /\.{png|jpe?g}$/i.test(f))
 .filter((f) => f === `${chapterId}.png` || f.startsWith(`${chapterId}.`))
 .sort()
 .map((f) => ({ file: path.join(SHOTS_DIR, f), name: f }));
}
```

```
/* ----- отрисовка PDF -----
```

```
class Renderer {
 constructor(doc, fonts) {
 this.doc = doc;
 this.fonts = fonts;
 this.footerTitle = "";
 this.pages = 0;
 this.cy = TOP;
 }
}
```

```
/**
```

```
 * Колонтитул текущей страницы (печатаем до перехода к следующей)
 *
```

```
 * Важно: на время печати нижнее поле страницы уменьшаем
 * решает, что текст не влез, и молча добавляет пустую строку
 */
```

```
stampFooter() {
 if (this.pages === 0 || !this.doc.page) return;
 const prevBottom = this.doc.page.margins.bottom;
 this.doc.page.margins.bottom = 16;
 const y = PAGE.height - PAGE.margin + 4;
 this.doc.font(this.fonts.regular).fontSize(7.6).fill('black').text(
 this.footerTitle, 0, y, { align: 'left', baseline: 'bottom', }
);
 this.doc.page.margins.bottom = prevBottom;
 this.pages++;
}
```

```
}
```

```
/**
```

```
* Режет текст на фрагменты не выше max пунктов: сначала
* предложений, слишком длинное предложение — по словам.
*
```

```
* Без этого pdfkit сам переносит остаток на новую строку,
* создаётся в обход рендерера — без нижнего колонтитула,
* из-за чего в файле оказывалось на одну страницу больше.
*/
```

```
splitToFit(text, opts, max) {
 const measure = (t) => this.height(t, opts);
 const source = String(text);
 if (measure(source) <= max) return [source];
 const chunks = [];
 let current = "";
 const flush = () => {
 if (current.trim()) chunks.push(current.trim());
 current = "";
 };
 for (const sentence of source.split(/(?<=[.!?…;:])\s+/)) {
 const withSentence = current ? `${current} ${sentence}` : sentence;
 if (measure(withSentence) <= max) {
 current = withSentence;
 continue;
 }
 flush();
 if (measure(sentence) <= max) {
 current = sentence;
 continue;
 }
 for (const word of sentence.split(/\s+/)) {
 const withWord = current ? `${current} ${word}` : word;
 if (measure(withWord) <= max) current = withWord;
 else {
 flush();
 current = word;
 }
 }
 }
}
```

```

 this.cy += 4;
 this.doc.font(this.fonts.bold).fontSize(size).fillColor(color);
 this.cy = Math.max(this.cy, this.doc.y) + 2;
 }

 /** Абзац: режим по высоте страницы сами – автоперенос
 para(rawText, { size = 10.0, indent = 0, color = "#1f77b4" });
 const text = plain(rawText);
 if (!text) return;
 const font = mono ? this.fonts.mono : italic && this.fonts.regular;
 const width = CONTENT_W - indent;
 const opts = { font, size, width, lineGap };
 const chunks = this.splitToFit(text, opts, BOTTOM - 10);
 chunks.forEach((chunk, index) => {
 const last = index === chunks.length - 1;
 const h = this.height(chunk, opts);
 this.ensure(h + (last ? gapAfter : 0), 6);
 const y0 = this.cy;
 this.doc.font(font).fontSize(size).fillColor(color);
 this.cy = Math.max(y0 + h, this.doc.y) + (last ? 0 : 10);
 if (this.cy > BOTTOM) this.newPage();
 });
 }

 bullet(rawText, { ordinal = null, depth = 0, size = 9 });
 const text = plain(rawText);
 if (!text) return;
 const indent = 16 + depth * 14;
 const marker = ordinal ? `${ordinal}.` : "*";
 const width = CONTENT_W - indent - 4;
 const opts = { font: this.fonts.regular, size, width, lineGap };
 const chunks = this.splitToFit(text, opts, BOTTOM - 10);
 chunks.forEach((chunk, index) => {
 const h = this.height(chunk, opts);
 this.ensure(h + 2.5, 6);
 const y0 = this.cy;
 if (index === 0) {
 this.doc.font(this.fonts.regular).fontSize(size)

```

```

 this.cy = y0 + boxH + (i < lines.length ? 2 : 7);
 if (i < lines.length) this.newPage();
 }
}

/** Таблица: строки режутся по страницам, заголовок по
table(rawRows, { size = 8.5 } = {}) {
 const rows = rawRows.map((r) => r.map((c) => plain(c)));
 if (!rows.length) return;
 const cols = Math.max(...rows.map((r) => r.length));
 const widths = new Array(cols).fill(CONTENT_W / cols);
 const pad = 5;
 const header = rows[0];

 const rowHeight = (cells) => {
 let max = 0;
 for (let i = 0; i < cols; i++) {
 const h = this.height(cells[i] ?? "", { font: this.font, size });
 max = Math.max(max, h);
 }
 return max + pad * 2;
 };

 const drawRow = (cells, y, isHeader, zebra) => {
 const h = rowHeight(cells);
 this.doc.rect(PAGE.margin, y, CONTENT_W, h).fill(zebra ? "#f2f2f2" : "white");
 for (let i = 0; i < cols; i++) {
 const x = PAGE.margin + widths.slice(0, i).reduce((a, b) => a + b, 0);
 this.doc
 .font(isHeader ? this.fonts.bold : this.fonts.regular)
 .fontSize(size)
 .fillColor("#1f2937")
 .text(String(cells[i] ?? ""), x, y + pad, { width: widths[i] });
 }
 this.doc.rect(PAGE.margin, y, CONTENT_W, h).lineWidth(1);
 return h;
 };
};

```

```

 if (!fs.existsSync(file)) return false;
 let img;
 try {
 img = this.doc.openImage(file);
 } catch {
 return false;
 }
 const capH = caption ? 14 : 0;
 const scale = Math.min(maxWidth / img.width, (maxHe
 const w = img.width * scale;
 const h = img.height * scale;
 this.ensure(h + capH + 10, 20);
 const y0 = this.cy;
 const x0 = PAGE.margin + (CONTENT_W - w) / 2;
 this.doc.image(file, x0, y0, { width: w, height: h
 this.doc.rect(x0, y0, w, h).lineWidth(0.6).strokeCo
 this.cy = y0 + h + 4;
 if (caption) {
 this.doc.font(this.fonts.regular).fontSize(8).fil
 enter" });
 this.cy = Math.max(this.cy, this.doc.y) + 6;
 }
 return true;
}

/** Бокс с заголовком и строками; длинные режутся на
box(rawTitle, rawLines, { color = "#eef2ff", border =
 const title = plain(rawTitle);
 const lines = rawLines.map((l) => plain(l)).filter(
 const inner = CONTENT_W - 20;
 let rest = lines.slice();
 let first = true;
 while (rest.length || first) {
 first = false;
 const headH = this.height(title, { font: this.fon
 this.ensure(headH + 24, 16);
 const y0 = this.cy;
 // сколько строк влезет на эту страницу

```



```

ensureDir(OUT_DIR);

const doc = new PDFDocument({
 size: [PAGE.width, PAGE.height],
 margin: PAGE.margin,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – алгоритмы и струк
 Author: "algv0 • автоэкспорт",
 Subject: "Текст всех тем + визуализации, без кода
 CreationDate: new Date(),
 },
});
const stream = fs.createWriteStream(GUIDE_PDF_PATH);
doc.pipe(stream);

const R = new Renderer(doc, fonts);
const today = new Date().toISOString().slice(0, 10);
const totalTopics = new Set(Object.values(chapterTopics));
const vizCount = chapters.filter((c) => VIZ_REGISTRY[c]).length;

/* — обложка — */
R.firstPage("Обложка");
R.cy = TOP + 90;
R.h1("Универсальное пособие", { size: 26, rule: false });
R.h1("Алгоритмы и структуры данных", { size: 17, color: "#4b5563" });
R.para(
 `Билеты 1-${totalTopics} • ${chapters.length} ${plusSign} ${vizCount}, "интерактивная демонстрация", "интерактивный код"`,
 { size: 10.5, color: "#4b5563" }
);
R.para(
 "Автоматический компактный сборник: весь текст тем + визуализации. Навигация приложения и код панели Python",
 { size: 9.6, color: "#374151" }
);
R.para(`Дата сборки: ${today}`, { size: 9, color: "#6b728b" });

```

```

const chapterTitle = chapter.title.replace(/^\\d+\\. /, "");
for (const b of htmlToBlocks(chapter.content)) {
 if (b.type === "p" && /^Билет\\s/i.test(b.text)) continue;
 if (b.type === "heading" && b.level <= 2 && b.text !== chapterTitle) {
 if (b.type === "heading") {
 if (b.level <= 2) R.h2(b.text);
 else if (b.level === 3) R.h3(b.text);
 else R.para(b.text, { size: 10.2, color: "#334155" });
 } else if (b.type === "summary") {
 R.para("► " + b.text, { size: 9.8, color: "#373737" });
 } else if (b.type === "li") {
 R.bullet(b.text, { ordinal: b.ordinal, depth: b.depth });
 } else if (b.type === "code") {
 R.code(b.text);
 } else if (b.type === "table") {
 R.table(b.rows);
 } else if (b.type === "quote") {
 R.quote(b.text);
 } else if (b.type === "caption") {
 R.para(b.text, { size: 8.3, color: "#6b7280", italic: true });
 } else {
 R.para(b.text);
 }
 }
}

/* иллюстрация страницы */
const img = CHAPTER_IMAGES[chapter.id];
if (img && fs.existsSync(path.join(ROOT, img))) {
 R.h2("Иллюстрация");
 R.image(path.join(ROOT, img), { maxHeight: 300, color: "#ccc" });
}

/* демонстрация: снимки + описание синхронизации с компилятором */
const viz = VIZ_REGISTRY[chapter.id];
const syncKeys = Object.keys(PAGE_SYNC).filter((k) => viz[k]);
if (viz || syncKeys.length) {
 R.h2("Демонстрация и связь с компилятором");
}

```

```

 quiz.forEach((q, qi) => {
 R.para(`${qi + 1}. ${q.question}`, { size: 9.6,
 q.options.forEach((option, oi) => {
 R.bullet(oi === q.correctIndex ? `${option} –
 });
 R.para(q.explanation, { size: 8.8, color: "#4b5
 });
 }
}

R.stampFooter();
doc.end();
await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

const size = fs.statSync(GUIDE_PDF_PATH).size;
console.log("[guide] PDF пособия собран");
console.log(`        тем: ${chapters.length} • снимков: ${pages}`);
console.log(`        выход: ${path.relative(ROOT, GUIDE_PDF_PATH)}`);

// Самопроверка: страниц в файле должно быть ровно столько, сколько
// рендерер. Расхождение означает, что pdftk создал файл с лишними
// в файл без нижнего колонтитула и без нумерации.
const pdfLib = await import("pdf-lib");
const physical = (await pdfLib.PDFDocument.load(fs.readFile(GUIDE_PDF_PATH))).getPages().length;
console.log(`        страниц: ${R.pages} (рендер) / ${physical} (файл)`);
if (R.pages > 100) console.warn(`        ВНИМАНИЕ: ${R.pages} страниц – слишком много`);
if (physical !== R.pages) {
 console.error(`        ОШИБКА: pdftk добавил ${physical - R.pages} страниц`);
 process.exitCode = 1;
}
}

main().catch((err) => {
 console.error("Ошибка сборки PDF пособия:", err.message);
 process.exit(1);
});

```

```
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
 });

 const stream = fs.createWriteStream(PDF_PATH);
 doc.pipe(stream);

 for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0 });
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
 }

 doc.end();
 await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
 });

 console.log("[3/3] png -> pdf");
 console.log(`      страниц: ${pages.length}`);
 console.log(`      выход:   ${path.relative(ROOT, PDF_PATH)}`);
}

main().catch((err) => {
 console.error("Ошибка сборки PDF:", err.message);
 process.exit(1);
});
```

```

 .replace(/^-+|-+$/g, "")
 .slice(0, 24);

async function vizIds() {
 const bundle = await build({
 stdin: {
 contents: `export { activeVizIds } from "../../src";`,
 resolveDir: path.join(ROOT, "scripts", "export"),
 loader: "ts",
 },
 bundle: true,
 write: false,
 format: "esm",
 platform: "node",
 logLevel: "warning",
 });
 const mod = await import("data:text/javascript;base64:" +
 btoa(bundle.code));
 return mod.activeVizIds;
}

fs.mkdirSync(OUT_DIR, { recursive: true });

const ids = await vizIds();
console.log(`[export:shots] страниц со визуализациями: ${ids.length}`);

const browser = await puppeteer.launch({
 headless: true,
 args: ["--no-sandbox", "--disable-setuid-sandbox", "--disable-dev-shm-usage"],
});
const page = await browser.newPage();
await page.setViewport({ width: 1400, height: 1000, deviceScaleFactor: 1 });

let done = 0;
let failed = 0;
for (const id of ids) {
 try {
 await page.goto(`${BASE_URL}/?topic=${id}`, { waitUntil: 'load' });
 await page.waitForSelector("#chapter-viz", { timeout: 10000 });
 const viz = await page.evaluate(() => {
 const viz = document.querySelector("#chapter-viz");
 return viz.innerHTML;
 });
 const vizId = id + ".html";
 const vizPath = path.join(OUT_DIR, vizId);
 fs.writeFileSync(vizPath, viz);
 done++;
 } catch (e) {
 console.error(`[export:shots] страница ${id} не визуализировалась: ${e}`);
 failed++;
 }
}

```

```

 .sort()
 .map((f) => ({ file: f, name: f.replace(/\.(png|jpe?g)/, ".txt") }));
fs.writeFileSync(path.join(OUT_DIR, "manifest.json"), JSON.stringify(manifest));

await browser.close();
console.log(`[export:shots] снимков: ${done}, пропусков: ${misses}`);
// Пропущенные страницы не повод ронять пайплайн: PDF документация
// текстовое описание демонстрации.
process.exit(0);

```

ФАЙЛ 95/111: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМЕР: 3.5 КБ

```

/**
 * ШАГ 1: КОД -> TXT
 *
 * Собирает ВСЕ исходный код репозитория в один текстовый файл
 * public/export/full_code_all_pages.txt
 *
 * Список файлов берётся из git (чтобы ничего не забыть)
 * с фоллбэком на обход файловой системы, если git недоступен.
 */
import fs from "node:fs";
import path from "node:path";
import { execFileSync } from "node:child_process";
import { ROOT, OUT_DIR, TXT_PATH, HR, SUBHR, ensureDir, ensureFile } from "scripts/export/constants";

/** Расширения, которые считаем «кодом» и включаем в документацию */
const CODE_EXT = new Set([
 ".ts", ".tsx", ".js", ".jsx", ".mjs", ".cjs",
 ".css", ".scss", ".html", ".json", ".md", ".yaml", ".yml", ".vue",
]);

/** Явные исключения: генерируемое, бинарное и просто файлы */
const EXCLUDE_FILES = new Set(["package-lock.json", "build", "dist", "node_modules"]);

```

```

 encoding: "utf8",
 })
 .split("\n")
 .filter(Boolean);
} catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path.sep);
 if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
 if (e.isDirectory()) walk(abs);
 else files.push(rel);
 }
 };
 walk(ROOT);
}

return files
 .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d)))
 .filter((rel) => !EXCLUDE_FILES.has(path.basename(rel)))
 .filter((rel) => CODE_EXT.has(path.extname(rel)))
 .filter((rel) => fs.existsSync(path.join(ROOT, rel)))
 .sort((a, b) => {
 const ia = ORDER.indexOf(categoryOf(a));
 const ib = ORDER.indexOf(categoryOf(b));
 return ia !== ib ? ia - ib : a.localeCompare(b);
 }));
}

```

/\*\* Достаём заголовки глав из данных пособия (без запуска)

```

function extractChapters(files) {
 const chapters = [];
 const seen = new Set();
 for (const rel of files.filter((f) => f.startsWith("src/"))) {
 const src = fs.readFileSync(path.join(ROOT, rel), "utf8");
 const re = /id:\s*"([^"]+)"\s*,\s*\n?\s*title:\s*"([^"]+)"/;
 let m;
 }
}

```

```
push(HR);
chapters.forEach((c, i) => {
 push(`  ${String(i + 1).padStart(3, " ")} ${c.title}`);
});
push();

push(HR);
push(" ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНИКОВ")
push(HR);
let current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push();
 push(`  # ${cat}`);
 }
 push(`  ${String(i + 1).padStart(3, " ")} ${rel}`);
});
push();
push();

current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push(HR);
 push(`РАЗДЕЛ: ${cat.toUpperCase()}`);
 push(HR);
 push();
 }
 const content = fs.readFileSync(path.join(ROOT, rel), "utf-8");
 const lines = content.split("\n");
 push(SUBHR);
 push(`ФАЙЛ ${i + 1}/${files.length}: ${rel}`);
 push(`КАТЕГОРИЯ: ${cat} | СТРОК: ${lines.length} | ВЕЛИЧИНА: ${content.length} байт`);
 push(SUBHR);
});
```



Универсальное пособие • полный исходный код

```

import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");

/** Куда кладём финальные артефакты (эта папка отдаётся клиенту) */
export const OUT_DIR = path.join(ROOT, "public", "export");
/** Промежуточные PNG-страницы (в .gitignore, в CI уезжают) */
export const PAGES_DIR = path.join(ROOT, "tmp", "export");

export const TXT_PATH = path.join(OUT_DIR, "full_code_and_comments.txt");
export const PDF_PATH = path.join(OUT_DIR, "full_code_and_comments.pdf");

/** Параметры вёрстки страницы (подобраны под A4 @150dpi) */
export const PAGE = {
 /** DPI раstra. Можно переопределить: EXPORT_DENSITY=100 */
 density: Number(process.env.EXPORT_DENSITY ?? 150),
 /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT_GRAYSCALE=1 */
 monochrome: process.env.EXPORT_GRAYSCALE !== "1",
 size: "A4",
 font: "DejaVu-Sans-Mono",
 fontFile: "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
 pointsize: 8,
 /** Максимум символов в строке (далее – жёсткий перебор) */
 cols: 138,
 /** Строк содержимого на странице (+2 служебные: шапка и подвал) */
 rows: 76,
};

export const HR = "=".repeat(PAGE.cols);
export const SUBHR = "-".repeat(PAGE.cols);

export function ensureDir(dir) {
 fs.mkdirSync(dir, { recursive: true });
}

export function humanSize(bytes) {
 if (bytes < 1024) return `${bytes} B`;
 if (bytes < 1024 * 1024) return `${(bytes / 1024).toFixed(2)} KB`;
 if (bytes < 1024 * 1024 * 1024) return `${(bytes / 1024 / 1024).toFixed(2)} MB`;
 return `${(bytes / 1024 / 1024 / 1024).toFixed(2)} GB`;
}
```

```

const SKIP = new Set([
 "svg", "script", "style", "button", "iframe", "video",
 "path", "circle", "rect", "line", "polyline", "polygon",
 "defs", "marker", "tspan", "ellipse", "use", "clipPath",
]);
```

```
/** Пустые элементы (без закрывающего тега). */
const VOID = new Set(["br", "hr", "img", "input", "meta",
```

```
/** Блочные теги: их содержимое собирается в отдельный блок. */
const BLOCK = new Set([
 "h1", "h2", "h3", "h4", "h5", "h6", "p", "li", "pre",
 "summary", "figcaption", "table", "tr", "td", "th", "thead",
 "article", "ul", "ol", "details", "figure", "header",
]);
```

```
const HEADING = new Set(["h1", "h2", "h3", "h4", "h5",
```

```
/**
 * Разбор HTML в дерево: { tag, attrs, children }, текст
 * Намеренно простой: страницы пособия генерируются нами
 */
```

```
export function parseHtml(html) {
 const root = { tag: "#root", attrs: {}, children: [] };
 const stack = [root];
 const re = /<!--[\s\S]*?-->|<\/([a-zA-Z0-9]+)\s*>|<([a-zA-Z0-9]+)\s*/;
 let m;
 while ((m = re.exec(html)) !== null) {
 const [full, closeTag, openTag, attrsStr, selfClose] = m;
 if (full.startsWith("<!--")) continue;
```

```
 if (closeTag) {
 const tag = closeTag.toLowerCase();
 // закрываем до совпадения (прощает незакрытые

 for (let i = stack.length - 1; i >= 1; i--) {
 if (stack[i].tag === tag) {
 stack.length = i;
 break;
```

```

 .replace(/≥/g, "≥")
 .replace(/≠/g, "≠")
 .replace(/∞/g, "∞")
 .replace(/→/g, "→")
 .replace(/←/g, "←")
 .replace(/&/g, "&")
 .replace(/&#(\d+);/g, (_, d) => String.fromCharCode(d))

const clean = (s) => decodeEntities(s).replace(/\s+/g, " ")

const isEl = (n) => n.tag !== "#text";
const isBlockEl = (n) => isEl(n) && BLOCK.has(n.tag) && !isTextEl(n);

/** Есть ли у узла блочные потомки (тогда его текст «разбит» на строки) */
const hasBlockChild = (node) => node.children.some(isBlockEl);

/** Текст инлайнового поддерева в порядке документа. */
function inlineText(node) {
 let out = "";
 for (const c of node.children) {
 if (c.tag === "#text") {
 out += c.text;
 } else if (SKIP.has(c.tag)) {
 continue;
 } else if (BLOCK.has(c.tag)) {
 continue;
 } else if (c.tag === "br") {
 out += "\n";
 } else {
 out += inlineText(c);
 }
 }
 return clean(out);
}

/** Весь текст поддерева (для ячеек таблиц и подписей). */
function collectText(node) {
 if (node.tag === "#text") return clean(node.text);
 let out = "";
 for (const c of node.children) {
 out += collectText(c);
 }
 return clean(out);
}

```

```

 if (prev && prev.type === block.type && prev.text !==
 out.push(block);
};

const emitListItem = (li, ordinal, depth) => {
 const text = collectText(li);
 if (text) push({ type: "li", text, ordinal: ordinal
 // вложенные списки внутри li
 li.children.forEach((c) => {
 if (c.tag === "ul" || c.tag === "ol") emitList(c,
 });
};

const emitList = (list, depth) => {
 let i = 0;
 for (const li of list.children) {
 if (li.tag !== "li") continue;
 i += 1;
 emitListItem(li, list.tag === "ol" ? i : null, de
 }
};

const walk = (node, listDepth = 0) => {
 if (node.tag !== "#root" && SKIP.has(node.tag)) ret

 if (node.tag === "table") {
 const rows = tableRows(node);
 if (rows.length) push({ type: "table", rows });
 return;
 }
 if (node.tag === "pre") {
 const text = decodeEntities(collectRawPre(node)).
 if (text.trim()) push({ type: "code", text: text.
 return;
 }

 // есть блочные дети: идём по порядку, инлайн-пробе
 if (hasBlockChild(node)) {

```

```

 root.children.forEach((c) => walk(c, 0));
 return out;
}

/** То же самое, но простым текстом (для .txt-дампа и в
export function htmlToPlainText(html) {
 const lines = [];
 for (const b of htmlToBlocks(html)) {
 if (b.type === "heading") lines.push("", "#".repeat
 else if (b.type === "summary") lines.push("", "▶ "
 else if (b.type === "li") lines.push(" ".repeat(b.
 else if (b.type === "code") lines.push("", ...b.tex
 else if (b.type === "table") {
 lines.push("");
 b.rows.forEach((r) => lines.push("| " + r.join("
 lines.push("");
 } else if (b.type === "quote") lines.push("> " + b.
 else if (b.type === "caption") lines.push(" " + b
 else lines.push(b.text, "");
 }
 return lines.join("\n").replace(/\n{3,}/g, "\n\n").tr
}

```

---

ФАЙЛ 98/111: scripts/export/render-pages.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 117 | РАЗМ

---

```

/**
 * ШАГ 2: TXT -> PNG
 *
 * Разбивает full_code_all_pages.txt на страницы формата
 * каждую страницу в PNG через ImageMagick (шрифт DejaVu
 *
 * Результат: tmp/export-pages/page-0001.png, page-0002
 */

```

```

if (!fs.existsSync(TXT_PATH)) {
 throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}
}

const im = imageMagickCmd();
const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

fs.rmSync(PAGES_DIR, { recursive: true, force: true });
ensureDir(PAGES_DIR);

const total = pages.length;
const jobs = pages.map((lines, idx) => {
 const num = String(idx + 1).padStart(4, "0");
 const header = `Универсальное пособие • полный исходный код
 Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 return { txtFile, pngFile, num };
});

const args = (job) => [
 "-density", String(PAGE.density),
 "-page", PAGE.size,
 "-font", fs.existsSync(PAGE.fontFile) ? PAGE.fontFile : "DejaVuSansMono",
 "-pointsize", String(PAGE.pointsize),
 `text:${job.txtFile}`,
 "-background", "white",
 "-flatten",
 "-colorspace", "Gray",
 ...(PAGE.monochrome ? ["-monochrome"] : []),
 "-strip",
 "-define", "png:compression-level=9",
 job.pngFile,
];

```

```
/**
 * DOM-харнес: настоящее приложение в jsdom + настоящий
 * Монтирует App, кликает кнопки, печатает в редакторе
 * визуализациям матрицы чужих размерностей и ловит пад
 */
```

```
import { act } from "react";
import { createRoot } from "react-dom/client";
import type { Root } from "react-dom/client";
import App from "../../src/App";
import { chapters, chapterTopics, SECTIONS, sectionOf }
import { quizzes } from "../../src/data/quizzes";
import { VIZ_REGISTRY } from "../../src/components/vizR
import { VizLiveStatus } from "../../src/components/Viz
import { PAGE_SYNC, buildInitTemplate } from "../../src
import { VizChapterContext, emitVizState, clearVizState
import { buildDebugRunner, type DebugResult } from "../../
```

```
type Py = { runPythonAsync: (src: string) => Promise<un
```

```
export interface TestResult {
 group: string;
 name: string;
 ok: boolean;
 ms: number;
 detail: string;
 consoleErrors: string[];
}
```

```
const results: TestResult[] = [];
let consoleErrors: string[] = [];
const realConsoleError = console.error;
```

```
/* -----
/* утилиты
/* -----
```

```
const sleep = (ms: number) => new Promise((r) => setTim
```

```

async function click(el: Element | undefined | null, label: string) {
 if (!el) throw new Error(`не найдена ${label}`);
 await act(async () => {
 el.dispatchEvent(new window.MouseEvent("click", { bubbles: true }));
 await sleep(20);
 });
}

/** Панель компилятора – все поиски внутри неё, чтобы не искать в документе
function panel(): HTMLElement | null {
 return document.querySelector<HTMLElement>('aside[aria-label="Compiler Panel"]');
}

function panelText(): string {
 return (panel()?.textContent ?? "").replace(/\s+/g, " ");
}

function textarea(): HTMLTextAreaElement | undefined {
 return panel()?.querySelector<HTMLTextAreaElement>("text-area");
}

function typeInto(ta: HTMLTextAreaElement, value: string) {
 const setter = Object.getOwnPropertyDescriptor(window.HTMLTextAreaElement.prototype, "value");
 if (!setter) throw new Error("нет нативного setter'a для HTMLTextAreaElement");
 act(() => {
 setter.call(ta, value);
 ta.dispatchEvent(new window.Event("input", { bubbles: true }));
 });
}

async function key(k: string, init: KeyboardEventInit = {}) {
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("keydown", init));
 await sleep(20);
 });
}

```



```

 }
 host = freshHost();
 window.history.replaceState({}, "", `/?topic=${encodeURIComponent(topic)}`);
 root = createRoot(host);
 await act(async () => {
 root!.render(<App />);
 });
 await settle(80);
 return host;
 }
}

```

```

/** Смонтировать одну визуализацию так, как это делает VizRegistry
 * @param vizId - идентификатор визуализации
 */
async function mountViz(vizId: string) {
 if (root) {
 await act(async () => {
 root?.unmount();
 });
 root = null;
 }
 if (host) host.remove();
 host = freshHost();
 const entry = VIZ_REGISTRY[vizId];
 if (!entry) throw new Error(`в реестре нет визуализации с id ${vizId}`);
 const Comp = entry.Component;
 root = createRoot(host);
 await act(async () => {
 root!.render(
 <VizChapterContext.Provider value={vizId}>
 <VizLiveStatus demoTitle={entry.title} />
 <Comp />
 </VizChapterContext.Provider>
);
 });
 await settle(60);
 return host;
}

```

```

const BAD_DOM = [

```

```

 detail = error instanceof Error ? `${error.message}
 }
 const ms = Date.now() - t0;
 const errs = consoleErrors.filter((e) => !/not wrapped
 if (errs.length > 0) {
 ok = false;
 detail = `${detail}\nconsole.error (${errs.length})
 }
 results.push({ group, name, ok, ms, detail, consoleEr
 const mark = ok ? "PASS" : "FAIL";
 realConsoleError === console.error;
 process.stdout.write(`${mark} [${group}] ${name}${ok
 if (!ok) process.stdout.write(detail.split("\n").map(
 return ok;
}

/* -----
/* сценарии
/* -----

/** Фильтры прогона: HAR_CHAPTERS=segment-trees,floyd и
const onlyChapters = (process.env.HAR_CHAPTERS || "").spl
const skipGroups = new Set((process.env.HAR_SKIP || "").split(" "))
/** HAR_SYNC=floyd,dijkstra – прогнать только эти записи
const onlySync = new Set((process.env.HAR_SYNC || "").split(" "))

export async function runSuite({ py }: { py: Py }): Promise<void> {
 const chapterList = onlyChapters.length ? chapters.filter(chapter =>
 const want = (group: string) => !skipGroups.has(group)
 // перехват console.error приложения (React пишет сюда)
 console.error = (...args: unknown[]) => {
 consoleErrors.push(args.map((a) => (a instanceof Error ? a.message : a)))
 };

 /* — 1. Панель Python: реальный запуск на каждой главе
 for (const chapter of chapterList) {
 const group = "А. страницы";
 await test(group, `${chapter.id}: монтируется, бейджик

```

```

 return `${msg} → визуализация переключилась на да
 });
}

```

/\* — 2. Флойд: матрицы ЛЮБОЙ размерности из кода пол

```

const floydCases: { name: string; vars: Record<string
 { name: "5×5 (меньше демо-7×7)", vars: { n: 5, k: 1
], [9, 6, 9, 9, 0]] }, expect: "5×5" },
 { name: "10×10 (больше демо – старый краш nodeNames
 from({ length: 10 }, (_, c) => (r === c ? 0 : (r
 { name: "3×3 с бесконечностями", vars: { n: 3, k: 0
 "3×3" },
 { name: "не квадратная 4×6", vars: { n: 4, k: 1, i:
 , 17, 18, 19, 0, 20]] }, expect: "Демо-схема граф
 { name: "словарь словарей d[0][1]", vars: { n: 4, k
 2: { 0: 9, 1: 9, 2: 0, 3: 1 }, 3: { 0: 3, 1: 9,
 { name: "ключи-пары \"(i, j)\", vars: { n: 3, k: 0
 "(1, 2)": 2, "(2, 0)": 1, "(2, 1)": 9, "(2, 2)":
 { name: "плоский список – это НЕ матрица", vars: {
 { name: "80×80 (обрезка до 64)", vars: { n: 80, k:
 (_, c) => (r === c ? 0 : 1 + ((r + c) % 50))) }
 { name: "пустая матрица", vars: { n: 0, k: 0, i: 0,
 { name: "матрица под именем D и k/i/j вне диапазона
 [10, 11, 12, 0]] }, expect: "4×4" },
];

```

```

for (const c of floydCases) {
 await test("В. Флойд: размерности", c.name, async () => {
 await mountViz("floyd");
 await act(async () => {
 emitVizState("floyd", { line: 3, func: "<module
 await sleep(30);
 });
 const txt = (host?.textContent ?? "").replace(/\\s
 if (!/данные из компилятора/.test(txt)) throw new
 if (!txt.includes(c.expect)) throw new Error(`в р
 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в

```

```

const A8 = Array.from({ length: 8 }, (_, i) => Array
await act(async () => {
 emitVizState("sparse-table", { line: 7, func: "<mod
 });
 await sleep(40);
});
const grids = () => Array.from((host ?? document).q
let g0 = grids()[0];
if (!g0) throw new Error("нет ни одной матрицы блок
const firstCell = () => (g0.children[0]?.textContent
// «Оригинал (1x1)» = сама матрица A: значения видн
if (firstCell() !== "1") throw new Error(`оригинал
if ((g0.children[7]?.textContent ?? "").trim() !==
// шаги кликаются даже под live-k из компилятора
await click(findButton("Шаг 2 (4x4)"), "вкладка шаг
const txt = (host?.textContent ?? "").replace(/\s+/
if (!/Матрица блоков 4x4/.test(txt)) throw new Erro
g0 = grids()[0];
const cls = (g0.children[0] as HTMLElement)?.classN
// пустая активная клетка (код смотрит на (0,0), st
if (!/border-dashed/.test(cls) || !/indigo/.test(cl
 throw new Error(`пустая активная клетка нарисован
}
if (domDefects(host).length) throw new Error(`артеф
return "оригинал из A с первого шага, вкладка «Шаг
});

```

```

/* — 4. Дейкстра / Форд-Беллман: чужие имена вершин
for (const [viz, vars] of [
 ["dijkstra", { u: 0, v: 3, w: 4, dist: [0, 4, 2, 6,
 ["bellman-ford", { n: 5, m: 4, i: 1, u: 2, v: 4, w:
] as [string, Record<string, unknown>][[]]) {
 await test("D. графы путей", `${viz}: вершины 0...4 в
 await mountViz(viz);
 const before = allButtons().length;
 const playEnabled = (findButton(/^Пуск|Пуск$/)) as
 await act(async () => {
 emitVizState(viz, { line: 5, func: "<module>",

```

```

 .filter({alts} => alts.some(ident))
 .filter({alts} => !alts.some({a} => ident(a) &&
 .map({alts} => alts.join(" / "));
 if (missing.length) throw new Error(`код не создан
 });

 if (!VIZ_REGISTRY[chapterId]) return `шагов ${result.steps.length}
 await mountViz(chapterId);
 const finalVars = result.finalValues ?? result.steps[0].finalValues;
 await act(async () => {
 emitVizState(chapterId, { line: result.steps[result.steps.length - 1].id,
 ed: Object.keys(finalVars) });
 await sleep(40);
 });
 const txt = (host?.textContent ?? "").replace(/\\s/g, " ");
 if (!/данные из компилятора/.test(txt)) throw new Error("ошибка в данных");
 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в визуализации: ${defects.length}
 return `шагов ${result.steps.length}${result.steps[result.steps.length - 1].id}
).join(", ")}`;
 });
}

```

/\* — 6. Поведение редактора: пустой код, ошибки, гла...  
const ch = "dijkstra";

```

await test("F. редактор", "только комментарии: редактор", async () => {
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "поиск");
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(ta, "# просто комментарий\n# и ещё один\n");
 const done = await waitFor(() => bodyText().include("# просто комментарий\n# и ещё один\n"));
 if (!done) throw new Error(`не дождался сообщения`);
 if (!textarea()) throw new Error("редактор исчез по ошибке");
 if (!/встроенное демо/.test(bodyText())) throw new Error("ошибка в тесте");
 return "редактор на месте, бейдж «встроенное демо»";
});

```

```

typeInto(ta, "");
const done = await waitFor(() => bodyText().include
if (!done) throw new Error(`на пустой редактор пане
if (!textarea()) throw new Error("редактор исчез по
if (!/встроенное демо/.test(bodyText())) throw new
if (allButtons().length < 5) throw new Error(`кноп
return `пустой код: кнопок ${allButtons().length},
});

```

```

await test("F. редактор", "глаз: эталон попадает в ре
await mountApp(ch);
await click(findButton("Боковая панель Python"), "п
const ta = textarea();
if (!ta) throw new Error("нет редактора");
const init = ta.value;
const editorButtons = allButtons();
const ref = PAGE_SYNC[ch]?.code ?? "";
if (!ref) throw new Error("у страницы нет эталонног
const done0 = await waitFor(() => RUN_DONE.some((m)
if (!done0) throw new Error(`инициализация не выпол

```

```

// — глаз ВКЛ: эталон лежит В РЕДАКТОРЕ, и редактор
await click(findButton("Вставить эталонный код"), "
const trace = await waitFor(() => !!findButton(/Ост
if (!trace) throw new Error(`эталон не запустился.
if (!textarea()) throw new Error("редактор исчез во
if (/Ошибка:/.test(panelText())) throw new Error(`э
const ta2 = textarea();
const firstRefLine = ref.split("\n").find((l) => l.
if (!ta2?.value.includes(firstRefLine)) {
 throw new Error(`выполняется не тот текст, что ле
 0, 100)} `);
}

```

```

const mustBe = [
 { what: "пуск/автопроход", ге: /Запустить код|Про
 { what: "копирование кода", ге: /Скопировать код|
 { what: "заккрыть панель", ге: /Заккрыть компилятор
 { what: "шаг вперёд", ге: /Шаг вперёд|вперёд/i },

```

```

typeInto(ta, "# MY-MARKER-14\nx = 41\nx += 1\nprint
await waitFor(() => RUN_DONE.some((m) => bodyText())
if (!bodyText().includes("42")) throw new Error("св
// переходим на другую страницу, не закрывая панель
await mountApp("floyd");
await click(findButton("Боковая панель Python"), "п
const ta2 = textarea();
if (!ta2) throw new Error("на новой странице нет ре
if (ta2.value.includes("MY-MARKER-14")) throw new E
if (!ta2.value.includes("# «")) throw new Error("на
await mountApp(ch);
await click(findButton("Боковая панель Python"), "п
const ta3 = textarea();
if (!ta3?.value.includes("MY-MARKER-14")) throw new
return "у каждой страницы свой код, возврат сохраня
));

```

```

await test("F. редактор", "редактор живёт и во время
await mountApp(ch);
await click(findButton("Боковая панель Python"), "п
const done = await waitFor(() => bodyText().include
if (!done) throw new Error(`трасса не собралась. Па
if (!textarea()) throw new Error("во время прогона
if (!findButton("/Остановить прогон трассы/")) throw
// Листинг сворачивается (освобождая место редактор
// и редактор остаются на месте.
const codeLines = () => panel()?.querySelectorAll("
if (codeLines() === 0) throw new Error("в раскрытой
await click(findButton("Свернуть листинг трассы"),
await settle(80);
if (!textarea()) throw new Error("после сворачивани
if (codeLines() !== 0) throw new Error(`листинг не
if (!findButton("В конец трассы")) throw new Error(
await click(findButton("Показать листинг трассы"),
await settle(80);
if (codeLines() === 0) throw new Error("листинг не
// Esc: прогон остановлен, трасса убрана, редактор
await key("Escape");

```

```

 начало трассы/, /В конец трассы/, /автопроход/i]
const btns = allButtons().join(" | ");
const missing = required.filter((re) => !re.test(btns));
if (missing.length) throw new Error(`в листинге нет
return `редактор: ${editorBtns} кнопок, листинг: ${
});

```

```

await test("Н. доступность", "бейдж честно различает
 await mountApp(ch);
 // 1) демо: компилятор ещё ничего не присылал
 if (!/встроенное демо/.test(bodyText())) throw new Error("демо не работает");
 if (/переменных пока нет/.test(bodyText())) throw new Error("переменные не пришли");
 await click(findButton("Боковая панель Python"), "показать панель");
 const listing = await waitFor(() => !!findButton(/0.1.0/));
 if (!listing) throw new Error("панель не показала трассу");
 // 2) код выполнен, отладчик на первой строке: переключить на код
 const awaiting = await waitFor(() => /переменных пока нет/);
 if (!awaiting) throw new Error(`после запуска бейдж
 ${bodyText().match(/встроенное демо|данные из компилятора/)}
 // 3) дошли до конца трассы – картинка обязана жить
 await click(findButton("В конец трассы"), "в конец трассы");
 const linked = await waitFor(() => /данные из компилятора/);
 if (!linked) throw new Error("в конце трассы визуализация не появилась");
 // 4) сняли связь (Esc + пустой код) – снова демо
 await key("Escape");
 await settle(80);
 const ta = textarea();
 if (!ta) throw new Error("редактор не вернулся по expected");
 typeInto(ta, "");
 const cleared = await waitFor(() => /встроенное демо/);
 if (!cleared) throw new Error("после очистки кода бейдж не обновился");
 if (/переменных пока нет/.test(bodyText())) throw new Error("переменные не пришли");
 return "демо → выполнено/переменных нет → данные из компилятора";
});

```

/\* — 7. Текстовый режим для PDF/парсеров —————

/\* — 9. Ссылки между билетами, разделы содержания, б.



```

 if (!py.getAttribute("aria-label")) throw new Error("у кнопки нет aria-label");
 if (!py.getAttribute("title")) throw new Error("у кнопки нет title");
 return "4 выгрузки на месте, href ведут на файлы, кнопки не работают";
 });

```

```

await test("I. ссылки", "упоминание «билет 5» кликабельно");
await mountApp("sparse-table");
const links = Array.from(document.querySelectorAll("a"));
if (links.length === 0) throw new Error("на странице нет ссылок");
const toPrefix = links.find((a) => a.dataset.goto === "5");
if (!toPrefix) throw new Error("нет ссылки на билет 5");
if (!/билет 5/i.test(toPrefix.textContent ?? "")) throw new Error("ссылка не содержит текст «билет 5»");
const href = toPrefix.getAttribute("href") ?? "";
if (!href.includes("topic=prefix-sums-2d")) throw new Error("ссылка не содержит «topic=prefix-sums-2d»");
await click(toPrefix, "ссылка на билет 5");
await settle(120);
const txt = bodyText();
if (!/Двумерная матрица префиксных сумм/.test(txt)) throw new Error("клик по «билет 5» не открыл страницу");
return `ссылок на странице: ${links.length}, переход по ссылке: ${toPrefix.textContent}`;
});

```

```

await test("I. ссылки", "содержание: разделы по порядку");
await mountApp("bridges-code");
const sidebar = document.querySelector("#sidebar-content");
if (!sidebar) throw new Error("нет сайдбара с содержанием");
const headers = Array.from(sidebar.querySelectorAll("h2"))
 .map((el) => (el.textContent ?? "").trim())
 .filter((t) => SECTIONS.some((sec) => sec.title === t));
if (headers.length !== SECTIONS.length) {
 throw new Error(`в содержании ${headers.length} разделов, а должно быть ${SECTIONS.length}`);
}
const expected = SECTIONS.map((sec) => sec.title).join(" | ");
if (headers.join(" | ") !== expected) throw new Error("содержание не соответствует ожидаемому");
// 11 «Мосты» и 12 «Точки сочленения» читаются парой элементов
const items = Array.from(sidebar.querySelectorAll("a"));

```

```

const goto = Array.from(document.querySelectorAll("a"));
if (!goto.includes("dijkstra")) throw new Error(`объект не найден`);
if (domDefects(document.querySelector("#chapter-viz"))) throw new Error("артефакты в DOM введения");
}
return "введение = куча/очередь/бинпоиск/сортировка";
});

```

```

await test("I. блиц", "вопросы отвечают, объясняют и ...");
await mountApp("sparse-table");
const list = quizzes["sparse-table"] ?? [];
if (list.length === 0) throw new Error("у билета 2 вопроса");
const section = Array.from(document.querySelectorAll("div"));
 (el.getAttribute("aria-label") ?? "").includes("Блиц");
};
if (!section) throw new Error("блиц не отрисован на странице");
const ask = (label: string | RegExp) =>
 Array.from(section.querySelectorAll("button")).find(b =>
 const hay = [b.getAttribute("aria-label"), b.textContent];
 return typeof label === "string" ? hay.includes(label) :
);
const shown = () => (section.textContent ?? "").replace(/<div>.*</div>/g, "");
if (!shown().includes(list[0].question.slice(0, 24)))
 throw new Error(`первый вопрос не показан: ${list[0].question}`);
const wrong = list[0].options.findIndex((_, i) => i !== 0);
await click(ask(list[0].options[wrong].slice(0, 20)));
await settle(80);
if (!/Неверно/.test(shown())) throw new Error("на нарисованном ответе не отмечено");
if (!shown().includes(list[0].explanation.slice(0, 24)))
 throw new Error("после ответа нет разбора – польза от блица не очевидна");
const nextBtn = ask("Следующий вопрос");
if (!nextBtn) throw new Error("нет кнопки «Следующий вопрос»");
await click(nextBtn, "дальше");
await settle(80);
if (!shown().includes(list[1].question.slice(0, 24)))
 // верный ответ

```



```

 "st = []",
 "for row in a:",
 " st.append([row] + [None] * (LOG - 1))",
 "for j in range(1, LOG):",
 " for i in range(n - (1 << j) + 1):",
 " st[i][j] = min(st[i][j - 1], st[i + (1 << j)][j - 1])",
 "print(n, st[0][2], st[1][1])",
].join("\n")
);
const ready = await waitFor(() => bodyText().includes("Готово"));
if (!ready) throw new Error(`своя таблица не построена`);
if (!/6 1 1/.test(panelText())) throw new Error(`отсутствует таблица`);
await click(findButton("В конец трассы"), "в конец трассы");
const linked = await waitFor(() => /данные из компилятора/);
if (!linked) throw new Error("демонстрация не принята");
const defects = domDefects(document.querySelector("#defects"));
if (defects.length) throw new Error(`артефакты в демонстрации`);
return `свой массив из 6 чисел: st[0][2] = 1, st[1][1] = 1`;
});

await test("К. свой код", "Косарайю: эталон сам считает");
await mountApp("scc-kosaraju");
await click(findButton("Боковая панель Python"), "панель Python");
if (!textarea()) throw new Error("нет редактора");
await click(findButton("Вставить эталонный код"), "вставить код");
const ready = await waitFor(() => bodyText().includes("Готово"));
if (!ready) throw new Error(`эталон Косарайю не выполнен`);
if (/Ошибка:/.test(panelText())) throw new Error(`эталон не выполнен`);
const out = panelText();
if (!/порядок выхода:/.test(out)) throw new Error(`неверный порядок`);
// граф на экране: 0→1→2→0, 2→3, 3→4 – ровно две SCC
if (!/компонент: 2/.test(out)) throw new Error(`неверное количество`);
if (!textarea()) throw new Error("редактор исчез, панель Python не появилась");
await click(findButton("В конец трассы"), "в конец трассы");
const linked = await waitFor(() => /данные из компилятора/);
if (!linked) throw new Error("демонстрация не принята");
const defects = domDefects(document.querySelector("#defects"));
if (defects.length) throw new Error(`артефакты в демонстрации`);

```

```

 const html = await buildBookHtml(chapters);
 if (!html.includes('<nav id="toc"')) throw new Error('нет оглавления');
 const missing = chapters.filter((c) => !html.includes(c.title));
 if (missing.length) throw new Error(`в книге потеряны главы: ${missing.map(c => c.title).join(', ')}`);
 const tocLinks = (html.match(/href="#ch-/g) ?? []).length;
 if (tocLinks < chapters.length) throw new Error(`содержится не все: ${chapters.length - tocLinks} глав`);
 if (!/Словарик терминов пособия/.test(html)) throw new Error('не найден словарь терминов');
 if (!/↑ к оглавлению/.test(html)) throw new Error('не найден ссылка на оглавление');
 if (html.length < 200_000) throw new Error(`книга слишком маленькая: ${html.length} байт`);
 return `${(html.length / 1024).toFixed(0)} КБ, ${chapters.length} глав`;
 });

 console.error = realConsoleError;
 return results;
}

```

```

/** Сводка для оркестратора. */
export function summary() {
 const byGroup = new Map<string, { ok: number; fail: number }>();
 for (const r of results) {
 const g = byGroup.get(r.group) ?? { ok: 0, fail: 0 };
 g[r.ok ? "ok" : "fail"]++;
 byGroup.set(r.group, g);
 }
 return { total: results.length, passed: results.filter(r => r.ok).length, failed: results.filter(r => !r.ok).length };
}

```

```

/** Инициализация страницы из vizSync – нужна для проверки
export const initTemplateOf = (id: string) => buildInitTemplateOf(id);

```

ФАЙЛ 100/111: scripts/verify/check-links.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 199 | РАЗМЕР: 1000 байт

/\*\*

```

 entryPoints: ["tmp/verify/links-entry.ts"],
 bundle: true,
 outfile: OUT,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
 logLevel: "error",
 define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/'',
 "import.meta.env.MODE": '"development"',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 "import.meta.env.SSR": "false",
 },
 loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl" },
 });
const { chapters, chapterTopics, SECTIONS, sectionOf, G
 await import(`${process.cwd()}/${OUT}?run=${Date.now()}`);

const chapterIds = new Set(chapters.map((c) => c.id));
const registryIds = Object.keys(VIZ_REGISTRY);

/** Демо без собственной страницы: открываются из подсказки */
const EXTRA_DEMOS = new Set(["stack-dfs", "queue-bfs", "graph-adjacency"]);

/* 1. билеты 1–24 */
const covered = new Set();
for (const [id, tickets] of Object.entries(chapterTopics)) {
 if (!ok(chapterIds.has(id), `chapterTopics ссылается на не существующий id`))
 tickets.forEach((n) => covered.add(n));
}
for (let n = 1; n <= 24; n += 1) {
 if (!ok(covered.has(n), `билет ${n} не закрыт ни одной страницей`))
 continue;
}

/* 2. разделы содержания */

```

```

 .replace(/<[^>]*>/g, " ");
 for (const m of textOnly.matchAll(TICKET_RE)) {
 const from = Number(m[1]);
 const to = m[2] ? Number(m[2]) : from;
 mentionsHere += 1;
 for (let n = from; n <= to; n += 1) {
 ok(n >= 1 && n <= 24 && covered.has(n), `«${chapter
 }
 }
 ticketMentions += mentionsHere;

// 4. явные переходы
for (const m of html.matchAll(/data-goto="([^\"]+)"/g)) {
 explicitLinks += 1;
 ok(chapterIds.has(m[1]), `«${chapter.id}»: data-goto
}
for (const m of html.matchAll(/href="\?(?:lite=1&)?to
 explicitLinks += 1;
 ok(chapterIds.has(decodeURIComponent(m[1])), `«${ch
}

// 5. якоря
for (const m of html.matchAll(/href="#([^\"]+)"/g)) {
 ok(html.includes(`id="${m[1]}"`), `«${chapter.id}»:
}

 if (mentionsHere === 0 && !/data-goto=/.test(html)) w
}

/* 6–7. глоссарий */
const demoKinds = new Set(MINI_DEMO_KINDS);
for (const term of GLOSSARY) {
 if (term.simulator) {
 ok(registryIds.includes(term.simulator), `глоссарий
 ok(
 chapterIds.has(term.simulator) || EXTRA_DEMOS.has
 `глоссарий «${term.id}»: simulator "${term.simula
 ые компилятора`
 }
}

```

```

if (NOTES.length) console.log(`  заметки:\n    ${NOTES}`);
if (noQuiz.length) console.log(`  без блица: ${noQuiz.j}`);

if (FAILS.length) {
 console.log(`\nПРОВАЛЕНО ${FAILS.length}:`);
 FAILS.slice(0, 40).forEach((f) => console.log(`  x ${f}`));
 process.exit(1);
}
console.log("\nвсе ссылки целы: 0 битых переходов");

```

-----  
ФАЙЛ 101/111: scripts/verify/check-reference-code.mjs  
КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 133 | РАЗМЕР: 1.5 КБ  
-----

```
/**
 * Контракт «эталонный код ↔ страница ↔ визуализация».
 *
 * npm run verify:code
 *
 * Каждый референсный Python-код из PAGE_SYNC выполняет
 * (pyodide из node_modules) под тем же трейсером, что и
 * проверяется:
 * 1. код не падает и не обрывается по лимиту шагов;
 * 2. трасса не пустая (иначе панель показала бы пустую);
 * 3. каждая переменная, обещанная странице, реально
 * существует, иначе визуализация не может за ним следовать;
 * 4. в снимках нет «мусорных» значений, которые уронят
 * вывод.
 */
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";
import { loadPyodide } from "pyodide";

const OUT_DIR = "tmp/verify";
const t0 = Date.now();
mkdirSync(OUT_DIR, { recursive: true });

```



```

for (const [key, entry] of Object.entries(mod.sync)) {
 if (only.length && !only.some((o) => key.includes(o)))
 printed.length = 0;
 let result;
 try {
 result = JSON.parse(String(await py.runPythonAsync(
 } catch (error) {
 problems++;
 rows.push({ key, steps: 0, status: `СБОЙ ЗАПУСКА: $
 continue;
 }
 const seen = new Set();
 for (const step of result.steps) for (const name of 0
 for (const name of Object.keys(result.finalValues ??

const issues = [];
if (result.error) issues.push(`ошибка Python: ${result
if (result.truncated) issues.push("трасса оборвана по
if (result.steps.length === 0) issues.push("нет ни од
const missing = (mod.declared[key] ?? [])
 .map(alts)
 .filter((a) => a.some(ident))
 .filter((a) => !a.some((x) => ident(x) && seen.has(
 .map((a) => a.join(" / "));
if (missing.length) issues.push(`код не создаёт обеща
/*
 * Обещанная переменная не должна быть пустой на ВСЕХ
 * объявил и не наполнил – визуализации нечего показы
 * «зашитый» результат вместо работы алгоритма. Именн
 * Косарайю: order был дан готовым списком, а comp та
 */
const isEmpty = (v) =>
 Array.isArray(v) ? v.length === 0
 : typeof v === "string" ? v.length === 0
 : typeof v === "object" && v !== null ? Object.keys
 : false;
const neverFilled = (mod.declared[key] ?? [])
 .map(alts)

```

ФАЙЛ 102/111: scripts/verify/dom-env.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 107 | РАЗМЕР: 1.5 КБ

```
/**
 * Браузерное окружение для проверок: jsdom + те API, к
 * Создаётся ПОСЛЕ загрузки Pyodide (иначе pyodide решит
 */
import { JSDOM, VirtualConsole } from "jsdom";

const OBSERVER_STUB = class {
 observe() {}
 unobserve() {}
 disconnect() {}
 takeRecords() {
 return [];
 }
};

export function createDom({ url = "http://localhost/?to"
 const virtualConsole = new VirtualConsole();
 const jsdomErrors = [];
 virtualConsole.on("jsdomError", (error) => {
 jsdomErrors.push(error?.message || String(error));
 onError?.(error);
 });
 // console.error/warn приложения собираем отдельно –
 const consoleErrors = [];
 virtualConsole.on("error", (...args) => consoleErrors
 virtualConsole.on("warn", () => {});

 const dom = new JSDOM(
 `<!doctype html><html lang="ru"><head><meta charset=
 <body><div id="root"></div></body></html>`,
 { url, pretendToBeVisual: true, runScripts: "outside
 });
 const { window } = dom;
```

```

 "Node",
 "Event",
 "CustomEvent",
 "MouseEvent",
 "KeyboardEvent",
 "InputEvent",
 "getComputedStyle",
 "requestAnimationFrame",
 "cancelAnimationFrame",
 "DOMParser",
 "SVGElement",
 "Text",
 "DocumentFragment",
 "NodeList",
 "HTMLCollection",
 "Range",
 "MutationObserver",
 "localStorage",
 "sessionStorage",
 "Blob",
 "File",
 "FileReader",
];
 for (const key of copy) {
 try {
 Object.defineProperty(globalThis, key, { value: w
 } catch {
 /* read-only в Node – не критично */
 }
 }
 globalThis.IS_REACT_ACT_ENVIRONMENT = true;

 return { dom, window, document: window.document, cons
}

```

```

 if (a === e) passed++;
 else failures.push(`${name}\n получено: ${a}\n
}
const m = (v) => liveMatrix(v);
const shape = (v) => {
 const r = m(v);
 return r ? { rows: r.rows, cols: r.cols, label: r.size
};
const cells = (v) => {
 const r = m(v);
 if (!r) return null;
 return Array.from({ length: r.rows }, (_, i) => Array
};

/* — liveMatrix: формы данных
check("квадратная 3x3 (список списков)", shape([[1, 2, 3], [4, 5, 6], [7, 8, 9]]), 3);
check("не квадратная 2x4", shape([[1, 2, 3, 4], [5, 6, 7, 8]], 2);
check("значения ячеек 2x2", cells([[0, 1], [2, 3]]), [[0, 1], [2, 3]]);
check("рванные строки: cols = максимум", shape([[1, 2, 3], [4, 5, 6], [7, 8, 9]], 3);
check("рванные строки: дырка = null", cells([[1, 2, 3], [4, 5, 6], [7, 8, 9]], 3);
check("at() вне диапазона → null", m([[1, 2], [3, 4]]).at(2), null);
check("at() с отрицательным индексом → null", m([[1, 2], [3, 4]]).at(-2), null);
check("список словарей с числовыми ключами", cells([{ 0: 1, 1: 2 }, { 0: 3, 1: 4 }]), [{ 0: 1, 1: 2 }, { 0: 3, 1: 4 }]);
check("словарь списков", cells({ 0: [1, 2], 1: [3, 4] }), { 0: [1, 2], 1: [3, 4] });
check("словарь словарей", cells({ 0: { 0: 7, 1: 8 }, 1: { 0: 9, 1: 10 } }), { 0: { 0: 7, 1: 8 }, 1: { 0: 9, 1: 10 } });
check('ключи-пары "(i, j)" после снапшота tuple', cells(tuple([0, 1], [2, 3])), tuple([0, 1], [2, 3]));
check('ключи-пары без скобок "0, 1"', cells({ "0, 0": 5, "0, 1": 6, "1, 0": 7, "1, 1": 8 }), { "0, 0": 5, "0, 1": 6, "1, 0": 7, "1, 1": 8 });

/* — liveMatrix: что матрицей НЕ является
check("плоский список чисел → null", m([1, 2, 3, 4, 5]), null);
check("словарь по именам → null", m({ dist: 1, prev: 2, cur: 3 }), null);
check("пустой список → null", m([]), null);
check("пустой словарь → null", m({}), null);
check("undefined → null", m(undefined), null);
check("null → null", m(null), null);
check("число → null", m(42), null);
check("строка → null", m("abc"), null);
check("массив строк-мусора → null", m(["a", "b"], ["c", "d"], ["e", "f"]]), null);

```

```

* Зонд состояния панели Python во времени: что происходит
* Запуск: node tmp/harness/run-probe.mjs
*/
import { act } from "react";
import { createRoot } from "react-dom/client";
import App from "../../src/App";

const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));
async function settle(ms = 40) {
 await act(async () => {
 await sleep(ms);
 });
}

function panel(): HTMLElement | null {
 return document.querySelector<HTMLElement>('aside[aria-label="panel"]');
}

function editor(): HTMLTextAreaElement | undefined {
 return panel()?.querySelector<HTMLTextAreaElement>('text-area');
}

function footer(): string {
 const divs = Array.from(panel()?.querySelectorAll("div"));
 const f = divs.find((d) => d.className.includes("text"));
 return (f?.textContent ?? "").replace(/\s+/g, " ");
}

function state(tag: string) {
 const p = panel();
 const ed = editor();
 const txt = (p?.textContent ?? "").replace(/\s+/g, " ");
 const step = txt.match(/war[^\s]*\s*\d+\s*\s*\s*\d+/i);
 process.stdout.write(
 `
 [${tag.padEnd(9)}] редактор: ${ed ? "ДА" : "нет"}
 step ? step[0] : "-" | футер: ${footer()}\n`
);
 process.stdout.write(`
 бейдж страницы: ${
 /встроенное демо/.test(document.body.textContent ?? "")
 }
 `);
}

function typeInto(ta: HTMLTextAreaElement, value: string) {
 const setter = Object.getOwnPropertyDescriptor(window, "document")?.value;
 setter?.typeInto(ta, value);
}
```

```

const ed = editor();
if (!ed) {
 process.stdout.write(" редактора нет – жмём Esc и ");
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("
 await sleep(30);
 }));
}
const ed2 = editor();
if (!ed2) throw new Error("редактор так и не появился");
typeInto(ed2, "# только комментарий\n# и ещё\n");
state("ввели");
for (const t of [200, 800, 2000, 6000]) {
 await settle(t);
 state(`t+${t}`);
}

process.stdout.write("\n=== 3. вводим рабочий код x=4");
const ed3 = editor();
if (!ed3) throw new Error("редактор пропал после комм");
typeInto(ed3, "x = 41\nx += 1\nprint(x)\n");
state("ввели");
for (const t of [200, 800, 2000, 6000]) {
 await settle(t);
 state(`t+${t}`);
}

process.stdout.write("\n=== 4. очищаем редактор полно");
const ed4 = editor();
if (ed4) {
 typeInto(ed4, "");
 state("очистили");
 for (const t of [200, 800, 2000]) {
 await settle(t);
 state(`t+${t}`);
 }
} else {
 process.stdout.write(" редактора нет (листинг) – жмём ");
}

```

Универсальное пособие • полный исходный код

-----  
список и словарь по именам (`dist`, `prev`): иначе визуализация чужие данные за сетку. Плюс `pickLiveVar` (приоритет имени (индексы пользователя вне диапазона демо)).

### `verify:code` – `check-reference-code.mjs`

Каждый референсный код из `PAGE\_SYNC` выполняется настоящим `sys.settrace`-трейсером, что и панель компилятора. Протокол трасса не оборвана по лимиту шагов, шагов больше нуля (пустой листинг), и \*\*каждая переменная, обещанная страницей кодом\*\* – иначе визуализация физически не может за ним следовать. Предупреждения интерпретатора, которые иначе попадают в «Результат».

### `verify:dom` – `browser-checks.tsx`

Настоящее приложение монтируется в jsdom (`App` целиком – глава, визуализация, панель Python), сценарии кликают кнопки в редакторе.

| Группа | Что делает |

| --- | --- |

| A. страницы (22 × 3) | каждая глава монтируется; бейдж автозапуска инициализации настоящим Python не роняет и не возвращает `[object Object]`; нет кнопок без имени |

| B. Флойд: размерности (10) | 5×5, 10×10 (больше демо – пустой список, 80×80 (обрезка), пустая матрица, чужое имя |

| C. матричные страницы (6) | префиксные суммы 2D и разности |

| D. графы путей (2) | Дейкстра/Форд-Беллман с вершинами, гаснущими и оживающими |

| E. эталонный код (46) | прогон кода в CPython → финальный результат и показать мусор |

| F. редактор (6) | код из одних комментариев, синтаксический анализ эталон и возвращает инициализацию; код не перетаскивается |

| G. lite-режим (1) | `?lite=1` отдаёт текст без навигации |

| H. доступность (2) | в панели подписаны все кнопки в меню и в меню |

Фильтры: `HAR\_CHAPTERS=floyd,dijkstra`, `HAR\_SKIP=A,E`,

```
const jsdomErrors = [];

log("гружаю CPython (pyodide из node_modules)...");
const py = await loadPyodide({ indexURL: "node_modules/" });
log(` Python готов за ${Date.now() - t0} мс`);

log("поднимаю jsdom...");
const { window, document, consoleErrors } = createDom({
 window.__algv0PythonRuntime = py;
});

log("собираю приложение esbuild'ом.");
mkdirSync(OUT_DIR, { recursive: true });
await esbuild.build({
 entryPoints: ["scripts/verify/browser-checks.tsx"],
 bundle: true,
 outfile: `${OUT_DIR}/bundle.mjs`,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
 sourcemap: true,
 // Vite-специфичные глобалы, которых нет вне dev-серв
 define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/'",
 "import.meta.env.MODE": '"development"',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 "import.meta.env.SSR": "false",
 },
 loader: { ".css": "empty", ".png": "dataurl", ".jpg":
 logLevel: "warning",
});
const mod = await import(`${process.cwd()}/${OUT_DIR}/b

process.on("unhandledRejection", (reason) => {
 jsdomErrors.push(`unhandledRejection: ${reason instan
});
```



```
/**
 * Зонд панели Python: печатает состояние редактора/тра
 * Не тест, а диагностика — когда нужно понять, что пан
 * ввода кода (автозапуск, переход в листинг, сообщения
 *
 * npm run verify:probe
 */
import * as esbuild from "esbuild";
import { mkdirSync } from "fs";
import { loadPyodide } from "pyodide";
import { createDom } from "../dom-env.mjs";

const OUT_DIR = "tmp/verify";
mkdirSync(OUT_DIR, { recursive: true });
const log = (m) => process.stdout.write(`[verify:probe]

log("гружаю CPython...");
const py = await loadPyodide({ indexURL: "node_modules/
log("поднимаю jsdom...");
const { window, document } = createDom();
window.__algv0PythonRuntime = py;

await esbuild.build({
 entryPoints: ["scripts/verify/probe-compiler.tsx"],
 bundle: true,
 outfile: `${OUT_DIR}/probe-bundle.mjs`,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
 sourcemap: true,
 define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/'',
 "import.meta.env.DEV": "true",
```

```
permissions:
 contents: read
 pages: write
 id-token: write

concurrency:
 group: pages
 cancel-in-progress: true

jobs:
 build:
 name: Build static site
 runs-on: ubuntu-latest
 steps:
 - name: Checkout
 uses: actions/checkout@v6

 - name: Setup Node.js
 uses: actions/setup-node@v7
 with:
 node-version: 24
 cache: npm

 - name: Install dependencies
 run: npm ci --no-audit --no-fund

 - name: Build for the repository subpath
 env:
 VITE_BASE: /algv0/
 run: npm run build

 - name: Make GitHub Pages fallback
 run: cp dist/index.html dist/404.html

 - name: Upload Pages artifact
 uses: actions/upload-pages-artifact@v5
 with:
```

## Универсальное пособие • полный исходный код

```

исходный код → full_code_all_pages.txt (script)
#
|
| → page-0001.png ... page-NNNN.png (
#
|
| → full_code_all_page
#
Готовые TXT и PDF коммитаются обратно в ветку (их отда
промежуточные PNG сохраняются как artifacts запуска.
```

```
on:
 push:
 branches:
 - "**"
 paths-ignore:
 # чтобы коммит самого workflow не запускал бесконеч
 - "public/export/**"
 - "**/*.md"
 workflow_dispatch:
 inputs:
 density:
 description: "DPI растеризации страниц (по умолч
 required: false
 default: "150"

concurrency:
 group: rebuild-pdf-`${{ github.ref }}`
 cancel-in-progress: true

permissions:
 contents: write

jobs:
 rebuild-pdf:
 name: код → txt → png → pdf
 runs-on: ubuntu-latest
 # не реагируем на собственный коммит бота
 if: "!contains(github.event.head_commit.message, '[
```

- ```
run: npm run export:pdf
```
- name: Step 4 – собрать приложение и поднять preview
run: |
 # headless-браузер для снимков: chrome ставит
 # этот вызов – страховка, если кэш оказался пустым
 npx puppeteer browsers install chrome || true
 npm run build
 mkdir -p tmp
 nohup npx vite preview --host 127.0.0.1 --port 4173 &
 for i in \$(seq 1 40); do
 if curl -sf http://127.0.0.1:4173/ > /dev/null
 then
 sleep 1
 fi
 done
 - name: Step 5 – снимки визуализаций (headless-браузер)
run: npm run export:shots
continue-on-error: true
 - name: Step 6 – PDF-конспект со снимками
run: |
 npm run export:guide
 # снимки публикуются вместе с приложением: HTML-снимки
 # в автономный файл прямо из /export/viz-shots
 mkdir -p public/export/viz-shots
 cp tmp/viz-shots/*.png tmp/viz-shots/manifest.json
 - name: Type-check приложения
run: npx tsc --noEmit
continue-on-error: true
 - name: Summary
run: |
 {
 echo "### Экспорт пересобран"
 echo ""
 echo "| Артефакт | Размер |"
 echo "| --- | --- |"

ФАЙЛ 110/111: `.github/workflows/verify.yml`

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 71 | РАЗМЕР: 2.3

```
name: Verify
```

```
# Проверки приложения на каждом пуше и PR:
```

```
#
```

```
#   verify:unit   – разбор «живых» данных компилятора (LSP)
```

```
#   verify:code   – все эталонные Python-коды страниц выгрузки
```

```
#                   под трейсером панели и создают обещания
```

```
#   verify:dom     – приложение монтируется в jsdom: страницы
```

```
#                   матрицы чужих размерностей, кнопки, и т.д.
```

```
#
```

```
# Headless-браузер не нужен: jsdom + pyodide из node_modules
```

```
on:
```

```
  push:
```

```
    branches:
```

```
      - "**"
```

```
    paths-ignore:
```

```
      - "public/export/**"
```

```
      - "**/*.md"
```

```
  pull_request:
```

```
  workflow_dispatch:
```

```
concurrency:
```

```
  group: verify-${{ github.ref }}
```

```
  cancel-in-progress: true
```

```
permissions:
```

```
  contents: read
```

```
jobs:
```

```
  verify:
```

```
    name: unit + эталонный код + DOM
```

РАЗДЕЛ: ПРОЧЕЕ

ФАЙЛ 111/111: .ai_skills/universal_guide_authoring/SKIL
КАТЕГОРИЯ: Прочее | СТРОК: 252 | РАЗМЕР: 13.3 KB

Universal Guide Authoring – шаблоны страниц пособия

Обязательный справочник для ****создания и правки контента репозитории**. На него ссылаются `AGENTS.md` и `GEMINI.md` собираем ****только**** из блоков ниже – не придумываем новый «голый» markdown.

Контент живёт в `src/data/\*\*/\*.ts` как массив объектов `src/data/content.ts` (при одинаковом `id` побеждает бо

1. Объект главы

```
```ts
import { Chapter } from "../../types";

export const myChapters: Chapter[] = [
 {
 id: "dijkstra", // = ключ VIZ_REGISTRY и
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html", // всегда "html": страни
 category: "Графы. Пути", // группа в навигации
 content: `...HTML ниже...`,
 },
];
```
```

```
-----
| --- | --- |
| `border-blue-500` + `text-blue-400` | определение, «о»,
| `border-emerald-500` + `text-emerald-400` | алгоритм,
| `border-indigo-500` + `text-indigo-400` | теория, док
| `border-amber-500` + `text-amber-400` | ловушки, част
| `border-rose-500` + `text-rose-400` | ограничения, «к
```

3. Библиотека блоков (точные классы)

3.1 Определение / строгое правило – всегда по центру

```
```html
<div class="bg-slate-900 p-4 rounded-lg mb-6 text-center">
 <p class="text-lg text-emerald-300 italic mb-2">Строг
 <p class="text-xl font-mono text-white">if d[v] > d[u]
</div>
```
```

3.2 Обычный абзац

```
```html
<p class="text-slate-300 text-sm mb-4">
 Текст с акцентом и инлайн-кодом
 dist[v]
</p>
```
```

Инлайн-«пилюля» для имени переменной/константы:

```
```html

 <code>v</code>

```
```

3.3 Сетка ключевых чисел (3 ячейки)

```
```html
```

```

```html
<div class="bg-slate-800/70 p-5 rounded-lg border border-slate-700" style="border: 1px solid #444; background-color: #1a202c80; padding: 10px; border-radius: 0.5rem;">
  <p class="text-sm font-bold text-white mb-2">Заголовок
  <p class="text-slate-300 text-sm mb-3">Пояснение перед таблицей
  <table class="w-full text-xs text-left border-collapse" style="width: 100%; border-collapse: collapse;">
    <thead>
      <tr class="text-slate-500 border-b border-slate-700" style="border-bottom: 1px solid #444; color: #a0aec0;">
        <th class="py-1.5 pr-2 font-bold">Колонка
        <th class="py-1.5 pr-2 font-bold">Колонка
      </tr>
    </thead>
    <tbody>
      <tr class="border-b border-slate-800 align-top" style="border-bottom: 1px solid #1a202c80;">
        <td class="py-2 pr-3 font-bold">Значение
        <td class="py-2 pr-3 font-mono">O(log n)
      </td>
    </tr>
  </tbody>
</table>
</div>
```

```

### 3.6 Спойлер ``<details>`` – **обязателен** для кода, который должен быть скрыт

```

```html
<details class="bg-slate-900/40 rounded-lg border border-slate-800" style="background-color: #1a202c40; border: 1px solid #1a202c80; border-radius: 0.5rem; padding: 10px;">
  <summary class="p-4 font-bold text-slate-400 outline-slate-700/50" style="padding: 5px 10px; color: #a0aec0; border: 1px solid #444; border-radius: 0.25rem; margin-bottom: 10px;">
    ▶ Код: Дейкстра на куче (Python)
  </summary>
  <div class="p-5 text-sm text-slate-300 space-y-4" style="padding: 10px 10px 0 10px;">
    <pre class="bg-slate-950 p-4 rounded overflow-x-auto" style="background-color: #0f172a; padding: 5px; border-radius: 0.25rem; margin-bottom: 10px;">
...</code></pre>
    <p class="text-slate-300 text-sm">Комментарий после кода
  </div>
</details>
```

```

- Вложенные списки внутри спойлера: ``<ul class="list-disc">`



- ``title``, опциональный ``hint``, ``Component``).
- ``src/data/vizSync.ts`` – запись ``PAGE_SYNC[id]`` или ``P``, где ``demoId`` – второй аргумент ``emitVizDemo(demoId, t`` (вкладки одной демонстрации).
- Поля записи: ``vizTitle``, ``stepNote`` (что считается од ``variables: [{ name, role, range }]`, ``code`` (референ
- Переменные, которые читает визуализация, должны быть ``int``, ``list``, ``dict``, ``str``): экземпляры классов сна до графики не доезжают.
- Стартовый шаблон в редакторе компилятора – ``buildInit`` инициализация переменных до строки ``# --- тело алгоритма``

Проверка перед коммитом: ``npm run dev`` → страница открыта, подставляет код, шаг демонстрации подсвечивает нужные п

---

## ## 6. Экспорт в PDF (``npm run export:guide``)

Текст страниц парсится ``scripts/export/html-to-blocks.m``, ``public/export/guide_lite.pdf`` (текст + визуализации, без компилятора и без навигации). Чтобы экспорт был полным:

- заголовки – только ``h2/h3/h4``; списки – ``ul/ol/li``; теги кода – ``pre``; определения – обычный ``p`` (см. 3.1);
- **\*\*смысл не должен держаться на эмодзи\*\***: в PDF используются эмодзи-глифы – пиктограммы вырезаются, остаётся «Операции на отрезках» доедет как «Операции на отрезках», а вот «значит да / значит нет» потеряет смысл, пиктограммы
- спойлеры в PDF раскрываются полностью – не прячьте ту часть, минимум, но и не дублируйте его в основном тексте;
- картинки страниц берутся из ``CHAPTER_IMAGES`` (``scripts/export/html-to-blocks.m``); снимки демонстраций – из ``tmp/viz-shots/*.png`` (их регенерация ``npm run export:shots`` в CI); локально без браузера в ``src`` описание вкладок и переменных из ``PAGE_SYNC``.

Цель по объёму: 40–60 страниц, жёсткий предел – 100.